Cardiovascular assistance system

Contents

Analysis:

My idea involves a diagnosis/risk assessment system for cardiovascular issues, followed by medically appropriate advice and next steps to help prevent further issues. The software aims to assist those who are unable to afford expensive diagnostic appointments at private institutes or those who do not wish to go through the long winded and tedious process of getting an NHS appointment. The program doesn't intend to professionally diagnose

someone, it more so acts as a prerequisite to further medical assistance, preventing the usual step of a costly or time- consuming check-up.

Since the program should be accessible to, the information needed as parameters for the diagnostic system should be easy to record/already known. This can be done by not including certain data that is accessed through blood tests or other further tests, but this should be an option to prevent limiting the system to less accurate results. The medical advice following the diagnosis will be tailored to the specific level of any cardiovascular issues, with more severe diagnosis's having the recommendation of visiting a hospital or a GP imminently to receive medication.

Other similar examples that predict the possibility of developing a cardiovascular disease in a certain period are useful as the diagnosis isn't certain or determined. If I were to do a diagnosis over a risk assessment, I would aim to have a low misdiagnosis rate, specifically in the cases where the user is at risk, but the program labelled them as healthy. This then may lead to misdiagnosis where the user is potentially healthy, but this is a much better and safer alternative to misdiagnosis's that could lead to worsened health.

I need to question biology students, teachers or experts like medical practitioners on which parameters are most important yet easily measured for this diagnostic system. I also need to check if certain parameters are necessary, even if they aren't easily accessible to create a successful and accurate diagnostic system. Additionally, to understand which format for presenting someone's cardiovascular health (a diagnosis, or 10-year probability of a heart event) I need to research and learn how users want their information to be presented to them, to help create an environment that is not overbearing and daunting.

Stakeholders:

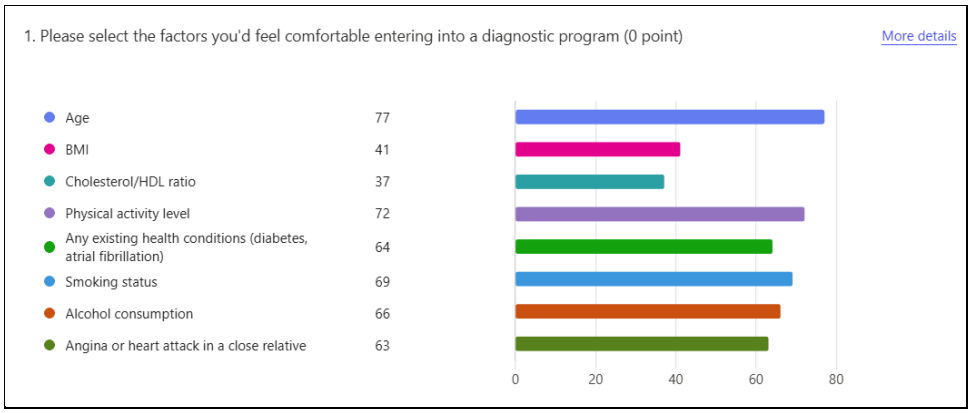
The stakeholders for my program will range from academics, particularly within medicine and biology, to individuals that could be at risk of developing cardiovascular diseases that are seeking clarity or potentially medical assistance and advice. However, the primary stakeholders will be those who are looking for a diagnosis or risk assessment. Therefore, my application should be tailored to users aged from 30 to 90, with a particular focus on those who are older than 50, as they are more likely to develop cardiovascular issues. I anticipate that the users will utilise the site alongside professional medical guidance or as a first step before consulting a medical practitioner.

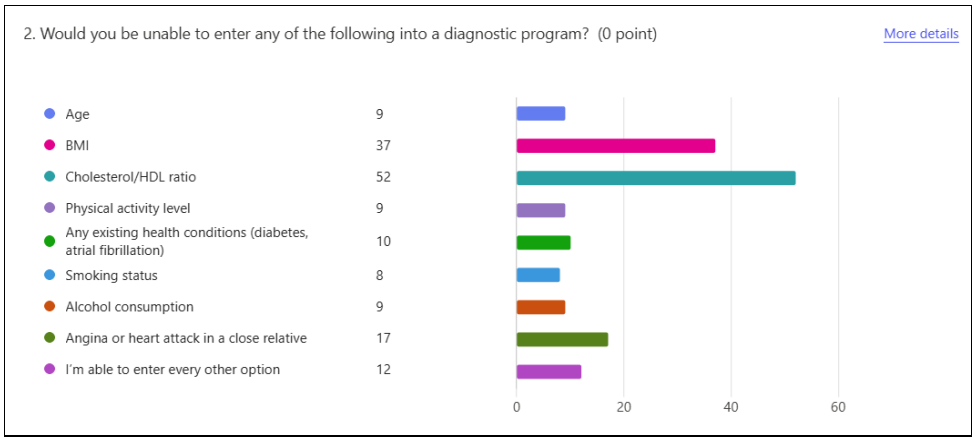
Given that cardiovascular health is often a sensitive topic, it's important that I display the results and any following advice in an appropriate and supportive manner to prevent users from taking further steps towards improving their health. To achieve this, I plan to conduct a survey to gain an insight into how users prefer sensitive information to be displayed.

Additionally, I will conduct further research related to which factors are most causal for cardiovascular diseases and how the program would situate in the medical field.

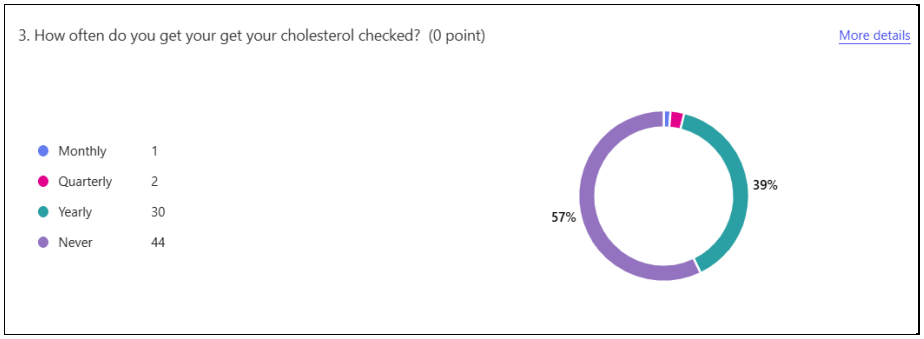
survey results and further research:

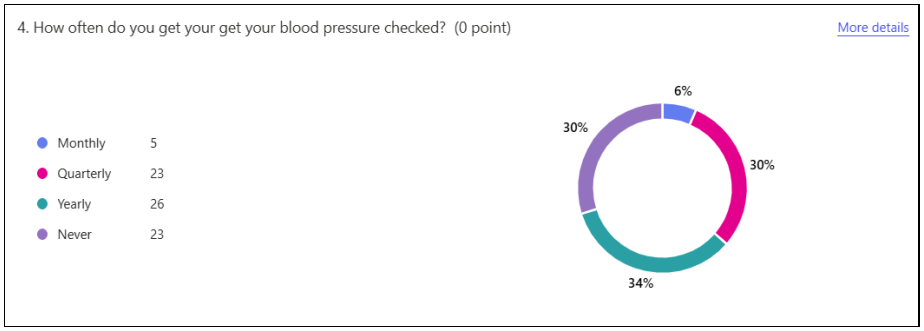
I conducted a survey with the goal of understanding user comfortability with a medical diagnosis/risk assessment program. The following questions aimed to collect quantitative and qualitative data based on different aspects of the program, allowing me to tailor my features towards the user's needs and wants.



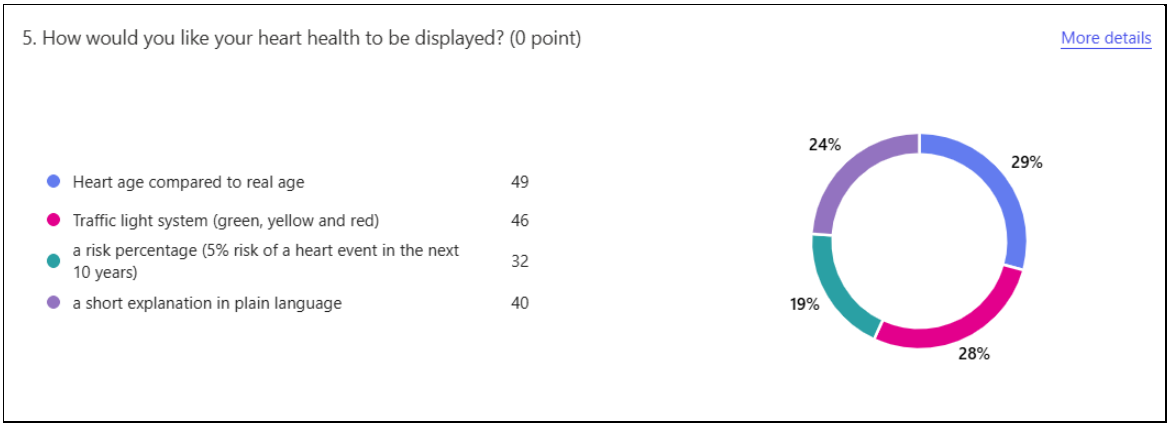


Question 1 and 2 both indicate a clear barrier in presenting cholesterol and BMI measurements. Due to the large disparity of information, I need to ensure that both are optional factors that you input. In terms of BMI, I can incorporate a height and weight input that keeps the calculations for BMI in the backend of the program, making it more accessible.

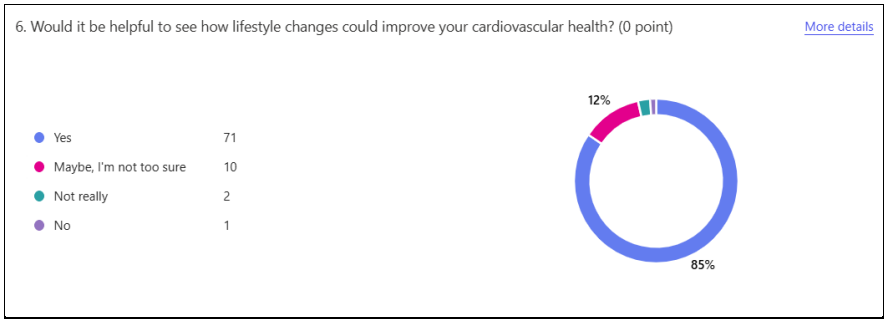


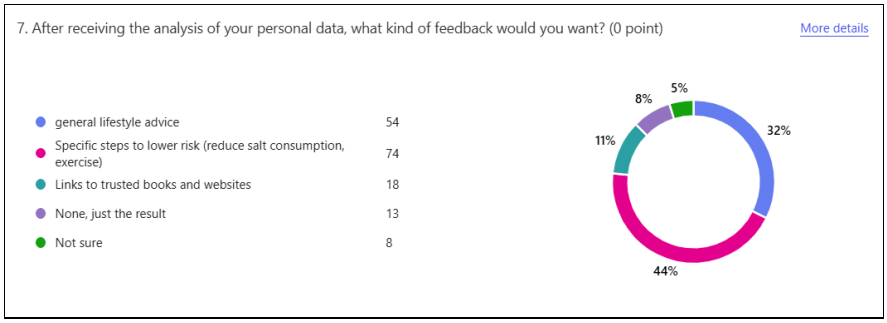


Both follow up questions corelate closely to the findings in the previous questions, users rarely get their cholesterol checked with 96% of sample users either checking yearly or never. This further supports the idea that cholesterol should be non-mandatory. However, blood pressure is a vital, causal factor for cardiovascular disease and unlike cholesterol, 70% of the sample user's measure their blood pressure either annually or even more frequently. This still indicates that blood pressure should be an optional input, but since it's more accessible for users meaning that it could have a larger contribution within the overall calculation than cholesterol.

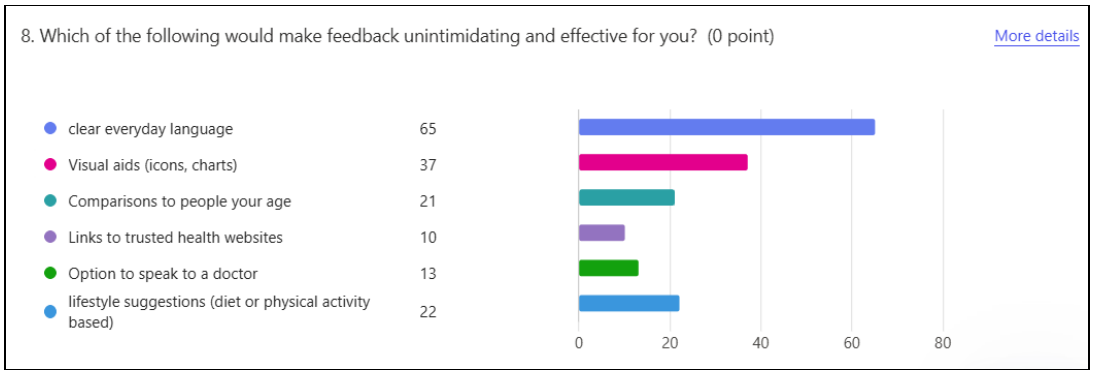


The fact that the options for the output of your heart health are quite evenly favoured is beneficial to me; whether I choose to display the analysis as a heart age or through a traffic light system won't likely reduce the accessibility of the program, meaning that whichever option is easiest to implement would be ideal as it allows me to prioritise other features that will benefit user engagement like a diary tool.





There's a clear majority of respondents that believe general advice would be valuable, with the favoured medium being potential lifestyle changes, as well as specific, possibly tailored, steps towards a healthier cardiovascular system. Considering this is the clear bulk of the sample users (76%), the other forms of feedback would be optional, and not essential for my program, their inclusion will depend on the complexity of the other necessary features.



It's evident that simple to understand, everyday language is an essential way of presenting any information, as medical jargon is a large barrier preventing varied groups of patients from truly understanding the root of their medical issue. This applies closely to my program; ensuring that the feedback doesn't lead to confusion is essential- this can be paired with visual aids to create an intuitive, user-friendly environment.

The response below to question 9 (next page) supports the previous statement, clearly showing that medical terminology can be more harmful than useful.

13	anonymous	No jargon. No acronyms. Tell it as it is. A lot of medical professionals speak to patients as if they are able to understand medical acronyms and jargon. A simple, honest and easy to understand explanation of their condition and the treatment available (if any) is what people are looking for. The truth too.

Question 9 allowed all the respondents to express their own opinions related to the functionality of the program and multiple varied responses provided further understanding on how the program should work.

9. Is there anything not mentioned or more specific that would lead to you trusting or being more willing to use a health tool like this?

The following responses indicate that user confidentiality is vital for a trustworthy user experience and since there's no harm in including this feature, even for those who are indifferent, it would be important to include in the program.

6	anonymous	A guarantee that the data will not be shared to third parties, it might impact health or travel insurance. Might even effect the wider family as often family history is asked.
7	anonymous	NA
8	anonymous	Responses kept confidential

Identifying which factors contribute most to your cardiovascular health is essential if I wish to provide accurate and relevant information during the analysis of your results as well as how I frame the feedback.

(CDC, 2025) provided valuable insights on the key risk factors for particularly coronary heart disease. **High blood pressure, high cholesterol and smoking** are the three key factors identified, with at least 47% of the population (United States) having one of the mentioned risk factors. Other vital but less significant impacts on your health include **diabetes, obesity, excessive alcohol use, lack of physical activity and an unhealthy diet (high salt intake)**. Additionally, other verified sources like (World Health Organisation, 2025) and (Teo & Rafiq, 2025) further support the previously mentioned risks as proven markers for cardiovascular disease.

Therefore, evaluating someone's obesity level, physical exertion and a presence of diabetes should be prioritised alongside alcohol consumption and smoking history. Considering high blood pressure and cholesterol levels are directly correlated to sustained unhealthy habits (as previously mentioned), which is supported by (Sabine van Oort, 2025), focusing on someone's lifestyle choices can be more beneficial than checking cholesterol and blood pressure, since these are both barriers for a wide range of my users.

User requirements:

Feature	Justification	Importance
An **opening page** containing text that details the function of the program in a large, easy-to-read font.	In the following examples you'll see a general trend of accessibility being a priority. Due to the stakeholders potentially struggling with technology, as they range from 30 to 90 years old, it's vital that visual accessibility is considered. Additionally, it's important that the language /font used is clear, and understandable for all; the survey recognised that a lot of potential users prioritise the use of non-medical terms to prevent confusion and misunderstanding. Creating a simple and intuitive display doesn't increase complexity during the programming stage, making it a definite essential.	Essential
A Simple, intuitive **user interface** that allows users to enter their data, with some boxes being optional.	To maximise accessibility, certain inputs, such as cholesterol and blood pressure, will be optional, as many users won't have access to these measurements. According to my survey, 59% of participants reported they would be unable to input either their blood pressure or cholesterol, clearly showing the need for these fields to be optional. Additionally, prioritising simplicity will help ensure that a wider range of users can interact with the program without feeling overwhelmed. This approach is consistent with other similar programs like Qrisk3 and NHS heart age. Making the inputs optional may lead to computational complications, as two different regression models may be created to consider those who have or don't have the optional measurements. However, implementing this feature if I use k-nearest-neighbour (KNN) would be simple, as it would only require selection.	Essential
A direct and logical **evaluation of their cardiovascular health**	Whether I choose to present the findings as a risk probability, a heart age comparison or through a traffic light system, it's essential that the format and presentation is easy to interpret and user friendly. The goal is to ensure that users feel encouraged rather than intimidated when engaging with and improving their cardiovascular health. Both QRisk3 and the NHS heart age use visual aids and minimal text to present your results, which is an approach that aligns closely with my survey findings. Most respondents showcased a preference for either heart age comparisons or a traffic light system, implying that these methods are more effective in	Essential

	promoting understanding and positive engagement. All the different heart health display options are simple to implement, as a traffic light system can just display the identified band of cardiovascular health for the user.	
Displaying which **specific factors likely contributed to the result**	Being able to outline which specific factor/factors relating to their cardiovascular health led to the result will likely provide clarity for the user, while simultaneously building trust with the user. Providing a 'why' behind the result (not just the 'what') makes the program more transparent and helps users feel more informed about their health. An example of this is in NHS' heart age where the program states how as your cholesterol varies (if you didn't input a reading), your heart age varies too, as cholesterol is largely correlated to your cardiovascular health. Not only will this provide clarity and build trust, but it will also make the program more scientifically robust, as it's important that you inform the users of the potential discrepancies with such an analysis. Doing this in an algorithm like KNN wouldn't be difficult, as you should be able to identify which factor linked the user to a group the most. You could also do this if I use logistic regression, as you can create baseline examples for what a 'healthy' person's measurements would look like and then you can compare the data to a user. This feature doesn't create a computational barrier and it's still efficient and reasonable to implement this in the program.	Essential
An appropriate follow up that provides **advice** and **guidance** related to your **cardiovascular health**.	Following the analysis of your data, what's arguably the most vital feature of the program is how potential lifestyle changes are presented to the user, ensuring that they don't feel pressured, which can result in a lack of change. Most respondents felt that general lifestyle advice would be beneficial with the guidance predominantly being small, incremental steps you can make towards healthier habits. Considering this approach doesn't involve drastic, sudden and extreme expectations, it's not surprising that this seemed to be the most user-friendly option.	Essential
Anonymous data storage or a **lack of storing user's data**	Multiple respondents said that they would want their data to be confidential, and not saved, which is understandable as the concern surrounding our digital footprints is prevalent in the modern day. This is very easy to implement, as a lack of storage is easy to program and	Essential

	reduces the processing power required for the client and server.	
Calculation display through mathematical equations and potentially graphs.	Displaying the inner workings of the algorithm as it calculates a user's risk will allow for computer science, biology and mathematics students to identify the underlying calculations that contribute to the result. Since the calculations are already being/have been done, it shouldn't be difficult to display them in a computational sense. This widens my user base to a potentially younger target audience, but it would contain college/ university level concepts in a medical context, meaning it doesn't change any of my GUI choices. Furthermore, my main target audience is still those who are at risk of developing cardiovascular diseases, so this feature would be desired but not necessary.	Important
A tool that recommends **diet changes**	A lot of respondents wish to see how small, incremental adjustments in their daily habits like your diet can alter your cardiovascular health. Allowing users to enter some of their general dietary habits, followed by small positive adjustments will lead to a maximised range of users (76% of survey respondents claim that general lifestyle advice and specific steps to lower risk would be the most beneficial). However, as you add more specificity and precision to the diet tool, the difficulty in developing it increases, meaning it must be quite general for me to implement it successfully.	Important
A **diary tool** to track your progress	Considering 83% of respondents would prefer general lifestyle advice that could improve their cardiovascular health, providing a diary tool that allows users to track their progress without any rigid restrictions surrounding what they can store provides freedom and accessibility. This aligns closely with myHealth, which provides a similar tool.	Important
The ability to **contact the NHS** through the program	Being able to directly contact a medical practitioner was requested as an additional feature by a respondent; Although this would be a useful utility, it would likely have to be localised for user's GPs, but most stakeholders are aware of how they can access their practitioners. However, the NHS is easy to access through an email or a phone call, making it easy to add contact information for	Optional

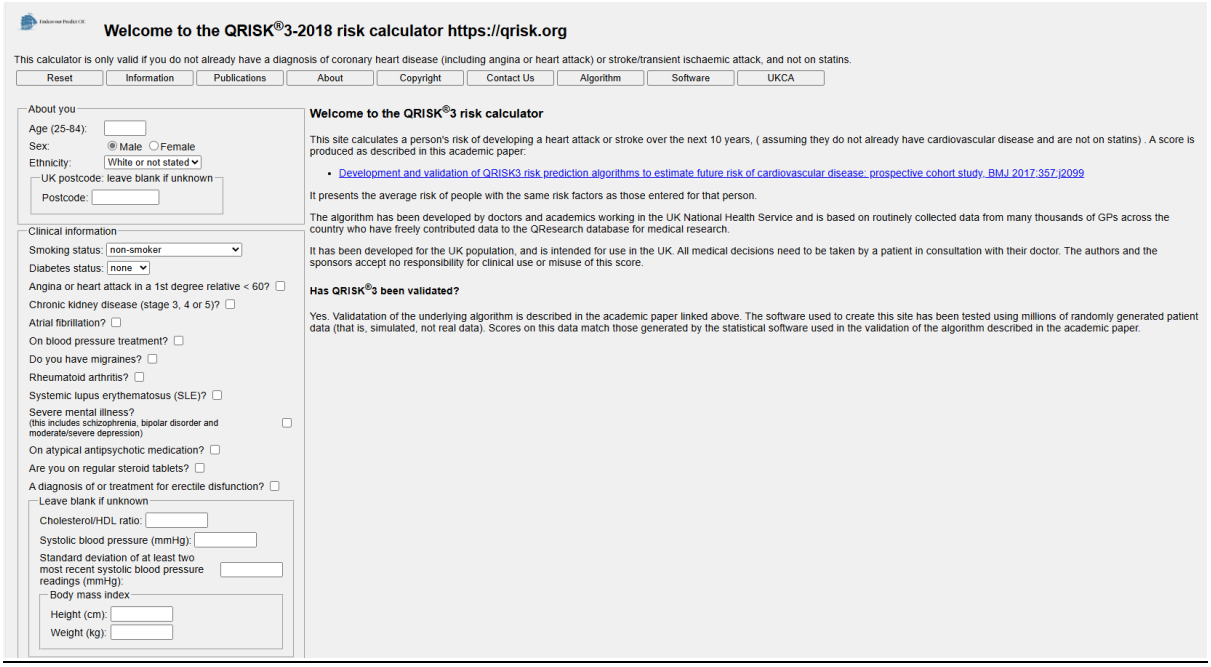
	the specific department correlating to cardiovascular issues. Program examples like myHealth include the ability to contact your doctor, but to sign up for this app you need clinical clearance, which reduces accessibility.	
The ability to **share your progress and results** with people of your choosing	Despite this feature somewhat clashing with the anonymous data storage feature mentioned earlier, it's possible that the data can be stored safely and securely while allowing you to share your progress and analysis with other users. However, this is more optional as you can easily share screenshots or simply explain your situation to family members or friends using social media, making this feature a non-essential. Additionally, implementing a social environment within the program massively increases the development complexity, meaning that if it would be implemented in the short time period allocated, it would have to be simple, reducing the appeal for the feature.	Optional

Limitations:

Feature	Justification
Bluetooth connection to certain medical devices like a blood pressure monitor or a scale that measure BMI	Although this would be a useful feature, it would be extremely difficult to implement successfully as Bluetooth is a difficult feature to implement. Additionally, it barely adds any extra functionality, as you can simply measure your blood pressure and BMI, then add it to the diary tool if you wish to track it, making implementing this feature a waste of time.
Using the program to directly contact and message a clinician	Being able to contact a clinician/practitioner is a feature I want to include, but not directly through the program. I hope to encourage users to contact their GP as well as providing an email or number to contact the NHS, but direct messaging is a redundant feature - considering it's very easy to use other messaging services - that would be difficult to implement.

direct prescription or medication recommendations	Considering direct medical recommendations would lead to the program being classified as a professional medical tool, resulting in regulation under MHRA. Additionally, since the program isn't medically certified and accurate, it would me immoral to recommend medication that could cause more harm than good.
real time, immediate, monitoring or care	Real time monitoring would require integration with wearable devices like a heart rate monitor. It would also require a 24/7 response system that connects you to medical practitioners, like the NHS. Including either of these features leads to legal liabilities of the program's users, which can lead to ethical and legal concerns.
access to NHS medical history for extra diagnosis support	Being able to access a user's medical history through NHS records would require confirmation from the NHS directly, or possibly confirmation from the user's local GP. Along with this, strict data handling policies would need to be in place, which is complicated and time consuming to set up correctly.

Similar existing programs:

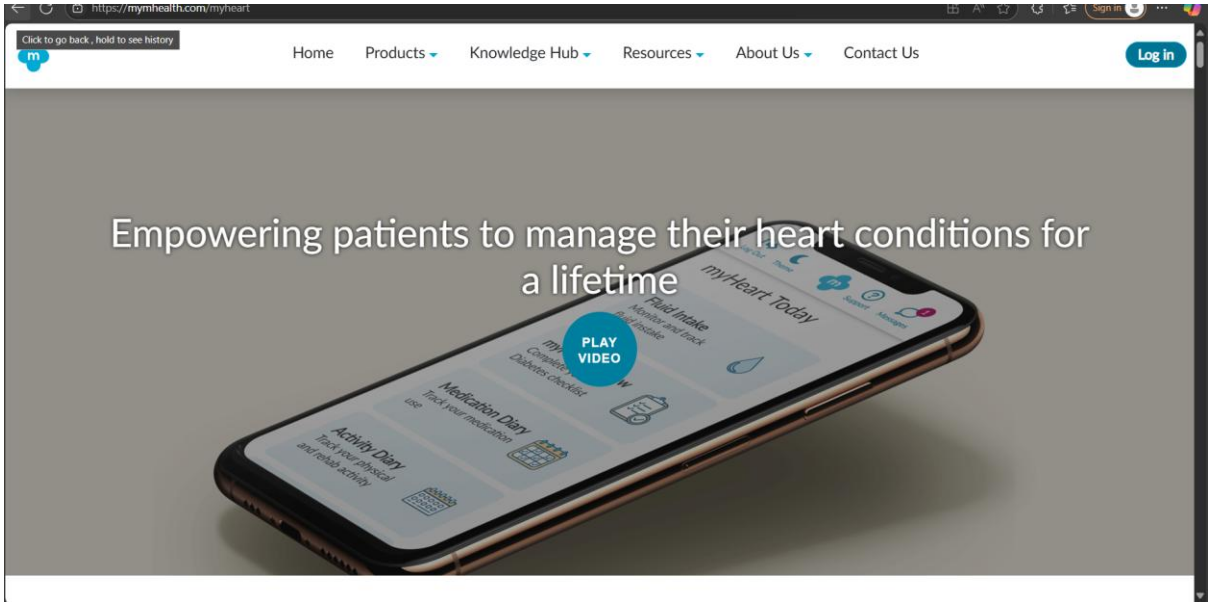


(Endeavour Predict CIC, 2025)

Qrisk3 – This program calculates the user's risk of developing a heart attack or stroke over the next 10 years, assuming they do not already have any implants or disease's relating to the cardiovascular system. It does this through the input of different parameters, like age, blood pressure and if you're using varied medication. Following this, it gives you a probability of the previously stated events occurring using a cox proportionality hazards model, created through the analysis of a large data set, specifically the Qresearch database.

Features I want to take forward	Features I don't want to take forward
At the top of the website, it lists the supporting information that holds medical validation of the program, open-source code and licensing and further resources they used to help develop the program in a well formatted website. This helps to prevent fear and confusion within users, which may be prevalent for stakeholders as they may be insecure about a diagnosis or fear the consequences.	Following the diagnosis, if the score/probability is quite high, for example 30% or more, there is no re-assurance or positive language used to help indicate that a lifestyle change is still possible, even if it has little effect. Some users may be isolated and older people who don't have the best mental wellbeing, and a lack of encouragement can be taxing and damaging on their mental health.
Following the estimation of the probability of a heart attack/stroke, it provides the same estimate of a healthy person with the same age, sex, and ethnicity as you, to	There's a lack of information related to finding out your readings and measurements for those who are insecure leading to potential wilful ignorance. If the website

provide as a benchmark to compare your cardiovascular health. This can help reduce fear and anxiety related to your score, potentially making it less daunting to work towards a healthier lifestyle.	provided encouraging information related to calculating the data necessary to get a reading, it could lead to more people learning about their health and how they could improve it.
Furthermore, after the estimation it additionally provides a graphic that represents the likelihood of x amount of people having a heart attack/stroke in the next 10 years to provide some context to the score you're given. This can help provide context to certain users who may not be able to contextualise a probability, especially the older generation who are likely to be my stakeholders.	For those who find it interesting and valuable, there isn't any information on how the probability is calculated. The program could not only be useful for those who have cardiovascular risks, it can act as an educational tool for aspiring medical students, so providing the context for the calculation creates an extra layer of functionality and broadens the potential stakeholders making the program more impactful.
Since some of the input data isn't very accessible, like cholesterol/HDL ratio, which would require a blood test, not all of them are necessary to get an estimation. This prevents certain users who haven't had professional readings from shying away from using the program, leading to a wider range of potential stakeholders.	The algorithm used to determine the probability of the risk is a cox proportional hazard model, which I likely would not want to do. Although it uses some form of regression, the hazard model is tailored to a time-based event, while I am doing a diagnosis.

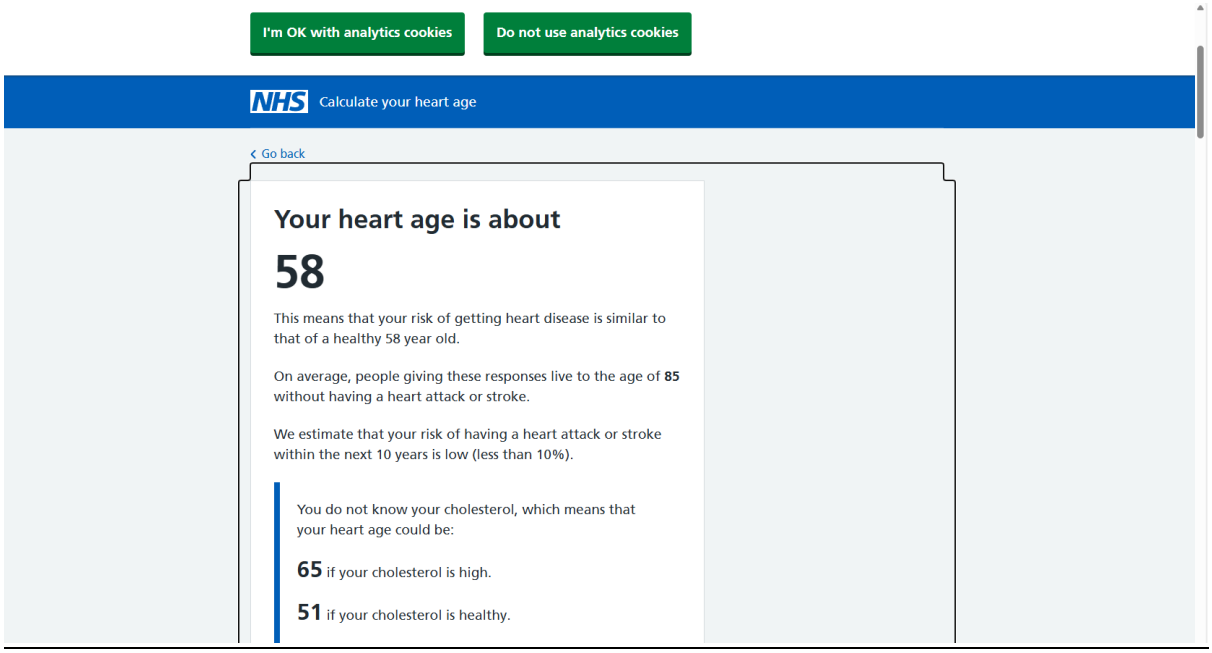


(mymhealth, 2025)

myHeart – myHeart is a part of the overarching website, mymhealth, which aims to support individuals dealing with various health conditions. myHeart specifically helps those experiencing heart failure, angina, valve replacement, valvular heart disease, and other cardiovascular-related issues. Unlike Qrisk3, myHeart focuses on providing helpful, easy-to-access, and encouraging educational videos on rehabilitation and diet, as well as features like a personal diary to track progress and direct messaging access to your clinician.

Features I want to take forward	Features I don't want to take forward
Although the website has a lot of information thrown at you, it opens with a video that explains what it is capable of. Since the stakeholders may be less capable with technology, this is a great way to cater to your user base while also getting your point across in a succinct and appropriate manner.	The app doesn't provide a diagnosis or estimation tool like Qrisk3. If you add this functionality, it can widen the potential range of stakeholders even further.
Every major thing you should consider when bettering your health after a diagnosis, like friendly exercises, a better diet and access to your practitioner is provided within this app. Not only is this available, but it's also presented in a supportive, and encouraging way that doesn't make it seem impossible to improve your lifestyle.	The £40 annual subscription may prevent some users from accessing the app. This could be due to either financial constraints or a lack of willingness to prioritise health-related spending.
Since it provides a diary tool that allows you to track your progress, plan your medication schedule and determine what exercises you need to do, it creates an environment where users can monitor their health while also holding themselves accountable. This can prevent the common "New Year's revolution" effect where a lack of visible progress can lead to demotivation.	The app requires clinical sign-up, which makes it difficult for self-motivated users to access its helpful resources. Many individuals can recognize the need to improve their heart health even without a formal diagnosis, so increasing accessibility for self-service users would make the app more inclusive and effective.
It uses large text with an easy-to-read font along with photos accompanying the small paragraphs providing information, which can be accessed by simply scrolling down the continuous website. This aligns with their stakeholders, who are likely above 50	

and may struggle to read certain text and control devices well.



(NHS, 2025)

NHS – The NHS provides a tool that allows you to determine your heart age based on certain parameters like your blood pressure, cholesterol, age, height and weight. Following your estimated heart age, it provides information surrounding potential causations for an elevated heart age and how you can reduce it through changes in your daily lifestyle.

Features I want to take forward	Features I don't want to take forward
This model uses the Joint British Societies model that uses logistic regression to determine the outcome based on your data, which is how I may determine a diagnosis or risk assessment.	The diagnosis doesn't use a lot of uncertain language showing that it isn't definite. The program has no way of determining an accurate heart age with this little amount of data, so it should repeatedly inform the user of the potential margin of error and that a doctor's diagnosis is always most accurate.
After the diagnosis, it educates you on the parameters that determine the calculated	Although it identifies which parameters are most likely contributing to a higher heart

heart age, like whether you've smoked on your blood pressure. This allows the user to understand what part of their lifestyle may need prioritising.	age, it doesn't use particularly encouraging language. This may discourage users from seeking assistance, especially when the parameters involved are ones, they're already aware of, such as weight.
If you don't know certain inputs, like cholesterol, it tells you your heart age as that unknown variable varies from low to high, providing some extra information for the user.	The logistic regression used, or calculations done for your heart age isn't accessible, so if you're wanting to learn about statistics in medicine or just statistics in real life scenarios, it can't be used on an educational standpoint.

Hardware and Software requirements:

I will use python as my chosen language, with the GUI being presented through the library 'tkinter' on Windows.

Hardware:

- Since this is a simple medical program, it does not require specific computer components to run, a general-purpose computer would suffice. Most medical programs in hospitals will have specific hardware to function, but my design makes it widely accessible to GPs and hospitals.

- If I use logistic regression The computer's CPU won't need to be very powerful, as there isn't any concurrent processing. The taxing computation will be done before the program is complete as regression will be used to determine the weights for the different inputs. However, if I use K-nearest-neighbour (KNN), some computational power will be required to compute the searching algorithm. Therefore, in the case of KNN being implemented, a more powerful CPU would be preferable, but it still isn't necessary as both algorithms use less processing power than neural networks.

- A keyboard will be needed for the program as it won't be touchscreen, so any laptop, desktop or tablet with a keyboard.

- It won't require any speakers as no sound is necessary. This widens the potential devices stakeholders can use to run the program.

- There won't be any storage requirements; if I do choose to store the data inside a database, it would be server side not client side as this improvements data security and keeps the user requirements low and accessible.

Software:

- Using python and tkinter means that software requirements are flexible and common among computer owners. This also reduces installation issues for my stakeholders, which is essential as they will likely be older than 50, meaning technological difficulties will be apparent.

- For the program to run specific software requirements won't be required, as long as you can display the GUI, which is possible if you have a screen and the python libraries downloaded.

- Python is free and available on the most utilised operating systems, like macOS, Windows and Linux, making it the most accessible language for my stakeholders.

- Despite python being available on some mobile devices, the program won't be designed for touchscreen input. This would only increase the build time for the program, leading to desired features possible being left out, reducing the program's quality.

- To run the program on your own device you must have python and tkinter downloaded, and potentially other libraries like numpy for the mathematical calculations, but all the modules mentioned are accessible and free, meaning it's very easy to access my program.

Computational approach:

This problem can be approached computationally due to its algorithmic nature and the amount of data analysis and processing required. Considering, a large database with over 50,000 records will be used to train the algorithm, analysing this depth of data by hand would be near impossible and extremely tedious.

Abstraction will be vital for this program, specifically **representational abstraction** and **problem reduction**. Removing unnecessary data like specific fields within large datasets that won't be needed allows me to focus on the contributing factors for cardiovascular disease. Although the calculations for the program may be on display for certain curious

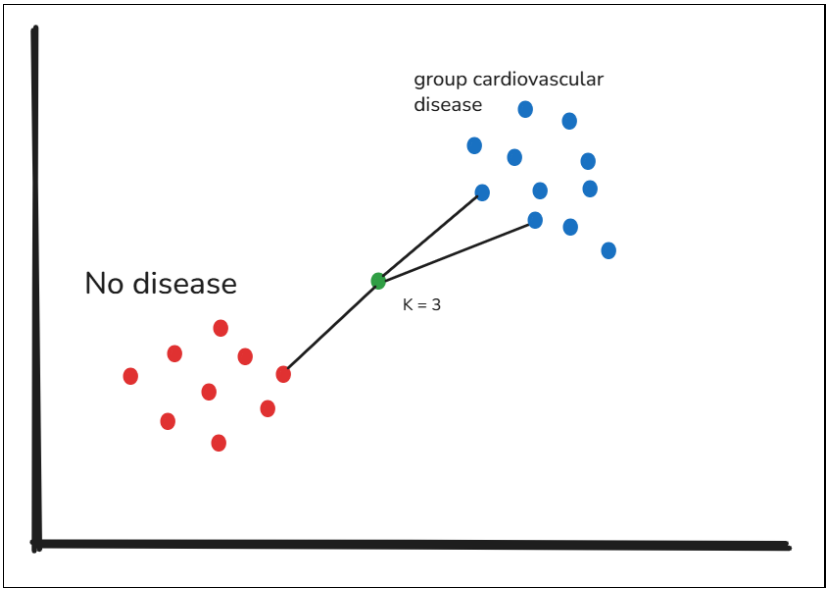
users, this wouldn't be displayed automatically to prevent crowding within the GUI, leading to confusion. Additionally, problem reduction is necessary for this specific program, as both logistic regression and K-nearest-neighbour (KNN) are well known, previously used algorithms that can be applied to this specific problem. This prevents unnecessary time loss creating an algorithm and it allows me to utilise the most efficient approach possible.

Decomposition through a **top-down design breakdown** will allow me to consider the different factors as individual weights that contribute to the final calculation. Additionally, breaking the whole problem down recursively into different and smaller sections like separating the diagnosis/risk assessment algorithm and the feedback relating to lifestyle changes into individual parts will allow me to better understand how they should be presented alone, leading to the full program's presentation being more well put together.

Algorithms

K-nearest neighbours (KNN)

KNN is one of the simplest machine learning algorithms you could use for classification. The algorithm involves using a sample data set which is labelled in different categories, and based on a user's input data, they're matched against the closest x amount of data points. For example, if I had a dataset containing 1000 patient records, my user's inputs would be paired with the 3 closest patient records based on the inputs and the most common classification is assigned to that user. If two of the 3 closest patients have coronary heart disease, the user would attain the same diagnosis.



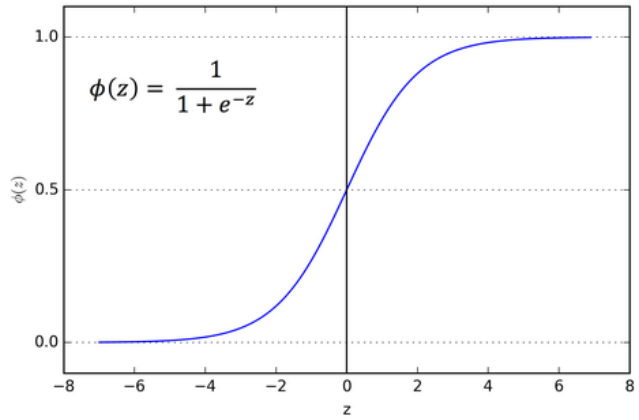
source: https://excalidraw.com/

On the left is a very simplified example of the K-nearest neighbour algorithm, where the blue group represents sampled patients with a cardiovascular disease and the red group represents sample patients without a cardiovascular disease.

We look at the 3 closest neighbours (as k is 3) and whichever group has more closest neighbours represents that data point. KNN is simple enough to understand for the students and medical practitioners using my program as an educational resource, as additional machine learning knowledge isn't required. Therefore, it would be a suitable and applicable choice for my scenario as it also works very well for cardiovascular health assessment. This is a much more suitable option compared to neural networks for example, as there is little computational overhead since only simple calculations are needed as the user runs the program. It's also a lot simpler to implement compared to a neural network, which requires complex mathematics and weight training, making it suitable for a short term NEA.

Logistic regression

Logistic regression is a statistical model used for binary classification, where data points are fitted to different classes against a shaped (logistic) curve. For example, different inputs are multiplied against weights (representing their importance in the final calculation) and summed. This sum is then converted to a number between 0 and 1 using a sigmoid function. This final probability can fall into different predetermined groups to identify the diagnosis. For example, any probability >0.6 are diagnosed with coronary heart disease.



On the left is a sigmoid function, where any value of z forms a number between 0 and 1.

Source: https://www.linkedin.com/pulse/understanding-sigmoid-function-logistic-regression-piduguralla/

Implementing logistic regression in this sense involves pre-determined weight assignment based on the dataset used, making it a more complex to implement compared KNN which is instance based and requires no training. However, logistic regression is an established technique used in modern medical fields for similar programs where transparency and reliability are essential. Therefore, the accuracy of the program will likely be higher if I use logistic regression, and the majority of the computational processing is done during the training of the program, making it easy for stakeholders to run.

Analysis References

CDC. (2025, 07 02). *About heart disease*. Retrieved from Heart disease: https://www.cdc.gov/heart-disease/about/index.html

Endeavour Predict CIC. (2025, 06 24). *Qrisk3*. Retrieved from Qrisk3: https://www.qrisk.org/

mymhealth. (2025, 06 26). *myHeart*. Retrieved from myHeart: https://mymhealth.com/myheart

NHS. (2025, 06 27). *Calculate your heart age*. Retrieved from NHS: https://www.nhs.uk/health-assessment-tools/calculate-your-heart-age/results

Sabine van Oort, J. W. (2025, 07 02). *Association of Cardiovascular Risk Factors and Lifestyle Behaviors With Hypertension*. Retrieved from ahajournals: https://www.ahajournals.org/doi/pdf/10.1161/HYPERTENSIONAHA.120.15761?

Teo, K. K., & Rafiq, T. (2025, 06 02). *Cardiovascular Risk Factors and Prevention: A Perspective From Developing Countries*. Retrieved from National Library of Medicine: https://pubmed.ncbi.nlm.nih.gov/33610690/

World Health Organisation. (2025, 07 02). *Cardiovascular diseases*. Retrieved from who: leading health emergency responses: https://www.who.int/health-topics/cardiovascular-diseases#tab=tab_2

Design

Development plan

Stage 1 – identifying the dataset and implementing logistic regression (31st September 2025)

Implementing the algorithm (logistic regression) based on a chosen dataset first allows me to design the user interface based on said data. Being able to identify how certain inputs should be taken in from the users based on how these parameters are stored within the database will allow me to cater the interface to the specifics of the dataset.

Using logistic regression also allows me to identify which factor is likely contributing to a poor cardiovascular health compared to k-nearest-neighbour, where identification is much more difficult. Additionally, the dataset will likely have over 50,000 records, so comparing the Eulerian distance to all of them with KNN will be computationally taxing for the users. I'll download the dataset as a csv file and scrape any of it using file handling to process the data.

Since some data will be optional and some won't be optional, I will need to create two different logistic regression models with the difference being what parameters are being considered. Both models will be tested the same, just one won't include all the data that the other is being tested with.

I'll use libraries like numpy and panda to do the complex mathematics required. This maths includes cost functions, gradient descents and sigmoid functions, which involve recursive differentiation and complex linear algebra. Creating a class in python for the regression model will allow me to separate all the functions and calculations from further manipulations I will do as I utilise the model.

Features to implement:

1. A regression model if all fields are used
2. A regression model if only necessary fields are used
3. A sigmoid function/another function to get a score between 0 and 1.

Test no.	Description + test data	Type of test	Expected outcome	outcome

1.	Enter exact data for a record which isn't boundary data	Valid	The corresponding value between 0 and 1 for their cardiovascular disease status.	
2.	Enter data that is close to a record in value but not exact (healthy)	Valid	The same cardiovascular disease status as that record	
3.	Enter data that is close to a record in value but not exact (unhealthy)	Valid	The same cardiovascular disease status as that record	
4.	Enter values on the lower range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	
5.	Enter values on the upper range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	
6.	Enter data values outside the range for every field in the dataset (unhealthy end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	
7.	Enter data values outside the range for every field in the dataset (healthier end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	
8.	Enter a negative value for 1 field	Error	This should still generate a number between 0 and 1, however the value may be skewed or inaccurate.	

Stage 2 – Creating a user interface (UI) main menu (15th of October 2025)

This will contain the section that takes user inputs for their cardiovascular analysis and diagnosis, so doing this stage after developing a regression line based on the data allows me to identify how certain inputs should be taken. It also allows me to identify which

parameters I should include, meaning I can decide if any unexpected parameters that are in the dataset can be added as optional inputs. Since optional inputs are now being incorporated, I need to ensure that there are two different regression models, one for optional inputs, and one without.

Additionally, I need to create a tab/slider that allows users to change the size of the font, and even change the font entirely, but this will be unlikely as ill use a general, basic font to prevent this issue. Alongside taking user inputs, this tab will contain some very general advice and information on what the program does, and where you can find any further information in other menus.

Since this menu takes user inputs, it's important that there's a lot of validation included. This involves preventing erroneous data (negative numbers or characters) as well as outlining data that is far outside of any given range, highlighting how this could be a major health risk or potentially a miss input.

Features to implement:

1. A large UI that displays information + a title alongside the input section
2. User input text boxes/drop downs for each input with the ability to only enter necessary inputs or optional ones too
3. A slider/setting to change the font size or font
4. A validation system to detect extreme/ erroneous data

Test no.	Description + test data	Type of test	Expected outcome	Outcome
1.	Enter exact data for a record which isn't boundary data	Valid	The corresponding value between 0 and 1 for their cardiovascular disease status.	
2.	Enter data that is close to a record in value but not exact (healthy)	Valid	The same cardiovascular disease status as that record	
3.	Enter data that is close to a record in value but not exact (unhealthy)	Valid	The same cardiovascular disease status as that record	
4.	Enter values on the lower range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	
5.	Enter values on the upper range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	
6.	Enter data values outside the range for	Boundary	Likely generate a number very close to 0 or 1 depending on	

	every field in the dataset (unhealthy end of each field)		what corresponds to cardiovascular disease.	
7.	Enter data values outside the range for every field in the dataset (healthier end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease Reject the inputs with an error message.	
8.	Enter a negative value for 1 field	Error	This should still generate a number between 0 and 1, however the value may be skewed or inaccurate.	
9.	Enter characters (abc) for a numerical input	erroneous	Reject the inputs with an error message	
10.	Enter negative numbers for an input that accepts positive numbers	erroneous	Reject the inputs with an error message	
11.	Enter an extreme value for a parameter	Extreme	Rejects the input and informs the user but also recommends seeking medical help	
12.	Leave an input blank	erroneous	Reject the inputs with an error message	
13.	Choose to include the optional input data as an option then leave them blank	erroneous	Reject the inputs with an error message	
14	Choose to not include any optional input data and enter valid, expected data	Valid	Gives a valid diagnosis that has correlation to their data.	
15.	Choose to not include any optional input data and enter that data as inputs	Erroneous	Rejects the inputs with an error message	

Stage 3 – Create the diagnosis + feedback UI (30th October 2025)

This UI/menu will follow the main menu directly, as once you've entered and submitted your data you will be directly taken here. This menu will provide the feedback and diagnosis based on your data.

The feedback will be presented either as a traffic light system or a heart to real age comparison, since these were the most picked options based on my survey. Both can be selected as options by the user, but both will be accompanied by a short text in plain language as this was also a top option by stakeholders. This short paragraph will detail the diagnosis but also outline any likely health factors that led to the diagnosis if they were deemed 'at risk'. If there was no single parameter causing this, it's also important to highlight that for the user, meaning that this feature must be properly implemented, validated and tested. There would need to be a cutoff for when certain factors are outlying compared to when a user is unhealthy for multiple reasons, making it difficult to identify the cause.

If the user didn't enter any optional data, I plan to offer an estimation on what their diagnosis would look like based on the rest of the data, although this feature won't be extremely accurate.

Accompanying this will be videos, graphs and photos is important to prevent confusion among users. General life advice, diet advice and potentially a recommendation to visit your GP or doctor based on the diagnosis to kickstart the user's journey to a healthier cardiovascular system will be used. Furthermore, the user will be granted the opportunity to create a diary to track their progress, but that's another stage that will be implemented after this one.

The main feature that must be tested is the diagnosis and input identification, as the other information will simply correlate to what diagnosis the user has received so it's easy to test it and implement it. Ensuring that the diagnosis not only matches the regression model, but also the identification of what factor is contributing to the diagnosis is vital.

Features I want to implement:

1. Traffic light/ heart age comparison diagnosis system
2. Outlying factor/s identification to identify cause for the diagnosis
3. Offer graphs, photos and videos to support the diagnosis
4. Offer diet and general lifestyle advice based on the factor/s identified.
5. Present the possible diagnosis if optional inputs were entered.

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data to a record in the data set (not boundary and healthy)	Valid	Heart age <= real heart age or green on traffic light system, with no outlying factor contributing to their health	
2.	Enter exact data to a record in the data set	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health	

	(not boundary and unhealthy).		factor led to this or if none are outlying.	
3.	Enter very similar data to a record in the data set (not boundary and healthy)	Valid	Same diagnosis as the first test with same factor identification.	
4.	Enter very similar data to a record in the data set (not boundary and unhealthy)	Valid	Same diagnosis as the second test with same factor identification.	
5.	Enter custom data for a healthy person with no outlying inputs	Valid	Green light or heart age <= real age without any factors identified.	
6.	Enter custom data for an unhealthy person with no outlying inputs	Valid	Heart age > real heart age or yellow/red on traffic light system. No are outlying factors identified	
7.	Enter custom data for an unhealthy person with 1 outlying input	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factor led to this.	
8.	Enter custom data for an unhealthy person with 2 outlying input	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factors led to this.	
9.	Enter healthy data without any of the optional inputs (non-boundary)	Valid	Green light or heart age <= real age with a estimate of what their health would be with the optional data	
10.	Enter unhealthy data without any of the optional inputs (non-boundary)	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify what their health would look like with optional inputs	

Stage 4 – Create the diary tool (15th of November)

This stage involves creating a tool that allows the user to track their progress as they hopefully work towards a better lifestyle by following the advice given. The tool should show the measurements entered by the user and allow them to track their progress by

seeing how measurements change over time alongside their diagnosis. It would use files on the persons computer which can be saved, accessed and updated easily.

Features I want to implement:

1. Allow the user to create a diary
2. Places their starting measurements in the diary as it's created
3. Saves whenever a user adds/ removes anything
4. Contains general diet and lifestyle advice initially based on their diagnosis

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	User selects to create a diary	Valid	If the user chooses to create the diary, the file is created accordingly	
1b.	User selects to create a diary	Valid	Upon opening the diary, the measurements and advice is already in it	
2.	User doesn't want to create a diary	Valid	No file is created	
3.	Add text to the diary	Valid	It saves next time and opens	
4.	Delete from the diary	Valid	It removes the text from the diary	

Stage 5 – Create the information and algorithm (UIs) (25th of November 2025)

These menus/UIs are there for the stakeholders who are confused on what they need to do to get an accurate reading or for stakeholders that are curious about how the program works.

The information menu will contain text that educates the user on how to get certain measurements and what they are, preventing any misconceptions and confusion. Additionally, it will tell them about cardiovascular diseases that the program is designed to diagnosis, raising awareness for those that may need it. The dataset that the program was trained against will be in the information section too, as it's important to be transparent about the program while providing credibility and reasoning for your conclusions.

Any general, common questions will also be identified and placed in this section, although most will likely be vague or just lead you to a practitioner as it's important to be medically verified to give advice on most situations.

The algorithm menu will be used to display how the algorithm works, with text-based explanations as well as graphical and mathematical explanations. I plan to allow the user to experiment with their inputs, but entering different values into a mock input section,

allowing them to see how this changes the diagnosis in real time. This allows students to study logistic regression in a practical situation while also exploring how different factors lead to different diagnoses.

Although extreme data would normally be rejected, allowing this to happen in this tab can show the user how extreme data can skew a result massively, so it is important this is allowed.

Features I want to implement:

1. An informative menu that educates the user
2. An algorithm tab that allows you to see how the algorithm works
3. Allows the user to enter custom values
4. Updates the result as the user enters values

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Open the information menu	Valid	Contains all the text required	
2.	enter erroneous data into any field in the algorithm tab	Erroneous	Rejects the inputs and provides an error message	
3.	enter valid data into the fields	Valid	A diagnosis is provided, with the calculations being displayed accordingly	
4.	Enter valid data into all fields then change a value to another valid data value	Valid	Diagnosis updates accordingly based on the regression calculations.	
5.	Enter characters into any field	Erroneous	Rejects the inputs and an error message is provided	
6.	Enter boundary data on the unhealthy range for all fields	Boundary	Accepts the data and provides a diagnosis, likely very high risk of cardiovascular disease	
7.	Enter boundary data on the healthy range for all fields	Boundary	Accepts the data and provides a diagnosis, but likely very high risk of cardiovascular disease	
8.	Enter extreme data outside the range of the dataset	Extreme	Accepts the data and provides a diagnosis	

GUI Design

Each of my GUI menus aim to optimise simplicity, readability and accessibility for my stakeholders so they're able to access the program without any issues. I've considered these factors throughout the design of my GUIs through font selection, colour selection, font size and text placement evaluation.

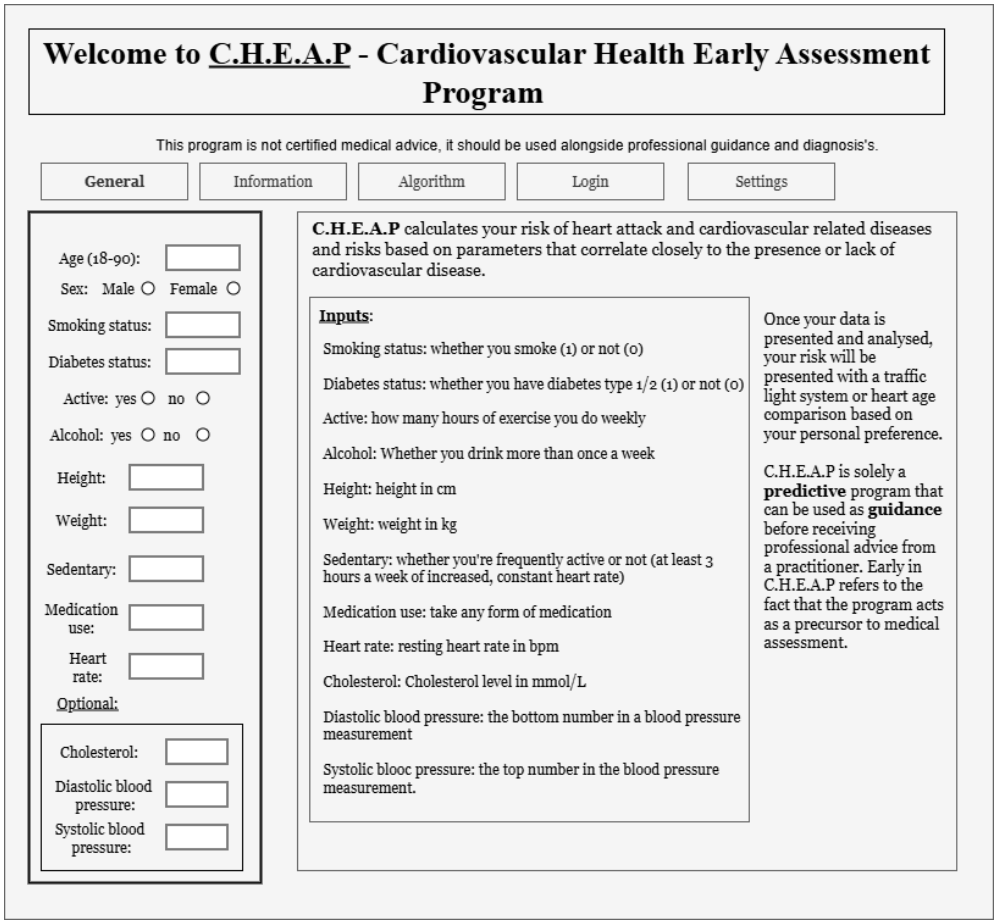
General

The following information contains constant details included in each of the GUI menus as well as the reasoning for that. Each of the menus use a consistent font to prevent confusion, as using multiple different fonts may lead to misunderstanding and it will increase the risk of some fonts being illegible to certain users. The fact that most of my stakeholders will be older than the general public, every consistent GUI consideration has this in mind.

The lack of colours is also used to prevent confusion and overwhelming the users, whilst simultaneously preventing any colour blindness issues. Additionally, the monotonous palette suggests that the program has a professional, serious tone, which is useful for the intentions of my program. Furthermore, the grey background contrasts well with the black text, making it a much more viable option than white, which I feel is too aggressive.

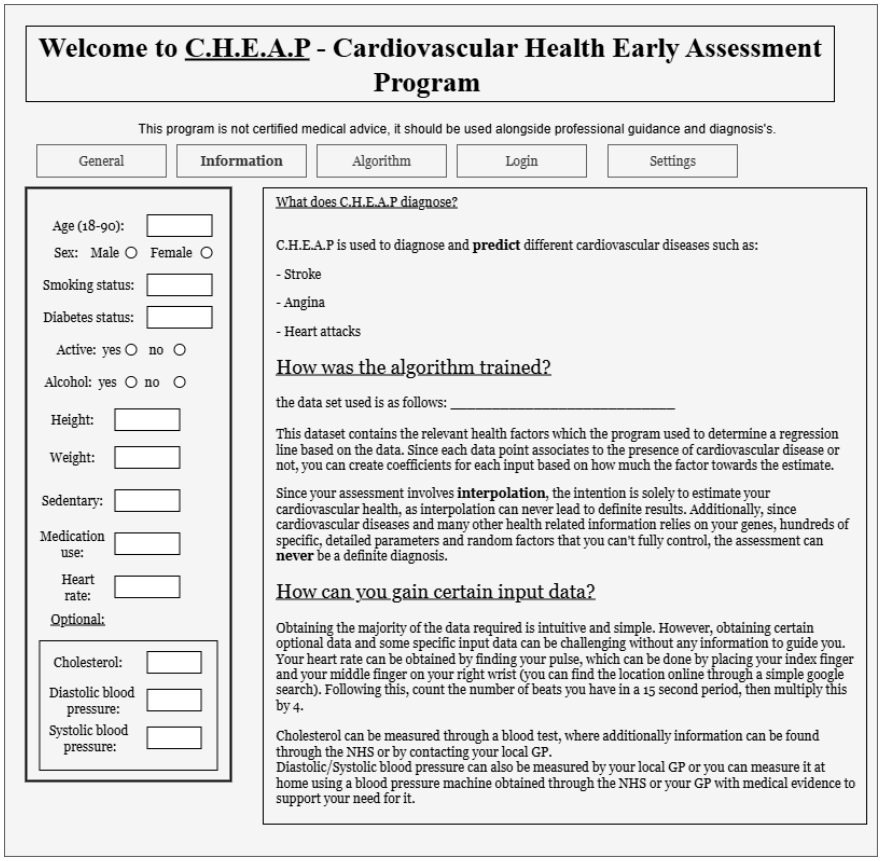
Using boxes for different sections of the program means that information that correlates can be identified easily and potentially subconsciously, as you likely attach anything within a box together. Placing the boxes at the top with your current tab in bold prevents further confusion as you could easily get lost since the tabs use the same colour scheme and font. Since they are high up on the screen, they should take priority as you scan the page and use the program.

Any key words I've identified are **bold**, as it's important that certain ideas are highlighted for my stakeholders, such as the analysis being a prediction rather than a definite diagnosis.

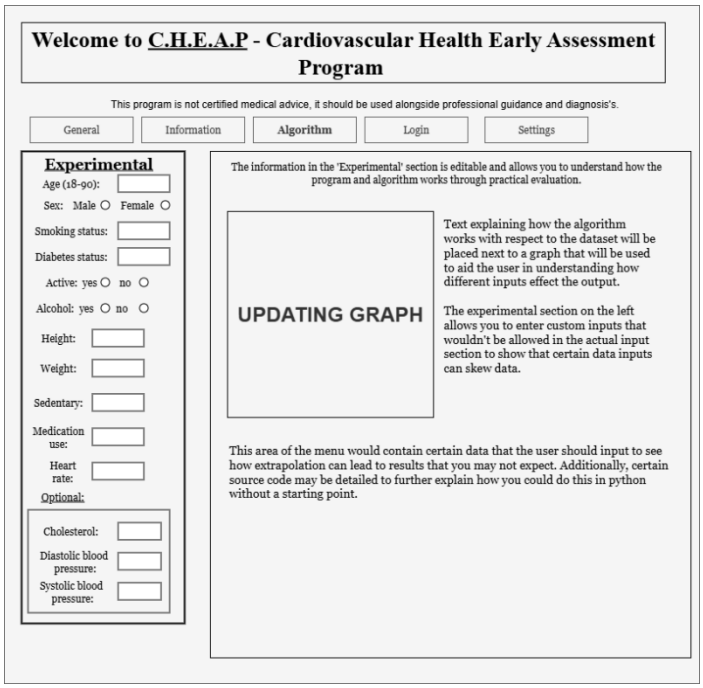


The main 'general' screen contains the input information as well as information surrounding the function of the program. The bold box on the left is where you can input your information; the placement on the left with the bold outline should make it easier to spot. Considering input information is on the general tab, the users shouldn't be confused on what the inputs are asking you for; if the information was placed in the 'information' tab, it may lead to confusion as it wouldn't not easily available.

As you will see, the input box on the left persists on all the menus. This just means that as you browse through the menus, you can gain information and then input your data. It also makes it even easier to understand that this is the box where your information is entered.



The 'information' menu outlines important information that a user will likely want to know. Each of the topics have their own headings in bold text, making it easier to understand than a block of text/information.

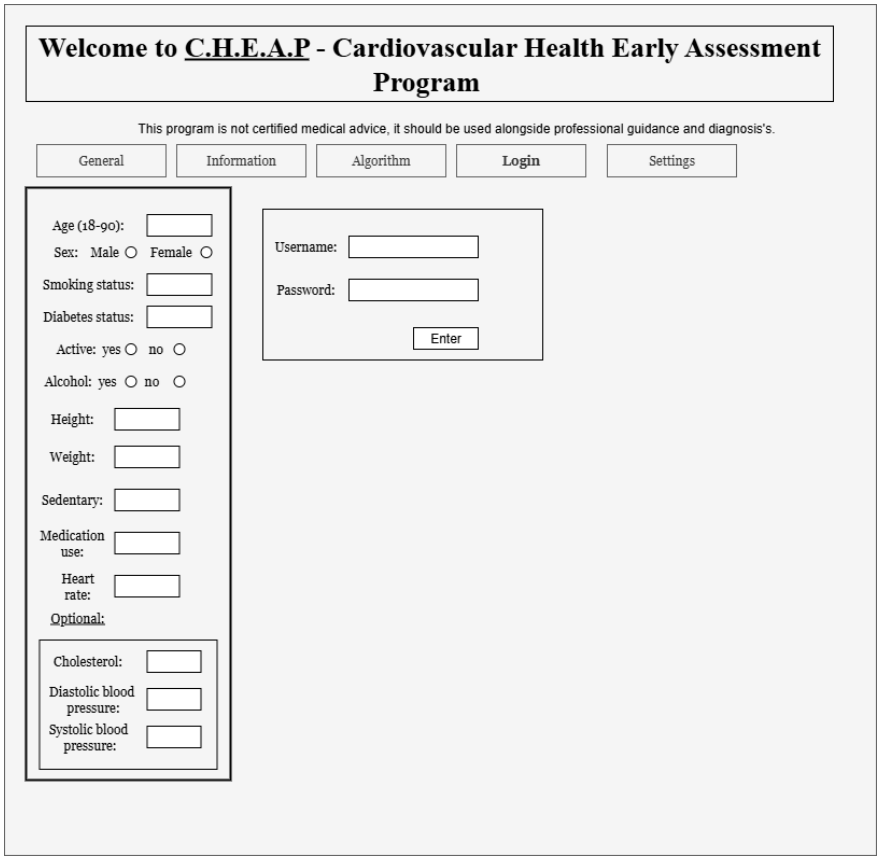


The algorithm tab is there for the portion of my stakeholders that may be students, doctors or curious users that want to understand how my program works.

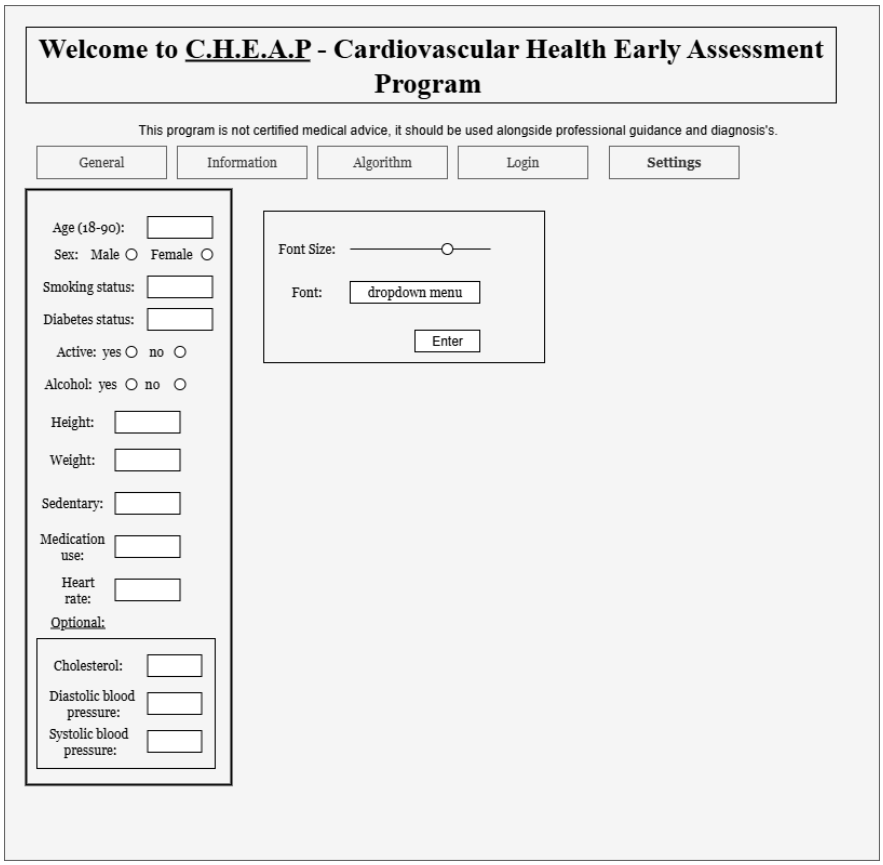
The updating graph is vital for this section, as explaining complicated mathematics without a visual guide would lead to a lot of confusion. The graph would be explained and kept simple since the intention isn't to be fully accurate, it's more to help the user visualise how logistic regression works.

The bold 'Experimental' tab aims to show that this box is no longer used for your personal analysis, it now acts as an experimental zone where the user can input any values in the boxes to understand how an output is reached.

Allowing the user to see certain source code that implements the algorithm widens the potential user base to computer scientists, as excluding this information targets this tab to the mathematical side of data analysis.



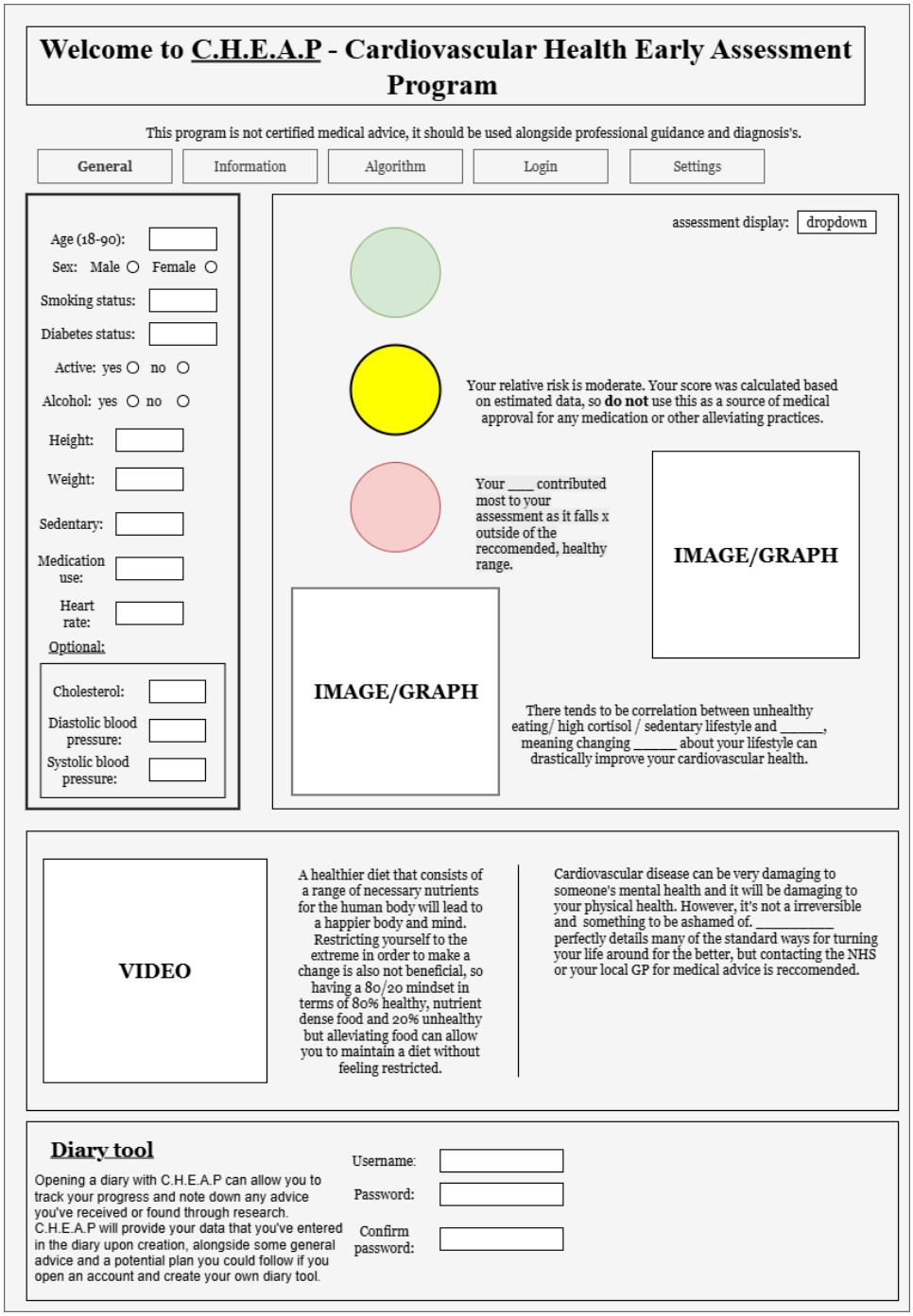
This tab is for anyone that has created a diary and wishes to access it. The simplicity makes it easy to follow and understand. I don't believe two factor authentication is necessary, as the information within the diary will likely be general and fitness orientated. Additionally, the diary will be a text file stored on the user's computer, making it hard to access for an unwanted user.



This tab is essential, as although the font is general, it's likely that some users will have visual problems, or would prefer a different font due to my older demographic (18 – 90 years old).

The slider is very intuitive to use, as numbers can seem arbitrary while a slider allows you to play around with the font sizing.

A drop-down menu also limits the user's options to prevent sizing issues, as some fonts can be much larger than others, and only including general fonts with some variety makes the most sense.



This display screen is very important from a user experience standpoint, since there's a lot of different forms of outputs for the user. Using a traffic light system is very intuitive for the users and it was one of the most requested forms of analysis display with a ≈ 28% selection rate. However, since the heart age comparison was also greatly favoured (≈30%), I'm offering a drop-down menu where the user can select how they want their analysis to be displayed for more user choice.

Accompanying the 'score' with a short explanation in plain language allows the user to fully understand their diagnosis. Placing this explanation near the centre of the screen makes it much easier to find compared to a text box placed further down that you must search for.

Each image/graph or video will be accompanied by adjacent text that explains the video while also providing further context and content, in case the video can't be rendered on the user's device, or if they have any hearing/vision issues.

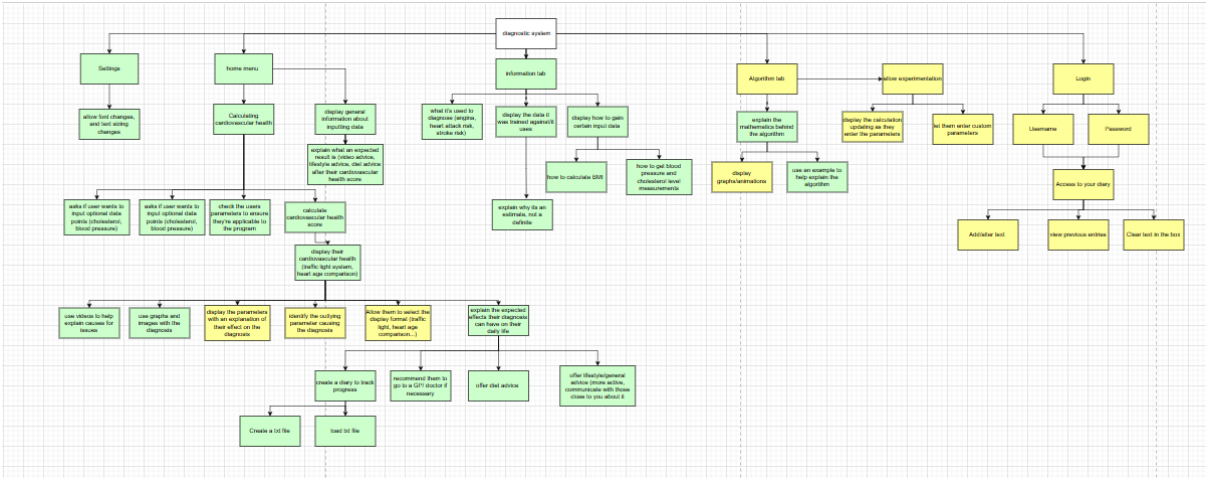
Although this page contains the most varied information, maintaining an intuitive and simple GUI throughout my program is essential, so I will space out the visual and text-based outputs to prevent overcrowding.

The diary tool can then be accessed at the bottom of this screen, as you are able to create an account. Again, the security isn't extensive as the threat is minimal and the information within the dairy isn't sensitive.

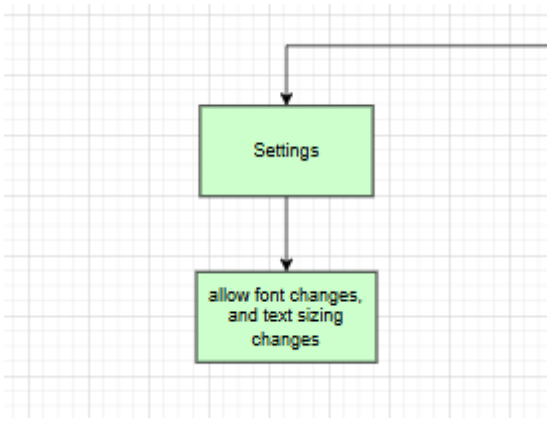
The diary tool will contain the data that you entered as you created your account, making it easy to understand where you currently lie on your journey. You can also update this as you progress.

On the right there will be some general plans for your diet, exercise and mental health care to help kickstart the user's journey towards a healthier cardiovascular system. Without the inclusion of general plans, the user may lack motivation for starting their fitness journey or they must simply be confused on where to start.

Structure Diagram

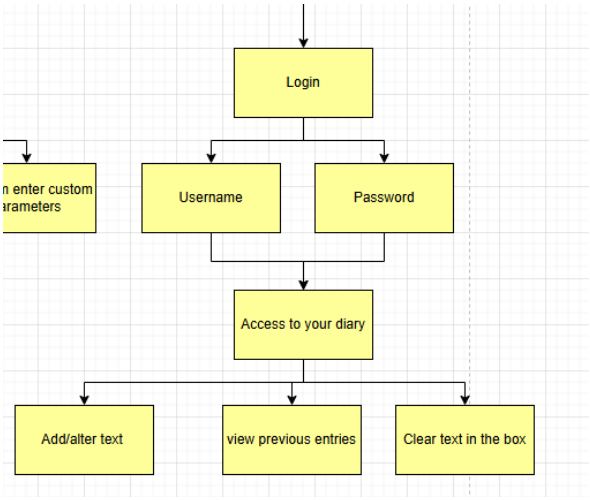


The screenshot above is my full structure diagram, and what follows is the different sections, each justified:

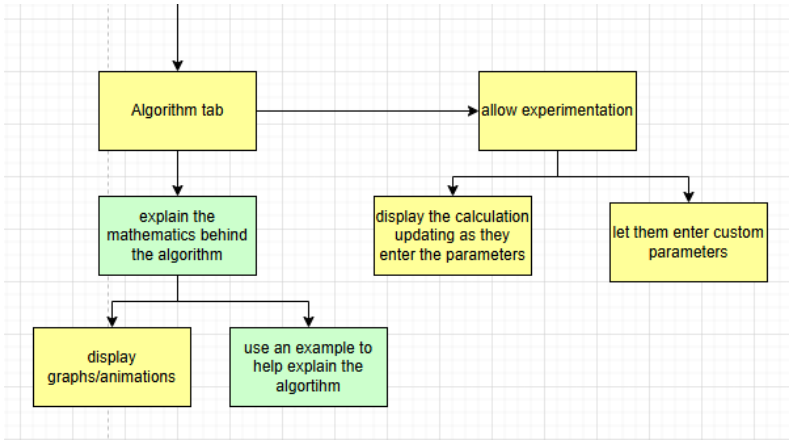


Here is my settings branch. This is a very simple menu that doesn't have a deeper layer of features to implement, it's only used to create a more usable environment where people are able to make the text easier to read via changing the default font or size of the text.

The login section is an optional, but useful menu that will allow the user to create a diary where they can track their progress. To enter the diary tool, you will need to enter a username and password you created after you've been provided your diagnosis. Once you enter your dairy, you can then alter/ add text and view your previous entries.



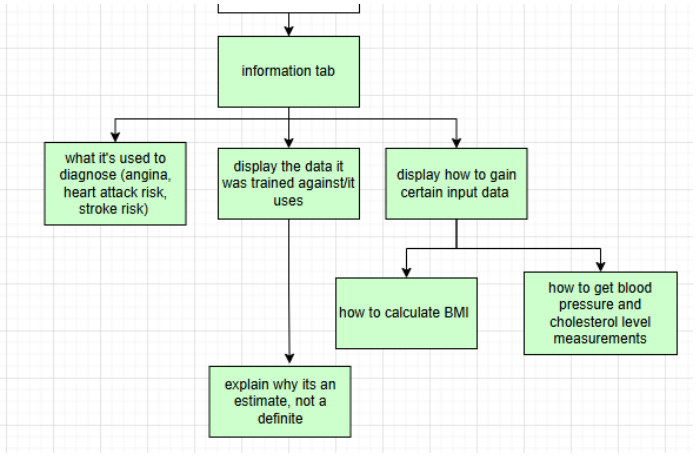
Being able to view your previous entries to your diary as well as their date of entry is essential, as the main idea of this tool is so that you can track your progress as you work towards a healthier lifestyle and healthier cardiovascular system.

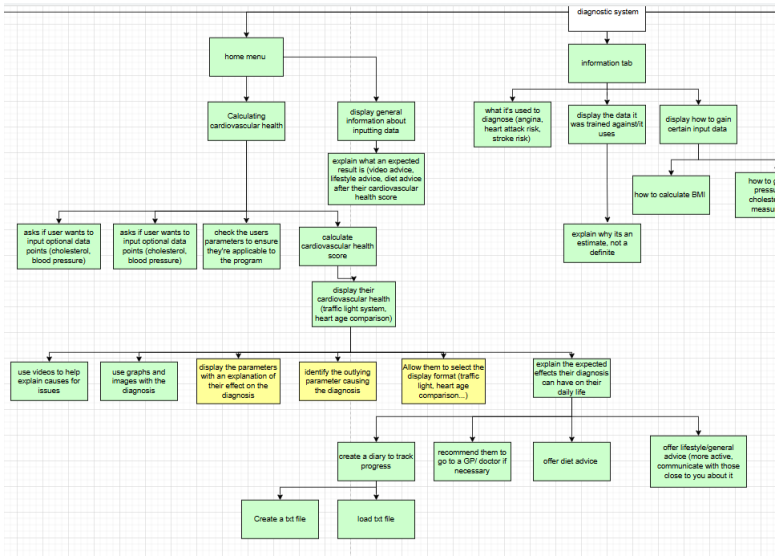


Although this menu is partially necessary, it's created for a different user base to my typical stakeholder. This section is designed for the students and medical practitioners that are curious how the logistic regression model works and how different measurements will lead to different diagnoses.

The reason this section isn't necessary, is because the intention of the program is to assist the older population in understanding their cardiovascular health more. These stakeholders won't be intrigued or understand how the logistic regression model works and it wouldn't be a feature that is required for them.

The information menu is very simple to implement and is essential for user clarity. Some of the inputs required for this program are hard to get and many users may be unsure how to measure their blood pressure or cholesterol levels. This menu therefore aims to clear up any confusion, making it ultimately easier for my stakeholders to access and utilise my program.





The home menu tab is the most important and largest tab in my program. Here, there is multiple layers of functionality, and this is where the outputs will follow your health metrics. As you can see, the features follow on from the user entering inputs that are valid and allow you into the diagnosis section. Within this section, certain features are optional, like

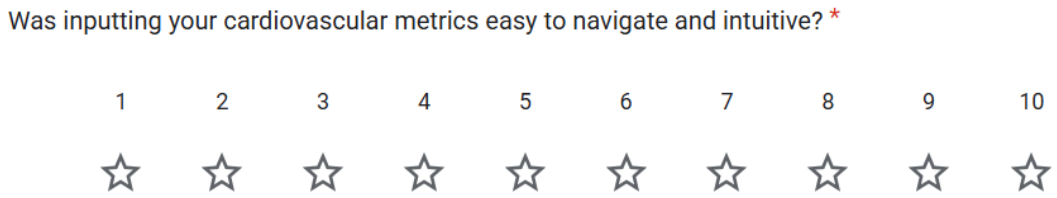
Post development plan

There are three different things I need to test for: functionality, usability and robustness.

Both functionality and robustness can be tested myself by running and testing features and evaluating how effectively they were implemented. Usability will require beta testing, where different stakeholders will view my program and provide feedback for me depending on how well they believe different features are implemented. To collect this feedback, I'll create a google forms that they can use. Since my program is designed for the older population and it can be run on my local device, I can only send the video of my program functioning to my stakeholders. Since this video includes all the features within the program, it should be enough to provide feedback on my program.

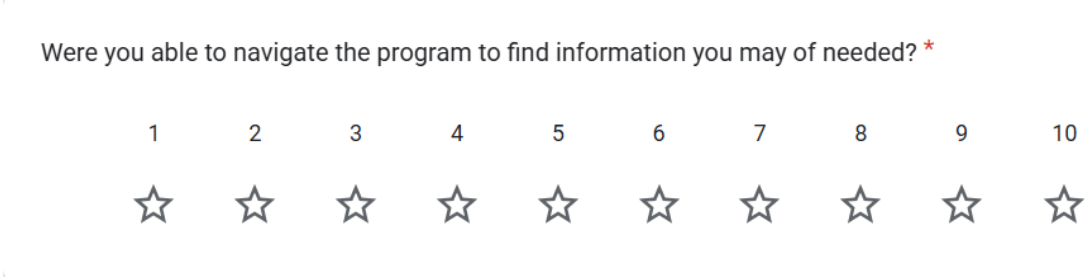
This form needs to collect mostly quantitative data, allowing me to see how features were rated on a scale from 1 to 10. On top of this, providing the users with free reign to provide qualitative data is important, as there may be something I didn't include that they wish to provide feedback on.

1.



My first question allows me to gain information on whether the input taking was intuitive. This feedback is useful because I need to know if you're able to easily enter inputs for my program.

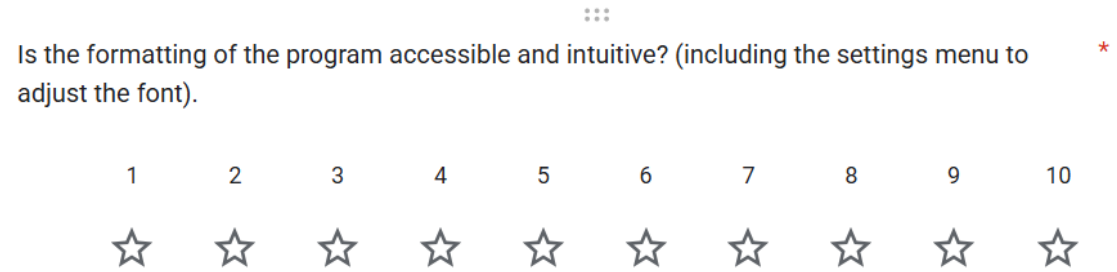
2.



This question then aims to see if the users were able to access the information tab easily if they needed information. It also therefore is about the general and input information tabs

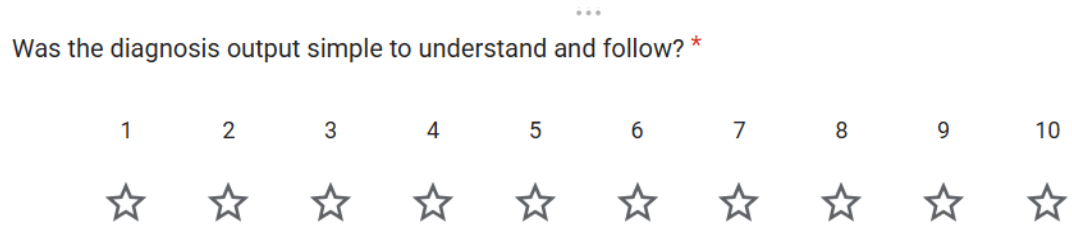
that the users can use if they're confused about what information to put into the inputs. On top of making the information menu reachable, the information within it has to be useful for the user.

3.



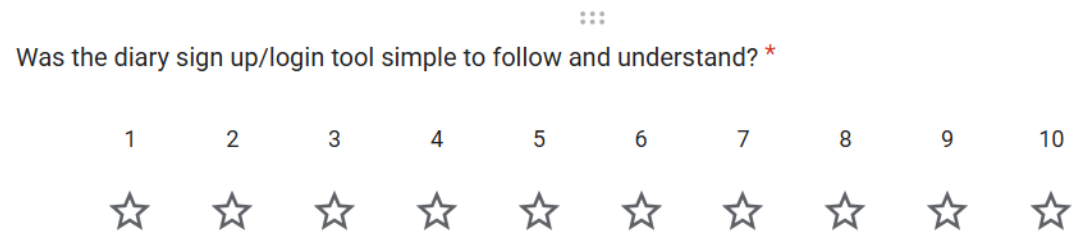
This question then aims to see if changing the fonts and their sizes allowed the user to gain a more accessible experience, and to see if the default display alone was accessible and intuitive for my stakeholders.

4.



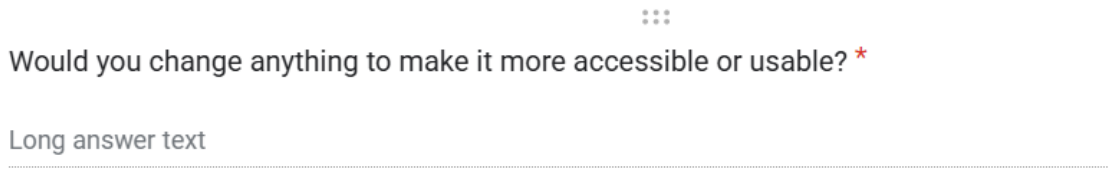
This question then aims to see if the output page provided information that was relevant and simple to follow; this also thus applies to the cardiovascular health video, and the dairy sign up tool.

5.



This question, therefore, aims to see if the diary tool is accessible and intuitive, as there isn't any information telling you how to explicitly operate it.

6.



This last question then is created to get qualitative feedback on previous questions that required concerns or any features that would have made the program more accessible. This question is very important as it allows me to understand what about the features led to a more/less accessible experience.

Now I can move onto functionality and robustness test plans. Both plans need to consider the program as an integrated, whole product, where features work alongside one another and the tests reflect this.

I'm going to create the functionality test plan using the essential, important and optional features I outlined during the analysis stage of my project. Since the optional features may not be implemented, I will still include them but indicate clearly that if they aren't implemented in the program, the tests can be disregarded. These plans will have the letter 'O' following them.

Functionality Test Plan:

Test no.	Description	Test Data	Expected outcome	outcome
1.	The program opens to a menu that displays general information.	N/A	Information is presented in a welcoming manner, making it intuitive to start.	
2a.	An intuitive and simple health metric input feature is implemented in the program.	Standard, healthy metric for a human, passing	The inputs are taken and used to determine the	

		the validation checks for the program.	user's cardiovascular health status.	
2b.	Following the previous test, there is an appropriate output outlining their cardiovascular health.	N/A	Their heart age/ traffic light score corresponds to a healthy individual with an appropriate text response.	
2c.	Alongside the information about the user's cardiovascular health, the health metrics that were likely the contributing factors are identified and outlined.	N/A	Since the user is healthy, there is likely no contributing factor identified.	
3.	Upon entering your health metrics, there is anonymous data storage, meaning that no data about the user is stored outside of the usage of the program.	N/A	There is no trace of data storage outside of logistic regression usage.	
4.(O)	A tool that allows you to see the view the mathematical side of the program, where you can enter health metrics and see visually how it effects the logistic regression model.	Various health metrics that the user can enter to visually analyse the logistic regression model.	The logistic regression model output (probability) varies corresponding to the altered health metrics.	
5.(O)	A tool that recommends diet changes to fit a healthier lifestyle.	N/A	Regardless of your health metrics it recommends what a healthy diet consists of.	
6.(O)	A diary tool that allows you to track your progress or record anything you wish.	Input information about you/ your progress and test if you're able to see previous entries	Previous entries are accessible to the user, and you can enter anything you wish to track your progress.	

The robustness test plan will then be made with the intentions of trying to break my program. This can be done in a multitude of ways to different features, so I need to ensure that my test plan covers all points of potential failure.

Robustness test plan:

Test no.	Description + Data	Test Type	Expected outcome	outcome
1.	Enter outside of range data 10 times, repeatedly encountering the error display	Erroneous	The message continuously displays and doesn't allow the user in	
2.	Enter a string 100 characters long as the age, height and weight of the user.	Erroneous	The inputs are still rejected.	
3.	Enter a number for the user's age, ending with ".0"	Erroneous	Whole integers only are accepted, so it is rejected	
4.	Repeatedly swap between menu buttons rapidly (10+ times).	Erroneous	The menus swap accordingly, leading to no errors	
5.	After a valid diagnosis, swap the health metrics drastically to replicate another user.	Valid	The new cardiovascular status based on this user is displayed	
6.	Enter a 100+ character string for the username and password for the diary tool.	Valid	The input is accepted and used as the username/ password	
7.	Attempt to change the font size of the program to 100.	Erroneous	The font size has an upper limit which doesn't allow this.	
8.	Copy pastes a 1000+ character string into the diary and save it as an entry.	Valid	The entry saves and is accessible.	
9.	Repeat the previous step 10+ times and attempt to visualise each entry to the diary.	Valid	All entries are accepted and visible to the user.	

10.	Attempt to enter non-English characters from another alphabet.	Valid	The entries are still accepted.	

Development

Stage 1 – identifying the dataset and implementing logistic regression (31st September 2025)

Implementing the algorithm (logistic regression) based on a chosen dataset first allows me to design the user interface based on said data. Being able to identify how certain inputs should be taken in from the users based on how these parameters are stored within the database will allow me to cater the interface to the specifics of the dataset.

Using logistic regression also allows me to identify which factor is likely contributing to a poor cardiovascular health compared to k-nearest-neighbour, where identification is much more difficult. Additionally, the dataset will likely have over 50,000 records, so comparing the Eulerian distance to all of them with KNN will be computationally taxing for the users. I'll download the dataset as a csv file and scrape any of it using file handling to process the data.

Since some data will be optional and some won't be optional, I will need to create two different logistic regression models with the difference being what parameters are being considered. Both models will be tested the same, just one won't include all the data that the other is being tested with.

I'll use libraries like numpy and panda to do the complex mathematics required. This maths includes cost functions, gradient descents and sigmoid functions, which involve recursive differentiation and complex linear algebra. Creating a class in python for the regression model will allow me to separate all the functions and calculations from further manipulations I will do as I utilise the model.

Features to implement:

4. A regression model if all fields are used
5. A regression model if only necessary fields are used
6. A sigmoid function/another function to get a score between 0 and 1.

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data for a record which isn't boundary data	Valid	The corresponding value between 0 and 1 for their cardiovascular disease status.	
2.	Enter data that is close to a record in value but not exact (healthy)	Valid	The same cardiovascular disease status as that record	
3.	Enter data that is close to a record in value but not exact (unhealthy)	Valid	The same cardiovascular disease status as that record	
4.	Enter values on the lower range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	
5.	Enter values on the upper range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	
6.	Enter data values outside the range for every field in the dataset (unhealthy end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	
7.	Enter data values outside the range for every field in the dataset (healthier end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	
8.	Enter a negative value for 1 field	Error	This should still generate a number between 0 and 1, however the value may be skewed or inaccurate.	

To start the first stage of my development I need to identify a dataset that I can use to create regression lines for cardiovascular disease. A successful dataset will contain the following features:

- Contains data for most of the medical information contained in my GUI
- Contains a lot (10,000 +) of varied records of patients
- The data is achieved through real, certified surveying or:

The cardiovascular disease status of each patient corresponds to their cardiovascular health in a synthesised dataset.

Cardiovascular disease dataset[1] found on Kaggle meets all the conditions. It was created through collecting information at medical examinations. Additionally, it contains 100,000 varied records and it only misses 3 parameters from the GUI design.

One parameter is the patient's diabetes status, which isn't necessary as the main effect it could have on cardiovascular health is a raised cholesterol level or blood sugar level, which can be identified through the other parameters available (glucose and cholesterol level). Heart rate isn't present in the dataset, but this also isn't necessary as it also correlates closely with other parameters related to your cardiovascular health, like your blood pressure. Lastly, the patient's medication usage also isn't present, this indicates the presence of other medical issues, however, if those diseases will impact the parameters we are taking from the user, it'll be redundant.

On the other hand, the dataset isn't perfect as the values stored for the parameters 'cholesterol' and 'glucose' aren't numeric. Both parameters are characterised by ranges, meaning it's impossible to know the definite numbers for each record. To fix this I plan to research what these ranges could possibly be in numeric form based on scientific publications. Although this isn't perfect, it will be constant for all the records and the variation for the ranges will be very small, making its effect minimal.

The ranges for healthy, above average and unhealthy cholesterol levels[2] are as follows:

Normal: 125-200 mg/dL

Borderline: 200 -239 mg/dL

High: 240+ mg/dL

For both men and women above the age of 20. This then perfectly represents the age demographic for my program.

The ranges for a healthy, pre-diabetic and diabetic glucose levels[3] are as follows:

Healthy: < 100 mg/dL

---

[1] (Ulianova)
[2] (Cervoni)
[3] (Seery)

Pre-diabetic: 100-125mg/dL

Diabetic: 126 mg/dL

These levels are taken while fasted, meaning that you haven't eaten for the last 8 hours. Because of this, the measurement is usually taken in the morning. The dataset I'm using labels the ranges as 'Normal', 'Above normal' and 'Well above normal', meaning that I'll match these ranges to the ones described above.

Additionally, some values for patient's height and blood pressure are extremely large/small. For example, 22 records were found with heights under 80 cm, and some blood pressure values are below 0 and greater than 200. To combat this, I plan to identify realistic ranges for each of the parameters (with some extra variation for extreme cases) and check each of the records against these ranges to remove any anomalous records.

All of the parameters are listed below, and I will provide a realistic range for each that each of the records will be checked against to clean the data.

1. Age: 30y – 75y → 9125d – 27375d

   The maximum value for the age of a record is ≈ 70 years old, so to prevent extreme extrapolation, I must limit the age to only around 75 years old. A positive to this is that people who are above 75 years old will likely have an understanding for their cardiovascular health as illness possibility only increases as you get older. If you're under 30, you're also very unlikely to suffer from a heart attack or other cardiovascular diseases, so the probability values will be very extreme and inaccurate.

2. Height: 100cm – 228cm

   Someone with dwarfism could be close to 105cm, so using 100cm is a realistic lower bound for data cleaning. 228 cm is ≈ 7 ft 6in, which is the height of one of the tallest NBA basketballer[4], meaning anyone taller will have to consult their own local GP as it is outside my set range.

3. Weight: 39.9kg – 225kg

   Some people in the data set are close to 140cm and below, and if you're very skinny on top of that you could fall close to 40kg. The upper bound is realistic as people can be over 250kg, but if that is the case I would want them to consult a practitioner already if they have not. If you're over 180cm, 200 kg is possible if you're very large.

4. Gender: N/A

   All the values are labelled as male or female.

5. Systolic blood pressure: $70 < x < 175$ mm Hg

[4] (Wikipedia)

6. Diastolic blood pressure: $50 < x < 110$ mm Hg

   Even though numbers outside these ranges are possible, any of these values would indicate a high risk of heart attack[5] or leaky valves[6] in the heart. This would indicate a need for immediate medical assistance, so I will make sure to highlight this if a user enters any values outside the given ranges.

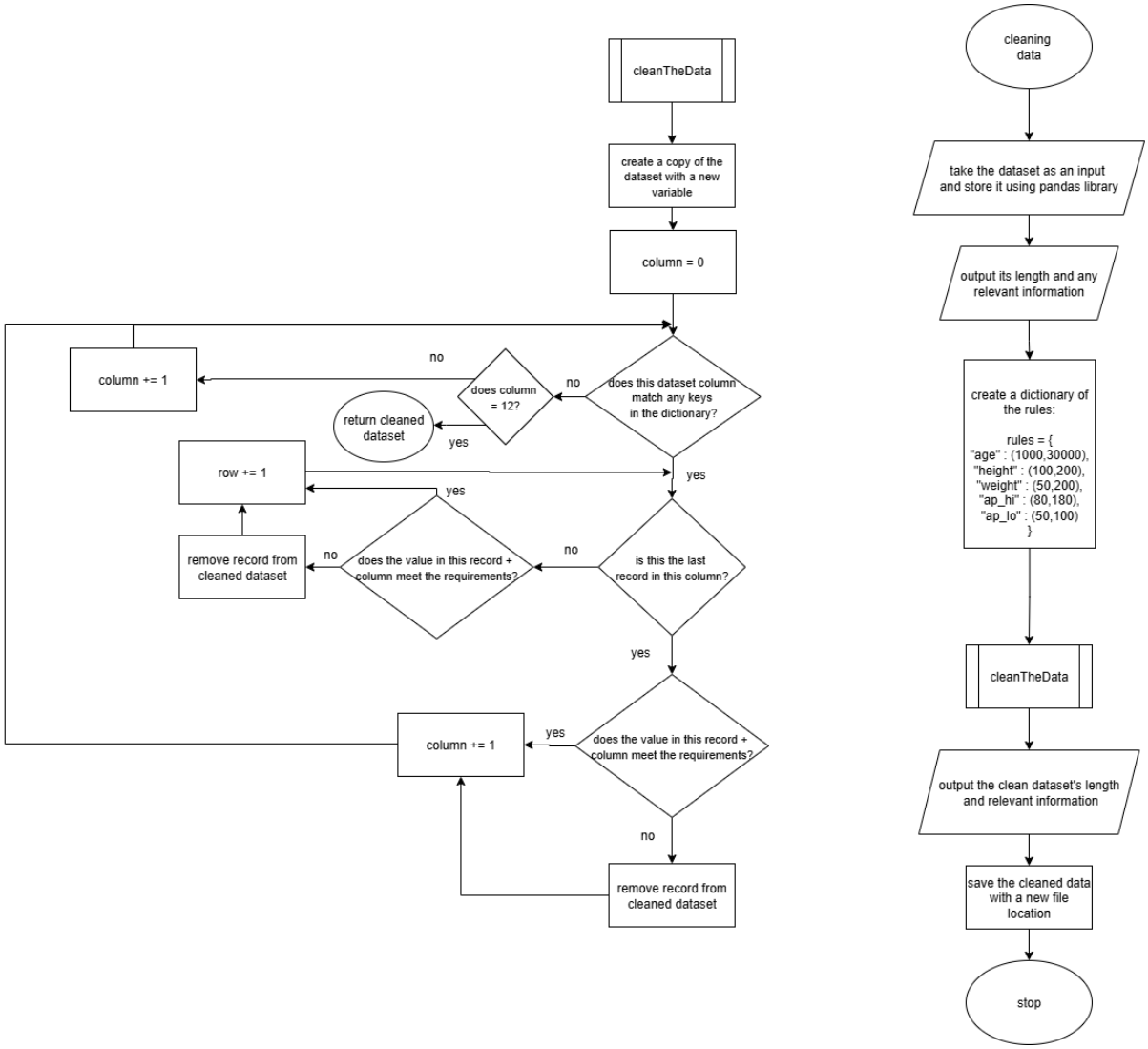
7. Cholesterol: N/A
8. Glucose: N/A
9. Smoking: N/A
10. Alcohol intake: N/A
11. Physical activity: N/A
12. Presence or absence of cardiovascular disease: N/A

The previous 4 parameters all have Boolean values, meaning I don't have to check or clean any of these values. Glucose and Cholesterol are both stored in a qualitative form, so cleaning this data is not necessary.

Cleaning the Data in Python:

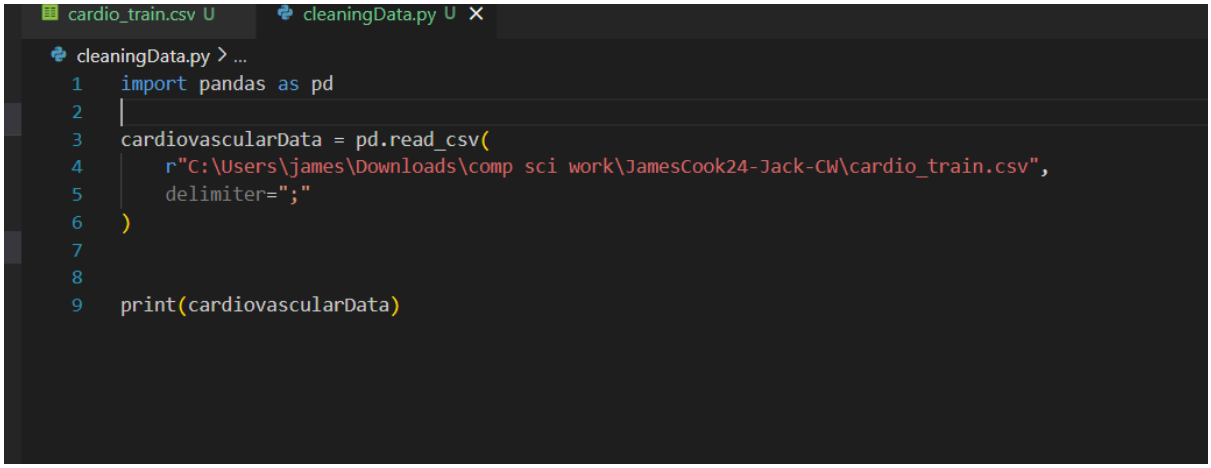
The following flowchart describes how I can clean the data using python libraries like pandas and numpy. I'm creating a function that loops through each column and each record in the database so I can check their values against defined rules. Although pandas allow you to loop through each column without directly coding the looping of each record, I included the record looping so that I understand the purpose of the function fully from the ground up.

[5] (American Heart Association)
[6] (Cleary)



Code Implementation

To successfully clean the data set, I will use pandas to help me use file manipulation to clean the dataset based on my set values.



Here is the code I've used to read the cardiovascular dataset, where I used the full source location of the dataset to prevent any location issues. Since the dataset doesn't use commas, I added the delimiter[7] ';', which will be what separates the columns in the dataset. Using pandas allows you to easily read the .csv file in a single line, while also using simple functions that allow you to save a new file (once cleaned), or manipulate records in a database using rules you can set. Since this section of code is only used to clean the data and not actually utilise the data to create regression models, it makes sense to use this library over simpler libraries, as wasting time at this stage of development is unnecessary.

Test:

```
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW> & C:/Users/james/AppData/Local/Microsoft/WindowsAp
c:/Users/james/Downloads/comp sci work/JamesCook24-Jack-CW/cleaningData.py"
         id    age  gender  height  weight  ap_hi  ap_lo  cholesterol  gluc  smoke  alco  active  cardio
0         0  18393       2     168    62.0    110     80            1     1      0     0       1       0
1         1  20228       1     156    85.0    140     90            3     1      0     0       1       1
2         2  18857       1     165    64.0    130     70            3     1      0     0       0       1
3         3  17623       2     169    82.0    150    100            1     1      0     0       1       1
4         4  17474       1     156    56.0    100     60            1     1      0     0       0       0
...     ...    ...     ...     ...     ...    ...    ...          ...   ...    ...   ...     ...     ...
69995  99993  19240       2     168    76.0    120     80            1     1      1     0       1       0
69996  99995  22601       1     158   126.0    140     90            2     2      0     0       1       1
69997  99996  19066       2     183   105.0    180     90            3     1      0     1       0       1
69998  99998  22431       1     163    72.0    135     80            1     2      0     0       0       1
69999  99999  20540       1     170    72.0    120     80            2     1      0     0       1       0

[70000 rows x 13 columns]
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

It showed that the columns were separated with the delimiter correctly and showed that the dataset was imported into the python final correctly.

[7] (Pykes)

```
cardiovascularData = pd.read_csv(
    r"C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\cardio_train.csv",
    delimiter=";"
)# retrieving the .csv file and seperating the columns based on the appearance of ';'

#creating a dictionary that stores all the bounds for the parameters

print(cardiovascularData.describe()) #tells you stuff like the min,max and mean for columns
#this will be useful to compare the data before and after cleaning it.
print("length:",len(cardiovascularData))#original length of the dataset

rules = {
    "age": (9125,27375),
    "height": (100,228),
    "weight": (39.9,225),
    "ap_hi": (70,175),
    "ap_lo": (50,110)
}
```

Here is the updated code that uses pandas to clean the data. I've established a dictionary that stores all the keys that match the description of each of the columns I wish to clean along with corresponding values. I chose to use a dictionary as each column has a lower bound and upper bound, so setting each of the values for the keys is intuitive and simple. It also can be stepped through using items()[8], making it useful when iteratively changing something. If I used a 2D array, accessing the correct indexes would be more compliacted and taxing to implement.

Additionally, printing the info and length of the dataset before altering and cleaning it will act as a test for the cleaning's success.

```
def cleaningData(ds,rules):#takes a dataset and a dictionary as parameters

    cleaning = ds.copy()#creates a copy of the dataset to clean

    for col, (min,max) in ds.items():#this line is very importnant.
        #the col acts as the column in the dataset, with 'min' and 'max' being established
        #items in the dataset that the records will be checked against.
        #items then allows you to go through the key and values stored in the dictionary as a tuple.

        cleaning = cleaning[(cleaning[col] >= min) & (cleaning[col] <= max)]
        #this line checks each column against value assigned to a matched key, making sure the values fall inside the range

    return cleaning # return the clean data
```

This function is simple yet effective, as I can utilise a for loop to check each record[9] against the rules and min/max values all in 1 line. the 'in' in the for loop ensures that the col is only checked against keys that match the string for the column, making it very easy to check if a record is safe. Furthermore, at first glance it would seem that I am only checking each column, not each row in each column. However, when pandas sees the [ ] used in a dataset, it checks each record in that column. This is why pandas is so useful for this step in creating regression models, if I use numpy only it would be unneccesarily fidgety and complicated to check each record and column of the dataset.

[8] (W3Schools)
[9] (Horvath)

```
cleanData = cleaningData(cardiovascularData,rules)

print(cleanData.describe())
print("length:",len(cleanData))
```

After this, I can check the new information and length of the dataset to see the efficacy of the cleaning.

Test:

```
                id           age        gender        height        weight         ap_hi         ap_lo   cholesterol          gluc         smoke          alco        active        cardio
count  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000
mean   49972.419900  19468.865814      1.349571    164.359229     74.205690    128.817286     96.630414      1.366871      1.226457      0.088129      0.053771      0.803729      0.499700
std    28851.302323   2467.251667      0.476838      8.210126     14.395757    154.011419    188.472530      0.680250      0.572270      0.283484      0.225568      0.397179      0.500003
min        0.000000  10798.000000      1.000000     55.000000     10.000000   -150.000000    -70.000000      1.000000      1.000000      0.000000      0.000000      0.000000      0.000000
25%    25006.750000  17664.000000      1.000000    159.000000     65.000000    120.000000     80.000000      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
50%    50001.500000  19703.000000      1.000000    165.000000     72.000000    120.000000     80.000000      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
75%    74889.250000  21327.000000      2.000000    170.000000     82.000000    140.000000     90.000000      2.000000      1.000000      0.000000      0.000000      1.000000      1.000000
max    99999.000000  23713.000000      2.000000    250.000000    200.000000  16020.000000  11000.000000      3.000000      3.000000      1.000000      1.000000      1.000000      1.000000
length: 70000
```

The information and the length of the list is displayed accordingly, with all the errors being spottable.

```
Traceback (most recent call last):
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\cleaningData.py", line 36, in <module>
    cleanData = cleaningData(cardiovascularData,rules)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\cleaningData.py", line 26, in cleaningData
    for col, (min,max) in ds.items():#this line is very importnant.
             ^^^^^^^^
ValueError: too many values to unpack (expected 2)
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

However, after calling the function and attempting to print out the cleaned data, I was hit with an error message. It suggests that the 'min' and 'max' values are too much to unpack, possibly due to the dictionary definition or how they're being used in the for loop.

After reading through my code, I realised that I have accidentally written 'ds.items()' over 'rules.items()', meaning that instead of 2 values being passed, multiple columns or records are likely being unpacked. To fix this I just need to change the like ds.items() to rules.items().

Test:

```
                id           age        gender        height        weight         ap_hi         ap_lo   cholesterol          gluc         smoke          alco        active        cardio
count  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000
mean   49972.419900  19468.865814      1.349571    164.359229     74.205690    128.817286     96.630414      1.366871      1.226457      0.088129      0.053771      0.803729      0.499700
std    28851.302323   2467.251667      0.476838      8.210126     14.395757    154.011419    188.472530      0.680250      0.572270      0.283484      0.225568      0.397179      0.500003
min        0.000000  10798.000000      1.000000     55.000000     10.000000   -150.000000    -70.000000      1.000000      1.000000      0.000000      0.000000      0.000000      0.000000
25%    25006.750000  17664.000000      1.000000    159.000000     65.000000    120.000000     80.000000      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
50%    50001.500000  19703.000000      1.000000    165.000000     72.000000    120.000000     80.000000      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
75%    74889.250000  21327.000000      2.000000    170.000000     82.000000    140.000000     90.000000      2.000000      1.000000      0.000000      0.000000      1.000000      1.000000
max    99999.000000  23713.000000      2.000000    250.000000    200.000000  16020.000000  11000.000000      3.000000      3.000000      1.000000      1.000000      1.000000      1.000000
length: 70000
                id           age        gender        height        weight         ap_hi         ap_lo   cholesterol          gluc         smoke          alco        active        cardio
count  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000  67594.000000
mean   49993.011155  19452.158668      1.347901    164.411516     74.000584    125.830917     81.001258      1.360209      1.223348      0.087493      0.052948      0.803562      0.489289
std    28860.168581   2469.951773      0.476308      7.946250     14.162775     15.240517      8.940224      0.675904      0.569379      0.282558      0.223932      0.397306      0.499889
min        0.000000  10798.000000      1.000000    100.000000     40.000000     70.000000     50.000000      1.000000      1.000000      0.000000      0.000000      0.000000      0.000000
25%    24988.250000  17640.250000      1.000000    159.000000     65.000000    120.000000     80.000000      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
50%    50052.500000  19695.000000      1.000000    165.000000     72.000000    120.000000     80.000000      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
75%    74906.500000  21316.000000      2.000000    170.000000     82.000000    140.000000     90.000000      1.000000      1.000000      0.000000      0.000000      1.000000      1.000000
max    99999.000000  23713.000000      2.000000    207.000000    200.000000    175.000000    110.000000      3.000000      3.000000      1.000000      1.000000      1.000000      1.000000
length: 67594
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

After editing the line and running the program again, the output is as expected. The minimum values now correspond to the ruleset I established, and the data now matches what I need to create regression models. Additionally, there are 2,406 records less in the cleaned dataset, resulting in a 3.44% change in the number of records. Since this dataset was created through collecting factual medical examinations, it's surprising that there were that many inaccuracies.

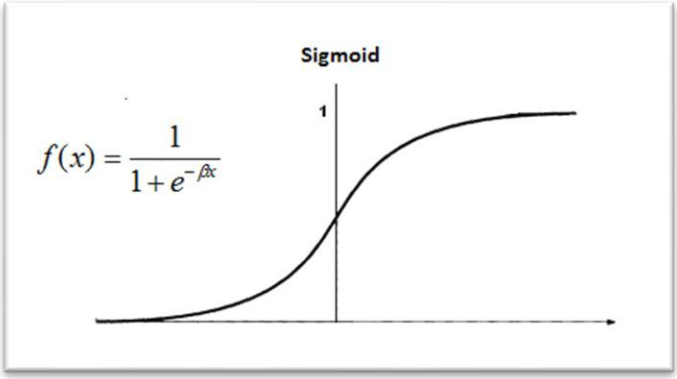
Now that the data is clean based on my ruleset, I can save[10] this version of the dataset as follows:

```
cleanData = cleaningData(cardiovascularData,rules)

print(cleanData.describe())
print("length:",len(cleanData))

cleanData.to_csv('cleaned_cardio_train.csv',index = False)
```

Since I have a saved, cleaned dataset, I can move onto developing regression models that are trained off the data.

Firstly, understanding the mathematics required for a multivariate discrete logistic regression model is essential.

The Maths behind multivariate discrete logistic regression



[11]

This is a sigmoid function, which is used to get a number between 0 and 1 no matter the input, so you can use probabilities. The regression line will be placed where 'β' is once the weights have been decided.

[10] (Mărginean)
[11] (Wikipedia)

Cost functions and Gradient Descent

A cost function is something that we wish to minimise so that our outputs are as close to the actual values as possible. The cost function works out the error of a model using log loss (cross-entropy loss), where the variation of the prediction from 0 and 1 leads to subtle or extreme punishments to the gradients.

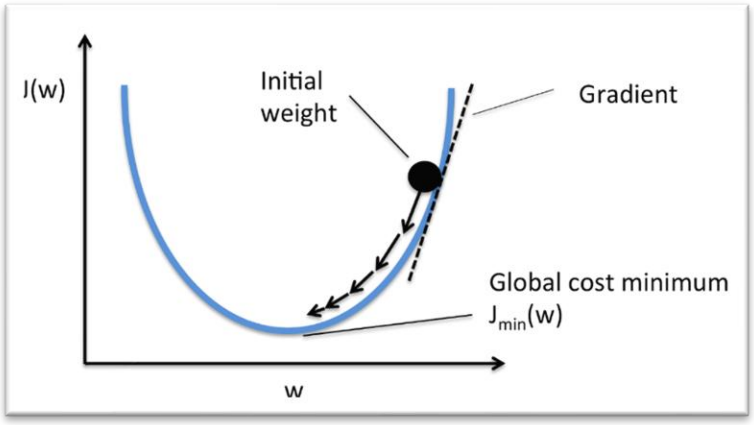
$$-\frac{1}{N}\sum_{i=1}^{N} y_i \cdot log(p(y_i)) + (1 - y_i) \cdot log(1 - p(y_i))$$

Binary Cross-Entropy / Log Loss

[12]

Where N is the number of observations, y_i is the actual value for that record.

To minimise the value of a cost function, we use gradient descent. Gradient descent involves differentiation, where you adjust the weight away from the direction of the gradient until the value converges in the local minimum[13].



[14]

Gradient descent and cost functions therefore work hand in hand, as the output of the cost function is used for the input of the gradient descent function.

The maths behind adjusting the weights is complicated, so to simplify imagine working out the derivative of an equation with respect to a weight (x_1) and based on the result you adjust this weight using learning rates and cost functions.

[12] (Banoula)
[13] (Kumar)
[14] (Low)

Developing the regression model:

Pseudocode:

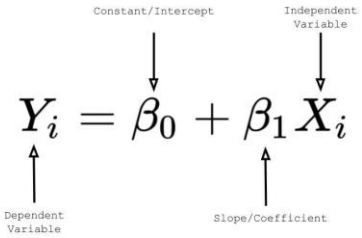
```
import numpy

class LogisticRegressionModel: #create a class that can be accessed in another python file


  def LogisticRegressionModel(self,learningRate= 0.1,IterationNumber = 10000):
    self.learningRate = learningRate
    self.IterationNumber = IterationNumber
    self.weights = 0
    self.bias = 0
    #ths will initialise all the attributes/variables I will need to use
```

Here is the class definition and the 'init' method, which is called upon class initiation. The constructor sets the learningRate and iterationNumber to default values of 0.1 and 10,000. It also sets the weights and bias to 0. Although this can be changed when the method is called, it creates a default value for each of the attributes/variables. A small learning rate and many iterations is ideal, as finding the true minimum is easier and possible with this ratio present.

```
def fittingModel(self,SampleVector,ValueVector):
  #create a function that takes a numpy vector as a parameter, where Sample is a vector
  #containing test data
  #ValueVector will contain the correct values as a numpy
  SampleNumber,ParameterNumber = SampleVector[x,y]
  #SampleNumber is then the number of records
  #ParameterNumber is number of columns
  self.weights = np.vector(numParameters)
  #this will make weights a vector based on the number of parameters used in the dataset
  for i in range(IterationNumber):
    Model = SampleVector * self.weights + self.bias

    Prediction = self.sigmoid(Model)#pass the linear model into sigmoid function

    dw = (1/SampleNumber) * transpose(SampleVector) * (Prediction - ValueVector)

    db = (1/SampleNumber) * sum(Prediction - ValueVector)

    self.weights = self.weights - learningRate * dw
    self.bias = self.bias - learningRate * db
    #follow the updating weights and bias formula
```

The 'fitting' method is used when training and developing the values of the weights based on the dataset vector passed into the method. The variables SampleNumber will hold the number of records in the dataset, while ParameterNumber stores the number of columns in the dataset. These variables can then be used when creating other vectors like the weights vector seen in the following lines. This vector will be the size of the number of parameters in the SampleVector, which is the dataset in vector /data frame form. Vectors in regression models are essential, as 2D vectors can be manipulated the same way as matrixes, meaning I can do matrix manipulation and transposition.

After this, you can start the loop, which loops until the IterationNumber is reached. This value can be altered, but a number too big will likely lead to overfitting during the gradient descent process, so finding the sweet spot is best for this value. The Model variable then follows the equation used for logistic regression found below:



[15]

'Prediction' then calls the sigmoid function, which will be displayed later, but it's a 1 line function that computes the sigmoid function mentioned and explained earlier in this passage.

'dw' and 'db' are then derived from the log loss formula, where dw and db are derivatives present in this formula:

$$J'(\theta) = \begin{bmatrix} \frac{dJ}{dw} \\ \frac{dJ}{db} \end{bmatrix} = [\dots] = \begin{bmatrix} \frac{1}{N}\sum 2x_i(\hat{y} - y_i) \\ \frac{1}{N}\sum 2(\hat{y} - y_i) \end{bmatrix}$$

[16]

Although this looks different to the cross-entropy function detailed earlier, this is derived from taking the derivates of that function, which is needed during gradient descent.

The 'weights' and 'bias' can then be adjusted by multiplying this value by the defined learning rate. Using a small learning rate is effective, as the weights are only slightly affected by each record in the dataset to prevent overfitting. Since the number of iterations is high and there are over 50,000 records, underfitting is also not an issue.

```
def sigmoid(self,x):
  return 1/(1+np.e^(-x))
  #e will refer to the mathematical constant 'e'
```

Here is the function 'sigmoid', which was used earlier. In python code this may be written differently, but there will only be variation found in the np. … part of the return line.

[15] (Arya)
[16] (Arya)

```
def prediction(self,SampleVector):
  #work out model again
  Model = SampleVector * self.weights + self.bias

  Prediction = self.sigmoid(Model)#pass the linear model into sigmoid function


  PredictionArray = [1 if i> 0.5 else 0 for i in Prediction]
  #using list comprehension i can work out the value between 0 and 1
  return PredictionArray
```

Finally, this prediction method will be used to predict the outcome of a record, which is then apply the sigmoid function and list comprehension to get a value between 0 and 1. This is currently binary logistic regression, and multinomial regression can be implemented later once this simplified version of the model is developed. Since binary logistic regression matches the cardiovascular classification of the dataset (0 or 1), testing its efficacy with binary prediction allows for accurate testing.

Now I need to write the pseudocode for the python file that will take the cardiovascular dataset as a numpy vector, which can be split up and used to train and test the regression model.

```
import pandas as pd
import numpy as np
import sklearn

from logisticModel import LogisticRegressionModel
#import the python class from the other python file

data = pd.read("cleaned_cardio_train.csv")
#reads the dataset as a vector

cardioData = data.remove(["id","cardio"]).values
#stores the values in the dataset except the id and cardio column
cardioValues = data["cardio"].values
#stores the values in the cardio column

cardioDataTrain,cardioValuesTrain,cardioDataTest,cardioValuesTest = np.split(cardioData,cardioValues, size = 0.8)
#splits the data using an np function into training and test data

```

I can import the class 'LogisticRegressionModel' from the other python file since they're in the same folder. After that, I can read the cleaned cardiovascular dataset using pandas, which will convert it into vector form. I can then remove the id column and cardio column and store this as another vector, since the 'id' values will only add noise to the training data and the cardio data is exclusive for the prediction part of the model. This can be seen as cardioValues takes the values inside the 'cardio' column.

I can then split the cardioData and cardioValues into training and test data, where 80% of the dataset will be used for training the weights in the regression model. Splitting the data into training and test data rather than using the full dataset for training means I can test the model on unseen data rather than data that was used for training the dataset. Additionally, testing the dataset on over 12,000 records will prevent bias and anomalous outputs skewing the result, leading to an inaccurate accuracy score. Even though I plan to test the model with the test plan established at the start of this development stage, testing the model on general data that can be inputted at a larger scale will be more effective when building the model to see if large trends appear which may lead to inaccurate results.

```
8
9  Regressionmodel = LogisticRegressionModel(lr = 0.01,IterationNumber = 10000)
0  Regressionmodel.fit(cardioDataTrain,cardioValueTrain)
1  #instantiates RegressionModel using the class from the other file
2  #calls the fit method to train the data
3
4  cardioValuePrediction = model.predict(cardioDataTest)
5  #gets a prediction using the test data
6
7  accuracy = sklearn.accuracy(cardioValueTest, cardioValuePrediction)
8
9  print("accuracy",accuracy)
```

Now, I just need to instantiate the regression model as an object, where I can call the fit method and the predict method to gain the values I need to determine the accuracy of the model.

Once I implement this pseudocode in python, most of the structure will remain the same, but certain function and variable names will differ due to numpy and Sklearn function names.

Data dictionary:

The following data dictionaries contain the key variables found in the two python files I plan to develop.

Full_logistic_model.py →

Variable	Data type	description
lr	Float	'lr' refers to the learning rate, which is used to scale down the adjustments of the weights during gradient descent.
numIters	Integer	How many times the gradient descent function will run and adjust the weights per record.
weights	Numpy vector	Will store the weights for each parameter using a numpy vector
bias	Float	A constant that is added onto the regression line to prevent any bias
numSamples	Integer	The number of records in the dataset
numParameters	Integer	The number of parameters/columns in the dataset
linearModel	Float	Stores the regression line in equation form; contains a constant (bias), variable (weights) and a coefficient (record values)
valuePrediction	Float	A value between 0 and 1 after a sigmoid function is applied to the linearModel variable

dw	Float	Represents the equation that is derived from finding the first derivative of the cross-entropy (log loss) function. This is used to update the weights.
db	Float	Represents the equation that is derived from finding the first derivative of the cross-entropy (log loss) function. This is used to update the bias.
valuePredictionClass	List	Using list comprehension, it will store the values predicted by the regression model, where it can be compared to the true values in the dataset.

Regression_test.py →

Variable	Data type	Description
data	Pandas vector	Reads the cleaned cardiovascular dataset as a pandas vector to be passed into the fit method found in the 'full_logistic_model.py' file.
cardioData	Pandas vector	Replicate of the 'data' variable, except the columns labelled 'id', 'cardio', 'height' and 'weight' have been dropped, with 'BMI' being added using the height and weight columns.
cardioValue	Pandas vector	Contains the values in the 'cardio' column within the dataset.
cardioData_train	Pandas vector	Contains 80% of cardioData, which is used to train the regression model
cardioData_test	Pandas vector	Contains 20% of cardioData, which is used to test the regression model.
cardioValue_train	Pandas vector	Contains 80% of cardioValue, which is used to train the regression model
cardioValue_test	Pandas vector	Contains 20% of cardioValue, which is used to test the regression model.
Model	Object	Instantiates the fullLogisticModel class found in 'full_logistic_model.py'.

cardioValue_pred	Pandas vector	Contains the predictions using the regression model which can be tested against the actual cardio values within the dataset.

Code Implementation:

the python file is called 'full_logistic_model', where 'full' refers to the usage of every parameter in the dataset, including the optional ones. I plan to develop separate models for non-optional parameters only, but initially I will develop the full model.

```
def __init__(self,lr = 0.01,numIters=1000):
    self.lr = lr
    self.numIters = noIters
    self.weights = None
    self.bias = None
```

'lr' refers to the learning rate used during gradient descent, which essentially controls the 'amount' that the weight will change as the gradient descent is used. 'numIters' will refer to the number of iterations the gradient descent function will do when the weight is adjusted for each record.

The weights and biases are instantiated with a null value, as they can be created during the training phase of development.

```
def fit(self, Sample,value):
    #Sample is a numpy nd vector of size NxM where N is the No. of samples
    #and M is the number of features in said sample

    #now we initialise the weights
    numSamples,numParameters = Sample.shape
    #this will unpack the shape of the vector into numSamples (N) and numParameters (M)
    self.weights = np.zeros(numParameters)
    #set the weights to a vector full of 0s with the size of
    self.bias = 0
    #sets the bias to be 0
```

The following section of the 'fit' function[17] is very intuitive; A vector is initialised based on the number of records and columns in a dataset. Additionally, the weight vector and bias value is set to 0 before any adjustments are made.

[17] (Loeber)

```python
def sigmoid(self,x):
    return 1/(1 + np.exp(-x))
    #sigmoid function of an input 'x'
```

Here is the sigmoid function mentioned earlier in practice written in python. It can now be called and used to get a probability between 0 and 1.

```python
#gradient descent function
for _ in range(self.numIters):
    linearModel = np.dot(Sample, self.weights) + self.bias
    #np.dot multiplies the vectors together, creating a linear regression line
    valuePrediction = self.sigmoid(linearModel)
    #approximation

    #Now we update our weights
    dw = (1/numSamples) * np.dot(Sample.T,(valuePrediction - value))
    db = (1/numSamples) * np.sum(valuePrediction-value)

    self.weights -= self.lr * dw #update the weight based on the derivative
    self.bias -= self.lr * db #update the bias based on the derivative
```

The image above contains the loop that is used when training the regression model, as the latter stage of the screenshot contains the formula used when adjusting the weights of the model[18]. Some functions are different compared to the pseudocode and mathematical notation, due to the numpy functions[19] used that make it easier to transpose and multiple vectors.

```python
def predict(self, Sample):
    linearModel = np.dot(Sample, self.weights) + self.bias
    valuePrediction = self.sigmoid(linearModel)
    #creates a probablity the same way that I did earlier in the 'fit' method

    #For simplicity and efficacy sakes, I will implement a binary model
    #1(cardiovascular disease) , 0(no cardiovascular disease)
    valuePredictionClass = [1 if i>0.5 else 0 for i in valuePrediction]
    #clist comprehension for every probability in valuePrediction
    return valuePredictionClass
```

This function is then used to finally predict the output based on the parameters analysed and a value (0 or 1) is evidently returned. When developing the section of the program that will display the model as it processed inputs, I can display the probabilities before list comprehension so that the user understands where the values come from. Displaying probabilities will also be good when using the test plan because I can see how 'confident' the model is.

[18] (Loeber)
[19] (NumPy)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

from full_logistic_model import fullLogisticModel#import the class containing the regression model

data = pd.read_csv("cleaned_cardio_train.csv")#read the data from the cleaned cardiovascular dataset
```

Sklearn is used to split, scale and calculate the accuracy of the predictions when implementing logistic regression in python, so I have imported it as seen above.

I have also imported the fullLogisticModel class from the full_logistic_model python file, so I can use the methods while also keeping maintainable code (separating the training and implementing code).

Lastly, the cleaned dataset is then imported so that it can be used.

```
cardioData = data.drop(["id", "cardio"], axis=1).values #contains all the records for every column except 'id' and 'cardio'
cardioValue = data["cardio"].values #contains all the values for the 'cardio' column

scaler = StandardScaler()
cardioData = scaler.fit_transform(cardioData)
#scales the data down using a standard scaler (Z-world)
```

cardioData then contains all the records for each column (besides 'id' and 'cardio'), allowing you to use it to as a nd vector in the 'fit' method. Dropping the 'id' column prevents the useless id values for each of the records skewing my regression line. Additionally, the cardio column contains the output that the model is tested against, so it must be removed.

Scaling certain column values down is essential, as values in the thousands (age in days) will overpower possibly more valuable columns like systolic blood pressure. Although I could have just changed the value of age from days to years, using a standardised scaling function creates a constant scale factor for each of the columns, which will lead to better results than leaving data in its current state. The standard scaler[20] uses the z-scale, a common scale which uses the mean and standard deviation of the column values. The formula is as follows:

$z = (x - \mu)/\sigma$, where μ refers to the mean and σ refers to the standard deviation of the data.

[20] (Skitlearn)

```
cardioData_train, cardioData_test, cardioValue_train, cardioValue_test = train_test_split(
cardioData, cardioValue, test_size=0.2, random_state=55)
#split the data into 80% training data, and 20% test data

# Train model
model = fullLogisticModel(lr=0.01, numIters=2000)
model.fit(cardioData_train, cardioValue_train)

# Test model
cardioValue_pred = model.predict(cardioData_test)
acc = accuracy_score(cardioValue_test, cardioValue_pred)

print(f"Accuracy: {acc*100:.2f}%")
```

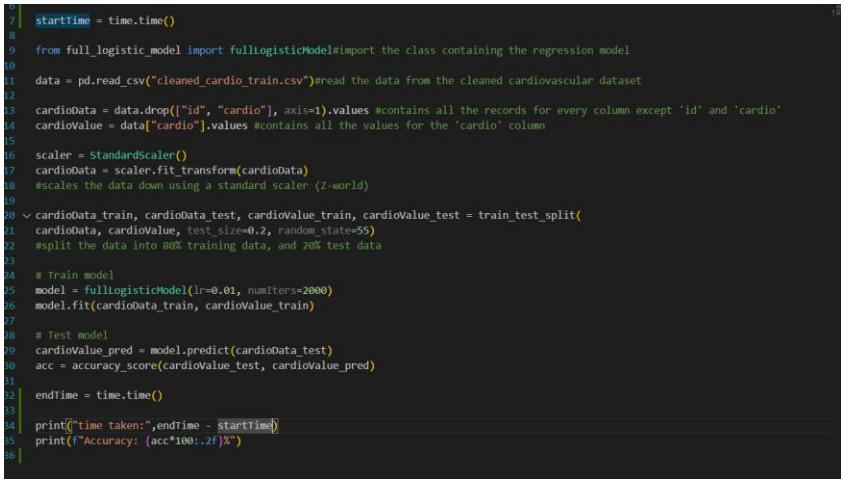
I then use the 'train_test_split' function from Sklearn to split the data into training data (80%) and test data (20%). This then allows me to test the model against unseen data to see if its reliable or not.

Now I can use the 'fit' method to train the data using the established variables cardioData_train and cardioValue_train. I've chosen to use 2000 as the number of iterations without any reasoning, but I can adjust this value and repeat the experiment to see if more iterations improve the accuracy of the regression model.

I can then predict the cardioValue using the method 'predict' established earlier, and I can calculate the accuracy of the prediction using the accuracy_score function from Sklearn. This function is simple, as it does the (number of correct predictions)/ (number of total predictions). Using it just removes the overhead needed to right such a function.

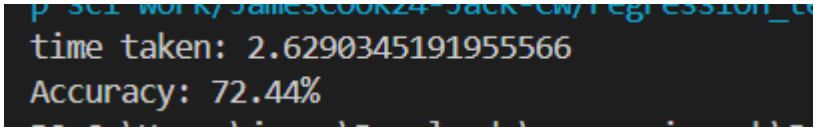
Test:

To see how long it takes to run, I will implement a timer using the time library.



```
startTime = time.time()

from full_logistic_model import fullLogisticModel#import the class containing the regression model

data = pd.read_csv("cleaned_cardio_train.csv")#read the data from the cleaned cardiovascular dataset

cardioData = data.drop(["id", "cardio"], axis=1).values #contains all the records for every column except 'id' and 'cardio'
cardioValue = data["cardio"].values #contains all the values for the 'cardio' column

scaler = StandardScaler()
cardioData = scaler.fit_transform(cardioData)
#scales the data down using a standard scaler (Z-world)

cardioData_train, cardioData_test, cardioValue_train, cardioValue_test = train_test_split(
cardioData, cardioValue, test_size=0.2, random_state=55)
#split the data into 80% training data, and 20% test data

# Train model
model = fullLogisticModel(lr=0.01, numIters=2000)
model.fit(cardioData_train, cardioValue_train)

# Test model
cardioValue_pred = model.predict(cardioData_test)
acc = accuracy_score(cardioValue_test, cardioValue_pred)

endTime = time.time()

print("time taken:",endTime - startTime)
print(f"Accuracy: {acc*100:.2f}%")
```

Here I just implemented startTime and endTime variables which can be used to test how long it took to run.



It is quick to the run the program, which is good as it may have to be run when booting up the full program. Additionally, it worked without any errors and provided an accuracy score.

However, the accuracy score is 72.44%, which could be better for a predictive program (80%+).

As a test, I will run the program while varying the number of iterations for each record, to see how it effects the accuracy, then I will plot this to see if the accuracy plateaus as the number of iterations scales up.

'Iterations' refers to the number of gradient adjustment iterations for each record.

Iterations	500	1000	2000	3000	4000	5000	6000	7000	8000	9000	10,000
Accuracy	72.15%	72.21%	72.44%	72.53%	72.58%	72.56%	72.53%	72.55%	72.56%	72.56%	72.56%
Time	1.06s	1.85s	3.38s	4.70s	5.56s	7.01s	13.68s	8.43s	16.72s	15.77s	24.34s

As you can see, the accuracy tends towards 72.56%, which seems to be the maximum value for accuracy with the current model.

There are multiple ways I can attempt to maximise accuracy before I continue with the development of my program. Firstly, I need to calculate the BMI of each record, as this is a more medically accredited measurement that correlates to cardiovascular health than height and weight alone; this can lead to relations being identified by the regression model that would otherwise go unnoticed. Your BMI is calculated by doing your height in metres squared / your weight in kg.

```
2
3   data['BMI'] = data['weight'] / ((data['height']/100) **2) #creates a column called BMI using the weight and height columns
4
5   cardioData = data.drop(["id", "cardio"], axis=1).values #contains all the records for every column except 'id' and 'cardio'
6   cardioValue = data["cardio"].values #contains all the values for the 'cardio' column
```

I've implemented this in the code so that a column called 'BMI' is created that uses the weight and height columns[21]. I plan to test the accuracy with 5000 iterations with and without the weight and height column data, to see if BMI alone improves the results.

[21] (Saturn Cloud)

Test:

```
time taken: 5.709251642227173
Accuracy: 72.56%
```

By simply adding BMI, the accuracy isn't improved. Now after removing height and weight:

```
cardioData = data.drop(["id", "cardio","height", "weight"], axis=1).values
```

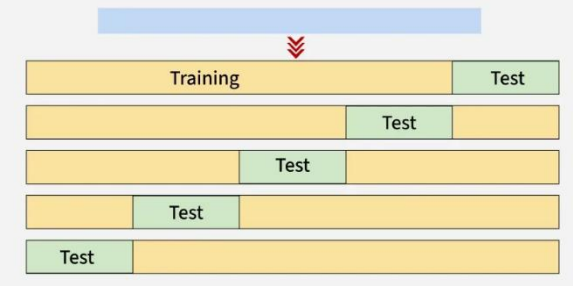
The results are as follows:

```
time taken: 6.708442687988281
Accuracy: 72.63%
```

Upon removing the height and weight for each record, the accuracy has improved for the logistic model.

On top of adding BMI to the dataset, there are two other notable ways to improve the accuracy of the program. The first method is altering how the model is trained and tested. Currently, the dataset is split into 80% training data and 20% testing data, but this is flawed since certain trends found in the other 20% will be missed and only 1 training cycle is done.

A method called cross validation[22] can be implemented, which will likely yield better accuracy metrics. The type of cross validation I plan to implement is k-fold validation.



19

This method involves splitting the dataset into k number of folds, where k represents the number of instances we'll do. If k = 5, we would split the dataset with an 80/20 split, where that 20% will change after each iteration, providing the model with different training and test data upon each iteration.

Implementing K-fold cross validation in my code is simple, as it makes use of other Sklearn functions. Here is the code as follows:

[22] (GeeksforGeeks)

```
import pandas as pd
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import numpy as np
import time

startTime = time.time()

from full_logistic_model import fullLogisticModel  # import your logistic regression class
```

Here the KFold function is imported from sklearn so it can be used below:

```
# K-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=55)  # 5 folds
accuracies = []

for train_index, test_index in kf.split(cardioData):
    cardioData_train, cardioData_test = cardioData[train_index], cardioData[test_index]
    cardioValue_train, cardioValue_test = cardioValue[train_index], cardioValue[test_index]

    # Train model
    model = fullLogisticModel(lr=0.01, numIters=5000)
    model.fit(cardioData_train, cardioValue_train)

    # Test model
    cardioValue_pred = model.predict(cardioData_test)
    acc = accuracy_score(cardioValue_test, cardioValue_pred)
    accuracies.append(acc)

endTime = time.time()

print("Time taken:", endTime - startTime)
print(f"Accuracies across folds: {[f'{a*100:.2f}%' for a in accuracies]}")
print(f"Mean Accuracy: {np.mean(accuracies)*100:.2f}%")
```

Now we can use the KFold function with 5 folds to train the data 5 times, each with different training data and test data with the goal of reducing bias, decreasing variance more effective data usage (uses the whole data set), which should be more effective than single split training and testing which I implemented beforehand. Lastly, I need to find the mean of the 5 accuracies, as each fold will produce its own accuracy.

Test:

```
Time taken: 31.15886425971985
Accuracies across folds: ['72.63%', '72.81%', '72.43%', '72.65%', '72.58%']
Mean Accuracy: 72.62%
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

The mean accuracy ended up being 0.01% lower than the accuracy of single test and train implementation after I added the BMI of each record.

As a test, I will increase the number of folds to see if the accuracy is affected.

```
Time taken: 74.38056445121765
Accuracies across folds: ['72.32%', '72.86%', '72.26%', '73.20%', '72.42%', '72.45%', '73.49%', '71.87%', '71.99%', '73.21%']
Mean Accuracy: 72.61%
```

The time taken for the program to run has significantly increased with a net negative impact, since the mean accuracy has decreased by 0.01% again. Due to this, there seems to be no benefit it to using K-fold cross validation, as the run time is simply longer, so I will use the 80-20% onetime train and test split.

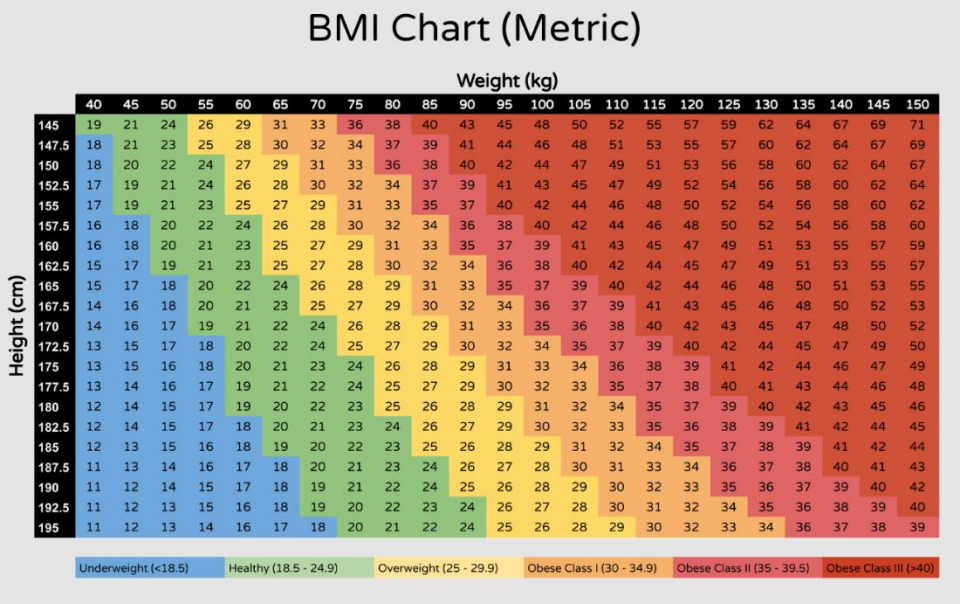
Considering I have likely optimised my parameters after cleaning and scaling while also optimising my training and testing methods, it seems the maximum accuracy for this dataset and logistic regression has been reached. Although an accuracy of 70% ±2.5% is regarded as a successful score for a logistic regression model, I need to ensure that my logistic model is truly maximised.

I've realised that although I've cleaned the dataset for impossible stand-alone values, certain combinations of values would be near impossible that would have passed my rule dictionary.

Firstly, a weight of 160kg and a height of 150cm would be accepted, but a BMI of 71.1 is near impossible for a human being. Additionally, a blood pressure of 80/90 mm Hg is impossible if you consider the systolic and diastolic blood pressure values alone, but systolic blood pressure should always be greater than diastolic blood pressure, causing anomalies.

Lastly, I need to check the dataset for any duplicates, as I never did this in my original cleaning of the cardiovascular dataset.

To fix this, I need to ensure that the ratio of height to weight is within a realistic range that leads to a realistic BMI reading.



[23]

Considering the very extremes of the BMI measurements are 11 and 70, I plan to remove the extreme values from my dataset, as they are unlikely to be physiologically possible (a weight of 40kg and a height of 190cm is near impossible). Since they would be extremely

[23] (Marathon Handbook)

rare cases, containing this in my dataset would skew the data for cases that are so rare they can be considered negligible.

Therefore, the lower bound and upper bound for a records BMI will be 14 and 58. 14 represents the bottom 5% of the BMI readings in the chart above and 58 represents the top 5% of the BMI readings, so eliminating numbers outside this range will remove any anomalous records.

Since BMI is calculated by doing a person's weight (kg) /height (m) ^2, I can calculate the acceptable weight to height ratio for a record.

w = weight (kg)

h = height (m)

$$\frac{w}{14} \geq h^2 \quad \cap \quad \frac{w}{58} \leq h^2$$

Implementing this in python is simple, and I can just add extra rules to the current 'cleaningData.py'.

I also need to do the same for systolic and diastolic blood pressure. Since systolic blood pressure represents the pressure in the arteries when the heart beats, and diastolic blood pressure represents the pressure in the arteries as the heart rests between beats, it is physiologically impossible for your pressure to be greater while your heart is resting over pumping blood.

A healthy difference is around 40 mm Hg, but anything greater than 60 can indicate artery malfunction. The same can be said about a 20 mm Hg difference[24]. Therefore, decreasing/increasing the gap by 12.5% will produce an acceptable difference of

$>15 \cup < 65$ mm Hg.

I also need to check for duplicate records in the dataset, which can be done using pandas' functions. Duplicates in a dataset can lead to bias weights, as the values in the columns for that record can affect the weights numerous times.

Pseudocode:

[24] (Scanlon)

```
for col, (min,max) in rules.items():
    #this line is very importnant.
    #the col acts as the column in the dataset, with 'min' and 'max' being established
    #items in the dataset that the records will be checked against.
    #items then allows you to go through the key and values stored in the dictionary as a tuple.

    height_m = cleaning["height"] / 100

    weight = cleaning["weight"] / 100

    condition1 = (weight/14) >= height_m ** 2
    condition2 = (weight/58) <= height_m ** 2
    #the height and weight conditions
    #applying the conditions to the cleaning dataset
    cleaning = cleaning[condition1]
    cleaning = cleaning[condition2]

    cleaning = cleaning[(cleaning[col] >= min) & (cleaning[col] <= max)]
    #this line checks each column against value assigned to a matched key, making sure the values fall inside the range
```

Implementing this in python is quite simple, as pandas' datasets allow you to apply conditions to datasets with []. Additionally, applying the next rules will also be simple for systolic and diastolic blood pressure:

```
ap_hi = cleaning["ap_hi"]
ap_lo = cleaning["ap_lo"]
diff = ap_hi - ap_lo

condition3 = diff >= 15
condition4 = diff <= 65

cleaning = cleaning[condition3]
cleaning = cleaning[condition4]

cleaning = cleaning[(cleaning[col] >= min) & (cleaning[col] <= max)]
#this line checks each column against value assigned to a matched key, makin
...
```

I also need to check for duplicates[25], which I can easily implement using pandas in python. The following code added to the cleaningData.py is below:

[25] (SaturnCloud)

```python

print(cardiovascularData.describe())
print("length:", len(cardiovascularData))

rules = {
    "age": (9125, 27375),    # days (25-75 years)
    "height": (100, 228),    # cm
    "weight": (39.9, 225),  # kg
    "ap_hi": (70, 175),      # systolic mm Hg
    "ap_lo": (50, 110)       # diastolic mm Hg
}

def cleaningData(ds, rules):
    cleaning = ds.copy()

    # Apply simple range rules
    for col, (min_val, max_val) in rules.items():
        cleaning = cleaning[(cleaning[col] >= min_val) & (cleaning[col] <= max_val)]

    # BMI-style rule (height in cm → m)
    h_m = cleaning["height"] / 100
    w = cleaning["weight"]
    bmi_condition = (w / 14 >= h_m**2) & (w / 58 <= h_m**2)
    cleaning = cleaning[bmi_condition]

    # Blood pressure consistency
    ap_hi = cleaning["ap_hi"]
    ap_lo = cleaning["ap_lo"]
    diff = ap_hi - ap_lo

    bp_condition = (ap_hi > ap_lo) & (diff >= 15) & (diff <= 65)
    cleaning = cleaning[bp_condition]

    return cleaning


cleanData = cleaningData(cardiovascularData, rules)
cleanData = cleanData.drop_duplicates()

print(cleanData.describe())
print("length:", len(cleanData))

# cleanData.to_csv('cleaned_cardio_train.csv', index=False)
```

Test:

Now, if I implement this in python and run the program, I should have a reduced number of records since the conditions are applied to the dataset.

```
                id           age        gender        height        weight         ap_hi  ...   cholesterol          gluc         smoke          alco        active        cardio
count  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  ...  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000  70000.000000
mean   49972.419900  19468.865814      1.349571    164.359229     74.205690    128.817286  ...      1.366871      1.226457      0.088129      0.053771      0.803729      0.499700
std    28851.302323   2467.251667      0.476838      8.210126     14.395757    154.011419  ...      0.680250      0.572270      0.283484      0.225568      0.397179      0.500003
min        0.000000  10798.000000      1.000000     55.000000     10.000000   -150.000000  ...      1.000000      1.000000      0.000000      0.000000      0.000000      0.000000
25%    25006.750000  17664.000000      1.000000    159.000000     65.000000    120.000000  ...      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
50%    50001.500000  19703.000000      1.000000    165.000000     72.000000    120.000000  ...      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
75%    74889.250000  21327.000000      2.000000    170.000000     82.000000    140.000000  ...      2.000000      1.000000      0.000000      0.000000      1.000000      1.000000
max    99999.000000  23713.000000      2.000000    250.000000    200.000000  16020.000000  ...      3.000000      3.000000      1.000000      1.000000      1.000000      1.000000

[8 rows x 13 columns]
length: 70000
                id           age        gender        height        weight         ap_hi  ...   cholesterol          gluc         smoke          alco        active        cardio
count  64163.000000  64163.000000  64163.000000  64163.000000  64163.000000  64163.000000  ...  64163.000000  64163.000000  64163.000000  64163.000000  64163.000000  64163.000000
mean   50004.844022  19407.159297      1.346648    164.446020     73.727831    124.274862  ...      1.350950      1.220002      0.086904      0.052367      0.803282      0.471456
std    28843.776131   2470.617792      0.475906      7.895926     13.907771     13.630605  ...      0.670011      0.566520      0.281696      0.222767      0.397520      0.499188
min        0.000000  10798.000000      1.000000    109.000000     40.000000     70.000000  ...      1.000000      1.000000      0.000000      0.000000      0.000000      0.000000
25%    25031.500000  17594.000000      1.000000    159.000000     65.000000    120.000000  ...      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
50%    50080.000000  19668.000000      1.000000    165.000000     72.000000    120.000000  ...      1.000000      1.000000      0.000000      0.000000      1.000000      0.000000
75%    74885.500000  21280.000000      2.000000    170.000000     81.000000    130.000000  ...      1.000000      1.000000      0.000000      0.000000      1.000000      1.000000
max    99999.000000  23713.000000      2.000000    207.000000    200.000000    175.000000  ...      3.000000      3.000000      1.000000      1.000000      1.000000      1.000000

[8 rows x 13 columns]
length: 64163
```

As you can see, the number of records has decreased by 5836 from the original dataset and 3431 records from the previous cleaned dataset.

Now if I save this dataset as a new cleaned dataset and I pull that file into the logistic model, I should have an increased accuracy.

```
cleanData.to_csv('cleaned_cardio_train2.csv', index=False)
```

```
data = pd.read_csv("cleaned_cardio_train2.csv")#read the data from the cleaned cardiovascular dataset
```

```
time taken: 6.206548452377319
Accuracy: 71.62%
```

The accuracy has decreased by 1% after cleaning the data further.

The accuracy wouldn't have decreased due to non-sensical blood pressure readings, as this isn't possible in humans. However, the BMI bounds may be too harsh, as extremely low and high BMI readings are physiologically possible. Additionally, the difference in blood pressure readings may be too harsh as well, as a difference of 70 in systolic blood pressure and diastolic blood pressure is possible in people who may have diabetes or cholesterol issues.

Therefore, I plan to change the BMI restrictions to >=10 and <= 75, with the blood pressure difference changing <= 90 and > 0.

Test:

```
[8 rows x 13 columns]
length: 67507
```

With this change there are more records (87 less than the original cleaned dataset), yet they still follow the capabilities of human anatomy, so the accuracy should remain close to the previous 72.62%, hopefully with a slight increase.

```
time taken: 6.742499351501465
Accuracy: 72.73%
```

Now you see an increase in the accuracy of the model, even though it is small.

This seems to be a maxed full logistic regression model, considering the dataset is as clean as possible and that the testing features optimise accuracy. Therefore, I can implement the partial models that exclude the optional parameters.

The parameters which will be optional due to the availability of systolic/diastolic blood pressure, cholesterol and glucose levels. If I don't implement this feature, the accessibility of my program drastically decreases, as 57% of my stakeholders in my survey (analysis stage) and 29.8% stated they never check their cholesterol or blood pressure respectively.

However, since the blood pressure measurements are measured simultaneously, there are only 3 different readings a user wouldn't have. Since you could have your blood pressure readings but not your cholesterol or glucose levels, there needs to be ways to remove the parameters that are left blank during the input stage of the program, however I can implement this in my second stage of development when I develop the GUI that takes the inputs from the user in tkinter. Since I am implementing validation at this stage, adding further validation alongside this but with the regression model makes sense, as I can align the validation for the user inputs with that of the validation for the regression model.

To conclude my first stage of development, I need to implement a way to take inputs in the terminal, which can then be passed into the regression model so that I am able to apply the test data I created.

Pseudocode:

I need to implement a way to temporary take inputs to test the regression model using my test data. The below pseudocode takes an input from the user:

```
...

age = input("enter age (years):\n")
height = input("enter height (cm):\n")
weight = input("enter weight (kg):\n")
gender = input("enter gender (Male/Female):\n")
sysBP = input("enter systolic blood pressure (mm Hg):\n")
diaBP = input("enter diastolic blood pressure (mm Hg):\n")
cholesterol = input("enter cholesterol levels (mg/dL):\n")
glucose = input("enter glucose levels (mg/dL):\n")
smoking = input("enter smoking status (0 or 1):\n")
alcohol = input("enter alcohol status (0 or 1):\n")
sedentary = input("are you physically active (150minutes a week or more):\n")
cardioStatus = input("do you have a cardiovascular disease (0 or 1):\n")
```

I can now create a pandas dataframe using dictionaries.

```
userInputs = {"age": age,
              "height": height,
              "weight": weight,
              "gender" gender,
              "ap_hi": sysBP,
              "ap_lo": diaBP,
              "cholesterol": cholesterol,
              "gluc": glucose,
              "smoke": smoking,
              "active": sedentary,
              "alco":alcohol}

userDataframe = pd.DataFrame(userInputs)
...
```

Using the cardiovascular dataset column names as the keys for the dictionary allows me to create a dataframe that uses the same columns as the cardiovascular dataset where the values in the dictionary represent the record (singular).

```
68  userDataFrame = pd.dataFrame(userInputs)
69
70  print(userDataFrame)
71
```

Now I can turn this into a data frame and print it out to see if it's formatted the same as the dataset.

Test:

```
1
do you have a cardiovascular disease (0 or 1):
0
Traceback (most recent call last):
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\regression_test.py", line 68, in <module>
    userDataFrame = pd.dataFrame(userInputs)
                    ^^^^^^^^^^^^
AttributeError: module 'pandas' has no attribute 'dataFrame'. Did you mean: 'DataFrame'?
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

main ⊗ 0 ⚠ 0 James Cook (17 min

After running the code, it seems I have written the data frame line incorrectly as the code crashed on that line. Additionally, when inputting the information, I realised that I put (Male/Female) in the brackets for gender, but the value of a Female corresponds to 1 and Male 2, so I need to change this in the program.

```
enter age (years):20
enter height (cm):181
enter weight (kg):90
enter gender (Female - 1/Male - 2 ):2
enter systolic blood pressure (mm Hg):140
enter diastolic blood pressure (mm Hg):90
enter cholesterol levels (mg/dL):140
enter glucose levels (mg/dL):140
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):1
do you have a cardiovascular disease (0 or 1):0
Traceback (most recent call last):
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\regression_test.py", line 68, in <module>
    userDataFrame = pd.DataFrame(userInputs)
                    ^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\james\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages\pandas\core\frame.py", line 778, in __init__
    mgr = dict_to_mgr(data, index, columns, dtype=dtype, copy=copy, typ=manager)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\james\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages\pandas\core\internals\construction.py", line 503, in d
ict_to_mgr
    return arrays_to_mgr(arrays, columns, index, dtype=dtype, typ=typ, consolidate=copy)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\james\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages\pandas\core\internals\construction.py", line 114, in a
rrays_to_mgr
    index = _extract_index(arrays)
            ^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\james\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages\pandas\core\internals\construction.py", line 667, in _
extract_index
    raise ValueError("If using all scalar values, you must pass an index")
ValueError: If using all scalar values, you must pass an index
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

After rerunning the code, I still have syntax errors, likely due to issues converting the dictionary into a pandas data frame.

```
userInputs = {"age": [age],
              "height": [height],
              "weight": [weight],
              "gender": [gender],
              "ap_hi": [sysBP],
              "ap_lo": [diaBP],
              "cholesterol": [cholesterol],
              "gluc": [glucose],
              "smoke": [smoking],
              "active": [sedentary],
              "alco":[alcohol]
              }
```

After reviewing how to implement data frames using dictionaries and pandas, it seems the values need to be stored as a list, where each list index stores the value for a key (column) in a record.

Test:

```
   age height weight gender ap_hi ap_lo cholesterol gluc smoke active alco
0  21    181     90      2   140    90         130  130     0      1    0
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

It now prints out the information as a data frame, meaning that I should be able to pass it into the prediction method for the full regression model.

```
print(model.predict(userDataFrame))
```

If I then pass userDataFrame into the predict method for the model instantiated earlier during training and testing its accuracy, I should get a value of either 0 or 1.

Test:

```
Traceback (most recent call last):
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\regression_test.py", line 72, in <module>
    print(model.predict(userDataFrame))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\full_logistic_model.py", line 40, in predict
    linearModel = np.dot(Sample, self.weights) + self.bias
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: shapes (1,11) and (10,) not aligned: 11 (dim 1) != 10 (dim 0)
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

Based on the syntax error provided, it seems the shape of the vector isn't the same as the shape of the cardiovascular dataset. This will be due to the lack of a BMI column and the presence of the height and weight column. Since I added a BMI column to my cardiovascular dataset, I must do the same to the test data to match it to the cardiovascular dataset.

Additionally, I need to scale the data the same way I scaled the cardiovascular dataset, to make sure the data fits within the bounds of the column values. Alongside this I need to convert the values from strings to floats and integers, because that's how it's stored in the csv file.

```
userInputs = {"age": [int(age)],
              "height": [float(height)],
              "weight": [float(weight)],
              "gender": [int(gender)],
              "ap_hi": [int(sysBP)],
              "ap_lo": [int(diaBP)],
              "cholesterol": [int(cholesterol)],
              "gluc": [int(glucose)],
              "smoke": [int(smoking)],
              "active": [int(sedentary)],
              "alco":[int(alcohol)]
              }

userDataFrame = pd.DataFrame(userInputs)

# Add BMI like in training
userDataFrame["BMI"] = userDataFrame["weight"] / ((userDataFrame["height"]/100) ** 2)

# Drop height and weight (to match training features)
userDataFrame = userDataFrame.drop(["height", "weight"], axis=1)

userDataFrame = scaler.transform(userDataFrame)
```

Here I have changed each of the userInputs variables into integers/floats, so that the BMI can be added properly. I then removed the height and weight columns. I have also scaled the data frame the same way I scaled the cardiovascular dataset.

Lastly, I need to apply the bounds decided earlier to glucose and cholesterol levels as well as changing the users age from years to days. I won't use any validation for these values as this is only used to check the regression model, not the efficacy of entering data into the program.

```
#tests for the regression model
age = input("enter age (years):") * 365
height = input("enter height (cm):")
weight = input("enter weight (kg):")
gender = input("enter gender (Female - 1/Male - 2 ):")
sysBP = input("enter systolic blood pressure (mm Hg):")
diaBP = input("enter diastolic blood pressure (mm Hg):")
cholesterol = input("enter cholesterol levels (mg/dL):")

if cholesterol <= 200:
    cholesterol = 1
elif cholesterol > 200 and cholesterol <= 240:
    cholesterol = 2
else:
    cholesterol = 3

glucose = input("enter glucose levels (mg/dL):")

if glucose <= 100:
    glucose = 1
elif glucose > 100 and glucose <= 125:
    glucose = 2
else:
    glucose = 3

smoking = input("enter smoking status (0 or 1):")
```

Here is the updated regression_test python dictionary:

Regression_test.py →

Variable	Data type	Description
data	Pandas vector	Reads the cleaned cardiovascular dataset as a pandas vector to be passed into the fit method found in the 'full_logistic_model.py' file.
cardioData	Pandas vector	Replicate of the 'data' variable, except the columns labelled 'id', 'cardio', 'height' and 'weight' have been dropped, with 'BMI' being added using the height and weight columns.
cardioValue	Pandas vector	Contains the values in the 'cardio' column within the dataset.
cardioData_train	Pandas vector	Contains 80% of cardioData, which is used to train the regression model
cardioData_test	Pandas vector	Contains 20% of cardioData, which is used to test the regression model.

cardioValue_train	Pandas vector	Contains 80% of cardioValue, which is used to train the regression model
cardioValue_test	Pandas vector	Contains 20% of cardioValue, which is used to test the regression model.
Model	Object	Instantiates the fullLogisticModel class found in 'full_logistic_model.py'.
cardioValue_pred	Pandas vector	Contains the predictions using the regression model which can be tested against the actual cardio values within the dataset.
userInputs	Dictionary	A dictionary that stores all of the user inputs that are passed into the regression model when testing.
userDataFrame	Pandas vector	Creates a pandas data frame for the user inputs.

Test:

```
enter gender (Female - 1/Male - 2 ):2
enter systolic blood pressure (mm Hg):140
enter diastolic blood pressure (mm Hg):90
enter cholesterol levels (mg/dL):130
enter glucose levels (mg/dL):130
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):1
do you have a cardiovascular disease (0 or 1):0
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
[0]
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

It now prints out a value between 0 and 1 below, which I need to test the model against the training data.

Test plan implementation

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data for a record which isn't boundary data	Valid	The corresponding value between 0 and 1 for their cardiovascular disease status.	Predicted 0 – value is 0 (correct prediction)

The record I am choosing to implement is as follows:

age: 21296d (58.3y)

gender: Female (1)

height: 170cm

Weight: 75kg

systolic blood pressure: 120 mm Hg

diastolic blood pressure: 80 mm Hg

cholesterol: Normal (1)

glucose: Normal (1)

smoke: No (0)

alcohol: No (0)

active: No (0)

cardiovascular disease: Not present (0)

Each of the selected values for the columns remain normal and within any bounds established throughout this stage of development. Due to this, they're a perfectly valid candidate for the valid data test.

Test:

```
, in <module>
    if cholesterol <= 200:
       ^^^^^^^^^^^^^^^^^^
TypeError: '<=' not supported between instances of 'str' and 'int'
```

When inputting the cholesterol, I realised that the casting is done after these if statements, so I need to make the variables integers upon their input.

```
cholesterol = int(input("enter cholesterol levels (mg/dL):"))

if cholesterol <= 200:
    cholesterol = 1
elif cholesterol > 200 and cholesterol <= 240:
    cholesterol = 2
else:
    cholesterol = 3

glucose = int(input("enter glucose levels (mg/dL):"))
```

I also need to ensure that the users cardioStatus is outputted alongside the model's prediction for comparison:

```
print(ans)
print("actual cardio status:", cardioStatus)
```

Test:

```
enter age (years):58
enter height (cm):170
enter weight (kg):75
enter gender (Female - 1/Male - 2 ):1
enter systolic blood pressure (mm Hg):120
enter diastolic blood pressure (mm Hg):80
enter cholesterol levels (mg/dL):150
enter glucose levels (mg/dL):80
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):0
do you have a cardiovascular disease (0 or 1):0
[[ 0.69573923 -0.73039938 -0.38265594 -0.11316605 -0.53296473 -0.39227175
  -0.30966572 -0.23636675 -2.02319804 -0.28096187]]
[0]
actual cardio status: 0
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

The code now works as intended and provides the predicted probability of 0 (lack of cardiovascular disease), which matches the records cardiovascular status. Although this prediction will be incorrect ≈ 28% of the time, in this case it predicted correctly, resulting in a valid test outcome.

Test no.	Description + test data	Type of test	Expected outcome	outcome
2.	Enter data that is close to a record in value but not exact (healthy)	Valid	The same cardiovascular disease status as that record	Predicted 0 – matches the records status of 0.

For this test, we can slightly alter the values of the previous record, without altering the overall expected diagnosis based on how parameters may correlate to cardiovascular disease.

age: 22630d (62y)

gender: Male (2)

height: 175cm

Weight: 70kg

systolic blood pressure: 120 mm Hg

diastolic blood pressure: 80 mm Hg

cholesterol: Normal (1)

glucose: Normal (1)

smoke: No (0)

alcohol: No (0)

active: Yes (1)

cardiovascular disease: Not present (0)

I have increased their age by 4 years, changed them to a male and slightly altered their height and weight. Additionally, I've changed how active they're. A change of sex shouldn't affect the cardiovascular diagnosis at all, yet increasing their age by 4 should increase their likelihood. However, reducing the record's weight will decrease their BMI, hopefully balancing the changes made by a greater age. Making them an active person should also decrease their likelihood of being predicted cardiovascular disease present.

Test:

```
enter age (years):62
enter height (cm):175
enter weight (kg):70
enter gender (Female - 1/Male - 2 ):1
enter systolic blood pressure (mm Hg):120
enter diastolic blood pressure (mm Hg):80
enter cholesterol levels (mg/dL):1
enter glucose levels (mg/dL):1
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):1
do you have a cardiovascular disease (0 or 1):0
[[ 1.28691198 -0.73039938 -0.38265594 -0.11316605 -0.53296473 -0.39227175
  -0.30966572  4.23071344 -2.02319804 -0.8744691 ]]
[0]
actual cardio status: 0
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

The prediction is still 0, which is the desired prediction considering the parameters aren't significantly different than the real, previous record.

Test no.	Description + test data	Type of test	Expected outcome	outcome
3.	Enter data that is close to a record in value	Valid	The same cardiovascular disease status as that record	Predicted 1 – matches the cardiovascular status of the

	but not exact (unhealthy)			record (present)

For this test I plan to find a record that has cardiovascular disease and seems to have health metrics that indicate the presence of cardiovascular disease as well.

age: 20228d (55.5y)

gender: Female (1)

height: 156 cm

Weight: 85 kg

systolic blood pressure: 140 mm Hg

diastolic blood pressure: 90 mm Hg

cholesterol: 3

glucose: 1

smoke: 0

alcohol: 0

active: 1

cardiovascular disease: Present (1)

They are over 50, which links to increased risk of cardiovascular disease[27]. Additionally, their BMI is 34.93, which is labelled as 'obese'. They also have a heightened blood pressure, linking the record to diabetes. Due to these factors, they're likely to have cardiovascular disease, despite them being active.

Test:

```
enter age (years):56
enter height (cm):156
enter weight (kg):85
enter gender (Female - 1/Male - 2 ):1
enter systolic blood pressure (mm Hg):140
enter diastolic blood pressure (mm Hg):90
enter cholesterol levels (mg/dL):450
enter glucose levels (mg/dL):50
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):1
do you have a cardiovascular disease (0 or 1):1
[[ 0.40015285 -0.73039938  0.93474977  1.00821207  2.42594395 -0.39227175
  -0.30966572  4.23071344 -2.02319804  1.44065419]]
[1]
actual cardio status: 1
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

The model predicted the record to have a cardiovascular disease, which is correct; therefore, the regression model passes the test.

[26] (Rodgers JL)

Test no.	Description + test data	Type of test	Expected outcome	outcome
4.	Enter values on the lower range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	A probability very close to 0.

For this test, I need to fabricate values for each column, where each value is placed close to the lower bound for each of the parameters. Additionally, I want to display the predicted probability between 0 and 1 alongside the output of 0 or 1 to test how 'confident' the model is. I also need to have this implemented for all the following tests.

To do this, I can create a method that returns the probability after it is passed into the sigmoid function but before list comprehension is provided to the output.

Pseudocode:

```
...

def probability(Sample)
  predictionModel = np.dot(sample,weights) + bias
  predictionValue = sigmoid(predictionModel)
  return predictionValue
...
```

This method will create a regression line the same way it was done earlier in the predict method, which can then be passed into a sigmoid function. After this, it then just must be returned without any adjustments, as we want the probability, not a value of 0 or 1.

Here is the implementation in python:

```
def probability(self,Sample):
    predictionModel = np.dot(Sample,self.weights) + self.bias
    predictionValue = self.sigmoid(predictionModel)
    return predictionValue
```

It is then easy to call this method as I output the predicted probability by the regression model:

```
ans = model.predict(userDataFrame)
print("actual probability:",model.probability(userDataFrame))
print(ans)
print("actual cardio status:", cardioStatus)
```

Record:

age: 7765g (21y)

gender: Female (1)

height: 110 cm

Weight: 40 kg

systolic blood pressure: 90 mm Hg

diastolic blood pressure: 50 mm Hg

cholesterol: 1

glucose: 1

smoke: 0

alcohol: 0

active: 0

cardiovascular disease: ?

Each of the values for the columns are based on the established bounds when cleaning the data and when establishing the goals of the program. Although a height and weight of 110cm and 40kg is unlikely, it's possible in people with dwarfism and the BMI is still realistic yet overweight (33). The systolic blood pressure and diastolic blood pressure are extremely low, with both values corresponding to serious issues like the risk of a leaky valve in the heart, but this test isn't trying to test the validation of user inputs, it's to test the regression model. The values of the cholesterol and glucose and normal as this is as low as it can go and that's the same with smoking, alcohol status and how active you're. Since this is an extreme made up record, the cardiovascular disease status is unknown.

Test:

```
enter age (years):21
enter height (cm):110
enter weight (kg):40
enter gender (Female - 1/Male - 2 ):1
enter systolic blood pressure (mm Hg):90
enter diastolic blood pressure (mm Hg):50
enter cholesterol levels (mg/dL):10
enter glucose levels (mg/dL):10
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):0
do you have a cardiovascular disease (0 or 1):0
[[-4.77260873 -0.73039938 -2.35876451 -3.47730042 -0.53296473 -0.39227175
  -0.30966572 -0.23636675 -2.02319804  1.08202206]]
actual probability: [0.01843606]
[0]
actual cardio status: 0
```

The probability was outputted before the number of 0 or 1, meaning my method worked. However, the probability was not close to 0.5 at all, as it is equal to 0 to 2 sig figs. However, after conducting research surrounding the metrics that correlate to cardiovascular disease and after understanding my dataset the more I utilise it, records with very low blood pressure are scarce and based on their cardiovascular status (often 0), the regression model

likely generated a positive correlation between blood pressure and cardiovascular disease status. Therefore, an extremely low blood pressure will lead to a very 'confident' diagnosis of 0 – lack of a cardiovascular disease.

So, despite the test results not matching my expected outcome, my expected outcome is at fault, not the regression model.

Test no.	Description + test data	Type of test	Expected outcome	outcome
5.	Enter values on the upper range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	A probability very close to 1.

Based on the information learnt from the previous test, I believe extreme data on the upper range for each column will lead to a very strong diagnosis close to 1, since a very high blood pressure will correlate to a cardiovascular disease presence.

age: 32850d (90y)

gender: Male (2)

height: 200 cm

Weight: 190 kg

systolic blood pressure: 170 mm Hg

diastolic blood pressure: 105 mm Hg

cholesterol: 3

glucose: 3

smoke: 1

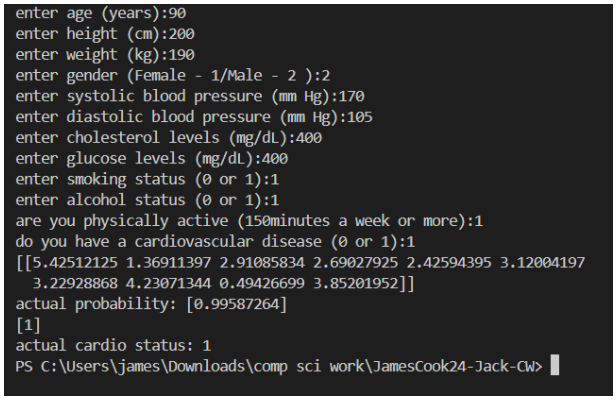
alcohol: 1

active: 1

cardiovascular disease: ?

Each of the values for the columns are based on the established bounds when cleaning the data and when establishing the goals of the program. Since this record is very old (90 years old), their cardiovascular disease probability is already significantly higher than those younger than him. Additionally, they have an extremely high BMI close to 50 (47.5) adding to this probability. Since their blood pressure metrics are extremely high, essentially indicating a heart attack, this isn't realistic, but I can use it to test the regression model. Since their cholesterol and glucose levels are labelled as above normal, they're even more likely to have a cardiovascular disease. Due to all the positive leaning factors, I believe the probability will be very close to 1 in its diagnosis.

Test:



The model predicted the user to have cardiovascular disease, with a probability of 0.99587. This conclusion aligns closely with my updated expectations, considering a high blood pressure is very closely correlated to a cardiovascular disease presence.

Test no.	Description + test data	Type of test	Expected outcome	outcome
6.	Enter data values outside the range for every field in the dataset (unhealthy end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	Generated number very close to 1

Here I need to similarly fabricate extreme values for a record, except each of the values must be on the unhealthy end of extremity.

age: 32850d (90y)

gender: Male (2)

height: 170 cm

Weight: 175 kg

systolic blood pressure: 200 mm Hg

diastolic blood pressure: 130 mm Hg

cholesterol: 3

glucose: 3

smoke: 1

alcohol: 1

active: 0

cardiovascular disease: ?

Each of the values for the columns are based on the established bounds when cleaning the data and when establishing the goals of the program. I don't need to change the age of this record compared to the previous since the older you are, the more likely you are to develop a cardiovascular disease. The gender/sex of the record is irrelevant, so it doesn't need to be altered. The height and weight for this record correspond to a BMI of 60, which is very high and is linked to extreme obesity. The blood pressure metrics are also very extreme and outside any realistic range for a living person. Lastly, the cholesterol and glucose levels are 3 (the maximum) and the record smokes and drinks regularly. A lack of exercise will increase the likelihood of developing a cardiovascular disease. Since this record is extremely unhealthy to the point of hyperbole, the output will be very close to 1.

Test:

```
enter age (years):90
enter height (cm):170
enter weight (kg):175
enter gender (Female - 1/Male - 2 ):2
enter systolic blood pressure (mm Hg):200
enter diastolic blood pressure (mm Hg):130
enter cholesterol levels (mg/dL):500
enter glucose levels (mg/dL):500
enter smoking status (0 or 1):1
enter alcohol status (0 or 1):1
are you physically active (150minutes a week or more):0
do you have a cardiovascular disease (0 or 1):1
[[ 5.42512125  1.36911397  4.88696691  5.49372456  2.42594395  3.12004197
   3.22928868 -0.23636675  0.49426699  6.35570035]]
actual probability: [0.99971392]
[1]
actual cardio status: 1
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

The prediction by the regression model matches my prediction. The probability was also extremely close to 1, likely due to the extreme blood pressure readings.

Test no.	Description + test data	Type of test	Expected outcome	outcome
7.	Enter data values outside the range for every field in the dataset (healthier end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	

For this test I must fabricate a record that is outside the range yet healthy for every parameter. Since values outside of my established ranges can't be considered healthy as a healthy measurement will fall quite close to the median value for that parameter, I will change this test slightly; I will create a record which has a textbook healthy value for every parameter with the goal of generating a probability very close to 0.

Test no.	Description + test data	Type of test	Expected outcome	outcome
7.	Enter data values that correspond to 'healthy' metrics for each parameter	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	Close to 0 – no cardiovascular disease

age: 10950d (30y)

gender: Male (2)

height: 180 cm

Weight: 72 kg

systolic blood pressure: 120 mm Hg

diastolic blood pressure: 80 mm Hg

cholesterol: 1

glucose: 1

smoke: 0

alcohol: 0

active: 1

cardiovascular disease: ?

Each of the values for the columns are based on the established bounds when cleaning the data and when establishing the goals of the program. I made this record quite young relative to the median for the dataset, since the younger you are the less risk, you have for developing a cardiovascular disease. The height and weight I chose correlate to a BMI close to 22, which is the standard, healthy goal for your BMI. Similarly, the blood pressure I chose correlates to healthy, normal blood pressure metrics. Their glucose and cholesterol levels are set to 1 (normal) and they do not smoke or drink alcohol regularly. Lastly, being active correlates with a healthier heart and a lower risk of developing a cardiovascular disease.

Test:

```
enter age (years):30
enter height (cm):180
enter weight (kg):72
enter gender (Female - 1/Male - 2 ):2
enter systolic blood pressure (mm Hg):120
enter diastolic blood pressure (mm Hg):80
enter cholesterol levels (mg/dL):1
enter glucose levels (mg/dL):1
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):1
do you have a cardiovascular disease (0 or 1):0
[[-3.44247004  1.36911397 -0.38265594 -0.11316605 -0.53296473 -0.39227175
  -0.30966572  4.23071344 -2.02319804 -0.99624658]]
actual probability: [0.13779134]
[0]
actual cardio status: 0
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

The result is as expected, the probability is close to 0 (0.13779), corresponding to a lack of a cardiovascular disease with high 'confidence'.

Test no.	Description + test data	Type of test	Expected outcome	outcome
8.	Enter a negative value for 1 field	Error	This should still generate a number between 0 and 1, however the value may be skewed or inaccurate.	Skewed the data massively – 46% to ≈0%

The final test is to see how the regression model responds to negative numbers. Since validation isn't accounted for yet, (being implemented in stage 2) I should still get a value, possibly close to 0 or 1 depending on what the parameter altered is.

I'm going to use the same record I used for the first test:

age: 21296d (58.3y)

gender: N/A (-1)

height: 170cm

Weight: 75kg

systolic blood pressure: 120 mm Hg

diastolic blood pressure: 80 mm Hg

cholesterol: Normal (1)

glucose: Normal (1)

smoke: No (0)

alcohol: No (0)

active: No (0)

cardiovascular disease: Not present (0)

The parameter I am choosing to alter is the record's gender. Altering any other parameter to a negative value will likely massively skew the result to 0 or 1, as all the other parameters likely have a strong coefficient for cardiovascular diseases. Altering their gender will allow me to see how the regression responds to negatives.

Test:

```
enter age (years):58
enter height (cm):170
enter weight (kg):75
enter gender (Female - 1/Male - 2 ):-1
enter systolic blood pressure (mm Hg):120
enter diastolic blood pressure (mm Hg):80
enter cholesterol levels (mg/dL):1
enter glucose levels (mg/dL):1
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):0
do you have a cardiovascular disease (0 or 1):0
[[ 0.69573923 -4.92942609 -0.38265594 -0.11316605 -0.53296473 -0.39227175
  -0.30966572 -0.23636675 -2.02319804 -0.28096187]]
actual probability: [0.44975755]
[0]
actual cardio status: 0
PS C:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW>
```

The probability outputted was close to 0.5, however I do not know if this value is skewed since I didn't implement the probability output when testing this record. For the sake of comparison and creating a normal, I'll input these parameters with the gender of the record being Female (1).

```
enter age (years):58
enter height (cm):170
enter weight (kg):75
enter gender (Female - 1/Male - 2 ):1
enter systolic blood pressure (mm Hg):120
enter diastolic blood pressure (mm Hg):80
enter cholesterol levels (mg/dL):1
enter glucose levels (mg/dL):1
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):0
do you have a cardiovascular disease (0 or 1):0
[[ 0.69573923 -0.73039938 -0.38265594 -0.11316605 -0.53296473 -0.39227175
  -0.30966572 -0.23636675 -2.02319804 -0.28096187]]
actual probability: [0.46806877]
[0]
actual cardio status: 0
```

Since the gender of the record is insignificant, there was very little skew found in the probability provided by the regression model. For the sake of the test, I will now change the record's age to a negative value to see how it skews the dataset, since gender is quite insignificant when it comes to reaching a diagnosis.

```
enter age (years):-10
enter height (cm):170
enter weight (kg):75
enter gender (Female - 1/Male - 2 ):1
enter systolic blood pressure (mm Hg):120
enter diastolic blood pressure (mm Hg):80
enter cholesterol levels (mg/dL):1
enter glucose levels (mg/dL):1
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):0
do you have a cardiovascular disease (0 or 1):0
[[-9.35419756 -0.73039938 -0.38265594 -0.11316605 -0.53296473 -0.39227175
  -0.30966572 -0.23636675 -2.02319804 -0.28096187]]
actual probability: [0.02788137]
[0]
actual cardio status: 0
```

As you can see, the probability was massively skewed to ≈ 0%. Considering a record's age and their cardiovascular status are likely positively correlated, this makes sense and shows that the regression model has identified a correlation between this parameter and cardiovascular statuses.

Now that I've concluded the basic test plan against the regression model, I can move onto implementing a main menu GUI and input system for the program. Since the stage 2 test plan is quite like this test plan, I will need to alter it significantly to account for validation I must implement and test. Additionally, the tests that I had to alter for this stage that are also found in the stage 2 test plan will be altered too so that I am using effective and efficient tests throughout the development of the program.

The final test plan is as follows:

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data for a record which isn't boundary data	Valid	The corresponding value between 0 and 1 for their cardiovascular disease status.	Predicted 0 – value is 0 (correct prediction)
2.	Enter data that is close to a record in value but not exact (healthy)	Valid	The same cardiovascular disease status as that record	Predicted 0 – matches the records status of 0.
3.	Enter data that is close to a record in value but not exact (unhealthy)	Valid	The same cardiovascular disease status as that record	Predicted 1 – matches the cardiovascular status of the record (present)
4.	Enter values on the lower range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	A probability very close to 0.
5.	Enter values on the upper range for all the data points in a field	Boundary	A number between 0 and 1, likely closer to 0.5 than 0 or 1.	A probability very close to 1.
6.	Enter data values outside the range for every field in the dataset (unhealthy end of each field)	Boundary	Likely generate a number very close to 0 or 1 depending on what corresponds to cardiovascular disease.	Generated number very close to 1
7.	Enter data values that correspond to 'healthy' metrics	Boundary	Likely generate a number very close to 0 or 1 depending on what	Close to 0 – no cardiovascular disease

	for each parameter		corresponds to cardiovascular disease.	
8.	Enter a negative value for 1 field	Error	This should still generate a number between 0 and 1, however the value may be skewed or inaccurate.	Skewed the data massively – 46% to ≈0%

Stage 1 References

American Heart Association. *Understanding Blood Pressure Readings*. 18 09 2025. <https://www.heart.org/en/health-topics/high-blood-pressure/understanding-blood-pressure-readings>.

Arya, Nisha. *Linear vs Logistic Regression: A succinct Explanation*. 21 03 2022. https://www.kdnuggets.com/2022/03/linear-logistic-regression-succinct-explanation.html. 24 09 2025.

Banoula, Mayank. *What is Cost Function in Machine Learning*. 23 02 2023. https://www.simplilearn.com.cach3.com/tutorials/machine-learning-tutorial/cost-function-in-machine-learning.html#google_vignette. 23 09 2025.

Cervoni, Barbie. *What is a healthy cholesterol level by age?* 2024. 18 09 2025. <https://www.verywellhealth.com/cholesterol-levels-by-age-chart-5190176>.

Cleary, Kevin. *The Facts About Low Blood Pressure*. 14 05 2025. https://www.healthproductsforyou.com/ar-facts-about-low-blood-pressure.html. 19 09 2025.

GeeksforGeeks. *cross validation in machine learning*. 04 08 2025. https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/. 24 09 2025.

GeeksForGeeks. *Different ways to create Pandas Dataframe*. 11 07 2025. https://www.geeksforgeeks.org/python/different-ways-to-create-pandas-dataframe/. 30 09 2025.

Horvath, Reka. *Using pandas and Python to Explore Your Dataset*. n.d. https://realpython.com/pandas-python-explore-dataset/#manipulating-columns. 18 09 2025.

Kumar, Sunil. *Implementing Logistic Regression From Scratch In Python*. 14 10 2024. https://www.analyticsvidhya.com/blog/2022/02/implementing-logistic-regression-from-scratch-using-python/#h-cost-function. 23 09 2025.

Loeber, Patrick. *Logistic Regression in Python - Python Tutorial*. 15 09 2019. https://www.youtube.com/watch?v=JDU3AzH3WKg.

Low, Rand. *Cost functions, gradient descent and gradient boosts*. 02 01 2019. https://randlow.github.io/posts/machine-learning/cost-func-gradient-descent-boost/. 23 09 2025.

Marathon Handbook. *BMI Calculator: Body Mass Index Healthy Weight Assessment*. 2025. https://marathonhandbook.com/bmi-calculator/. 25 09 2025.

Mărginean, Tudor. *How to Save a Pandas DataFrame to CSV*. 26 06 2022. https://www.datacamp.com/tutorial/save-as-csv-pandas-dataframe?utm_cid=19589720821&utm_aid=157098104775&utm_campaign=230119_1-ps-other~dsa~tofu_2-b2c_3-emea_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9045819-&utm_mtd=-c&utm_kw=&utm_source=g. 18 09 2025.

NumPy. *NumPy documentation*. 2025. https://numpy.org/doc/2.3/index.html. 23 09 2025.

Pykes, Kurtis. *pandas read_csv() Tutorial: importing data*. 02 12 2022. https://www.datacamp.com/tutorial/pandas-read-csv?utm_cid=19589720821&utm_aid=157156375191&utm_campaign=230119_1-ps-other~dsa~tofu_2-b2c_3-emea_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9045819-&utm_mtd=-c&utm_kw=&utm_source=google&utm_med. 18 09 2025.

Rodgers JL, Jones J, Bolleddu SI. *Cardiovascular Risks Associated with Gender and Aging*. 27 04 2019. http://pmc.ncbi.nlm.nih.gov/articles/PMC6616540/. 01 10 2025.

Saturn Cloud. *How to Change All the Values of a Column in a Pandas DataFrame*. 24 10 2023. https://saturncloud.io/blog/pandas-how-to-change-all-the-values-of-a-column/. 24 09 2025.

SaturnCloud. *How to Find All Duplicate Rows in a Pandas Dataframe*. 17 06 2023. https://saturncloud.io/blog/how-to-find-all-duplicate-rows-in-a-pandas-dataframe/#:~:text=To%20find%20duplicate%20rows%20in%20a%20pandas%20dataframe%2C%20we%20can,get%20all%20the%20duplicate%20rows. 27 09 2025.

Scanlon, Amy. *Systolic Vs Diastolic Blood Pressure: What Causes Big Difference*. 15 10 2024. https://www.cardiovasculardiseasehub.com/archives/9868. 25 09 2025.

Seery, Conor. *Blood Sugar level Ranges*. 2022. 18 09 2025. <https://www.diabetes.co.uk/diabetes_care/blood-sugar-level-ranges.html>.

Skitlearn. *StandardScaler*. n.d. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. 24 09 2025.

Ulianova, Svetlana. *Cardiovascular disease dataset*. 16 09 2025. <https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset?resource=download>.

W3Schools. *Loop Through a Dictionary*. 2025. https://www.w3schools.com/python/python_dictionaries_loop.asp. 18 09 2025.

Wikipedia. *Logistic Regression*. n.d. https://en.wikipedia.org/wiki/Logistic_regression. 23 09 2025.

—. *Yao Ming*. 2025. 18 09 2025. <https://en.wikipedia.org/wiki/Yao_Ming>.

Stage 2 – Creating a user interface (UI) main menu (15th of October 2025)

This will contain the section that takes user inputs for their cardiovascular analysis and diagnosis, so doing this stage after developing a regression line based on the data allows me to identify how certain inputs should be taken. It also allows me to identify which parameters I should include, meaning I can decide if any unexpected parameters that are in the dataset can be added as optional inputs. Since optional inputs are now being incorporated, I need to ensure that there are two different regression models, one for optional inputs, and one without.

Additionally, I need to create a tab/slider that allows users to change the size of the font, and even change the font entirely, but this will be unlikely as ill use a general, basic font to prevent this issue. Alongside taking user inputs, this tab will contain some very general advice and information on what the program does, and where you can find any further information in other menus.

Since this menu takes user inputs, it's important that there's a lot of validation included. This involves preventing erroneous data (negative numbers or characters) as well as outlining data that is far outside of any given range, highlighting how this could be a major health risk or potentially a miss input.

Features to implement:

1. A large UI that displays information + a title alongside the input section

2. User input text boxes/drop downs for each input with the ability to only enter necessary inputs or optional ones too
3. A slider/setting to change the font size or font
4. A validation system to detect extreme/ erroneous data

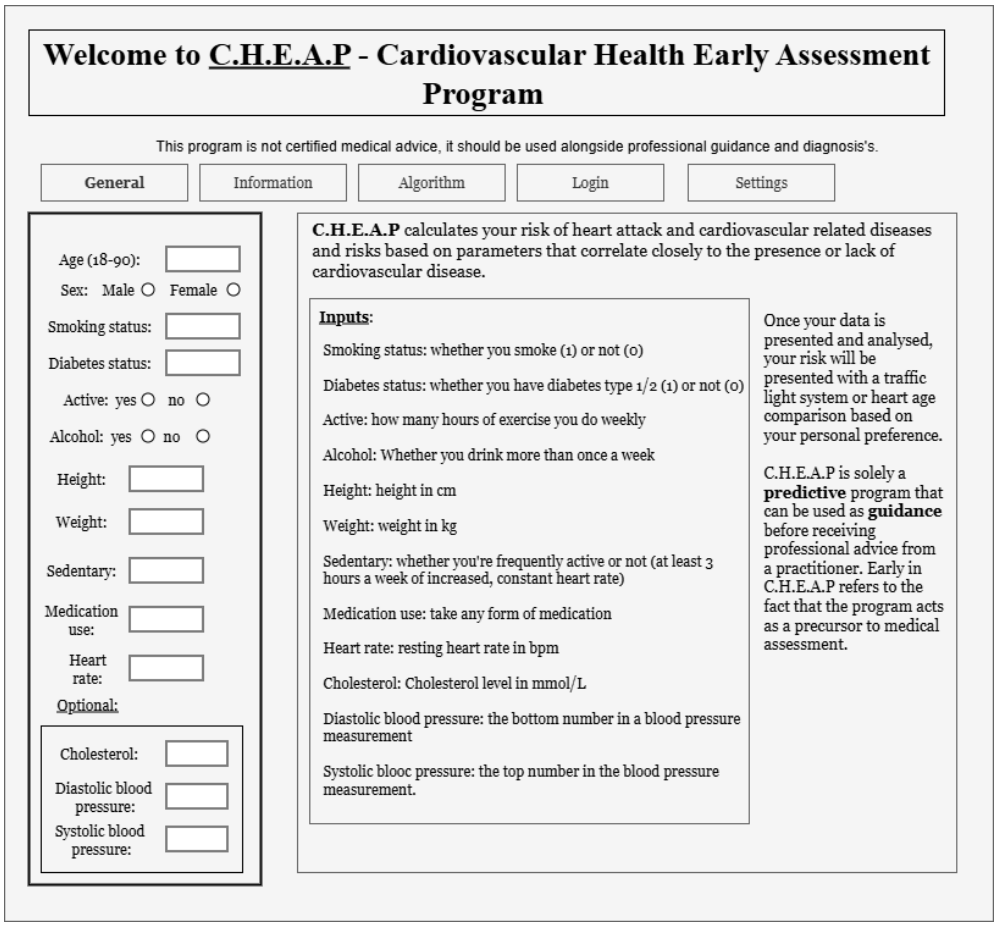
As I mentioned when I finished my first stage of development, I need to alter this test plan to properly test the features I plan to implement in this stage. Stages 6 to 8 should be rejected, since I will implement validation that rejects data that is outside a realistic range for each parameter. Additionally, stage 4 and 5 should be altered as my predicted output should match the output in stage 1, as the output is still a number between 0 and 1. Additionally, I will need to test each of the optional parameters separately as you can enter 1 optional parameter but not the other. This can be done using the same test No. and I will just use it repeatedly. Here is the updated stage 2 test plan:

Test no.	Description + test data	Type of test	Expected outcome	Outcome
1.	Enter exact data for a record which isn't boundary data	Valid	The corresponding value between 0 and 1 for their cardiovascular disease status.	
2.	Enter data that is close to a record in value but not exact (healthy)	Valid	The same cardiovascular disease status as that record.	
3.	Enter data that is close to a record in value but not exact (unhealthy)	Valid	The same cardiovascular disease status as that record.	
4.	Enter values on the lower range for all the data points in a field	Boundary	Matches the output from stage 1, test 4.	
5.	Enter values on the upper range for all the data points in a field	Boundary	Matches the output from stage 1, test 5.	

6.	Enter data values outside the range for every field in the dataset (unhealthy end of each field)	Boundary	Reject the inputs with an error message.	
7.	Enter data values outside the range for every field in the dataset (healthier end of each field)	Boundary	Reject the inputs with an error message.	
8.	Enter a negative value for 1 field	Error	Reject the inputs with an error message.	
9.	Enter characters (abc) for a numerical input	erroneous	Reject the inputs with an error message.	
10.	Enter negative numbers for an input that accepts positive numbers	erroneous	Reject the inputs with an error message.	
11.	Enter an extreme value for a parameter	Extreme	Rejects the input and informs the user but also recommends seeking medical help	
12.	Leave an input blank	erroneous	Reject the inputs with an error message	
13.	Choose to include the optional input data as an option then leave them blank	erroneous	Reject the inputs with an error message	
14	Choose to not include any optional	Valid	Gives a valid diagnosis that has correlation to their data.	

	input data and enter valid, expected data			
15.	Choose to not include any optional input data and enter that data as inputs	Erroneous	Rejects the inputs with an error message	

This stage involves GUI creation, so I initially need to create a tab that opens using tkinter The following image is the main menu I intend to create during this stage of development:



Pseudocode:

```
import tkinter as tk

root = tk.Tk()#this instantiates the class that creates a screen

root.mainloop()# starts a loop
```

This simple code will create a blank GUI menu using a main loop[27] within the tkinter library.

```
1  import tkinter as tk
2
3  root = tk.Tk()#this instantiates the class that creates a screen
4
5  root.geometry("1000x1000")#pass the dimensions of the window as a string
6
7  root.title("C.H.E.A.P")#creates a title for the menu
8
9  root.mainloop()# starts a loop
```

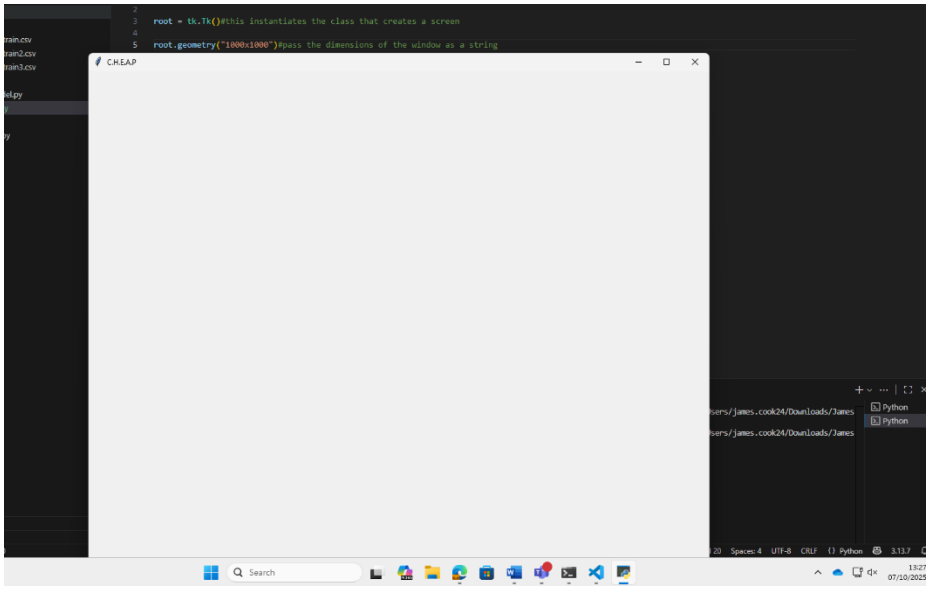
Now I've added a title and dimensions to the menu. To make sure that this works I will implement this in python:

mainMenuGUI.py > ...

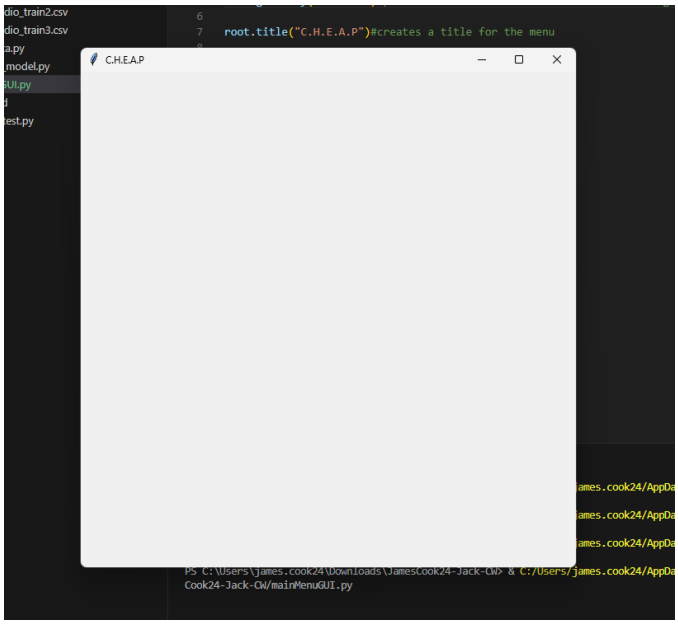
```
1  import tkinter as tk
2
3  root = tk.Tk()#this instantiates the class that creates a screen
4
5  root.geometry("1000x1000")#pass the dimensions of the window as a string
6
7  root.title("C.H.E.A.P")#creates a title for the menu
8
9  root.mainloop()# starts a loop
```

Test:

[27] (NeuralNine, 2025)



It created a menu too big to fit my screen, so I will reduce the dimensions to 600x600.



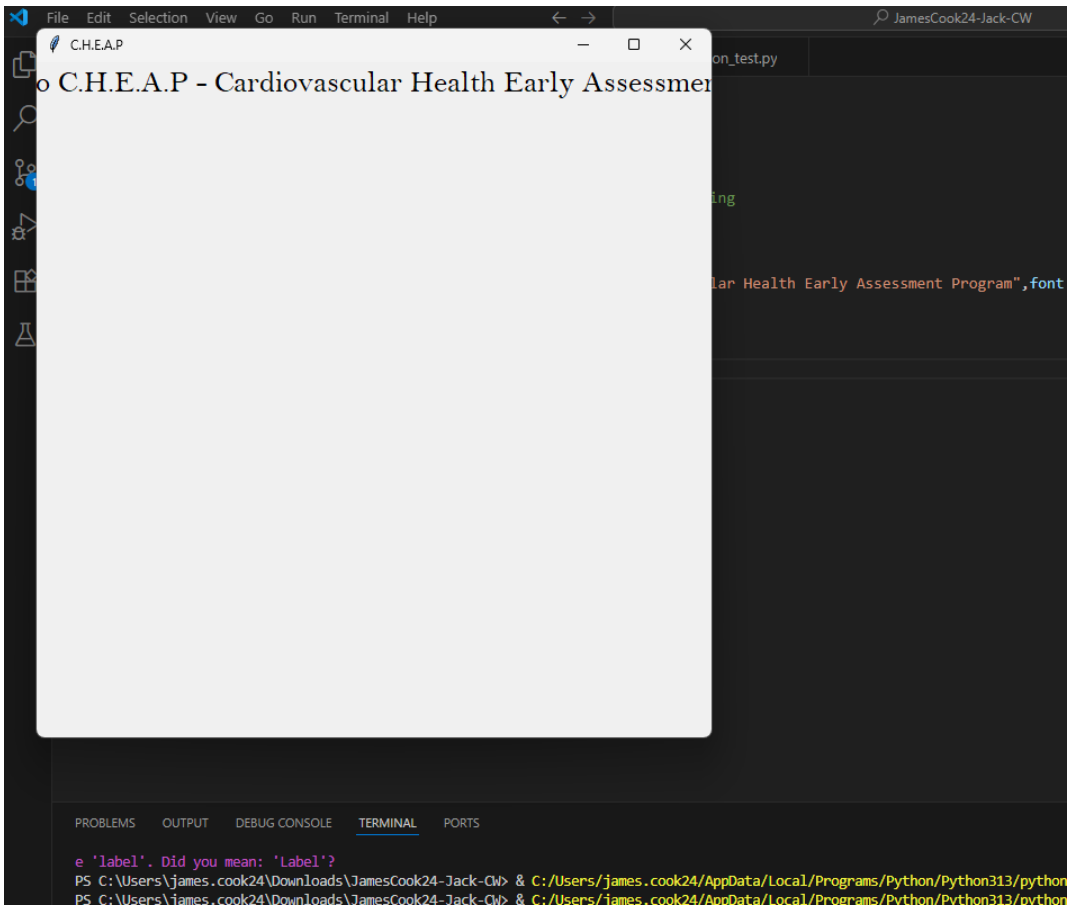
The menu now fits a lot better and uses a standard sizing. For a GUI menu. From this point forward I plan to only use pseudocode and/or flowcharts for code that has logic in it, as any other code will be implementing features that are so basic that allocating time to flowchart/pseudocode design is unnecessary.

Now that I have a basic menu set up, I will start implementing the text boxes found in the main menu that contain vital information surrounding the purpose of the program. After this, I will implement the inputs and buttons that link to other screens. Adding the features that don't require inputs first allows me to familiarize myself with the tkinter module while also implementing the easier features before I tackle the harder functions.

```
root.title("C.H.E.A.P")#creates a title for the menu

mainTitle = tk.Label(root, text="Welcome to C.H.E.A.P - Cardiovascular Health Early Assessment Program",font = ("Bell MT", 20))
mainTitle.pack() # adds it to the menu


root.mainloop()# starts a loop
```

The mainTitle variable stores the tkinter label that contains the Title at the top of every page for the program. The font is 'Bell MT', the font I have repeatedly used throughout my program. Lastly, the font size is 20, as it needs to be large. The settings menu will allow the user to change the font if they find the text difficult to discern. The pack() functions adds the label to the screen.

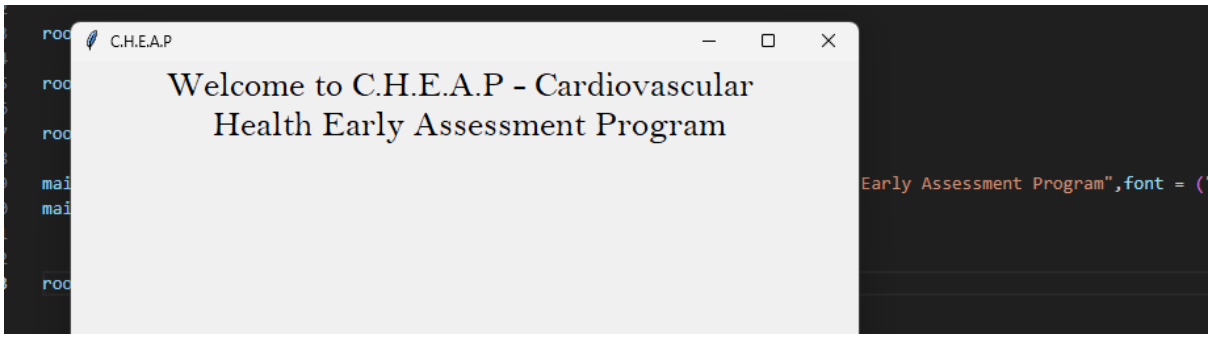
Test:



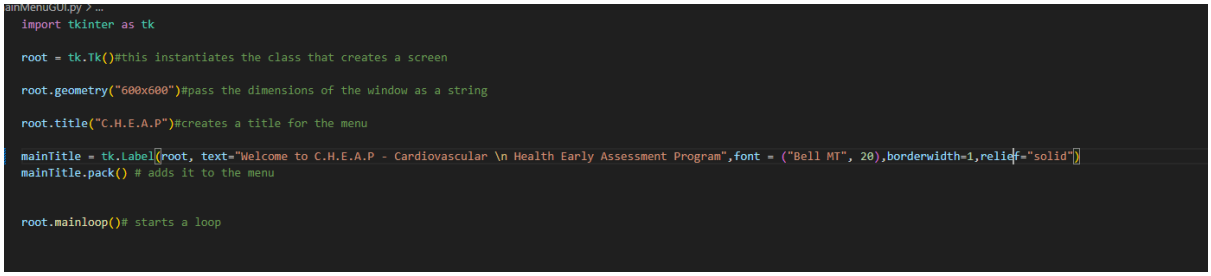
The text appeared on screen with the correct font and sizing, however since the text is too long it followed on outside the screen. I plan to fix this by adding '\n' to the text, which should create a new line, thus putting the rest of the text below the previous text in the GUI menu.

```
mainTitle = tk.Label(root, text="Welcome to C.H.E.A.P - Cardiovascular \n Health Early Assessment Program",font = ("Bell MT", 20))
mainTitle.pack() # adds it to the menu
```

Test:

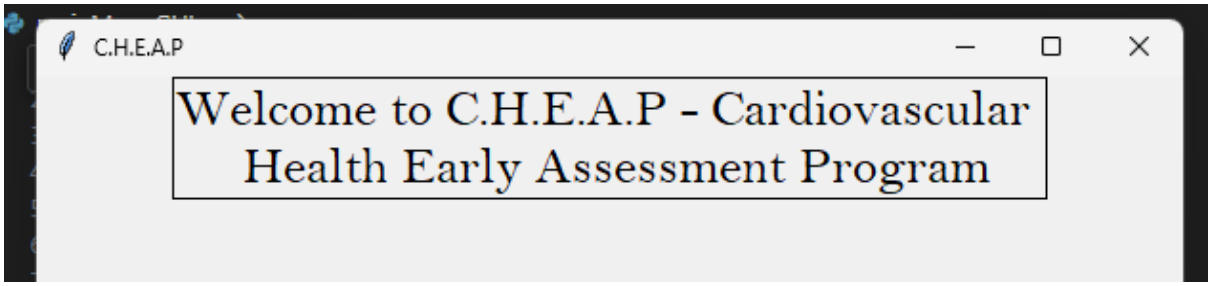


The text now displays below properly. I want to underline C.H.E.A.P to highlight it and have it stand out by adding a border around this headline. To do this I can change the relief of the label[28] during its instantiation. Additionally, I am unable to make the acronym 'C.H.E.A.P' underlined as applying the underlined function to the label underlines the whole text. Underlining just the acronym for the program would draw the user's eyes away from the expanded name, which may serve more purpose. Furthermore, making this label bold is unnecessary, as it's already positioned at the top of the page with a border around it, signifying that it is a title. I may bolden this label in the future, but adding essential features first is more important.



```
import tkinter as tk

root = tk.Tk()#this instantiates the class that creates a screen

root.geometry("600x600")#pass the dimensions of the window as a string

root.title("C.H.E.A.P")#creates a title for the menu

mainTitle = tk.Label(root, text="Welcome to C.H.E.A.P - Cardiovascular \n Health Early Assessment Program",font = ("Bell MT", 20),borderwidth=1,relief="solid")
mainTitle.pack() # adds it to the menu


root.mainloop()# starts a loop
```

Test:



As you can see, the title is now boxed in with a border, which I believe helps indicate the titles importance.

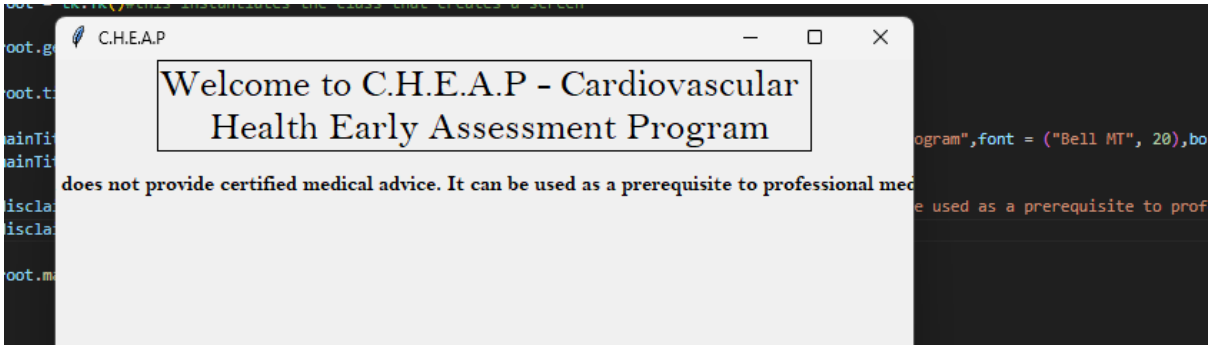
[28] (GeeksforGeeks, 2025)

Now I will add the small disclaimer underneath the title, and I will bolden it[29] to make it pop more. It's important that the users are aware of the intention behind the program, so that it isn't used with mis intent.

```
disclaimer = tk.Label(root, text = "This program does not provide certified medical advice. It can be used as a prerequisite to professional medical diagnosis",font = ("Bell MT",11,"bold"))
disclaimer.pack(pady=10)
```

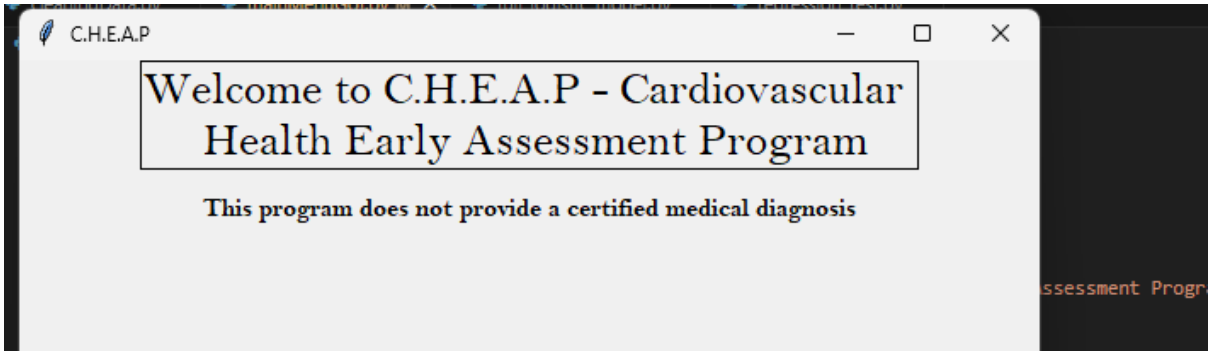
As you can see, within the font parameter to the Label object I can make the text bold. I want to make sure this label isn't clumped next to the title, so I need to pad it as I pack it.

Test:



Although the text is bold and padded as I intended, the text goes off the screen the same way the title did. I could add a line break to have the text go onto a new line, but I want this disclaimer to only occupy one line in the GUI. Due to this, I can either decrease the font size or change the text, and since decreasing the font size for this label could make text hard to distinguish, I will alter the text in the disclaimer. Since the main idea that I want to get across is that the program isn't certified and verified by medical professionals, I only need to include the sentence: 'This program does not provide a certified medical diagnosis'.

Test:



The text now fits the screen and conveys the message I wish to convey.

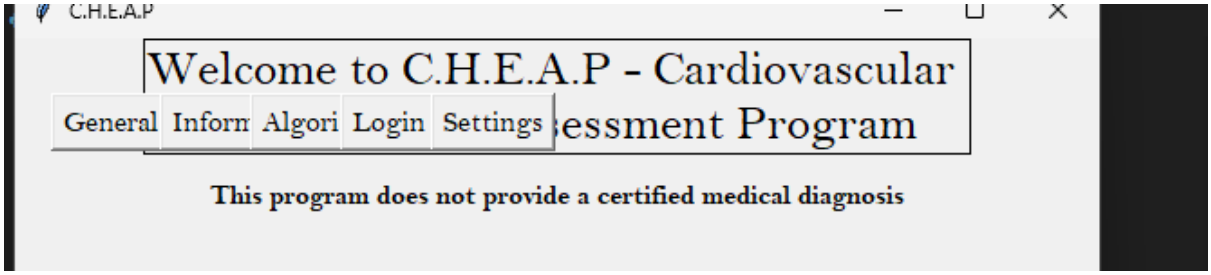
[29] (GeeksForGeeks, 2025)

I can now add the buttons that allow you to navigate around the different tabs in the program. However, I won't add the functionality to the buttons until all pages are developed. This just means that I need to create button objects[30] now. Using the 'place' [31]function I am able to choose the x and y positions for each of the buttons, allowing me to place them alongside each other.

```
#creating the buttons for each page

generalButton = tk.Button(root,text = "General",font = ("Bell MT",12))
informationButton = tk.Button(root,text = "Information",font = ("Bell MT",12))
algorithmButon = tk.Button(root,text = "Algorithm",font = ("Bell MT",12))
loginButton = tk.Button(root,text = "Login",font = ("Bell MT",12))
settingsButton = tk.Button(root,text = "Settings",font = ("Bell MT",12))

#placing the buttons
generalButton.place(x = 20,y = 30)
informationButton.place(x = 80,y = 30)
algorithmButon.place(x = 130,y = 30)
loginButton.place(x = 180,y = 30)
settingsButton.place(x = 230,y = 30)
root.mainloop()# starts a loop
```

At first, I need to experiment with the placement of the buttons as I have no other reference points.

Test:

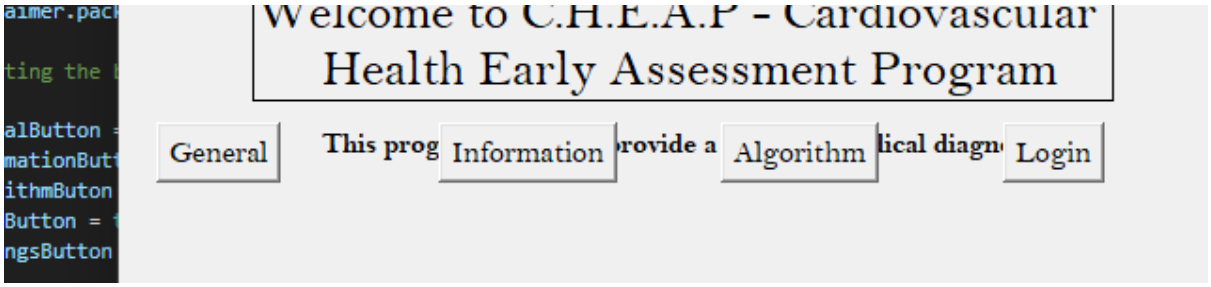


As you can see, the buttons are placed on top of one another, and the y position is too low. Since I incremented the x positions by 50 (first increment was 60 by mistake), I need to increase this increment to potentially 150, to ensure that there is a substantial gap between the buttons. The y positions of the button need to be at least doubled, so to be safe I will change it to 75.

[30] (NeuralNine, 2025)
[31] (GeeksforGeeks, 2025)

```
#placing the buttons
generalButton.place(x = 20,y = 75)
informationButton.place(x = 170,y = 75)
algorithmButon.place(x = 320,y = 75)
loginButton.place(x = 470,y = 75)
settingsButton.place(x = 620,y = 75)
root.mainloop()# starts a loop
```

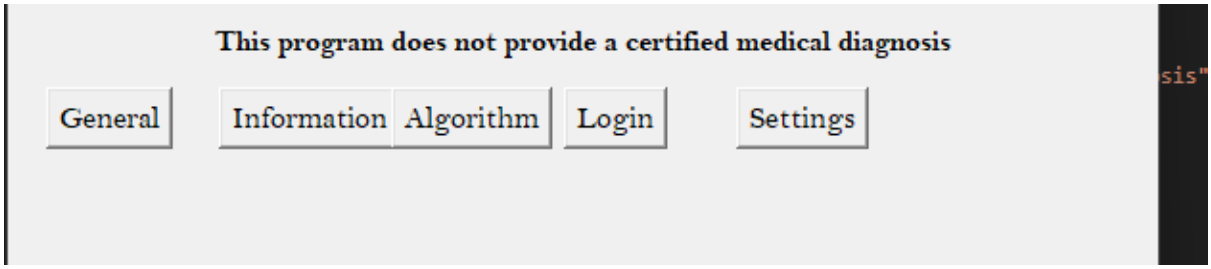
Test:



The y position needs to be increased further, possibly to 110. Furthermore, the gap between the buttons is too large now, so I will decrease the increments to 90.

```
#placing the buttons
generalButton.place(x = 20,y = 110)
informationButton.place(x = 110,y = 110)
algorithmButon.place(x = 200,y = 110)
loginButton.place(x = 290,y = 110)
settingsButton.place(x = 380,y = 110)
root.mainloop()# starts a loop
```

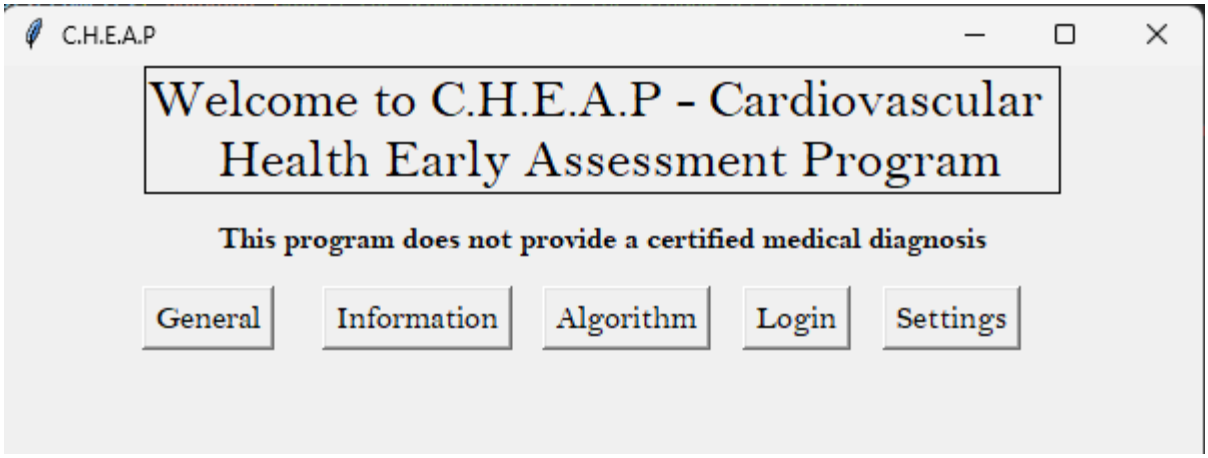
Test:



The y position for the buttons seems to be good now, but incrementing the x position of the buttons by a constant won't work since the text within the buttons are different lengths. Now I need to alter the positions myself using intuition.

```
#placing the buttons
generalButton.place(x = 70,y = 110)
informationButton.place(x = 160,y = 110)
algorithmButon.place(x = 270,y = 110)
loginButton.place(x = 370,y = 110)
settingsButton.place(x = 440,y = 110)
root.mainloop()# starts a loop
```

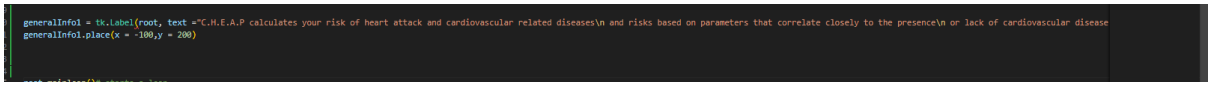
After altering the x positions based on the length of the words within the buttons, it now displays like this:



The buttons are now centred in the GUI menu and the distance between each button remains somewhat constant, although it is not perfect.

Next, I will add the text box that will display the essential information that helps you understand how to operate the program. Initially, I will use the same text from the GUI design to create a box that is the right size. After the sizing is correct, ill adjust the text to match the parameters I'm taking in. For simplicity reasons, I will decompose the information found on the right of the main menu into 3 different boxes of text.

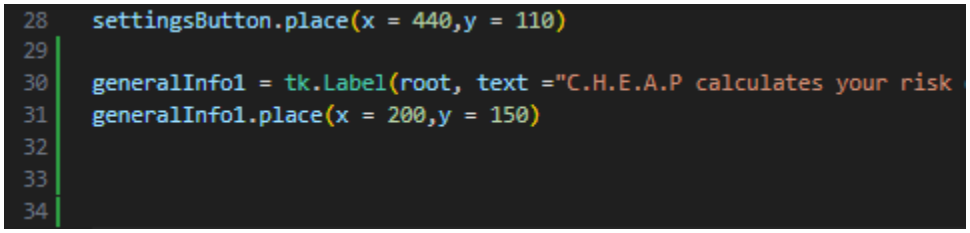
Here is the first one:

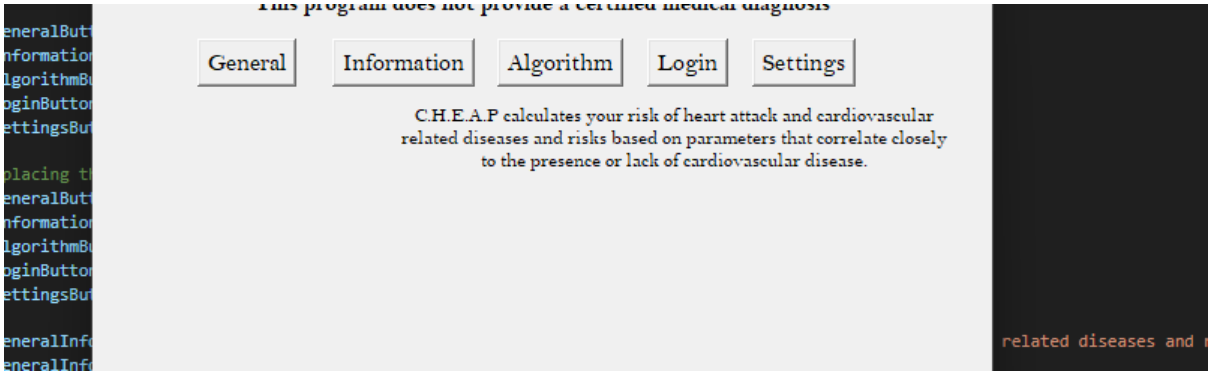


The x and y position of this box is random, but I can adjust it once I see where it appears.

<u>Test:</u>

As you can see, the text box was placed quite far to the left of the menu, but the y position isn't too bad. I can decrease the y slightly and increase the x co-ordinate.





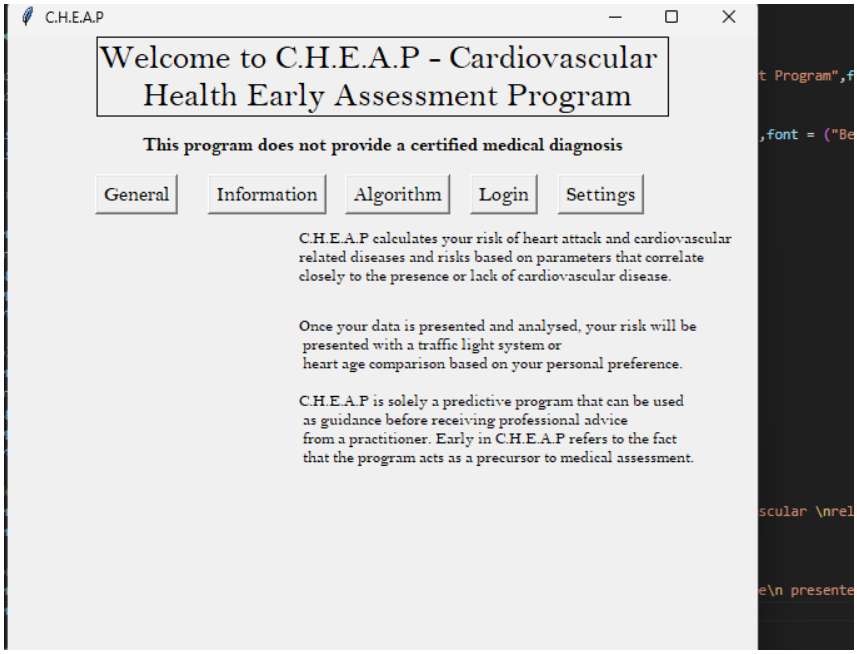
As you can see after adjusting the text box's x co-ordinate it is placed a lot better. I will use a similar x co-ordinate for the other two boxes. One concern is that I am forcing myself to create new lines so that the lines in the box all start with a similar x co-ordinate, but if I use the 'justify' (geeksforgeeks, 2025) parameter for the tkinter label I can, orientate the text differently (start from the left not centre). The anchor parameter is also used to position the label to the east of the Gui, creating more structure.

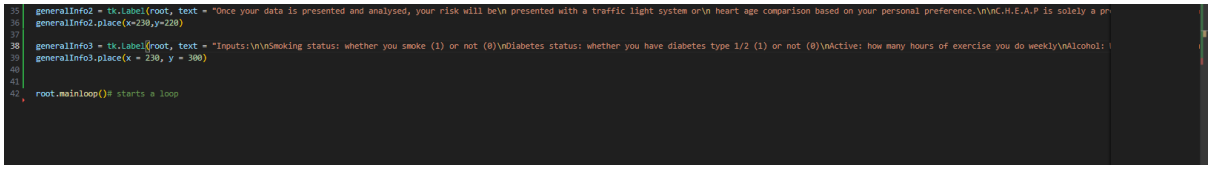
```
scular disease.", font = ("Bell MT",10),justify="left",anchor="e")


is solely a predictive program that can be used\n as guidance before receiving professional ad
```

```
generalInfo1.place(x = 230,y = 150)#places this label

#provides more information about the intentions of the program
generalInfo2 = tk.Label(root, text = "Once your data is presented and analysed, your risk will be\n presented with a traffic light system or\n
generalInfo2.place(x=230,y=220)

root.mainloop()# starts a loop
```

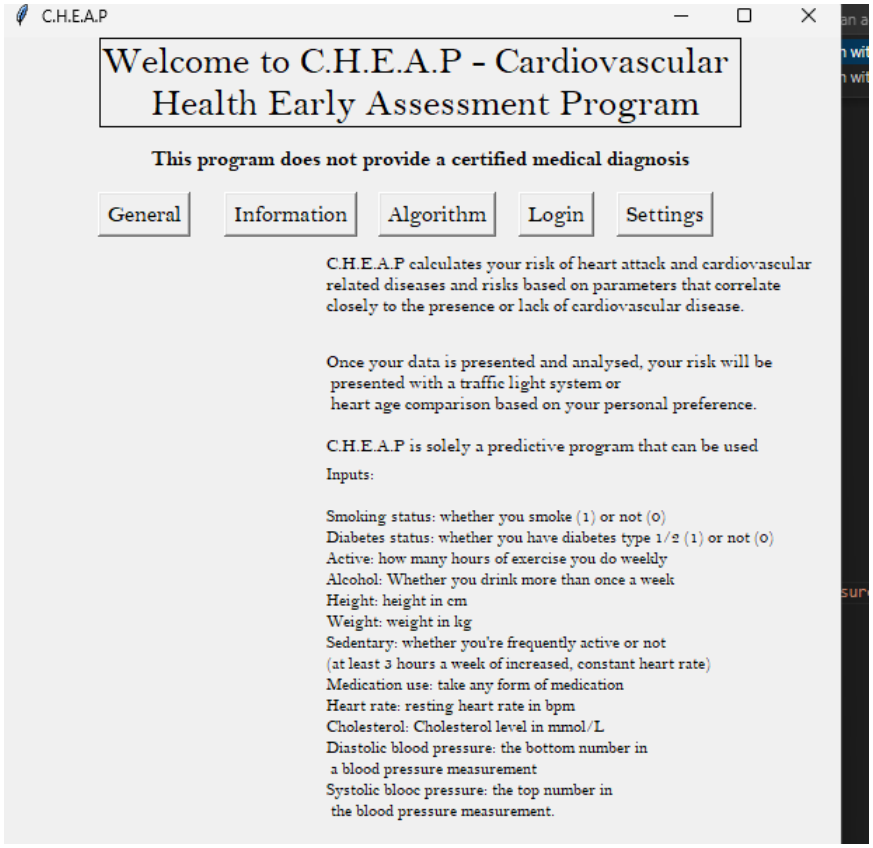
Test:

As you can see below, the information is aligned and positioned the right-hand side of the GUI menu, leaving space for the inputs on the left. There is one more text label required, which will display what the inputs mean.


C.H.E.A.P
Welcome to C.H.E.A.P - Cardiovascular Health Early Assessment Program
This program does not provide a certified medical diagnosis
General
Information
Algorithm
Login
Settings
C.H.E.A.P calculates your risk of heart attack and cardiovascular related diseases and risks based on parameters that correlate closely to the presence or lack of cardiovascular disease.
Once your data is presented and analysed, your risk will be presented with a traffic light system or heart age comparison based on your personal preference.
C.H.E.A.P is solely a predictive program that can be used as guidance before receiving professional advice from a practitioner. Early in C.H.E.A.P refers to the fact that the program acts as a precursor to medical assessment.


generalInfo2 = tk.Label(root, text = "Once your data is presented and analysed, your risk will be\n presented with a traffic light system or\n heart age comparison based on your personal preference.\n\nC.H.E.A.P is solely a pr
generalInfo2.place(x=230,y=220)
generalInfo3 = tk.Label(root, text = "Inputs:\n\nSmoking status: whether you smoke (1) or not (0)\nDiabetes status: whether you have diabetes type 1/2 (1) or not (0)\nActive: how many hours of exercise you do weekly\nAlcohol:
generalInfo3.place(x = 230, y = 300)
root.mainloop()# starts a loop

Test:

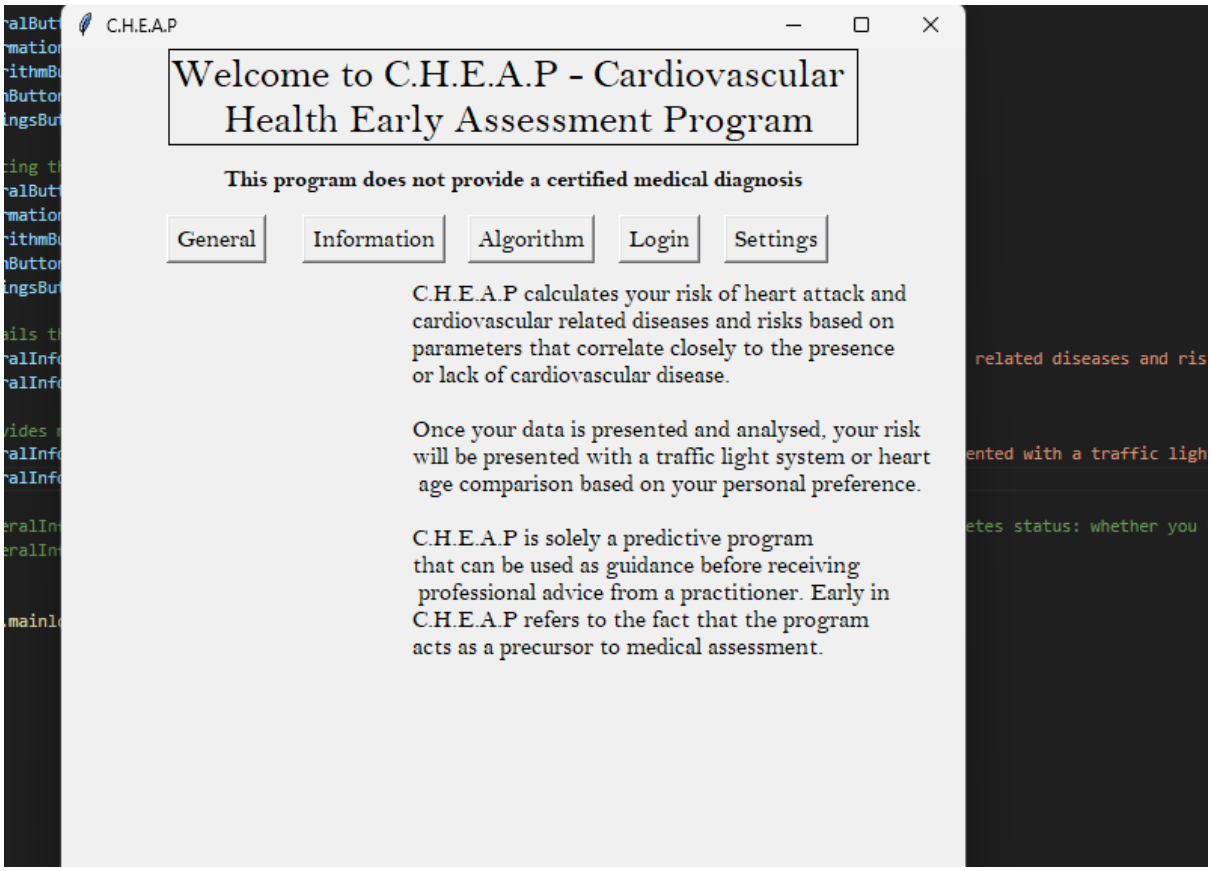

C.H.E.A.P
Welcome to C.H.E.A.P - Cardiovascular Health Early Assessment Program
This program does not provide a certified medical diagnosis
General
Information
Algorithm
Login
Settings
C.H.E.A.P calculates your risk of heart attack and cardiovascular related diseases and risks based on parameters that correlate closely to the presence or lack of cardiovascular disease.
Once your data is presented and analysed, your risk will be presented with a traffic light system or heart age comparison based on your personal preference.
C.H.E.A.P is solely a predictive program that can be used
Inputs:
Smoking status: whether you smoke (1) or not (0)
Diabetes status: whether you have diabetes type 1/2 (1) or not (0)
Active: how many hours of exercise you do weekly
Alcohol: Whether you drink more than once a week
Height: height in cm
Weight: weight in kg
Sedentary: whether you're frequently active or not
(at least 3 hours a week of increased, constant heart rate)
Medication use: take any form of medication
Heart rate: resting heart rate in bpm
Cholesterol: Cholesterol level in mmol/L
Diastolic blood pressure: the bottom number in a blood pressure measurement
Systolic blooc pressure: the top number in the blood pressure measurement.

As you can see, the inputs are being displayed, but they are quite clunked together. Additionally, with the current font size of the 'inputs' Label it overlaps a little with the previous label. Although I need to increase my font size to make it accessible and legible for all users, Increasing the font size would cause more overlap. Additionally, if I move the labels left to create more space for text, I will reduce the space allowed for the constant input menus found on the left of the GUI. To fix this, I will use a button that will change the text to the input information instead of the general information.

This also allows me to increate the font size to 12, making the text more readable:

```
at the program \nacts as a precursor to medical assessment.",font = ("Bell MT",12),justify="left",anchor="e")

 hours a week of increased, constant heart rate)\nMedication use: take any form of medication\nHeart rate: resting heart
```

Test:



It's a lot more readable now, making it more accessible for all my users.

Now I need to add the button/buttons that will swap the text in this Label. The pseudocode below highlights the way I plan to algorithmically implement this.

```
button1 = tk.button(...)
button2 = tk.button(...)

if(button.pressed = True):
    generalInfo1.remove()
    generalInfo2.remove()
    generalInfo3.pack()

if(button2.pressed = True):
  generalInfo1.pack()
  generalInfo2.pack()
  generalInfo3.remove()
```

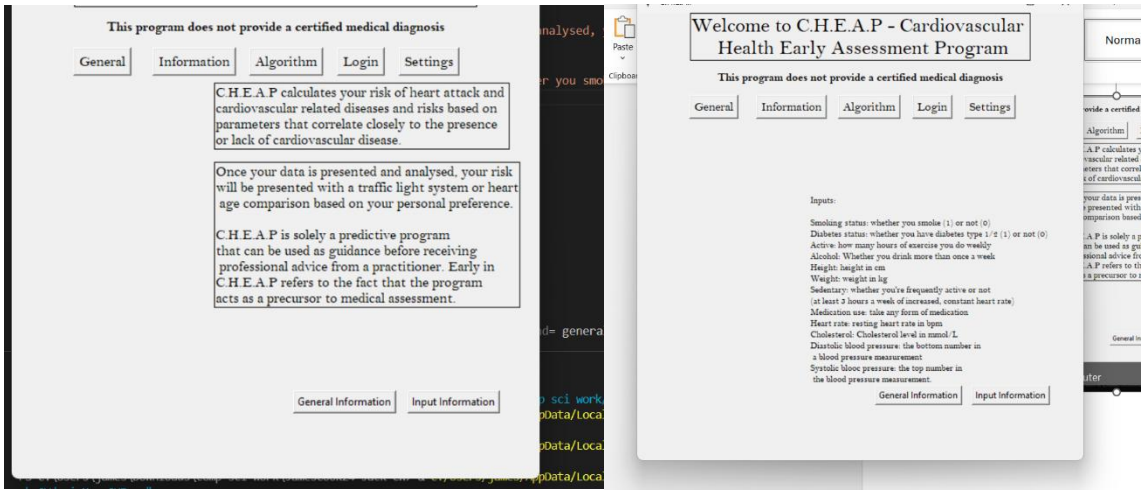
The logic behind this sequence of code is that when 1 of two buttons is pressed, the labels that display general information will be removed from the GUI using some form of tkinter function that manipulates labels, and the other button would do the opposite. I can then add a small sentence above the buttons detailing their functionality.

The code below uses widgets and the 'place and 'place_forget'[32] functions built into tkinter. This allows me to create buttons that use the functions 'generalInfoButtonCommand' and 'inputsInfoButtonCommand' as their personal tkinter button commands. These buttons then utlitise the previously mentioned tkinter functions to essentially 'hide' the label/s so that the other information can be displayed. This resolves the legibility issue where font size would have to be very small to achieve no overlap.

```
Run and Debug (Ctrl+Shift+D)
ace(x=230,y=240)
generalInfo3 = tk.Label(root, text = "Inputs:\n\nSmoking status: whether you smoke (1) or not (0)\nDiabetes status: whether you have diab

#creates the general information display button
def generalInfoButtonCommand():
    generalInfo3.place_forget()
    generalInfo1.place(x = 230,y = 150)#places this label
    generalInfo2.place(x=230,y=240)


def inputsInfoButtonCommand():
    generalInfo1.place_forget()
    generalInfo2.place_forget()
    generalInfo3.place(x = 230,y = 240)

generalInfoButton = tk.Button(root,text = "General Information", command= generalInfoButtonCommand)
generalInfoButton.place(x=320,y=500)

inputsInfoButton = tk.Button(root, text = "Input Information",command= inputsInfoButtonCommand)
inputsInfoButton.place(x =450 ,y =500 )
root.mainloop()# starts a loop
```

[32] (geeksforgeeks, 2025)

Test:



Before and after pressing the buttons, the labels do appear and appear respectively to the correct button. However, the placement of the input is off, and I can make the font and orientation of the inputs better. Additionally, I can add a border to the inputs.

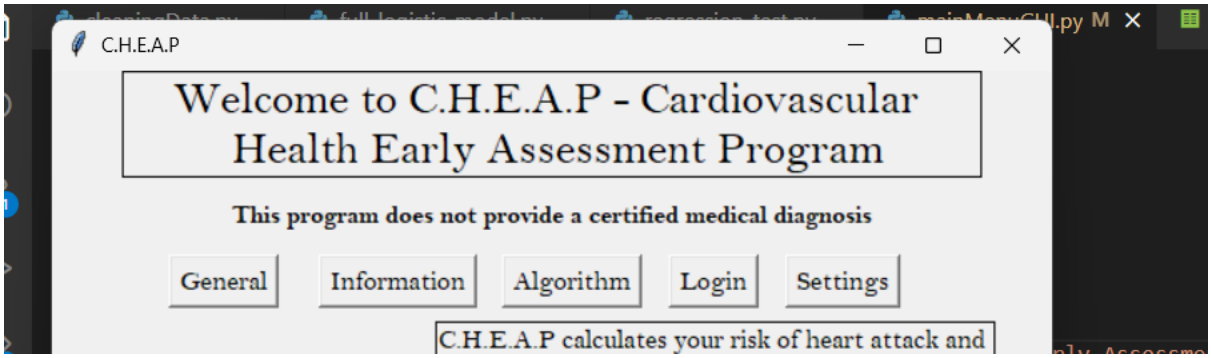
```
def inputsInfoButtonCommand():
    generalInfo1.place_forget()
    generalInfo2.place_forget()
    generalInfo3.place(x = 230,y = 160)

generalInfoButton = tk.Button(root,text = "General Informat
```

```
urement.",font = ("Bell MT",11),justify = "left",anchor = "e",borderwidth=1,relief = "solid")
```

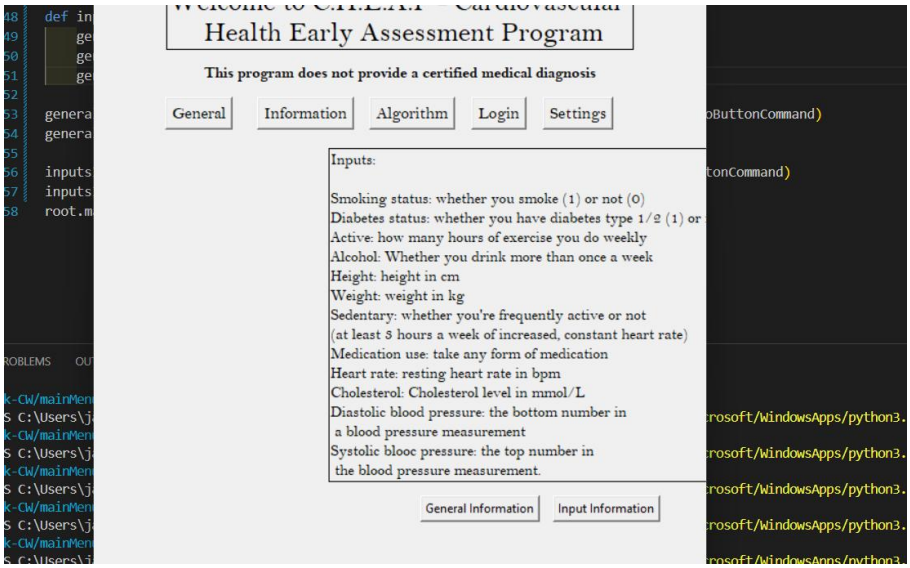
While playing around with borders for the input label, I found the 'width' parameter, which sets the width of the border and fixes it[33]. I can use this on the title to make it wider and fill out more of the top of the page.

[33] (GeeksforGeeks, 2025)

Test:



The width of the border is now greater, making it look larger and more important.

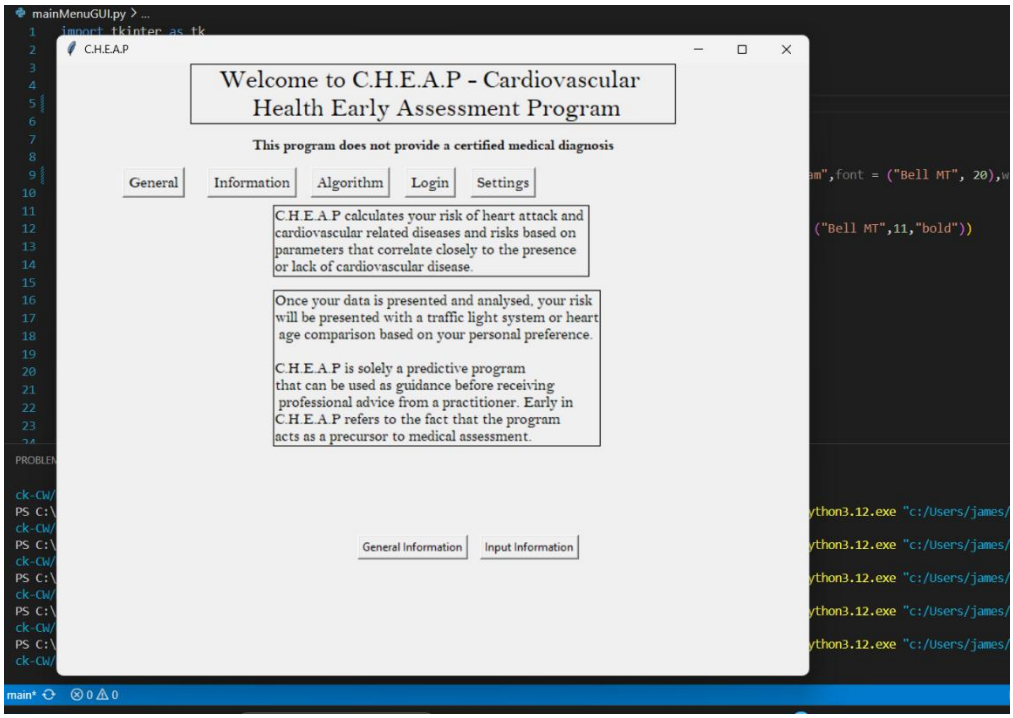


As you can see the input text is slightly off and the border falls outside of the GUI. There are two ways that I can fix this, and I plan to do both. Firstly, there is no harm in making the GUI dimensions larger; it leads to more visible and therefore more accessible text boxes, and it gives me more space to add words for extra information. Additionally, if adding other features leads to me making the GUI dimensions larger, altering more stuff to then fit the new dimensions will take more effort than altering the features now.

Secondly, I need to change the text displayed for the input information section as it isn't accurate. It could also lead to more packable text.

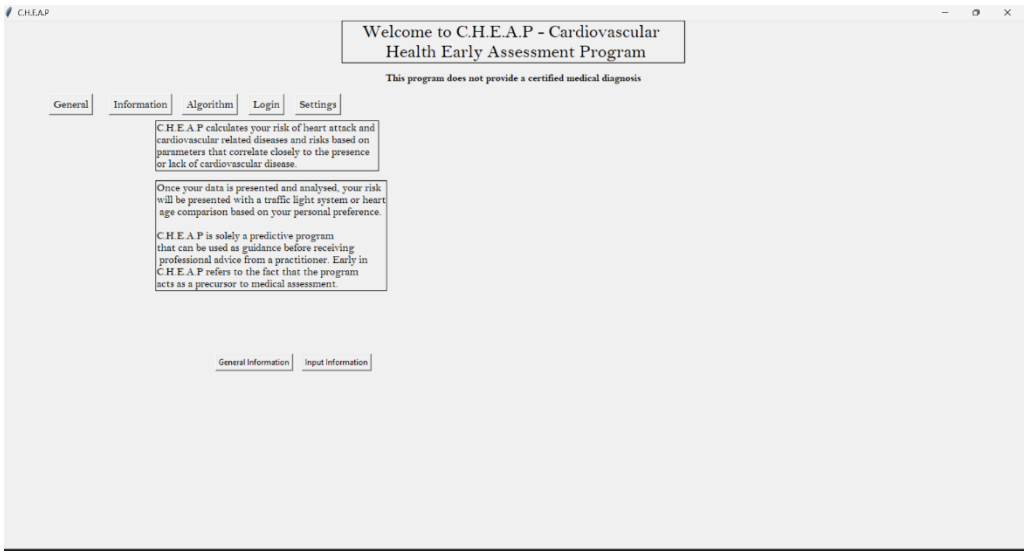
```
3   root = tk.Tk()#this instantiates the class that creates a screen
4
5   root.geometry("800x650")#pass the dimensions of the window as a string
6
7   root.title("C.H.E.A.P")#creates a title for the menu
8
9   mainTitle = tk.Label(root, text="Welcome to C.H.E.A.P - Cardiovascular \n H
10  mainTitle.pack() # adds it to the menu
```

After changing the dimensions to 800x650 (my screen is sized to 1591 x 695, any larger could lead to accessibility issues), this is the output:



The buttons and text boxes aren't positioned correctly, but this was expected. Doing this also made me realise that if users make the window full screen the boxes and labels will likely be offset. To fix this, I need to change the way labels and buttons are placed on the GUI. Instead of using place, I need to use pack, which places objects based on padding and centring parameters.

Test:



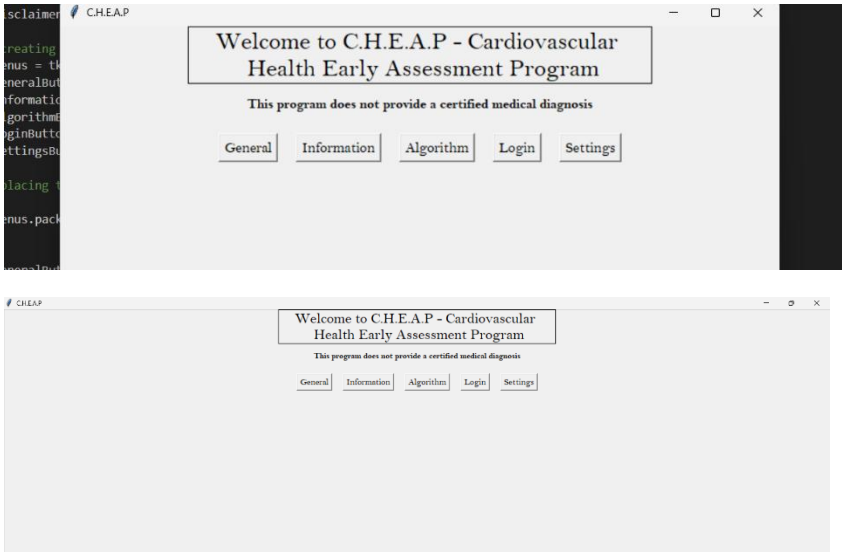
Since the title and the disclaimer are placed using packing, you can see that their relative position remained the same, but the buttons and text boxes use the same x and y position before full screen was activated, which is an issue.

Therefore, I will firstly alter how labels and buttons are placed within the GUI, then I will change the input text to correlate to the dataset I chose.

```
14
15  #creating the buttons for each page
16  menus = tk.Frame(root)
17  generalButton = tk.Button(menus,text = "General",font = ("Bell MT",12))
18  informationButton = tk.Button(menus,text = "Information",font = ("Bell MT",12))
19  algorithmButon = tk.Button(menus,text = "Algorithm",font = ("Bell MT",12))
20  loginButton = tk.Button(menus,text = "Login",font = ("Bell MT",12))
21  settingsButton = tk.Button(menus,text = "Settings",font = ("Bell MT",12))
22
23  #placing the buttons
24
25  menus.pack(pady=10)
26
27
28  generalButton.pack(side="left",expand=True,padx= 10)
29  informationButton.pack(side="left",expand=True,padx= 10)
30  algorithmButon.pack(side="left",expand=True,padx= 10)
31  loginButton.pack(side="left",expand=True,padx= 10)
32  settingsButton.pack(side="left",expand=True,padx= 10)
33
34  #details the functionality of the program
```

Firstly, I am altering how the different buttons at the top are displayed. I have created a frame for the buttons so that I can pack the frame and pack the buttons within the frame. Since they all have similar purposes and their placement are dependent on one another, placing them in the same frame allows me to manipulate their placement simultaneously, which will save time and make the code more manageable.

Test:

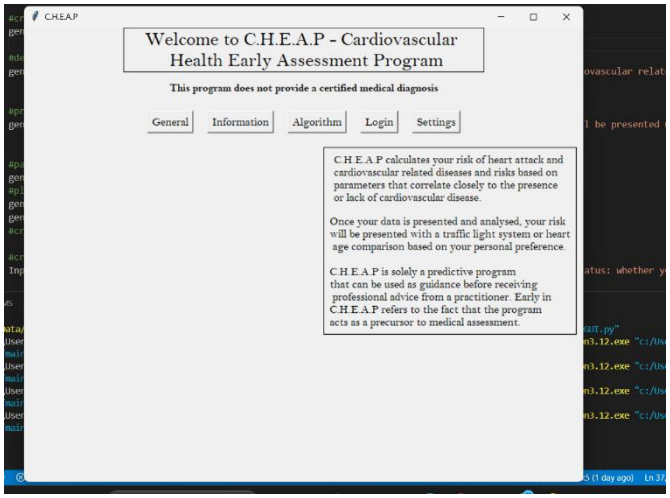


As you can see, whether I'm in full screen or not, the buttons are placed next to each other with some horizontal padding. I don't know if I want the buttons and text boxes to display centrally or as left as possible. However, this is a non-essential consideration right now and altering this later wouldn't be difficult. The main priority is creating the main menu with adequate spacing and scalable packing.

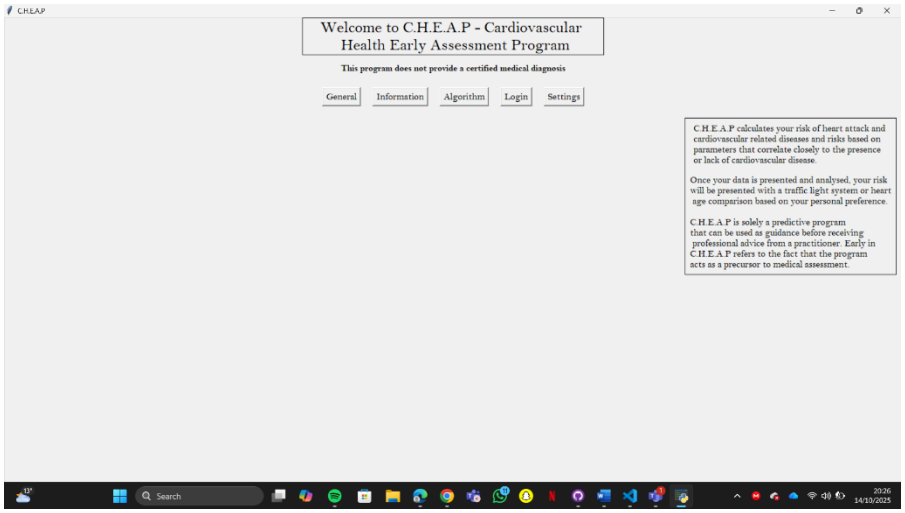
```
38  #details the functionality of the program
39  generalInfo1 = tk.Label(generalInfoFrame, text ="C.H.E.A.P calculates your risk of heart attack and \ncardiovas
40
41
42  #provides more information about the intentions of the program
43  generalInfo2 = tk.Label(generalInfoFrame, text = "Once your data is presented and analysed, your risk \nwill be
44
45
46  #pack the frame and general info
47  generalInfoFrame.pack(pady = 10,padx = 10,side="right",anchor="ne")
48  #placed the frame to the right and north east of the GUI.
49  generalInfo1.pack(side="top",padx = 5,pady = 5)
50  generalInfo2.pack(side="bottom",padx = 5,pady = 5)
51  #creates a horizontal and vertical pad with width and height 5 and packs it.
52
```

As you can see, I can create a general information frame, where I can put both labels that contain general information about the program into. Frames are a much better way to pack and manage labels, it allows for easier manipulation and more manageable code. This is essential and a massive positive, as when the code gets more complicated as more menus are created, having easy to manipulate objects is important.

Test:



As you can see the text box still as towards the right of the screen when you enter full screen. Although this doesn't seem proportional, altering this can be done later, the most important thing is that no overlap happens for different screen sizes, and this prevents that.

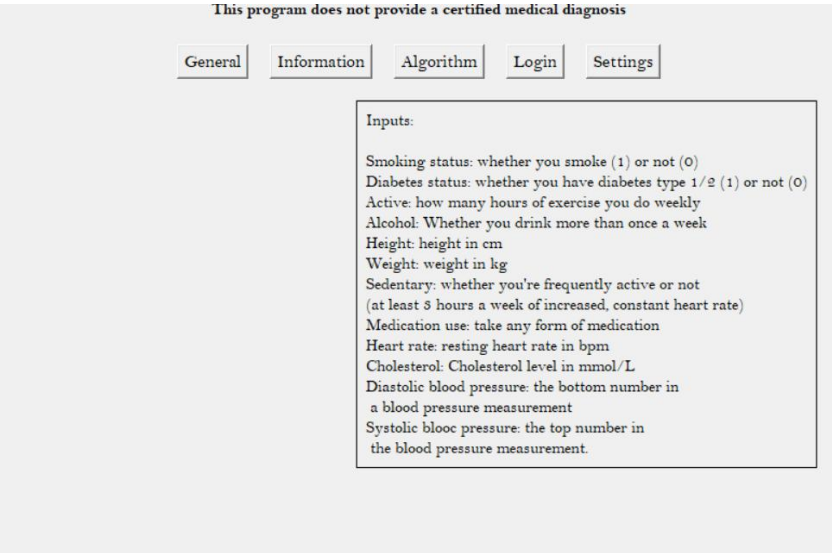


Now I want to test where the input information should be packed so that I know where it should be placed once the buttons are reimplanted using frames.

```
52
53    #create the frame and label for input info
54    inputInfoFrame = tk.Frame(root,borderwidth=1,relief="solid")
55    inputInfo = tk.Label(inputInfoFrame, text = "Inputs:\n\nSmoking status: whether you smoke (
56
57    inputInfoFrame.pack(pady = 10,padx = 10,side="right",anchor="ne")
58    inputInfo.pack(side = "left",pady = 5,padx = 5)
59
60
61    #creates the general information display button
```

Here I can do a similar thing I did with the general info frames, as they both wont be present at the same time.

Test:



It is outputted the same way the general info frame is, which is perfect.

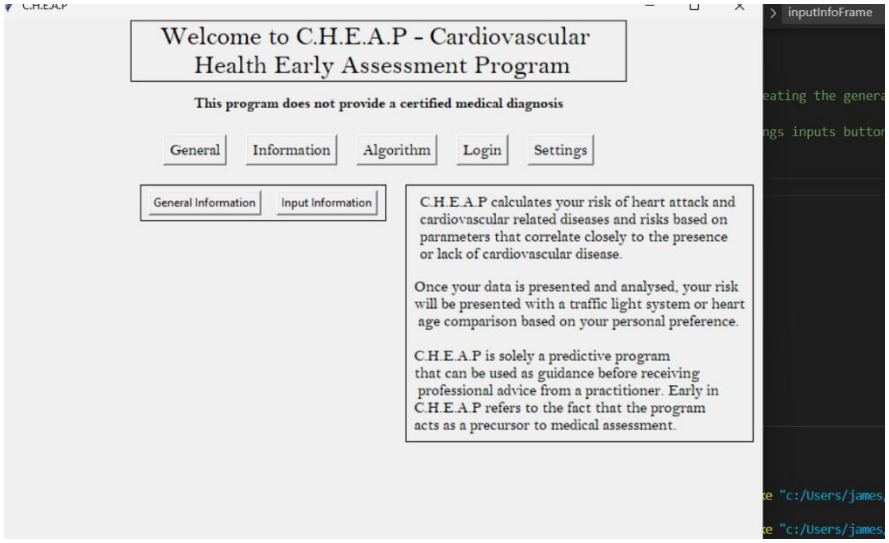
So now I can change the functions that act as commands for the buttons as such:

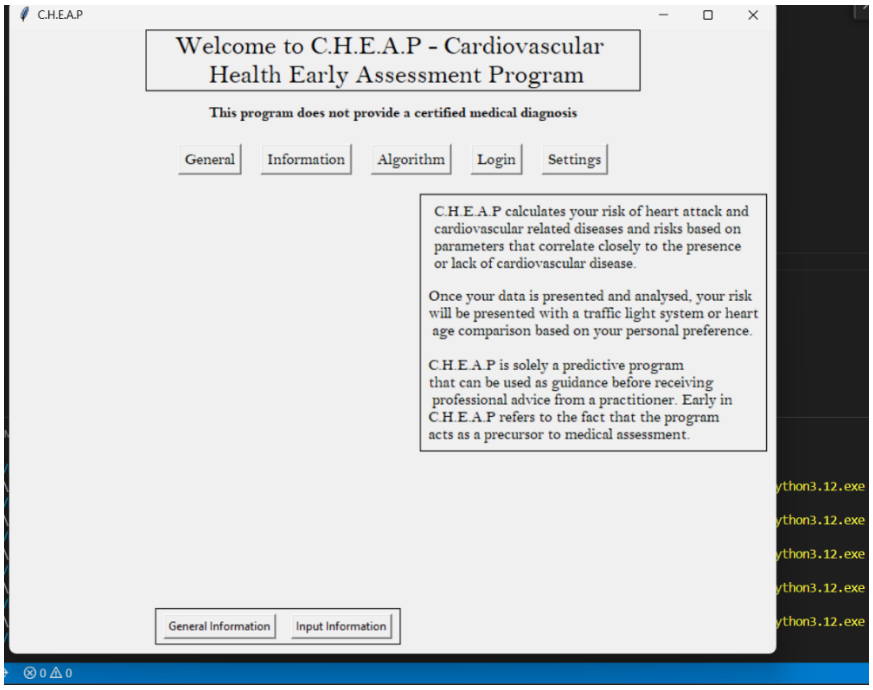
```
#creates the general information display button
def generalInfoButtonCommand():
    inputInfoFrame.pack_forget()
    #forget the inputInfoFrame
    generalInfoFrame.pack(pady = 10,padx = 10,side="right",anchor="ne")
    #pack the generalInfo Frame into the GUI


def inputsInfoButtonCommand():
    generalInfoFrame.pack_forget()
    inputInfoFrame.pack(pady = 10,padx = 10,side="right",anchor="ne")

```

This should have the same functionality as the previous implementation, except its not forgetting and packing frames.

```
#creating and packing the information buttons
InfoButtonsFrame = tk.Frame(root,borderwidth=1,relief="solid")#creating a frame for the info and general buttons

generalInfoButton = tk.Button(InfoButtonsFrame,text = "General Information", command= generalInfoButtonCommand)#creating the general button

inputsInfoButton = tk.Button(InfoButtonsFrame, text = "Input Information",command= inputsInfoButtonCommand)#creatings inputs button


InfoButtonsFrame.pack(side = "right",anchor = "ne",pady = 10,padx = 10)
generalInfoButton.pack(side="left",padx = 8,pady = 5)#packing the general button
inputsInfoButton.pack(side = "right",padx = 8,pady = 5)#packing info button

root.mainloop()# starts a loop
```

Now I have a frame for the buttons, but when I run the program, the buttons are either placed to the left as seen below or at the very bottom of the GUI. To fix this, I can create a frame that contains all the boxes to the right of the GUI. I could also create two frames, one that contains the general info and one that contains the input info so that I can swap the display of the frames as the button commands.






```
# Create a container for the right side (for info + buttons)
rightSideContainer = tk.Frame(root)
rightSideContainer.pack(side="right", anchor="ne", padx=10, pady=10)

#create a frame for the general info
generalInfoFrame = tk.Frame(rightSideContainer,borderwidth=1,relief="solid")
```

Now if I have a container for the right side of the main menu, I can add the general and input info frames to this frame, creating a frame hierarchy.

```
#create the frame and label for input info
inputInfoFrame = tk.Frame(rightSideContainer,borderwidth=1,relief="solid")
inputInfo = tk.Label(inputInfoFrame, text = "Inputs:\n\nSmoking status: whether
```

As I've implemented this into the program, I've realised that the commands for the buttons need to be altered. At the moment they both remove the inputInfo/generalInfo frames then pack the latter, but this would place the buttons at the top of this rightsidecontainer.

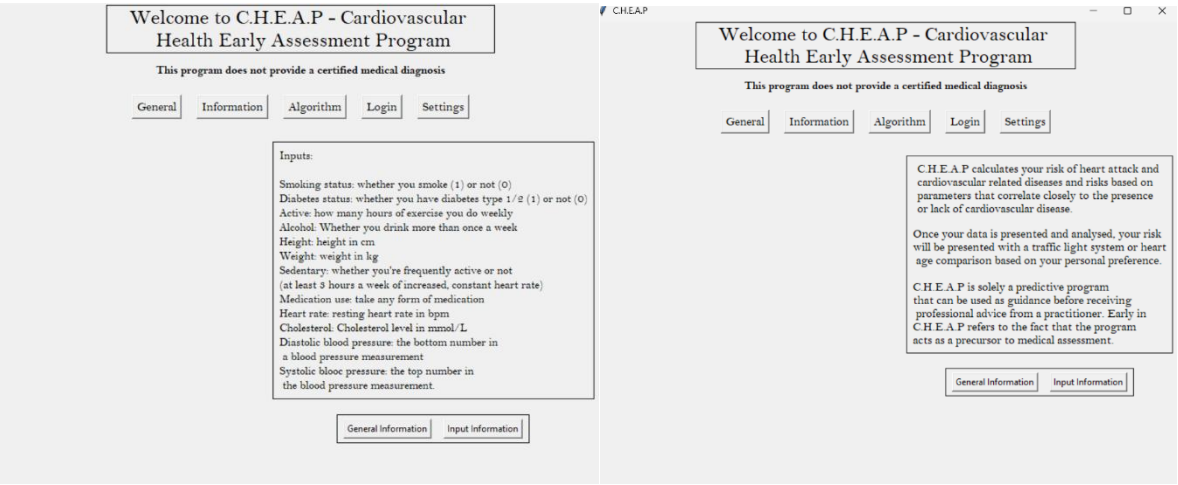
```
#creates the general information display button
def generalInfoButtonCommand():
    inputInfoFrame.pack_forget()
    generalInfoFrame.pack(side="top", pady=10)


def inputsInfoButtonCommand():
    generalInfoFrame.pack_forget()
    inputInfoFrame.pack(side="top", pady=10)
```

To combat this, I will forget the button frame within both functions at the start of the sequencing, then at the end of each function I will pack the buttons again, meaning that it will always be at the bottom of the rightsidecontainer.

```
#creates the general information display button
def generalInfoButtonCommand():
    #remove the buttons frame
    InfoButtonsFrame.pack_forget()

    inputInfoFrame.pack_forget()
    generalInfoFrame.pack(side="top", pady=10)#add the general and inputs info frames


    InfoButtonsFrame.pack(side = "top",pady = 10,padx = 10)#re add the buttons frame

def inputsInfoButtonCommand():
    #remove the buttons frame
    InfoButtonsFrame.pack_forget()

    generalInfoFrame.pack_forget()
    inputInfoFrame.pack(side="top", pady=10)


    InfoButtonsFrame.pack(side = "top",pady = 10,padx = 10)#re add the buttons frame
```

Now the buttons should work accordingly, where when pressed they alter the text being displayed while remaining at the bottom of the rightsidecontainer.

Test:



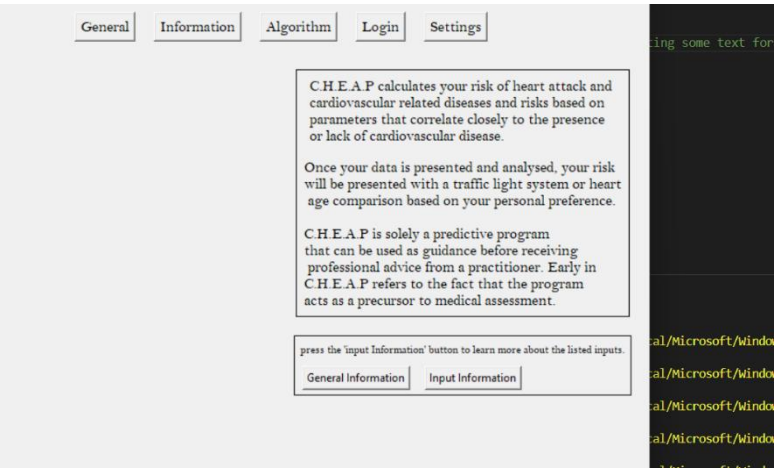
After pressing the buttons, the text boxes change, and the buttons remain at the bottom of the container.

Lastly, I will add some text to the buttons frame so that they know what they do.

```
94
95  infoButtonstext = tk.Label(InfoButtonsFrame,text = "press the 'input Information' button to learn more about the listed inputs.",font = ("Bell MT
96
97  generalInfoButton = tk.Button(InfoButtonsFrame,text = "General Information", command= generalInfoButtonCommand)#creating the general button
98
99  inputsInfoButton = tk.Button(InfoButtonsFrame, text = "Input Information",command= inputsInfoButtonCommand)#creatings inputs button
00
01
02  InfoButtonsFrame.pack(side = "top",pady = 10,padx = 10)
03  infoButtonstext.pack(side = "top",pady = 3,padx = 3)
04  generalInfoButton.pack(side="left",padx = 8,pady = 5)#packing the general button
05  inputsInfoButton.pack(side = "left",padx = 8,pady = 5)#packing info button
06
```

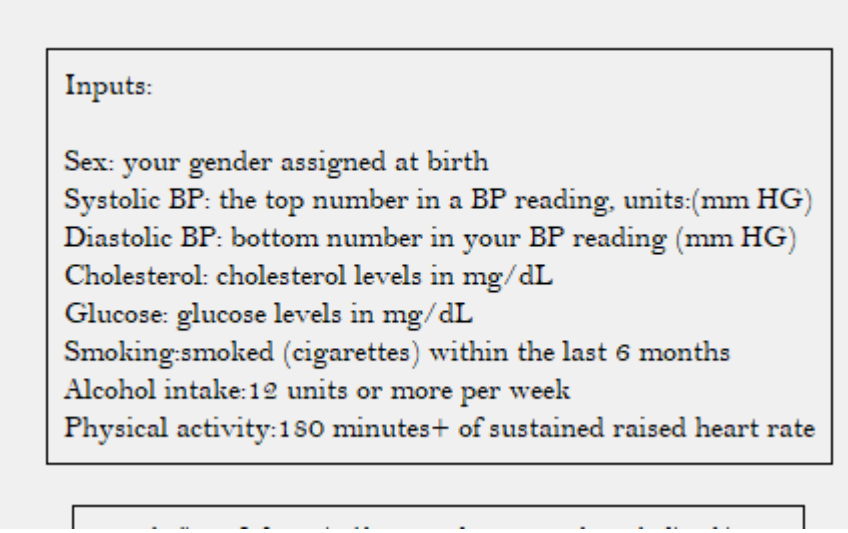
As you can see, I have created a label for the info buttons frame so there is some context provided on the purpose of the button. I chose to not detail what the general Information button does as it's very simple to understand if you press each button (it's already being displayed upon opening the GUI).

Test:



The buttons and text are both displays properly below the general information, allowing you to swap between the two easily.

I will now update the text for the input information, as it was sourced from the outdated GUI design that doesn't account for the dataset I chose. The following parameters don't need a description, as they're extremely intuitive if you provide the units required: age (years), height (cm), weight (kg). I will detail what I mean by sex/gender, as it's important that they provide their sex not the gender they identify by. I'll explain what systolic and diastolic blood pressure means as these terms can be confusing. Additionally, since 30% of my stakeholders from my survey never check their blood pressure, there's even more of a reason to explain what it is. Smoking, alcohol and physical activity status all need to be explained as they can be interpreted arbitrarily since they could be qualitative questions. Lastly, cholesterol and glucose levels require explaining as they're even less measured than blood pressure (57%).



Here is the new input information screen. I chose to use the word Sex as it entails more of a biological assessment rather than the word gender. Additionally, the smoking, alcohol intake and physical activity information was sourced from medically approved articles[34], making them useful classifiers.

Now that the input information is sufficient, I can work on making the tab buttons work. Now I will just implement the settings and general buttons, as these are the two menus I am working on during this stage of development.

I plan creating multiple rightsidecontainers, where each one contains the different menus. This means that the commands can delete all the other rightsidecontainers then pack their own one.

[34] (WHO, 2025)

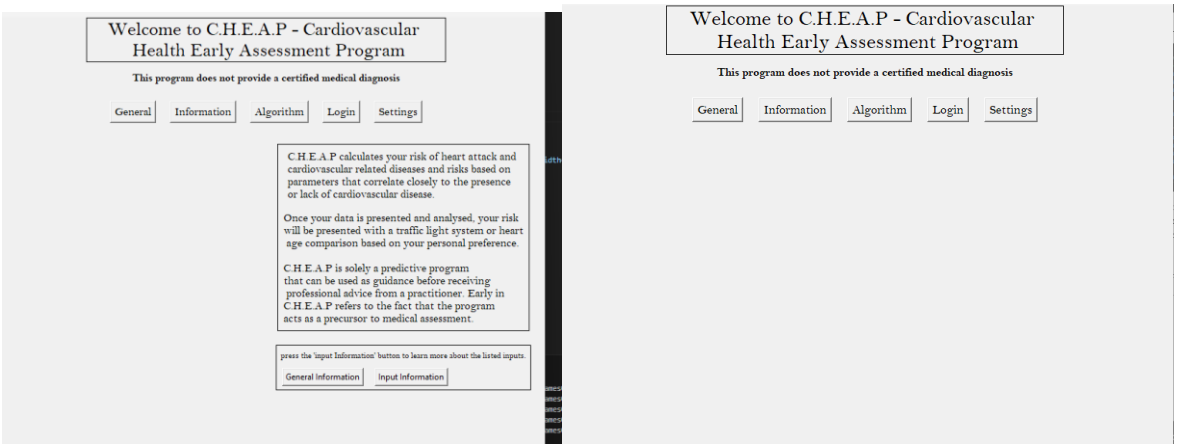
```
# Create a container for the right side (for info + buttons)
rightSideContainer1 = tk.Frame(root)#general menu
rightSideContainer1.pack(side="right", anchor="ne", padx=10, pady=10)

rightSideContainer2 = tk.Frame(root)# information menu


rightSideContainer3 = tk.Frame(root)#algorithm menu


rightSideContainer4 = tk.Frame(root)#login menu


rightSideContainer5 = tk.Frame(root)#settings menu
```

As you can see, each of the menus have their own container for their respective information.

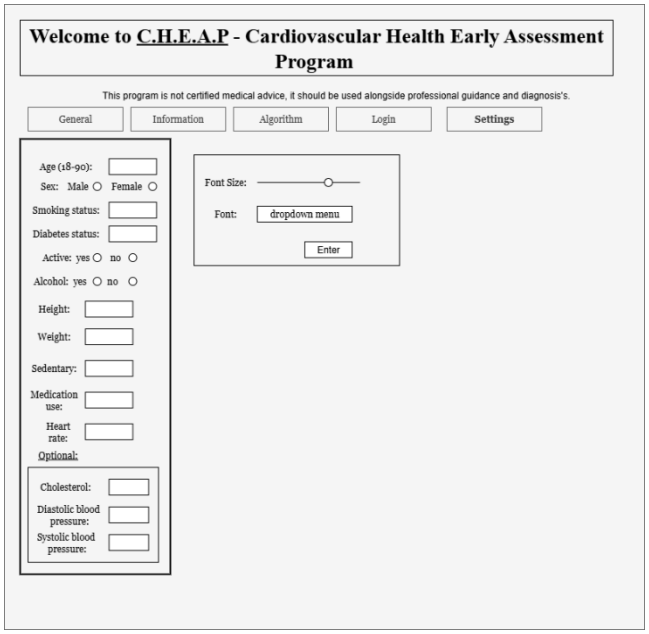
```
26
27  def generalButtonCommand():
28      #forgets all the menus except the general menu
29      rightSideContainer2.pack_forget()
30      rightSideContainer3.pack_forget()
31      rightSideContainer4.pack_forget()
32      rightSideContainer5.pack_forget()
33
34      rightSideContainer1.pack(side="right", anchor="ne", padx=10, pady=10)#packs the general menu
35
36  def settingsButtonCommand():
37      rightSideContainer1.pack_forget()
38      rightSideContainer2.pack_forget()
39      rightSideContainer3.pack_forget()
40      rightSideContainer4.pack_forget()
41
42      rightSideContainer5.pack(side="right", anchor="ne", padx=10, pady=10)#packs the settings menu
43  # FUNCTIONS ------------------------------------------------------------------
```

because of this, the commands for the general and settings buttons can be created as seen above. This should forget all the other menus (even if only 1 is currently packed) and then pack their respective menu.

Test:



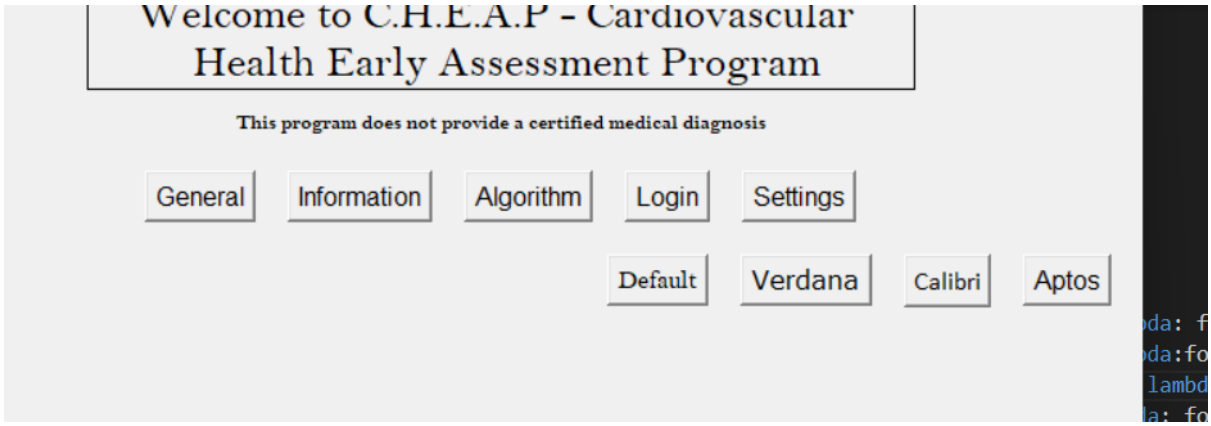
After pressing the settings button, the general menu information disappeared and the general button had it reappear again, as I intended.



Now that those buttons work, I can work on the settings menu. Firstly, I will implement different font choices using buttons. I can create multiple buttons that each represent different font choices.

```
#different font choices
DefaultFont = tk.Button(rightSideContainer5,text = "Default",font = ("Bell MT",12))
VerdanaFont = tk.Button(rightSideContainer5,text = "Verdana",font = ("Verdana",12))
ProductSansFont = tk.Button(rightSideContainer5,text = "Calibri",font = ("Calibri",12))
MontserratFont = tk.Button(rightSideContainer5,text = "Aptos",font = ("Aptos",12))


DefaultFont.pack(side="left",expand=True,padx= 10)
VerdanaFont.pack(side="left",expand=True,padx= 10)
ProductSansFont.pack(side="left",expand=True,padx= 10)
MontserratFont.pack(side="left",expand=True,padx= 10)
```

I chose the following fonts: Bell MT, Verdana, Calibri and Aptos. The 3 other fonts were chosen due to their renowned accessibility, and the variation should lead to the removal of reading issues like dyslexia. This is it implemented in the settings menu below.



Now I need to implement the button commands so that upon clicking them they change the font throughout the program. To do this I need to make a variable called Font which

will store the current font being used. Currently, all the labels have ‘Bell MT’ as their designated font, but If I assign the variable ‘Font’ with this string during initialisation, it will have the same effect.

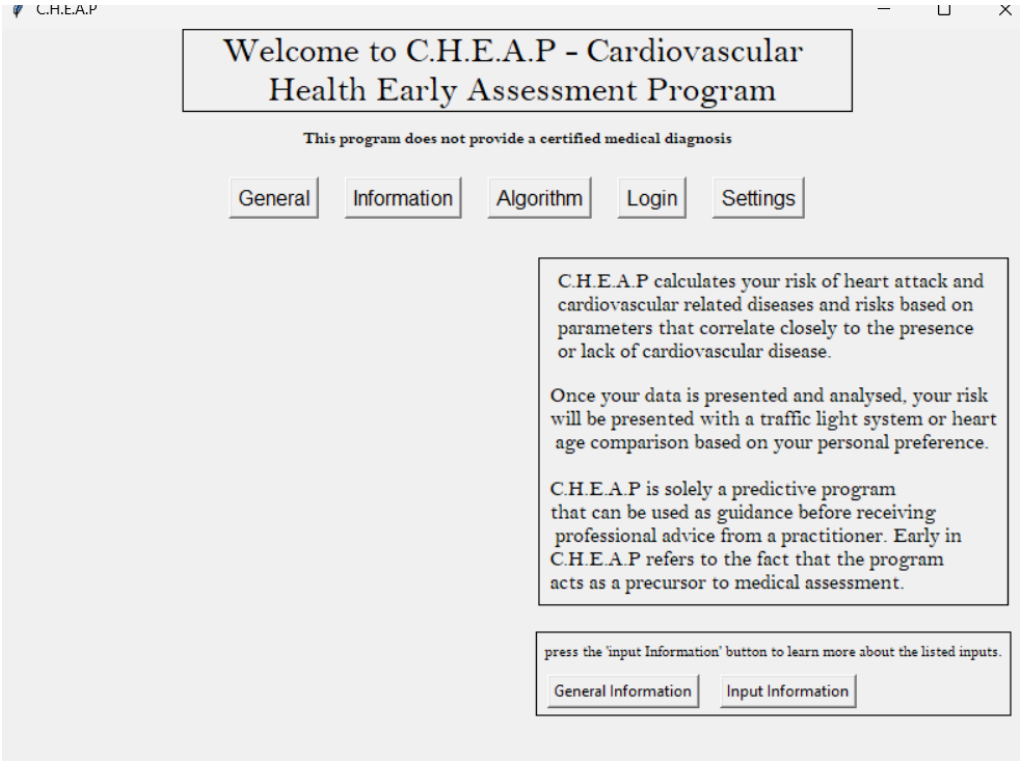
```
52
53  titleFont = tkFont.Font(family="Bell MT", size=20)
54  bodyFont = tkFont.Font(family="Bell MT", size=12)
55  smallFont = tkFont.Font(family="Bell MT", size=9)
56  smallFontBold = tkFont.Font(family="Bell MT", size=9,weight = "bold")
57
58  mainTitle = tk.Label(root, text="Welcome to C.H.E.A.P - Cardiovascular \n
```

I’ve firstly established a set of baseline fonts that I will use throughout the program. Creating fonts outside their labels and then calling these fonts within the instantiation of the labels allows to me continuously access and change the font family or size without dealing with the labels directly. I’ve then changed how labels access fonts, they know just take the defined fonts above as a parameter. The following as examples:

```
lack of cardiovascular disease.", font = (bodyFont),justify="left",anchor="e")
```

```
mainTitle = tk.Label(root, text="Welcome to C.H.E.A.P - Cardiovascular \n Health Early Assessment Program",font = (titleFont),width= 32,borderwidth=1,relief="solid")
mainTitle.pack() # adds it to the menu
```

This then led to this in the GUI:



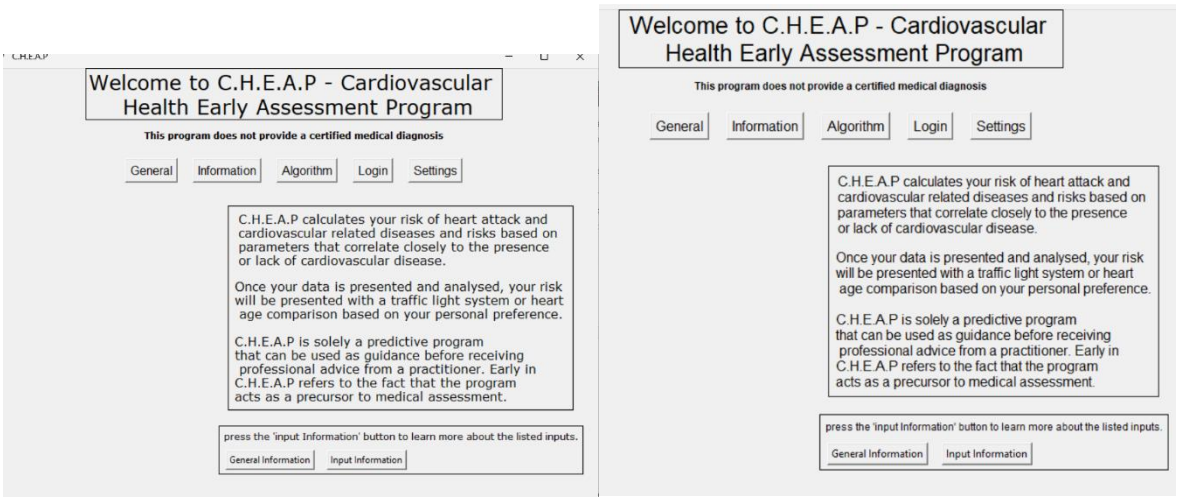
The GUI appears the same as before.

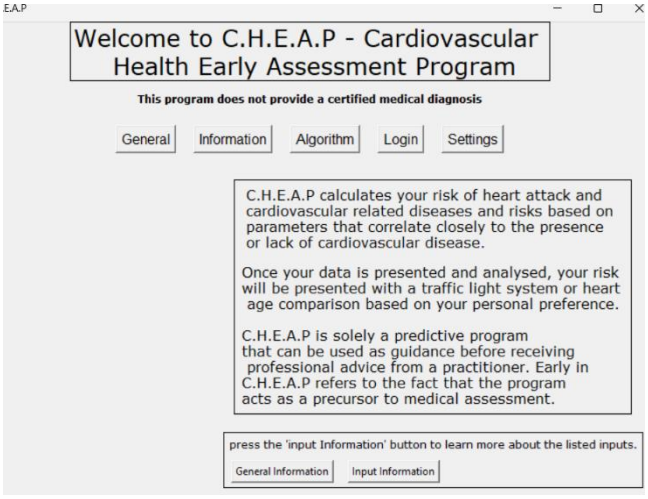
```
#settings menu font change function
def fontChange(newFont):
    smallFont.config(family=newFont)
    smallFontBold.config(family=newFont)
    titleFont.config(family = newFont)
    bodyFont.config(family = newFont)


#different font choices
DefaultFont = tk.Button(rightSideContainer5,text = "Default",font = ("Bell MT",12),command = lambda: fontChange("Bell MT"))
VerdanaFont = tk.Button(rightSideContainer5,text = "Verdana",font = ("Verdana",12),command = lambda:fontChange("Verdana"))
ProductSansFont = tk.Button(rightSideContainer5,text = "Calibri",font = ("Calibri",12),command = lambda: fontChange("Calibri"))
MontserratFont = tk.Button(rightSideContainer5,text = "Aptos",font = ("Aptos",12),command = lambda: fontChange("Aptos"))
```

Now I can implement the commands for the fonts I've chosen. They've each been instantiated as buttons on the settings page seen above. Each of the buttons use the same function in their command, which is a lambda command to prevent instant function calling; if it wasn't a lambda command the button would instantly be called as the GUI is generated.

Each of the fonts take their own respective font as a parameter on the command, so when the button is pressed the config is changed (as you can see in the fontChange function) for each of the fonts used throughout the GUI.

Test:





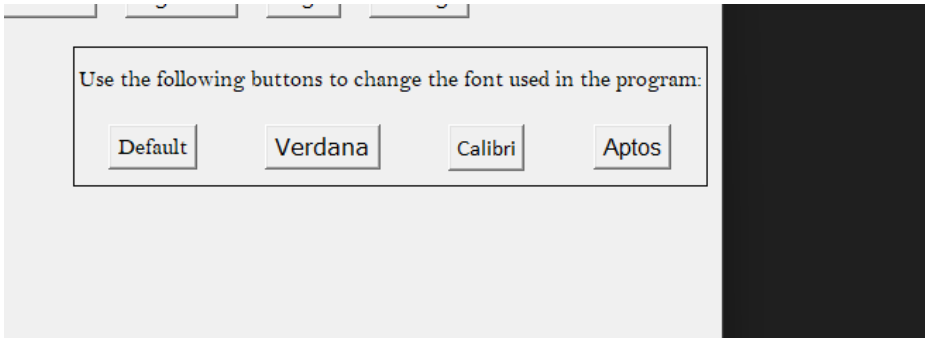
As you can see, after pressing the buttons and returning to the general page, the fonts have changed correctly, showing that the font family accessibility settings is working.

```
    smallFont.config(family=newFont)
    smallFontBold.config(family=newFont)
    titleFont.config(family = newFont)
    bodyFont.config(family = newFont)

FontChangeText = tk.Label(rightSideContainer5, text = "Use the following buttons to change the font used in the program:", font = bodyFont)
#different font choices
DefaultFont = tk.Button(rightSideContainer5,text = "Default",font = ("Bell MT",12),command = lambda: fontChange("Bell MT"))
VerdanaFont = tk.Button(rightSideContainer5,text = "Verdana",font = ("Verdana",12),command = lambda:fontChange("Verdana"))
ProductSansFont = tk.Button(rightSideContainer5,text = "Calibri",font = ("Calibri",12),command = lambda: fontChange("Calibri"))
MontserratFont = tk.Button(rightSideContainer5,text = "Aptos",font = ("Aptos",12),command = lambda: fontChange("Aptos"))

FontChangeText.pack(side = "top",expand = True,pady = 10)
DefaultFont.pack(side="left",expand=True,padx= 10, pady = 10)
VerdanaFont.pack(side="left",expand=True,padx= 10, pady = 10)
ProductSansFont.pack(side="left",expand=True,padx= 10, pady = 10)
MontserratFont.pack(side="left",expand=True,padx= 10, pady = 10)
```

I've added some supporting text before the buttons, so their functionality is obvious to the user. I've also added a border to the rightsidecontainer5 as seen below:

```

rightSideContainer5 = tk.Frame(root,border= 1,relief= "solid")#settings menu
```

Test:



It works as I expected it to.

The implementation of dynamic font sizes requires a bit more logic so I will plan with pseudocode. I will also likely use a text box for the font size instead of a slider as sliders can be very fidgety to use and they are harder to implement in tkinter.

One issue with implementing font scaling is that different labels have different font sizes to begin with, so I can't just adjust all the fonts with the same font size. Instead, I plan to hold all the font sizes in an array. I will then assume that the font size the user inputs is referring to the text that is found on the general menu rightsidecontainer (they are very likely to not be referring to the title or the disclaimer). With this new font size from the reader, I will increase all the font sizes in the array until the rightsidecontainer font sizes match that of the user's input. This will then lead to all the fonts increasing together but not to the same number.

Additionally, my current test plan doesn't test the user input validation I will need in the font sizing section of the settings menu. Since this involves user input which can vary a lot, a test plan is needed to ensure that all cases are covered.

Test no.	Description + test data	Type of test	Expected outcome	Outcome
1.	Enter an integer within the bounds: 14	Valid	Changes all font sizes (scaled) to font size 14	
2.	Enter an integer outside range: 5	Erroneous	Doesn't accept the user's input	
3.	Enter integer outside range: 20	Erroneous	Doesn't accept the user's input	
4.	Enter a float: 10.5	Erroneous	Doesn't accept the user's input	
5.	Enter no data	Erroneous	Doesn't accept the user's input	
6.	Enter non-numerical character: "a"	Erroneous	Doesn't accept the user's input	
7.	Enter non-numerical character: "/"	Erroneous	Doesn't accept the user's input	
8.	Enter non-numerical and numerical character: "-7"	Erroneous	Doesn't accept the user's input	

```
fontSizes = [20,12,9,8]#represents the font sizes used throughout the program

userInput = INPUT("Enter desired font size of body text:")#get user input using some form of input taking in tkinter


if(fontSizes[1] > userInput):#need to reduce the font sizes
  difference = fontSizes[1] - userInput
  for fontSize in fontSizes:
    fontSize = fontSize - difference

else:
  difference = userInput - fontSizes[1]#need to increase font sizes
  for fontSize in fontSizes:
    fontSize = fontSize + difference#adjusts all the fonts by same scale factor

def fontSizeChange:#command that the font size change button or input will use
  smallFont.config(size = fontSizes[2])
  smallFontBold.config(size = fontSizes[3])
  titleFont.config(size = fontSizes[0])
  bodyFont.config(fontSizes[1])
```

Above is the pseudocode that would change the font sizes dynamically and to the same scale. If I request only a body text change from the user, I can scale every font size using the difference from the current size and the users input size.

I've now created frames for the font change buttons, and then the font size change input box. This allows me to place the font size change frame below the font change frame, so that they appear separately.

```
FontChangeText = tk.Label(rightSideContainer5, text = "Use the following buttons to change the font used in the program:", font = bodyFont)
#different font choices
FontChangeText.pack(side="top", expand=True, pady=10)

# create a frame to hold all font buttons horizontally
fontButtonsFrame = tk.Frame(rightSideContainer5)
fontButtonsFrame.pack(side="top", pady=10)

DefaultFont = tk.Button(fontButtonsFrame, text="Default", font=("Bell MT", 12), command=lambda: fontChange("Bell MT"))
VerdanaFont = tk.Button(fontButtonsFrame, text="Verdana", font=("Verdana", 12), command=lambda: fontChange("Verdana"))
ProductSansFont = tk.Button(fontButtonsFrame, text="Calibri", font=("Calibri", 12), command=lambda: fontChange("Calibri"))
MontserratFont = tk.Button(fontButtonsFrame, text="Aptos", font=("Aptos", 12), command=lambda: fontChange("Aptos"))

# pack all buttons side by side in the sub-frame
DefaultFont.pack(side="left", expand=True, padx=10)
VerdanaFont.pack(side="left", expand=True, padx=10)
ProductSansFont.pack(side="left", expand=True, padx=10)
MontserratFont.pack(side="left", expand=True, padx=10)

#new frame below for the font size label and input box
fontSizeFrame = tk.Frame(rightSideContainer5)
fontSizeFrame.pack( pady=10)

fontSizeTextLabel = tk.Label(fontSizeFrame, text="Enter new font size:", font=smallFont)
fontSizeTextLabel.pack(side="left", pady=5)

fontSizeText = tk.Text(fontSizeFrame, width=10, height=1)
fontSizeText.pack(side="left", pady=5)
```

Test:

As you can see, the font size box and text appeared below the fonts. However, I think the "enter new font size" is too small, so I will change the font that it uses to a larger font.

Test:

It now appears at a more legible size.

To get user inputs using text boxes, I should replace the Label 'fontSizeText' with an Entry object[35], which will take the user's input:

```
177    fontSizeEntry = tk.Entry(rightSideContainer5, width=10, font=bodyFont)
178    fontSizeEntry.pack(pady=5)
```

Now I can implement the pseudocode in python using tkinter.

I need to first actually take and store the user's input inside a variable of some kind, so I can use a bind function[36] which uses a certain key as a command to receive an input.

```
fontSizeEntry.bind("<Return>", lambda event: fontSizeChange())
```

This will use the 'enter' button key bind for applying the font size change. I can now implement the pseudocode I wrote earlier using tkinter libraries.

```
def fontSizeChange():
    #stores the title font, body font and small font sizes respectively
    #index 1 is comparison index
    userInput = int(fontSizeEntry.get().strip())#retrieve the user input from Entry object

    fontSizes = [titleFont.cget("size"),bodyFont.cget("size"),smallFont.cget("size"),smallFontBold.cget("size")]

    if(fontSizes[1] > userInput):
        #need to reduce font sizes
        difference = fontSizes[1] - userInput
        for i in range(len(fontSizes)):
            fontSizes[i] -= difference
    else:
        difference = userInput - fontSizes[1]
        for i in range(len(fontSizes)):
            fontSizes[i] += difference

    titleFont.config(size = fontSizes[0])
    bodyFont.config(size = fontSizes[1])
    smallFont.config(size = fontSizes[2])
    smallFontBold.config(size = fontSizes[3])
```

[35] (GeeksforGeeks, 2025)
[36] (GeeksforGeeks, 2025)

As you can see, I've retrieved the user's input using the .get() method, I've also stripped users input to remove any leading or following white spaces from the input; further validation will be done but for the case of implementation I wish to test the program with valid inputs initially.

I've then created the array of font sizes, and I must create it using the cget("size"), otherwise the array won't dynamically retrieve the current font sizes, it will just default to the numbers I place inside it. Additionally, I have changed the for loop to go through the numbers 0,1,2.... Until the end of the length of the fontSizes array. This means I can alter the data within the indexes directly. In the pseudocode, there was no direct altercations, it was creating new variables and not actually changing the necessary integers. Programming it this way leads to an actual impact as you make an input.

Test:

Recording: fontSizeTest.mp4

It accurately and dynamically updates the font sizes.

Currently you can enter non-numerical characters, floats and integers that are way too small or big. I need to implement validation for inputs for the font size. Firstly, I can implement bounds for the inputs, as this is easy to do.

```
def fontSizeChange():
    #stores the title font, body font and small font sizes respectively
    #index 1 is comparison index
    userInput = int(fontSizeEntry.get().strip())#retrieve the user input from Entry object

    #validating userInput
    userValidation = True #currently the input is accepted
    if(userInput < 6 or userInput > 18):
        userValidation = False



    fontSizes = [titleFont.cget("size"),bodyFont.cget("size"),smallFont.cget("size"),smallFontBold.cget("size")]

    if userValidation ==True:
        if(fontSizes[1] > userInput):
            #need to reduce font sizes
            difference = fontSizes[1] - userInput
            for i in range(len(fontSizes)):
                fontSizes[i] -= difference
        else:
            difference = userInput - fontSizes[1]
            for i in range(len(fontSizes)):
                fontSizes[i] += difference

    titleFont.config(size = fontSizes[0])
    bodyFont.config(size = fontSizes[1])
    smallFont.config(size = fontSizes[2])
    smallFontBold.config(size = fontSizes[3])
```

As you can see, I've added a Boolean variable that I can use to track whether the user's input should be accepted or not. I've chosen to use 18 as the maximum font size as any bigger becomes hard to use even at full screen.

Test:

Test video: TestFontSize1_2_3.mp4

The following test video shows that inputs outside my chosen range are not accepted, but numbers within it are.

1.	Enter an integer within the bounds: 14	Valid	Changes all font sizes (scaled) to font size 14	Worked: 0-4 seconds
2.	Enter an integer outside range: 5	Erroneous	Doesn't accept the user's input	Worked: 4-14 seconds
3.	Enter integer outside range: 20	Erroneous	Doesn't accept the user's input	Worked: 14 – 20 seconds

Now I can implement type checks, where I make sure that the input is only an integer.

```
def fontSizeChange():
    #stores the title font, body font and small font sizes respectively
    #index 1 is comparison index
    userInput = fontSizeEntry.get().strip()#retrieve the user input from Entry object

    #validating userInput
    userValidation = True #currently the input is accepted

    if(userInput.isdigit() == False):
        userValidation = False
    else:
        userInput = int(userInput)

    if(userValidation == True and (int(userInput) < 6 or int(userInput) > 18)):
        userValidation = False
```

Because I'm doing a type check I can't assume that the user entered an integer, so I must remove the casting on the userInput instantiation. Additionally, isdigit() is great because it checks if the user inputted an integer, so any inputs including alpha characters or mathematical symbols are instantly rejected. Valid inputs then must be casted as you can see in the else statement.

This then allows me to test the rest of the remaining tests in the test plan, and this should lead to all the required validations working.[37]

<u>Test:</u>

Test video: fontSizeTest4_5_6_7_8.mp4

The test video shows that I have passed all the tests in the test plan, indicating a successful settings menu.

4.	Enter a float: 10.5	Erroneous	Doesn't accept the user's input	Worked: 5 – 12 seconds

[37] (Kumar, 2025)

5.	Enter no data	Erroneous	Doesn't accept the user's input	Worked: 0 – 4 seconds
6.	Enter non-numerical character: "a"	Erroneous	Doesn't accept the user's input	Worked: 13 – 20 seconds
7.	Enter non-numerical character: "/"	Erroneous	Doesn't accept the user's input	Worked: 20 – 24 seconds
8.	Enter non-numerical and numerical character: "-7"	Erroneous	Doesn't accept the user's input	Worked: 24 - 31 seconds

I can now start working on the left side of the GUI- the input section. This section will contain the constant menu that allows users to enter inputs. This menu will remain constant until they submit a valid set of data which can be passed into the logistic regression model.

Before I start the development of the left side container, I want to update my test plan so that it fully tests the validation I will need for the program effectively and efficiently.

This is my current test plan:

Test no.	Description + test data	Type of test	Expected outcome	Outcome
1.	Enter exact data for a record which isn't boundary data.	Valid	The corresponding value between 0 and 1 for their cardiovascular disease status.	
2.	Enter data that is close to a record in value but not exact (healthy).	Valid	The same cardiovascular disease status as that record.	
3.	Enter data that is close to a record in.	Valid	The same cardiovascular disease status as that record.	

	value but not exact (unhealthy).			
4.	Enter values on the lower range for all the data points in a field.	Boundary	Matches the output from stage 1, test 4.	
5.	Enter values on the upper range for all the data points in a field.	Boundary	Matches the output from stage 1, test 5.	
6.	Enter data values outside the range for every field in the dataset (unhealthy end of each field).	Boundary	Reject the inputs with an error message.	
7.	Enter data values outside the range for every field in the dataset (healthier end of each field).	Boundary	Reject the inputs with an error message.	
8.	Enter a negative value for 1 field	Error	Reject the inputs with an error message.	
9.	Enter characters (abc) for a numerical input.	erroneous	Reject the inputs with an error message.	
10.	Enter negative numbers for an input that accepts positive numbers.	erroneous	Reject the inputs with an error message.	
11.	Enter an extreme value for a parameter	Extreme	Rejects the input and informs the user but also recommends seeking medical help	
12.	Leave an input blank	erroneous	Reject the inputs with an error message	

13.	Choose to include the optional input data as an option then leave them blank	erroneous	Reject the inputs with an error message	
14	Choose to not include any optional input data and enter valid, expected data	Valid	Gives a valid diagnosis that has correlation to their data.	
15.	Choose to not include any optional input data and enter that data as inputs	Erroneous	Rejects the inputs with an error message	

Firstly, this test plan seems to test whether the output from the inputs matches the outputs from when I created and tested the logistic regression model, but the purpose of this test plan is to check if my validation is correct for the inputs I have implemented. The next stage of my development will focus on the output of the program, so I can remove the tests that target this within the test plan. Additionally, the tests need to be more tailored to how I will implement the validation in the program. When I created the test plan I didn't know exactly how I would implement the validation system, but now I'm more aware of how this will be done in my program so I can make the tests more specific and targeted.

Since a lot of the inputs will use the exact same form of validation, I only need to test this form of validation on one input. For example, the inputs age, height and weight all check if an input has been provided, if it's numeric and if it's within an established bound.

Test no.	Description + test data	Type of test	Expected outcome	Outcome
1.	Enter an integer value for age between 18 and 90.	Valid	No age error is added to the error outputs and displayed in the message.	
2.	Enter nothing in the age section.	Erroneous	"Age must be a number" added to error outputs and displayed.	

3.	Enter the character “a”.	Erroneous	“Age must be a number” added to error outputs and displayed.	
4.	Enter 16 as the age input.	Erroneous	“Age must be between 18 and 90” added to error outputs and displayed.	
5.	Select ‘Male’ as your sex.	Valid	Nothing added to the error outputs and not displayed in the error output message.	
6.	Select nothing for the ‘sex’ input.	Erroneous	“Please select your sex.” Added to the error outputs and displayed.	
7.	Enter nothing in Systolic blood pressure	Valid	Nothing added to the error outputs and not displayed in the error output message.	
8.	Enter the character “g”.	Erroneous	“Systolic blood pressure must be numeric.” added to the error outputs and displayed.	
9.	Enter “180”.	Erroneous	“Systolic blood pressure must be between 70 and 175 mm/Hg” added to the error outputs and displayed.	
10.	Enter “100”	Valid	Nothing added to the error outputs and not displayed in the error output message.	
11.	Enter “valid” inputs using the bounds in the program for all the compulsory inputs, leave the optional one’s blank.	Valid	“All inputs are valid!”	
12.	Enter “valid” inputs using the bounds in the program for all compulsory and optional inputs.	Valid	“All inputs are valid!”	

I only need to create one left side container as it is permanent and can persist on all menus.

```
#creating the left side container for the users input
leftSideContainer = tk.Frame(root,borderwidth=2,relief="solid")
rightSideContainer1.pack(side="left", anchor="ne", padx=10, pady=10)
```

I can then create separate entry boxes for each of the parameters I need to accept from the user and place them into a frame that is placed within this container.

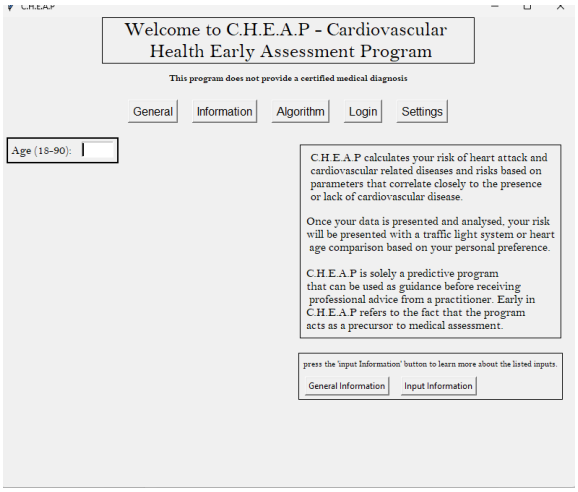
Since I will add the compulsory inputs into 1 bordered box and optional inputs into another bordered box within the left side container, I can create two frames for these two brackets of inputs:

```
compulsoryInputParameters = tk.Frame(leftSideContainer)
optionalInputParameters = tk.Frame(leftSideContainer)
```

Next, I need to create the Entry objects and the labels for the user's inputs, and as you can see, I've created this for the age input. I plan on creating a submit button that will then read all the user's inputs, rather then enter being necessary for each input.

```
compulsoryInputParameters = tk.Frame(leftSideContainer)
optionalInputParameters = tk.Frame(leftSideContainer)

compulsoryInputParameters.pack(side = "top",pady = 2)

userAge = 0

#########################################################
#functions for taking the inputs

#########################################################

ageEntryLabel = tk.Label(compulsoryInputParameters,text = "Age (18-90): ",font=bodyFont)#create the label displaying the age text
ageEntry = tk.Entry(compulsoryInputParameters, width = 5,font = bodyFont)#create the entry box for taking users age


ageEntryLabel.pack(padx=2,pady=2,side = "left")
ageEntry.pack(padx= 5,pady = 2)

root.mainloop()# starts a loop
```

Test:



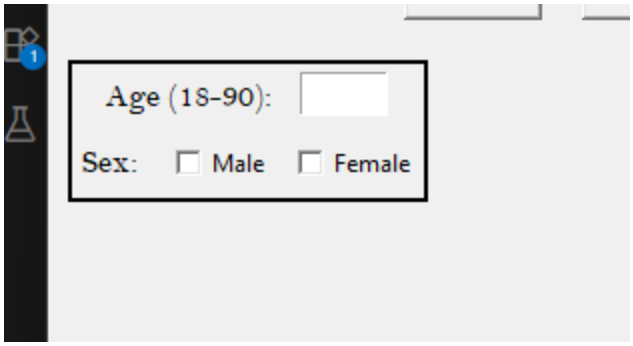
The label and entry box have both displayed appropriately, I now just need to do the same for the other user inputs.

However, since I want the sex input to be a checkbox, I must go about it differently; I've used the tkinter CheckButton object[38], which then allows you to restrict the user's inputs to the check boxes that you create.

```
#objects for users sex
sexEntryLabel = tk.Label(compulsoryInputParameters,text = "Sex: ",font = bodyFont)
sexButtonMale = tk.Checkbutton(compulsoryInputParameters,text = "Male",onvalue=1,offvalue=0)
sexButtonFemale = tk.Checkbutton(compulsoryInputParameters,text = "Female",onvalue=1,offvalue=0)

sexEntryLabel.pack(padx=2,pady=5,side ="left")
sexButtonMale.pack(padx=5,pady = 5,side = "left")
sexButtonFemale.pack(pady = 5,padx =2,side = "left")
```

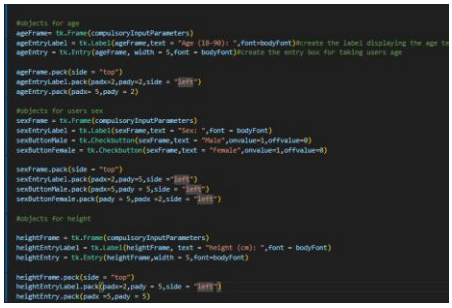
Test:

[38] (GeeksforGeeks, 2025)

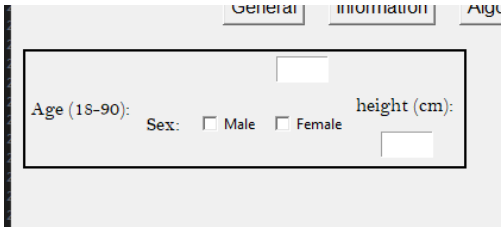


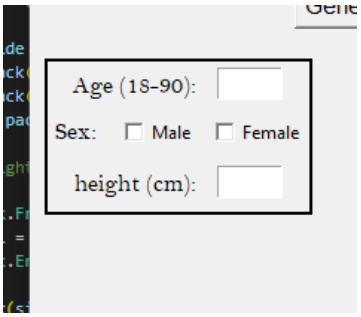
I have to make sure that the user is unable to select both checkboxes, but using checkboxes is better than an open text box, as this could lead to values that are ambiguous needing to be checked and it's less appearing for the user.

After adding the height objects, placing all the input boxes and their labels properly in the left side container is proving to be an issue. I will thus create a frame for each input and place these frames in the left side container. Here's the result without implementing these frames.



Test:



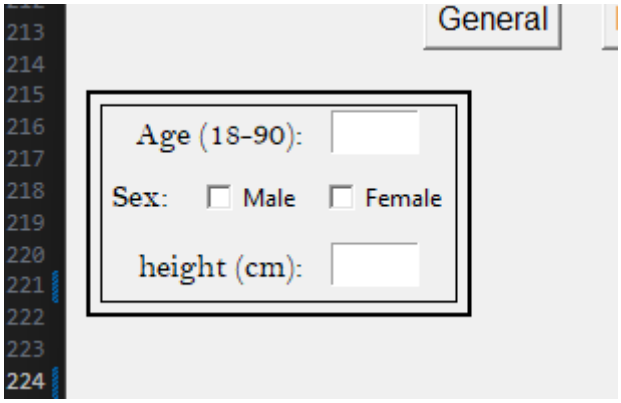


The input boxes are now orientated a lot better!

I have also added a border to the compulsory inputs frame so that these inputs and the optional inputs will appear differently.

```
#adding the input boxes in the left side container

compulsoryInputParameters = tk.Frame(leftSideContainer,borderwidth = 1, relief = "solid")
optionalInputParameters = tk.Frame(leftSideContainer)

compulsoryInputParameters.pack(side = "top",pady = 5, padx = 5)
```

Test:



After finishing the compulsory inputs, I will now move onto the optional inputs, which will just be placed in a separate frame. Here's the following code:

```
#objects for height

heightFrame = tk.Frame(compulsoryInputParameters)
heightEntryLabel = tk.Label(heightFrame, text = "Height (cm): ",font = bodyFont)
heightEntry = tk.Entry(heightFrame,width = 5,font=bodyFont)

heightFrame.pack(side = "top")
heightEntryLabel.pack(padx=2,pady = 5,side = "left")
heightEntry.pack(padx =5,pady = 5)

#objects for weight

weightFrame = tk.Frame(compulsoryInputParameters)
weightEntryLabel = tk.Label(weightFrame, text = "Weight (kg): ",font = bodyFont)
weightEntry = tk.Entry(weightFrame,width = 5,font = bodyFont)

weightFrame.pack(side = "top")
weightEntryLabel.pack(padx =2,pady = 5,side = "left")
weightEntry.pack(padx = 5, pady = 5)

#objects for smoking
smokingFrame = tk.Frame(compulsoryInputParameters)
smokingEntryLabel = tk.Label(smokingFrame, text = "Do you smoke?: ",font = bodyFont)
smokingEntry= tk.Checkbutton(smokingFrame,text = "Yes", onvalue=1,offvalue=0,font = bodyFont)


smokingFrame.pack(side = "top")
smokingEntryLabel.pack(padx = 5,pady = 5, side = "left")
smokingEntry.pack(padx=5,pady =5)

#objects for alcohol
alcoholFrame = tk.Frame(compulsoryInputParameters)
alcoholEntryLabel = tk.Label(alcoholFrame,text = "Do you drink alcohol?: ",font = bodyFont)
alcoholEntry = tk.Checkbutton(alcoholFrame,text = "Yes",onvalue=1,offvalue=0,font = bodyFont)

alcoholFrame.pack(side = "top")
alcoholEntryLabel.pack(padx = 5, pady = 5, side = "left")
alcoholEntry.pack(padx = 5,pady = 5)

#objects for sedentary
sedentaryFrame = tk.Frame(compulsoryInputParameters)
sedentaryEntryLabel = tk.Label(sedentaryFrame, text = "Are you physically active?: ",font = bodyFont)
sedentaryEntry = tk.Checkbutton(sedentaryFrame,text = "Yes",onvalue=1, offvalue=0,font = bodyFont)

sedentaryFrame.pack(side = "top")
sedentaryEntryLabel.pack(padx = 2, pady =5, side = "left")
sedentaryEntry.pack(padx = 5, pady =5)
root.mainloop()# starts a loop
```

Test:

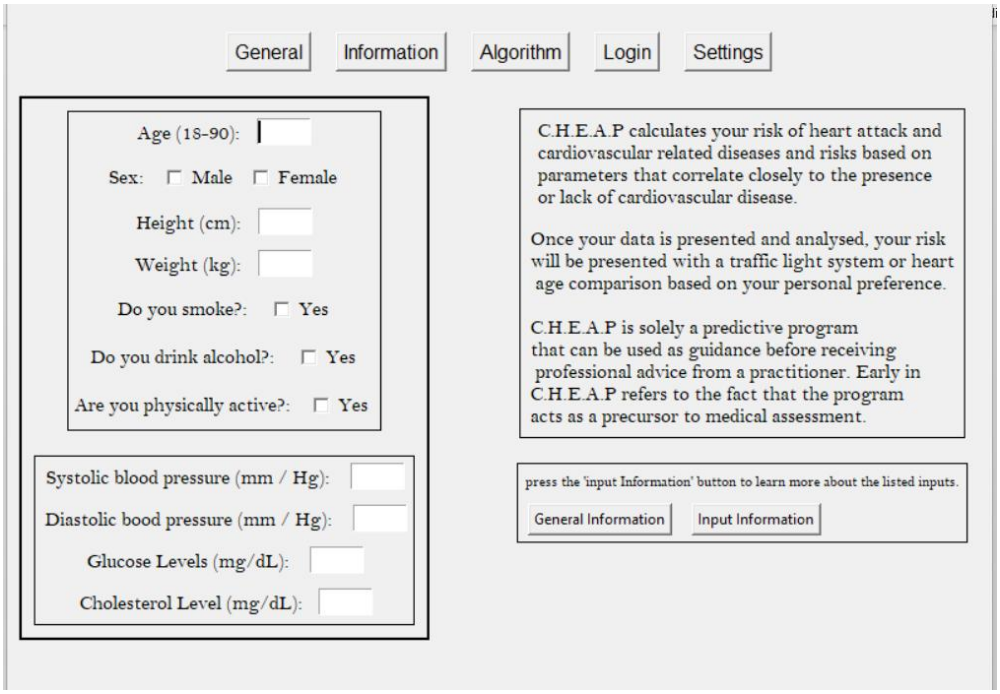
I felt that I only needed to include the Yes button, as you can simply not tick it if your answer is No. to make this clear I will change the input information section so that it accurately supports the user through the input process, as it can be confusing without guidance.

Before I change the input information, I will add the optional inputs that the user can enter to make their diagnosis better so that I can also alter the input information for these inputs as well.

I simply have to do the same thing I've done for the compulsory inputs with the optional input frame:

```
#objects for systolic blood pressure
systolicFrame = tk.Frame(optionalInputParameters)
systolicEntryLabel = tk.Label(systolicFrame,text = "Systolic blood pressure (mm / Hg): ",font = bodyFont)
systolicEntry = tk.Entry(systolicFrame,width = 5, font = bodyFont)

systolicFrame.pack(side = "top")
systolicEntryLabel.pack(padx = 5,pady = 5, side = "left")
systolicEntry.pack(padx = 5, pady =5)

#objects for diastolic blood pressure
diastolicFrame = tk.Frame(optionalInputParameters)
diastolicEntryLabel = tk.Label(diastolicFrame, text = "Diastolic bood pressure (mm / Hg): ", font = bodyFont)
diastolicEntry = tk.Entry(diastolicFrame,width = 5,font = bodyFont)

diastolicFrame.pack(side = "top")
diastolicEntryLabel.pack(padx=5,pady= 5,side = "left")
diastolicEntry.pack(padx = 5,pady =5)

#objects for glucose
glucoseFrame = tk.Frame(optionalInputParameters)
glucoseEntryLabel = tk.Label(glucoseFrame,text = "Glucose Levels (mg/dL): ",font = bodyFont)
glucoseEntry = tk.Entry(glucoseFrame,width = 5,font = bodyFont)

glucoseFrame.pack(side = "top")
glucoseEntryLabel.pack(padx = 5, pady = 5, side = "left")
glucoseEntry.pack(padx = 5,pady = 5)

#objects for cholesterol
cholesterolFrame = tk.Frame(optionalInputParameters)
cholesterolEntryLabel = tk.Label(cholesterolFrame,text = "Cholesterol Level (mg/dL):",font = bodyFont)
cholesterolEntry = tk.Entry(cholesterolFrame,width = 5, font = bodyFont)

cholesterolFrame.pack(side = "top")
cholesterolEntryLabel.pack(padx =5,pady = 5, side = "left")
cholesterolEntry.pack(padx=5, pady =5)
```

This here results in this in the input section:



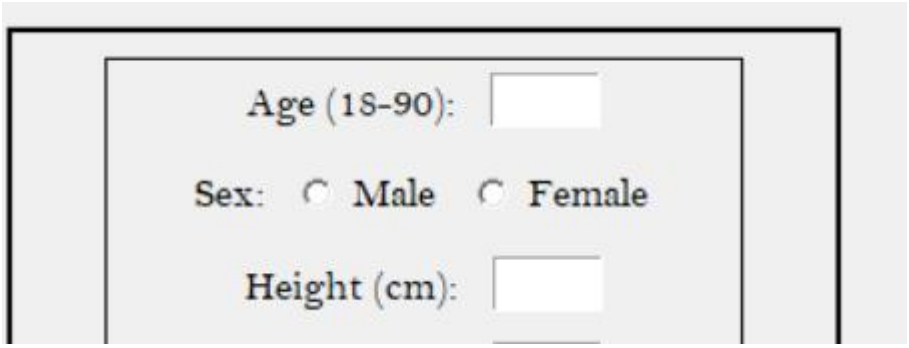
Currently, all the inputs are being displayed accordingly, however, I need to make sure the menu is intuitive and simple. To do this I can add an optional title to the optional input Frame and alter the input information as mentioned to accurately depict the inputs I have.

After further research on different objects in Tkinter, I learnt that you could use radiobuttons[39] instead of checkboxes so that only 1 input is accepted for the sex of the user. This is a lot better than using checkboxes as the validation to check if 1 box is selected is made redundant:

```
# Use an IntVar (or StringVar) for shared state
sexVar = tk.StringVar(value="None")

sexButtonMale = tk.Radiobutton(
    sexFrame, text="Male", variable=sexVar, value="Male", font=bodyFont
)
sexButtonFemale = tk.Radiobutton(
    sexFrame, text="Female", variable=sexVar, value="Female", font=bodyFont
)

sexFrame.pack(side="top")
sexEntryLabel.pack(padx=2, pady=5, side="left")
sexButtonMale.pack(padx=5, pady=5, side="left")
sexButtonFemale.pack(padx=2, pady=5, side="left")
```

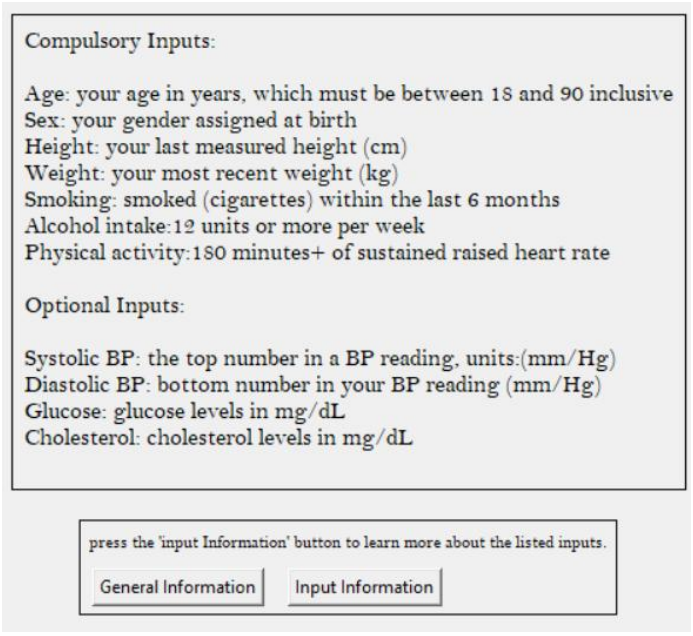
Test:

[39] (RadioButton in Tkinter | Python, 2025)



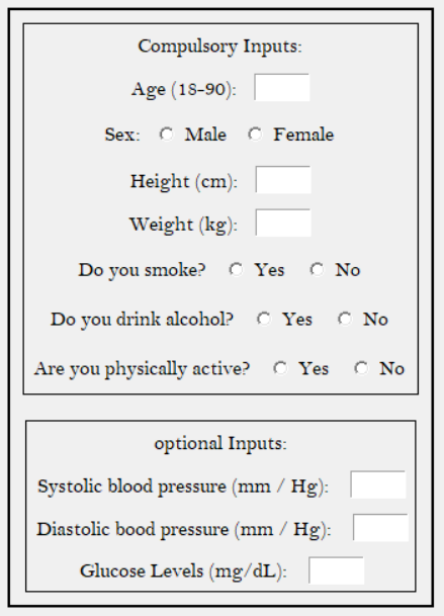
When I test the validation for each of the inputs, I can see the efficacy of the radiobuttons. Before I implement this validation, I will change the input information as mentioned.

Firstly, I will add the titles “Compulsory” and “Optional” under the corresponding inputs so that it matches how they appear in the input frame. Additionally, I need to reorder the inputs, so they match how the appear in the input frame to create a simpler and cohesive program. Lastly, I originally didn’t include inputs that I deemed as “too simple”, but to prevent all possible issues with input taking, I should explain every input even if it seems very simple.



As you can see, I’ve adjusted the input information to match what I mentioned, and now it’s much easier to follow and utilise.

Furthermore, on top of the sex input, making the other three inputs (smoking, alcohol and physical activity) should also be radiobuttons as it makes validation much easier and it’s simpler for the user.



Now I can move onto validating the inputs, which is a very important feature for the program. I need to ensure that every input is provided in the compulsory input section and that certain inputs follow specific bounds and rules. Firstly, I will apply the validation to the compulsory inputs as it is not difficult to implement the validation I will need. Since there is a lot of logic required though, I will create some pseudocode first. The only inputs I will write in pseudocode are inputs that require different types of validation:

```
def validateInputs():
  errors = [] # create an array of what inputs are present

  age = ageEntry.get()
  if not age.isdigit():#checks if its a number
    errors.append("Age must be a number")
  elif not (18 <= int(age) <= 90):
    errors.append("Age must be between the given bounds")

  if sexVar.get() not in ["Male","Female"]:#checks if either option was chosen
    errors.append("Please select your sex")

  .....

  if errors != []:
    print(errors)#some form of tkinter function to display the errors

  else:
    print("no issues!")
```

As you can see, I've included the age input, and sex input. Every other input will be checked the same way the other two are checked. Checking to see if an input is an integer is as simple as using the isdigit() function, which can check if all the characters in an input are digits. Then checking bounds is very easy once the numbers are established.

Checking the variables defined for radiobuttons is simple too. Whatever I defined the values are defined as in the radio buttons will be stored if it's selected, so checking to see if either is selected can be done as seen above.

Lastly, if the user inputs a blank input this would flag that case too, as a blank input isn't a numeric input and it wouldn't be in any of the lists I'm checking against.

I can now implement this in Python:

```
def validateCompulsoryInputs():
    errors = []# array of errors

    #age
    ageVal = ageEntry.get().strip()
    if not ageVal.isdigit():
        errors.append("Age must be a number.")
    elif not (18 <= int(ageVal) <= 90):
        errors.append("Age must be between 18 and 90.")

    #sex
    if sexVar.get() not in ["Male", "Female"]:
        errors.append("Please select your sex.")

    #height
    heightVal = heightEntry.get().strip()
    if not heightVal.isdigit() or (90 <= int(heightVal)<= 250):
        errors.append("Height must be a integer within the programs given range.")

    #weight
    weightVal = weightEntry.get().strip()
    if not weightVal.isdigit() or (40 <= int(weightVal) <= 240):
        errors.append("Weight must be a integer within the programs given range.")

    #smoking
    if smokingVar.get() not in ["Yes", "No"]:
        errors.append("Please indicate if you smoke.")

    #alcohol
    if alcoholVar.get() not in ["Yes", "No"]:
        errors.append("Please indicate if you drink alcohol.")

    #activity
    if activityVar.get() not in ["Yes", "No"]:
        errors.append("Please indicate if you are physically active.")

    if errors:
        messagebox.showerror("Input Error", "\n".join(errors))
    else:
        messagebox.showinfo("Success", "All compulsory inputs are valid!")
```

I've chosen to use message boxes[40] as they act as pop up tabs that display text that can't be removed without pressing the red X at the top right of the menu. Creating an additional pop-up menu makes the issue seem more imperative and important, which is the case for this scenario.

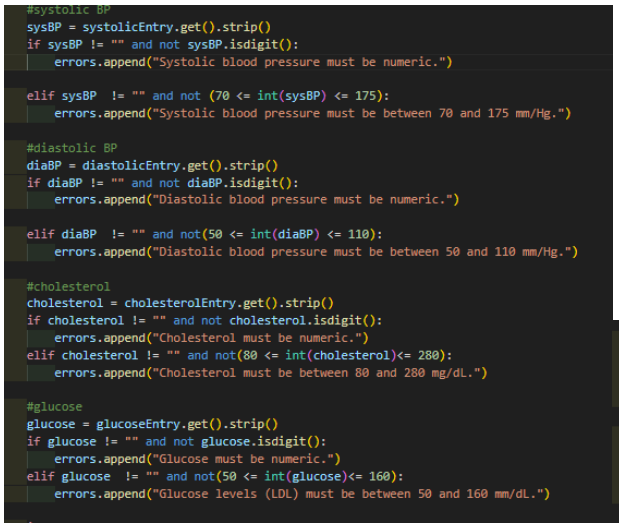
I've now added the optional validation checks and placed the error output below these inputs.

Since these are optional, to do the number validation checks I need to make sure there is content being inputted, as they can be empty while the compulsory inputs cannot be:

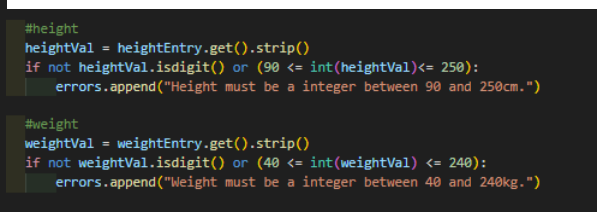
[40] (GeeksForGeeks, 2025)

```
    #optional inputs
    #systolic BP
    sysBP = systolicEntry.get().strip()
    if sysBP != "" and not sysBP.isdigit():
        errors.append("Systolic blood pressure must be numeric.")

    elif sysBP  != "" and not (70 <= int(sysBP) <= 175):
        errors.append("This systolic blood pressure value is outside the program's range.")

    #diastolic BP
    diaBP = diastolicEntry.get().strip()
    if diaBP != "" and not diaBP.isdigit():
        errors.append("Diastolic blood pressure must be numeric.")

    elif diaBP  != "" and not(50 <= int(diaBP) <= 110):
        errors.append("This diastolic blood pressure value is outside the program's range.")

    #cholesterol
    cholesterol = cholesterolEntry.get().strip()
    if cholesterol != "" and not cholesterol.isdigit():
        errors.append("Cholesterol must be numeric.")
    elif cholesterol != "" and not(80 <= int(cholesterol)<= 280):
        errors.append("This cholesterol is outside the program's range.")

    #glucose
    glucose = glucoseEntry.get().strip()
    if glucose != "" and not glucose.isdigit():
        errors.append("Glucose must be numeric.")
    elif glucose  != "" and not(50 <= int(glucose)<= 160):
        errors.append("This glucose level is outside the programs range.")

    if errors:
        messagebox.showerror("Input Error", "\n".join(errors))
    else:
        messagebox.showinfo("Success", "All compulsory inputs are valid!")
```

After implementing these checks, I've realised that I should state these ranges in the error message, instead of just saying the number given was outside the 'program's range'. This information is very vague and not very helpful for the user. Therefore, I've added the numbers I included in the range checks:

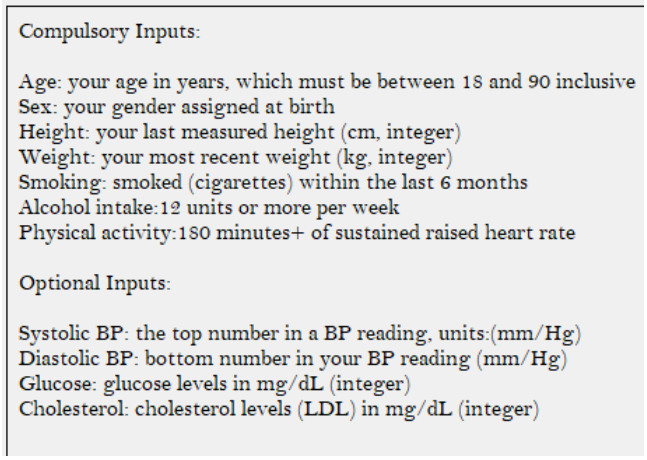


```
#systolic BP
sysBP = systolicEntry.get().strip()
if sysBP != "" and not sysBP.isdigit():
    errors.append("Systolic blood pressure must be numeric.")

elif sysBP  != "" and not (70 <= int(sysBP) <= 175):
    errors.append("Systolic blood pressure must be between 70 and 175 mm/Hg.")

#diastolic BP
diaBP = diastolicEntry.get().strip()
if diaBP != "" and not diaBP.isdigit():
    errors.append("Diastolic blood pressure must be numeric.")

elif diaBP  != "" and not(50 <= int(diaBP) <= 110):
    errors.append("Diastolic blood pressure must be between 50 and 110 mm/Hg.")

#cholesterol
cholesterol = cholesterolEntry.get().strip()
if cholesterol != "" and not cholesterol.isdigit():
    errors.append("Cholesterol must be numeric.")
elif cholesterol != "" and not(80 <= int(cholesterol)<= 280):
    errors.append("Cholesterol must be between 80 and 280 mg/dL.")

#glucose
glucose = glucoseEntry.get().strip()
if glucose != "" and not glucose.isdigit():
    errors.append("Glucose must be numeric.")
elif glucose  != "" and not(50 <= int(glucose)<= 160):
    errors.append("Glucose levels (LDL) must be between 50 and 160 mm/dL.")
```



```
#height
heightVal = heightEntry.get().strip()
if not heightVal.isdigit() or (90 <= int(heightVal)<= 250):
    errors.append("Height must be a integer between 90 and 250cm.")

#weight
weightVal = weightEntry.get().strip()
if not weightVal.isdigit() or (40 <= int(weightVal) <= 240):
    errors.append("Weight must be a integer between 40 and 240kg.")
```

On top of that, since I am suing int() for all the numeric inputs, I need to ensure that the input information tells the user to enter integers, not floats:



I believe I can now move onto testing the validation that I have done using the text plan I adjusted accordingly:

Test Video: inputValidation1_2_3_4.mp4

Test no.	Description + test data	Type of test	Expected outcome	Outcome
1.	Enter an integer value for age between 18 and 90.	Valid	No age error is added to the error outputs and displayed in the message.	No age error is added to the error outputs and displayed in the message. 0-6s
2.	Enter nothing in the age section.	Erroneous	“Age must be a number” added to error outputs and displayed.	“Age must be a number” added to error outputs and displayed. 6-12s
3.	Enter the character “a”.	Erroneous	“Age must be a number” added to error outputs and displayed.	“Age must be a number” added to error outputs and displayed. 13-18s
4.	Enter 16 as the age input.	Erroneous	“Age must be between 18 and 90” added to error outputs and displayed.	“Age must be between 18 and 90” added to

				error outputs and displayed. 18-24s

The previous linked clip shows that all the tests were successful.

I can now test all the inputs that use radio boxes together as their validation is all the same:

Tests: inputValidation5.mp4 and inputValidation6.mp4

Test no.	Description + test data	Type of test	Expected outcome	Outcome
5.	Select 'Male' as your sex.	Valid	Nothing added to the error outputs and not displayed in the error output message.	Nothing added to the error outputs and not displayed in the error output message.
6.	Select nothing for the 'sex' input.	Erroneous	"Please select your sex." Added to the error outputs and displayed.	"Please select your sex." Added to the error outputs and displayed.

I can move onto testing the optional inputs now, and I have slightly different test plans as leaving them blank is possible, So I need to make sure the validation works for that as well:

Test: inputValidation7_8_9_10.mp4

Test no.	Description + test data	Type of test	Expected outcome	Outcome
7.	Enter nothing in Systolic blood pressure	Valid	Nothing added to the error outputs and not displayed in the error output message.	Nothing added to the error outputs and not displayed in the error output message.

				0-6s
8.	Enter the character “g”.	Erroneous	“Systolic blood pressure must be numeric.” added to the error outputs and displayed.	“Systolic blood pressure must be numeric.” added to the error outputs and displayed. 6-11s
9.	Enter “180”.	Erroneous	“Systolic blood pressure must be between 70 and 175 mm/Hg” added to the error outputs and displayed.	“Systolic blood pressure must be between 70 and 175 mm/Hg” added to the error outputs and displayed. 11-17s
10.	Enter “100”	Valid	Nothing added to the error outputs and not displayed in the error output message.	Nothing added to the error outputs and not displayed in the error output message. 17-22s

For the last two tests I just need to ensure that the validation functions when all the inputs are correct. Once the inputs are being used in the regression model during stage 3 of development I can use them properly.

As I was checking the inputs for height and weight, I realised I didn’t include a not before the range checks, so I’ve added it in. I also realised that the regression model considers the user’s BMI, so I should potentially check the height and weight against each other to make sure that it’s valid, however, the user isn’t attempting to mislead the program so as long as the inputs are actually valid (no characters or insanely large or small numbers), the weight and height will very likely be accurate to the person, so checking for a BMI is redundant for validation.

```
#height
heightVal = heightEntry.get().strip()
if not heightVal.isdigit() or not (90 <= int(heightVal)<= 250):
    errors.append("Height must be a integer between 90 and 250cm.")

#weight
weightVal = weightEntry.get().strip()
if not weightVal.isdigit() or not (40 <= int(weightVal) <= 240):
    errors.append("Weight must be a integer between 40 and 240kg.")

#smoking
```

Test: inputValidation11.mp4 and inputValidation12.mp4

Test no.	Description + test data	Type of test	Expected outcome	Outcome
11.	Enter “valid” inputs using the bounds in the program for all the compulsory inputs, leave the optional one’s blank.	Valid	“All inputs are valid!”	“All inputs are valid!”
12.	Enter “valid” inputs using the bounds in the program for all compulsory and optional inputs.	Valid	“All inputs are valid!”	“All inputs are valid!”

This concludes stage 2 since all the tests were successful in both my test plans.

Stage 2 References

geeksforgeeks. (2025, 10 14). *How To Align Text In Tkinter Label?* Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/python/how-to-align-text-in-tkinter-label/

GeeksforGeeks. (2025, 11 04). *How To Get The Input From A Checkbox In Python Tkinter?* Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/python/how-to-get-the-input-from-a-checkbox-in-python-tkinter/

geeksforgeeks. (2025, 10 14). *How to Hide, Recover and Delete Tkinter Widgets?* Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/python/how-to-hide-recover-and-delete-tkinter-widgets/

GeeksForGeeks. (2025, 10 09). *How to make A Label Bold Tkinter?* Retrieved from GeeksForGeeks: https://www.geeksforgeeks.org/python/how-to-make-a-label-bold-tkinter/

GeeksforGeeks. (2025, 10 09). *How to place a button at any position in Tkinter?* Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/python/how-to-place-a-button-at-any-position-in-tkinter/

GeeksforGeeks. (2025, 10 09). *How to Set Border of Tkinter Label Widget*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/python/how-to-set-border-of-tkinter-label-widget/

GeeksforGeeks. (2025, 10 21). *Python | Binding function in Tkinter* . Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/python/python-binding-function-in-tkinter/

GeeksforGeeks. (2025, 10 21). *Python Tkinter - Entry Widget*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/python/python-tkinter-entry-widget/

Kumar, B. (2025, 10 23). *How to Check if Input is a Number in Python?* Retrieved from PythonGuides: https://pythonguides.com/check-if-input-is-a-number-in-python/

NeuralNine. (2025, 10 07). *Tkinter Beginner Course - Python GUI Development*. Retrieved from https://www.youtube.com/watch?v=ibf5cx221hk

RadioButton in Tkinter | Python. (2025, 11 10). Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/python/radiobutton-in-tkinter-python/

WHO. (2025, 10 16). *WHO guidelines on physical activity and sedentary behaviour*. Retrieved from World Health Organisation: https://www.who.int/publications/i/item/9789240015128

Stage 3 – Create the diagnosis + feedback UI (30th October 2025)

This UI/menu will follow the main menu directly, as once you've entered and submitted your data you will be directly taken here. This menu will provide the feedback and diagnosis based on your data.

The feedback will be presented either as a traffic light system or a heart to real age comparison, since these were the most picked options based on my survey. Both can be selected as options by the user, but both will be accompanied by a short text in plain language as this was also a top option by stakeholders. This short paragraph will detail the diagnosis but also outline any likely health factors that led to the diagnosis if they were deemed 'at risk'. If there was no single parameter causing this, it's also important to highlight that for the user, meaning that this feature must be properly implemented, validated and tested. There would need to be a cutoff for when certain factors are outlying compared to when a user is unhealthy for multiple reasons, making it difficult to identify the cause.

If the user didn't enter any optional data, I plan to offer an estimation on what their diagnosis would look like based on the rest of the data, although this feature won't be extremely accurate.

Accompanying this will be videos, graphs and photos is important to prevent confusion among users. General life advice, diet advice and potentially a recommendation to visit your GP or doctor based on the diagnosis to kickstart the user's journey to a healthier cardiovascular system will be used. Furthermore, the user will be granted the opportunity to create a diary to track their progress, but that's another stage that will be implemented after this one.

The main feature that must be tested is the diagnosis and input identification, as the other information will simply correlate to what diagnosis the user has received so it's easy to test it and implement it. Ensuring that the diagnosis not only matches the regression model, but also the identification of what factor is contributing to the diagnosis is vital.

Features I want to implement:

1. Traffic light/ heart age comparison diagnosis system
2. Outlying factor/s identification to identify cause for the diagnosis
3. Offer graphs, photos and videos to support the diagnosis
4. Offer diet and general lifestyle advice based on the factor/s identified.
5. Present the possible diagnosis if optional inputs were entered.

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data to a record in the data set (not boundary and healthy)	Valid	Heart age <= real heart age or green on traffic light system, with no outlying factor contributing to their health	
2.	Enter exact data to a record in the data set (not boundary and unhealthy).	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factor led to this or if none are outlying.	
3.	Enter very similar data to a record in the data set (not boundary and healthy)	Valid	Same diagnosis as the first test with same factor identification.	
4.	Enter very similar data to a record in the data set (not boundary and unhealthy)	Valid	Same diagnosis as the second test with same factor identification.	
5.	Enter custom data for a healthy person with no outlying inputs	Valid	Green light or heart age <= real age without any factors identified.	
6.	Enter custom data for an unhealthy person with no outlying inputs	Valid	Heart age > real heart age or yellow/red on traffic light system. No are outlying factors identified	
7.	Enter custom data for an unhealthy person with 1 outlying input	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factor led to this.	
8.	Enter custom data for an unhealthy person with 2 outlying input	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factors led to this.	
9.	Enter healthy data without any of the optional inputs (non-boundary)	Valid	Green light or heart age <= real age with a estimate of what their health would be with the optional data	
10.	Enter unhealthy data without any of the optional inputs (non-boundary)	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify what their health would look like with optional inputs	

The main feature I need to add right now is integrating the regression model with the UI, and to do this I firstly need to alter validating inputs function to store the inputs so that they can be passed into the regression model.

I believe the best way to do this is using an array that can store the values for each of the inputs. This then allows me to manipulate and access these values using 1 data structure instead of 10 individual variables, making them faster and easier to access.

However, I need to understand that the optional inputs can be left blank, so during the definition of the array I will set all of the indexes to the value '-1' which will indicate a NULL value.

```
def validateCompulsoryInputs():
    errors = []# array of errors
    inputs = [-1] * 11
```

I can then add the values for the inputs as they're validated. I don't have to worry about checking the compulsory inputs for null values as this array will only be passed to the regression model if all the validation checks are valid.

I can do this by adding else statements to all the validation checks, and if that else is true then I can add the value to the inputs array.

```
    errors.append("Age must be between 18 and 90.")
else:
    inputs[0] = int(ageVal)

#sex
if sexVar.get() not in ["Male", "Female"]:
    errors.append("Please select your sex.")
else:
    inputs[1] = sexVar.get()

#height
heightVal = heightEntry.get().strip()
if not heightVal.isdigit() or not (90 <= int(heightVal)<= 250):
    errors.append("Height must be a integer between 90 and 250cm.")
else:
    inputs[2] = int(heightVal)

#weight
weightVal = weightEntry.get().strip()
if not weightVal.isdigit() or not (40 <= int(weightVal) <= 240):
    errors.append("Weight must be a integer between 40 and 240kg.")
else:
    inputs[3] = int(weightVal)

#smoking
if smokingVar.get() not in ["Yes", "No"]:
    errors.append("Please indicate if you smoke.")
else:
    inputs[4] = smokingVar.get()

#alcohol
if alcoholVar.get() not in ["Yes", "No"]:
    errors.append("Please indicate if you drink alcohol.")
else:
    inputs[5] = alcoholVar.get()

#activity
if activityVar.get() not in ["Yes", "No"]:
    errors.append("Please indicate if you are physically active.")
else:
    inputs[6] = activityVar.get()
```

This will repeat for all the inputs where inputs [10] will store the glucose.

However, when doing the optional inputs I need to make it an else if as they can still be empty:

```
        errors.append("Systolic blood pressure must be between 70 and 175 mm/Hg.")

    elif sysBP != "":
        inputs[7] = int(sysBP)

    #diastolic BP
    diaBP = diastolicEntry.get().strip()
    if diaBP != "" and not diaBP.isdigit():
        errors.append("Diastolic blood pressure must be numeric.")

    elif diaBP  != "" and not(50 <= int(diaBP) <= 110):
        errors.append("Diastolic blood pressure must be between 50 and 110 mm/Hg.")

    elif diaBP != "":
        inputs[8] = int(diaBP)


    #cholesterol
    cholesterol = cholesterolEntry.get().strip()
    if cholesterol != "" and not cholesterol.isdigit():
        errors.append("Cholesterol must be numeric.")
    elif cholesterol != "" and not(80 <= int(cholesterol)<= 280):
        errors.append("Cholesterol must be between 80 and 280 mg/dL.")
    elif cholesterol != "":
        inputs[9] = int(cholesterol)

    #glucose
    glucose = glucoseEntry.get().strip()
    if glucose != "" and not glucose.isdigit():
        errors.append("Glucose must be numeric.")
    elif glucose  != "" and not(50 <= int(glucose)<= 160):
        errors.append("Glucose levels (LDL) must be between 50 and 160 mm/dL.")
    elif glucose != "":
        inputs[10] = int(glucose)
```

Now for the regression model I only use 10 inputs, compared to the 11 stored in this array. That's because the height and weight are converted into BMI, which I can do as I'm using the array within the regression model.

```
regression_test.py > ...
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import time

startTime = time.time()

from full_logistic_model import fullLogisticModel#import the class containing the regression model

data = pd.read_csv("cleaned_cardio_train3.csv")#read the data from the cleaned cardiovascular dataset

data['BMI'] = data['weight'] / ((data['height']/100) **2) #creates a column called BMI using the weight and height columns


cardioData = data.drop(["id", "cardio","height", "weight"], axis=1).values #contains all the records for every column except 'id', 'cardio', 'height' and 'weight'
cardioValue = data["cardio"].values #contains all the values for the 'cardio' column

scaler = StandardScaler()
cardioData = scaler.fit_transform(cardioData)
#scales the data down using a standard scaler (Z-world)

cardioData_train, cardioData_test, cardioValue_train, cardioValue_test = train_test_split(
cardioData, cardioValue, test_size=0.2, random_state=55)
#split the data into 80% training data, and 20% test data

# Train model
model = fullLogisticModel(lr=0.01, numIters=5000)
model.fit(cardioData_train, cardioValue_train)

# Test model
cardioValue_pred = model.predict(cardioData_test)
acc = accuracy_score(cardioValue_test, cardioValue_pred)

endTime = time.time()

print("time taken:",endTime - startTime)
print(f"Accuracy: {acc*100:.2f}%")
```

This is how the regression model is currently called, tested and evaluated. As you can see, I can drop columns at will and I needed to drop the cardio column, so the model doesn't know the actual cardiovascular status of the user. I can use this feature to drop the optional columns which aren't being used.

So, to only run the regression model if the inputs have been fully validated, I can make the validate inputs procedure a function but adding return statements as follows:

```
            inputs[10] = int(glucose)

        if errors:
            messagebox.showerror("Input Error", "\n".join(errors))
            return False
        else:
            return inputs

    #submit button
    submitButton = tk.Button(optionalInputParameters, text="Submit", font=bodyFont, command = validateCompulsoryInputs)
    submitButton.pack(pady=5)


    #IMPLEMENTING THE REGRESSION MODEL
```

This then allows me to do a check to see if the return statement is false. After that I can use the inputs in the regression model.

```
validationResult = validateCompulsoryInputs()
if(validationResult != False):
    #IMPLEMENTING THE REGRESSION MODEL
    from full_logistic_model import fullLogisticModel#import the class containing the regression model

    data = pd.read_csv("cleaned_cardio_train3.csv")#read the data from the cleaned cardiovascular dataset

    data['BMI'] = data['weight'] / ((data['height']/100) **2) #creates a column called BMI using the weight and height columns


    cardioData = data.drop(["id", "cardio","height", "weight"], axis=1).values #contains all the records for every column except 'id', 'cardio', 'height' and 'weight'
    cardioValue = data["cardio"].values #contains all the values for the 'cardio' column

    scaler = StandardScaler()
    cardioData_train = scaler.fit_transform(cardioData)
    #scales the data down using a standard scaler (Z-world)

    # Train model
    model = fullLogisticModel(lr=0.01, numIters=5000)
    model.fit(cardioData_train, cardioValue)

    # Test model
    cardioValue_pred = model.predict(cardioData_test)
```

I can now create a dictionary that will use the inputs array. I did this before when I tested the regression model in the terminal, so I can now do something similar.

```
validationResult = validateCompulsoryInputs()
if(validationResult != False):

    userInputs = {
        "age": validationResult[0],
        "gender": validationResult[1],
        "height": validationResult[2],
        "weight": validationResult[3],
        "smoke": validationResult[4],
        "alco": validationResult[5],
        "active": validationResult[6],
        "ap_hi": validationResult[7],
        "ap_lo": validationResult[8],
        "cholesterol": validationResult[9],
        "gluc": validationResult[10]
    }
    userDataFrame = pd.DataFrame(userInputs)

    userDataFrame['BMI'] = userDataFrame['weight'] / ((userDataFrame['height']/100) ** 2)
```

As you can see the code is essentially identical to what I had before, except the values for each key come from the inputs array created during the validate inputs function.

I now need to drop columns that have a value of -1 in the userDataFrame from the userDataFrame and the 'data' data frame.

```
colsToDrop = [col for col in userDataFrame.columns if userDataFrame[col].iloc[0] == -1]
userDataFrame = userDataFrame.drop(columns=colsToDrop)


#IMPLEMENTING THE REGRESSION MODEL
from full_logistic_model import fullLogisticModel#import the class containing the regression model

data = pd.read_csv("cleaned_cardio_train3.csv")#read the data from the cleaned cardiovascular dataset

data['BMI'] = data['weight'] / ((data['height']/100) **2) #creates a column called BMI using the weight and height columns


cardioData = data.drop(["id", "cardio","height", "weight"], axis=1).values #contains all the records for every column except 'id', 'cardio', 'height' and 'weight'
CardioData = data.drop(columns=colsToDrop).values
cardioValue = data["cardio"].values #contains all the values for the 'cardio' column
```

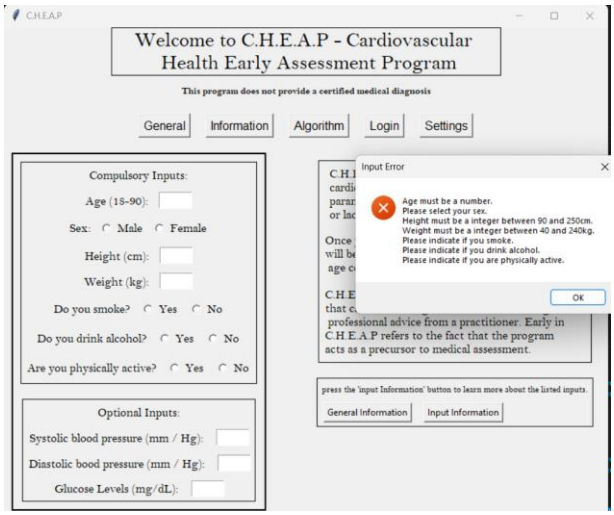
I can firstly loop through the data frame and add the columns with the value -1 to an array colsToDrop, allowing me to easily drop this from userDataFrame and the cardioData data frame.

I can therefore now add userDataFrame to the prediction call:

```
#IMPLEMENTING THE REGRESSION MODEL
from full_logistic_model import fullLogisticModel#import the class containing the regression model

data = pd.read_csv("cleaned_cardio_train3.csv")#read the data from the cleaned cardiovascular dataset

data['BMI'] = data['weight'] / ((data['height']/100) **2) #creates a column called BMI using the weight and height columns


cardioData = data.drop(["id", "cardio","height", "weight"], axis=1).values #contains all the records for every column except 'id', 'cardio', 'height' and 'weight'
CardioData = data.drop(columns=colsToDrop).values
cardioValue = data["cardio"].values #contains all the values for the 'cardio' column

scaler = StandardScaler()
cardioData_train = scaler.fit_transform(cardioData)
#scales the data down using a standard scaler (Z-world)

userDataFrame = scaler.transform(userDataFrame.values)
# Train model
model = fullLogisticModel(lr=0.01, numIters=5000)
model.fit(cardioData_train, cardioValue)

# Test model
cardioValue_pred = model.predict(userDataFrame)
```

I can do a temporary test where I print the prediction score below to see if it matches the output of a test I did earlier in stage 1.

Firstly, I need to check if the program runs as intended when I run it to ensure the extra code didn't alter anything:



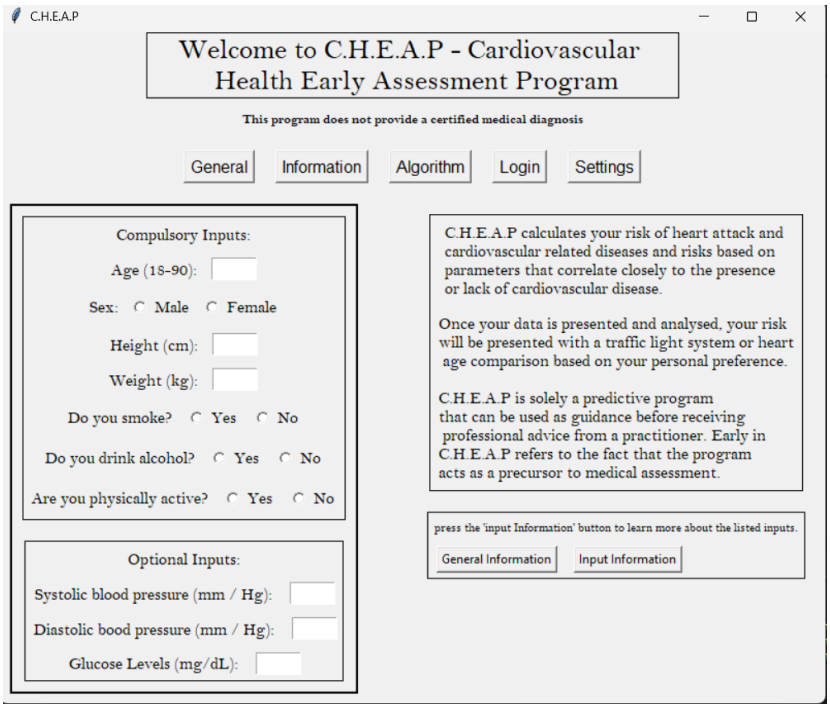
When I run the program, the error menu pops up, indicating the function is being called and instantly displaying the error message with all the errors added to the array.

To fix this I plan on making the inputs array a global variable outside of the function. This then allows me to edit the inputs variable without needing to return it. I can then create a Boolean variable which can be made true once I've made sure that the inputs are accurate and validated.

```
375
376 inputs = [-1] * 11
377 state = False
378 #Validation
```

```
    if errors:
        messagebox.showerror("Input Error", "\n".join(errors))
    else:
        state = True

#submit button
submitButton = tk.Button(optionalInputParameters, text="Submit", font=bodyFont, command = validateCompulsoryInputs)
submitButton.pack(pady=5)


if(state == True):
    userInputs = {
```

This then should fix the issue and upon running the code it should not display the errors message:



There is now no errors message being displayed.

This then means I can move onto providing inputs for the program and testing if it produces the same output as when I tested the regression model in stage 1. Here is the data I used on test 1 for stage 1:

age: 21296d (58.3y)

gender: Female (1)

height: 170cm

Weight: 75kg

systolic blood pressure: 120 mm Hg

diastolic blood pressure: 80 mm Hg

cholesterol: Normal (1) -120

glucose: Normal (1) - 60

smoke: No (0)

alcohol: No (0)

active: No (0)

cardiovascular disease: Not present (0)

Each of the selected values for the columns remain normal and within any bounds established throughout this stage of development. Due to this, they're a perfectly valid candidate for the valid data test.

After looking at these values, I need to code the inputs to match how the data is stored in the dataset. For example, the sex of the user is stored as a 1 (Female) or 2 (Male) and the cholesterol and glucose are stored in ranges. Luckily, I've already established the ranges in stage 1, meaning I can reimplement this:

```
elif cholesterol != "":
    if int(cholesterol) <= 200:
        inputs[9] = 1
    elif int(cholesterol) > 200 and cholesterol <= 240:
        inputs[9] = 2
    else:
        inputs[9] = 3

#glucose
glucose = glucoseEntry.get().strip()
if glucose != "" and not glucose.isdigit():
    errors.append("Glucose must be numeric.")
elif glucose  != "" and not(50 <= int(glucose)<= 160):
    errors.append("Glucose levels (LDL) must be between 50 and 160 mm/dL.")
elif glucose != "":
    if int(glucose) <= 100:
        inputs[10] = 1
    elif int(glucose) > 100 and glucose <= 125:
        inputs[10] = 2
    else:
        inputs[10] = 3

if errors:
```

```
else:
    if sexVar.get() == "Female":
        inputs[1] = 1
    else:
        inputs[1] = 2
```

```
if smokingVar.get() not in ["Yes", "No"]:
    errors.append("Please indicate if you smoke.")
else:
    if smokingVar.get() == "No":
        inputs[4] = 0
    else:
        inputs[4] = 1

#alcohol
if alcoholVar.get() not in ["Yes", "No"]:
    errors.append("Please indicate if you drink alcohol.")
else:
    if alcoholVar.get() == "No":
        inputs[5] = 0
    else:
        inputs[5] = 1

#activity
if activityVar.get() not in ["Yes", "No"]:
    errors.append("Please indicate if you are physically active.")
else:
    if activityVar.get() == "No":
        inputs[6] = 0
    else:
        inputs[6] = 1
```

Now that I've coded the data the same as how I did in stage 1, I can implement this test. This isn't the test that I need for my test plan as I haven't implemented the heart health output features yet, this to just check if the regression model is working.

```
# Test model
cardioValue_pred = model.predict(userDataFrame)
print(cardioValue_pred)
print("actual probability:",model.probability(userDataFrame))
```

Firstly, since when I did this test in stage 1 I only cared about the output of the regression model not the probability it deduced I need to input this again in the regression model to see what probability it provides. I can then check this against the UI that has the model integrated into it.

```
Time taken: 10.698505151484985
Accuracy: 72.73%
enter age (years):58
enter height (cm):170
enter weight (kg):75
enter gender (Female - 1/Male - 2 ):1
enter systolic blood pressure (mm Hg):120
enter diastolic blood pressure (mm Hg):80
enter cholesterol levels (mg/dL):120
enter glucose levels (mg/dL):60
enter smoking status (0 or 1):0
enter alcohol status (0 or 1):0
are you physically active (150minutes a week or more):0
do you have a cardiovascular disease (0 or 1):0
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
actual probability: [0.49245536]
[0]
actual cardio status: 0
```

The output was 0 with a probability of 0.492, so quite a boundary result that would've almost output 1, corresponding to the presence of cardiovascular disease. However, this isn't important, I just need to ensure that this output is matched when I use the UI.

To check if the code is being ran or not, I'll add an else statement that checks will display "did not run" if the regression model if statement was False.

```
53
54  else:
55      print("did not run")
56
```

Test:

```
PS C:\Users\james\Downloads\c
/Microsoft/WindowsApps/python
k-CW/mainMenuGUI.py"
did not run
```

After adding the input data, “did not run” was displayed, indicating that the regression model if statement wasn’t passed.

Instead of using if statements I can create a function that contains the regression model code, which can be called once the inputs are validated.

```
def regressionModel():
    userInputs = {
        "age": inputs[0],
        "gender": inputs[1],
        "height": inputs[2],
        "weight": inputs[3],
        "smoke": inputs[4],
        "alco": inputs[5],
        "active": inputs[6],
        "ap_hi": inputs[7],
        "ap_lo": inputs[8],
        "cholesterol": inputs[9],
        "gluc": inputs[10]
    }
    userDataFrame = pd.DataFrame(userInputs)

    userDataFrame['BMI'] = userDataFrame['weight'] / ((userDataFrame['height'

    colsToDrop = [col for col in userDataFrame.columns if userDataFrame[col].
    userDataFrame = userDataFrame.drop(columns=colsToDrop)
```

This is much better, as I can call this function once I know the input array stores validated inputs, compared to using more variables that store the states of the program.

```
490                 inputs[10] = 3
491
492         if errors:
493             messagebox.showerror("Input Error", "\n".join(errors))
494         else:
495             regressionModel()
496
```

After running the program and adding the values for the inputs I got this error message:

```
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2800.0_x64__qbz5n2kfra8p0\Lib\tkinter\__init__.py", line 1968, in __call__
    return self.func(*args)
           ^^^^^^^^^^^^^^^^
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\mainMenuGUI.py", line 495, in validateCompulsoryInputs
    regressionModel()
  File "c:\Users\james\Downloads\comp sci work\JamesCook24-Jack-CW\mainMenuGUI.py", line 516, in regressionModel
    userDataFrame = pd.DataFrame(userInputs)
                    ^^^^^^^^^^^^^^^^^^^^^^^^
```

After reviewing my code against the regression test I created in stage 1, the dictionary should have values stored in arrays using "[ ]".

Here's the fix:

```
def regressionModel():
    userInputs = {
        "age": [inputs[0]],
        "gender": [inputs[1]],
        "height": [inputs[2]],
        "weight": [inputs[3]],
        "smoke": [inputs[4]],
        "alco": [inputs[5]],
        "active": [inputs[6]],
        "ap_hi": [inputs[7]],
        "ap_lo": [inputs[8]],
        "cholesterol": [inputs[9]],
        "gluc": [inputs[10]]
    }
    userDataFrame = pd.DataFrame(userInputs)
```

Additionally, I've realised that the userDataFrame calculates the BMI using height and weight but doesn't drop these columns afterwards as it only searches for columns with a value of -1 in it.

```
    userDataFrame = pd.DataFrame(userInputs)

    userDataFrame['BMI'] = userDataFrame['weight'] / ((userDataFrame['height']/10

    colsToDrop = [col for col in userDataFrame.columns if userDataFrame[col].iloc
    userDataFrame = userDataFrame.drop(columns=colsToDrop)

    userDataFrame = userDataFrame.drop(["height", "weight"]).values
```

After running the program, it appears that the output was 0 as the cardiovascular status but the probability was ≈0.

```
k-CW/mainMenuGUI.py"
[0]
actual probability: [7.11890226e-16]
```

This indicates that somewhere the scaling is off, or the inputs are being altered incorrectly.

This is most likely due to a scaling issue, like the user Inputs data frame not being scaled properly causing issues. After checking the scaling, I applied to the data frames it seems that I need to fit and transform the frames rather than just transforming them:

```
scaler = StandardScaler()
cardioData_train = scaler.fit_transform(cardioData)
#scales the data down using a standard scaler (Z-world)

userDataFrame = scaler.fit_transform(userDataFrame)
# Train model
```

I also need to fix the way columns are dropped and then data frames are scaled. Currently, frames in the userDataFrame and the cardio data frame are dropped depending on what is left blank and what isn't but this results in column ordering that doesn't correspond with the cardio data, causing inputs checking against incorrect columns.

```
549     #scales the data down using a standard scaler (Z-world)
550
551     userDataFrame = userDataFrame[[
552     "age", "gender", "ap_hi", "ap_lo",
553     "cholesterol", "gluc",
554     "smoke", "alco", "active", "BMI"
555     ]]
556
557     userDataFrame = scaler.transform(userDataFrame)
558     # Train model
559     model = fullLogisticModel(lr=0.01, numIters=5000)
```

This fixes this issue, as it reorders the userDataFrame **before** it is scaled and transformed.

Test:

testingRegression.mp4

This video shows that the UI generated the same output and probability as the regression model in the terminal.

From the video you can see that it took 10 seconds to generate the probability from the UI, so ill need to add a loading pop up until it finishes. However, this feature isn't a priority, and I will implement once the traffic light system is set up. Firstly, I need to get the traffic light images I will use:

[41]

I will use these traffic lights to display the cardiovascular status of the user if they choose the traffic light system. I then don't need any images to do the heart age comparison for the user. Before I add any supporting information, I need to make it so that the correct information is displayed when the submit button is pressed.

What I can do is remove the right-side container and display the traffic light score there. Firstly, to determine which tier in the traffic light system the user is I can use the probability generated by the model.

```
49  def hideAllRightSide():
50      rightSideContainer1.pack_forget()
51      rightSideContainer2.pack_forget()
52      rightSideContainer3.pack_forget()
53      rightSideContainer4.pack_forget()
54      rightSideContainer5.pack_forget()
55
56  # FUNCTIONS ---------------------------------------------------------------
57
```

Firstly, I've created a function that hides all the potential right-side containers open which can be called when the submit button is pressed.

[41] (School, 2025)

```
#need to remove right side container and add the result frame
resultContainer = tk.Frame(root)

from tkinter import PhotoImage#need this to add traffic lights

greenImg = PhotoImage(file="greenLight.png")
amberImg = PhotoImage(file="amberLight.png")
redImg = PhotoImage(file="redLight.png")
#load the images correctly

trafficLightLabel = tk.Label(resultContainer)
trafficLightLabel.pack(padx=10, pady=10)


def showRiskLight(prob):
    # prob is the prediction probability from your model (0 to 1)

    hideAllRightSide()  # remove the right-side menu completely

    # choose light
    if prob < 0.33:
        trafficLightLabel.config(image=greenImg)
    elif prob < 0.66:
        trafficLightLabel.config(image=amberImg)
    else:
        trafficLightLabel.config(image=redImg)

    # show the result container
    resultContainer.pack(side="left", anchor="nw", padx=10, pady=10)
```

I can then use the images and the resultContainer I created to display the correct traffic light using the probability generated by the regression model. Since this isn't a binary output, I simply need to divide 1 by 3 to get the 3 ranges for the traffic lights.

These can then both be called as the regression model runs:

```
    # Test model
    cardioValue_pred = model.predict(userDataFrame)
    hideAllRightSide()
    actualProb = model.probability(userDataFrame)
    showRiskLight(actualProb)
    print(cardioValue_pred)
    print("actual probability:",actualProb)
```

After running the program, the amber traffic light was displayed, as it should.


```
[0]
actual probability: [0.4954207]
```

This matches the probability that I entered for the same record. I now need to work on the heart age comparison. To do this I plan to create a "healthy" and "average" user and fix all their measurements except the age. I can then apply this data frame to the regression model for all potential ages, generating their risk and storing it. This then allows me to compare the users risk with the sample user and compare their probabilities. I can then also drop the columns which aren't used from this healthy user so that it matches the data frame of the user of the program.

<u>"Healthy" person:</u>

age: varies

sex: Male – 2

systolic blood pressure: 120

diastolic blood pressure: 80

cholesterol: normal (1)- 150

glucose: normal (1) – 90

smoking: No (0)

alcohol: No (0)

active: Yes (1)

BMI: 22

The chosen values for the inputs all correspond to a healthy person. The sex of the sample is redundant as we proved in stage 1 that someone's sex doesn't significantly contribute to their cardiovascular status. The blood pressure measurements are the standard, healthy readings, and the cholesterol and glucose measurements fall into the normal range I established. This sample user also doesn't smoke or drink, which would result in an overall healthier cardiovascular system. Furthermore, a BMI of 22 is very standard and healthy.

```
#can create the heart age real age comparison
resultContainerHeartAge = tk.Frame(root)
n = 18
healthyUserProbabilities = []

while (n <= 90):
    healthyUser = {
        "age": [int(n*365)],
        "gender": [int(2)],
        "ap_hi": [int(120)],
        "ap_lo": [int(80)],
        "cholesterol": [int(150)],
        "gluc": [int(90)],
        "smoke": [int(0)],
        "alco": [int(0)],
        "active": [int(1)],
        "BMI": [int(22)]
    }
```

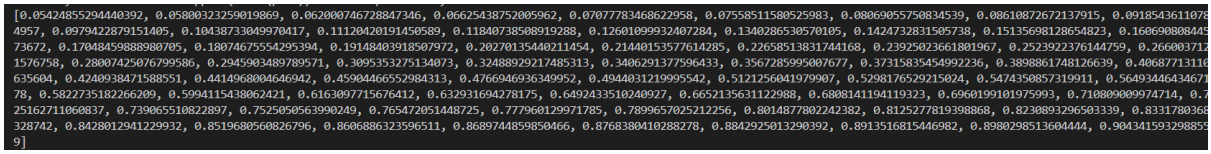
If I create a variable 'n', I compute the probabilities for all n values between 18 and 90 by increasing n by 1 until it is equal to 90. I can then store these probabilities and do a search to find the probability that is closest to the user's probability.

```
resultContainerHeartAge = tk.Frame(root)
n = 18
healthyUserProbabilities = []

while (n <= 90):
    healthyUser = {
        "age": [int(n*365)],
        "gender": [int(2)],
        "ap_hi": [int(120)],
        "ap_lo": [int(80)],
        "cholesterol": [int(1)],
        "gluc": [int(1)],
        "smoke": [int(0)],
        "alco": [int(0)],
        "active": [int(1)],
        "BMI": [int(22)]
    }

    healthyUserFrame = pd.DataFrame(healthyUser)
    #healthyUserFrame = healthyUserFrame.drop(colsToDrop, axis=1)
    healthyUserFrame = healthyUserFrame.reindex(columns = cardioData.columns)
    healthyUserFrame = scaler.transform(healthyUserFrame)


    prob = model.probability(healthyUserFrame)
    healthyUserProbabilities.append(float(prob))
    n += 1
```

This then should compute all the probabilities between ages 18 and 90 for the healthy sample user. I've added the line:

print(healthyUserProbabilites) to test this.

Test:

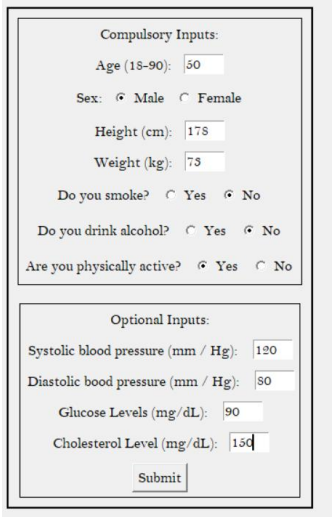


As you can see, it has printed the probabilities, and they seem to match what I've intended, as they gradually increase as the age of the healthy sample user increases.

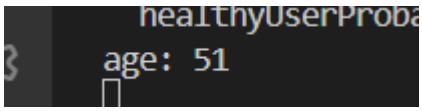
Now to search through these probabilities for the closest match I can use a linear search, where I try and minimise a variable- "difference". I can initially set this difference as 1, meaning that any difference I calculate will be less than this. Once the search is complete, I can use the index of the smallest difference to know what the heart age should be for that user.

```
        healthyUserProbabilities.append(float(prob)) #add the probability to the list
        n+=1 #increment n by 1
    print(healthyUserProbabilities)

    probDifference = 1
    heartAge = -1
    for i in range(0,72):
        newDifference = abs(actualProb - healthyUserProbabilities[i])
        if newDifference < probDifference:
            probDifference = newDifference
            heartAge = i + 18
    print("age:",heartAge)
```

To test if this is accurate, I will enter the exact same parameters of the healthy user for the age '50' to see if the heart age produced is the same or slightly off 50. A BMI of 22 can be generated from a user with a height of 178cm and weight of 73 kg, so I will use these as the inputs.



The age outputted was 51, which is only off by 1 age compared to the healthy user I created, so this strategy seems to work! Now that I have created the traffic light system and heart age comparison, I can work on making them both more presentable as well as finding the information that will support the diagnosis's.



Firstly, I need to write the paragraphs that will be placed next to the traffic lights and the heart age to real age comparison. Each traffic light will have its own designated short paragraph to outline what the colour means for their cardiovascular health.

Red: Your relative risk for developing a cardiovascular disease is high. If you haven't already, consult a medical practitioner for an assessment and potentially further guidance to better your cardiovascular health.

I need to ensure that the message is firm; although I don't want to intimidate my users, it is important that someone with a high risk of developing a cardiovascular disease is told that they should receive professional guidance.

Amber: Your relative risk for developing a cardiovascular disease should be brought to your attention. Although you don't fall within the dangerous range for cardiovascular disease development, you should actively work towards a healthier lifestyle for your future.

Green: Your relative risk for developing a cardiovascular disease is low. Your health markers do not correlate with measurements that factor into the development of cardiovascular diseases.

I can now write the paragraphs for the heart age comparisons. I plan on breaking it into three ranges.

1. $|\text{heart age} - \text{real age}| \leq 3$
2. $3 < |\text{heart age} - \text{real age}| <= 10$
3. $|\text{heart age} - \text{real age}\ | > 10$

With these ranges I can display different messages for the user, similarly to the traffic light system.

1. Your relative risk for developing cardiovascular disease is low, and your health markers do not correlate with measurements that factor into the development of cardiovascular diseases.
2. Your relative risk for developing a cardiovascular disease should be brought to your attention. Although you don't fall within the dangerous range for cardiovascular disease development, you should actively work towards a healthier lifestyle for your future.
3. Your relative risk for developing a cardiovascular disease is high. If you haven't already, consult a medical practitioner for an assessment and potentially further guidance to better your cardiovascular health.

I also need to add a button that swaps the display between traffic light and heart age comparison, which should be easy to do.

```
577     print("actual probability:",actualProb)
578
579     #can create the heart age real age comparison
580     resultContainer = tk.Frame(root,borderwidth = 1, relief = "solid")
581     resultContainerHeartAge = tk.Frame(resultContainer)
582
583
```

I can create the resultContainer and add the traffic light and heart age container to this container when they need to be displayed.

I can then use these functions (I will need to upgrade them for usability) to display the different outputs.

```
#the functions that are used to display the diagnosis
def showHeartAge():
    hideAllRightSide()
    resultContainerHeartAge.pack_forget()
    resultContainerTraffic.pack_forget()
    print(heartAge)


#add the traffic light to the label once
trafficLightLabel = tk.Label(resultContainerTraffic)
def showRiskLight():
    # prob is the prediction probability from your model (0 to 1)
    resultContainerHeartAge.pack_forget()
    resultContainerTraffic.pack_forget()
    hideAllRightSide()  # remove the right-side menu completely
    # show the result container
    # choose light
    if actualProb < 0.33:
        trafficLightLabel.config(image=greenImg)
    elif actualProb < 0.66:
        trafficLightLabel.config(image=amberImg)
    else:
        trafficLightLabel.config(image=redImg)

    trafficLightLabel.pack()
    resultContainerTraffic.pack(side="left",anchor = "w", padx=10, pady=10)
```

I'm only using these to test that the buttons work.

```
#adding buttons to swap displays

trafficLightButton = tk.Button(resultContainer, text = "Traffic Light Display", font = bodyFont, command = showRiskLight)
heartAgeButton = tk.Button(resultContainer,text = "Heart Age Display",font = bodyFont, command = showHeartAge)

trafficLightButton.pack(side="left", expand=True, padx=10,pady = 10)
heartAgeButton.pack(side="left", expand=True, padx=10,pady = 10)


#packing results container
resultContainer.pack(padx = 10,pady = 10)
root.mainloop()# starts a loop
```

I can then make the buttons that will display the traffic light system and the heart age comparison. I can now test that these buttons work and the functions I created display the different available outputs:

Test video: testingButtonsDisplay.mp4

The video shows that the buttons do as intended, I just need to change the functions so that the information displays better.

Firstly, I need to make the traffic lights display below the buttons, the buttons, then I can make it the default display when you receive your diagnosis. After this I can add the paragraphs for both the traffic lights and the heart age comparison.

To make the buttons appear above the traffic lights, I can create a frame for the buttons specifically and add this to the results container.

```
# The result container
resultContainerTraffic = tk.Frame(resultContainer)

# Create a row for the buttons
buttonRow = tk.Frame(resultContainer)
buttonRow.pack(fill="x", pady=10)
```

This then allows me to pack the traffic lights below the buttons:

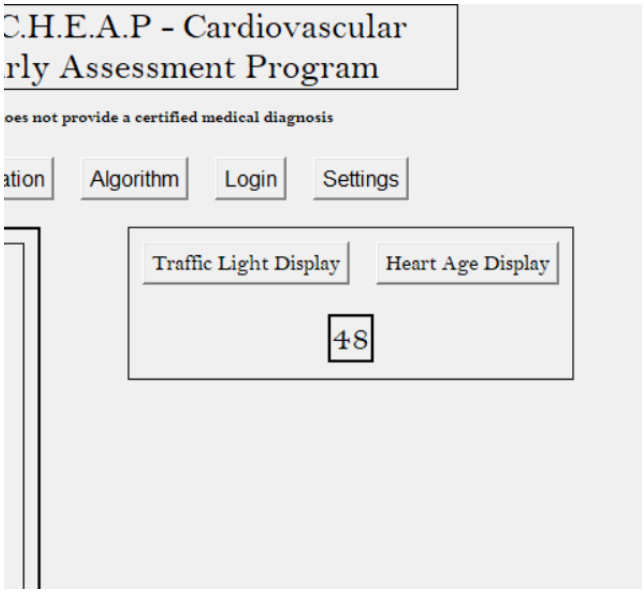
```
    # Pack below buttons
    resultContainerTraffic.pack(fill="both", pady=10)
    trafficLightLabel.pack(pady=10)


#buttons

trafficLightButton = tk.Button(buttonRow, text="Traffic Light Display",font=bodyFont, command=showRiskLight)
heartAgeButton = tk.Button(buttonRow, text="Heart Age Display", font=bodyFont, command=showHeartAge)

trafficLightButton.pack(side="left", expand=True, padx=10)
heartAgeButton.pack(side="left", expand=True, padx=10)


#packing main container
resultContainer.pack(padx=10, pady=10)
```

I can now make the age compared to heart age more presentable. I can display the person heart age using a large font size.

```
# Traffic light label inside its own container
trafficLightLabel = tk.Label(resultContainerTraffic)
heartAgeDisplay = tk.Label(resultContainerHeartAge, font=titleFont, borderwidth=2, relief="solid")


#functions for display

def showHeartAge():
    hideAllRightSide()
    resultContainerTraffic.pack_forget()
    resultContainerHeartAge.pack_forget()
    heartAgeDisplay.config(text = heartAge)

    #packing containers
    resultContainerHeartAge.pack(pady=10)
    heartAgeDisplay.pack(padx=2,pady=2)
```

Creating a label outside the function ensures that it doesn't get duplicated when the showHeartResult button is pressed. This button now displays the users heart age using the Title Font (largest font available).

Test:



I can now add the paragraphs I wrote earlier to the display. I can do this using if statements and the bounds I have already established. Doing this with the traffic light system is simple because I already have done range checks to determine what traffic light the user is.

```
# The result container
resultContainerTraffic = tk.Frame(resultContainer)

# Create a row for the buttons
buttonRow = tk.Frame(resultContainer)
buttonRow.pack(fill="x", pady=10)

from tkinter import PhotoImage

greenImg = PhotoImage(file="greenLight.png")
amberImg = PhotoImage(file="amberLight.png")
redImg = PhotoImage(file="redLight.png")

# Create horizontal row for (image + text)
lightRow = tk.Frame(resultContainerTraffic)

# Traffic light label inside its own container
trafficLightLabel = tk.Label(lightRow)
heartAgeDisplay = tk.Label(resultContainerHeartAge , font=titleFont, borderwidth=2, relief="solid")

#explanationLabelTraffic
explanationTrafficLabel = tk.Label(lightRow, font=bodyFont, borderwidth=1, relief="solid")
```

I can then create an explanationTrafficLabel with no text within in it, which I can then add as I configure this label.

```
def showRiskLight():
    hideAllRightSide()
    resultContainerHeartAge.pack_forget()

    # Reset previous contents
    resultContainerTraffic.pack_forget()

    # Pack below buttons
    resultContainerTraffic.pack(fill="both", pady=10)


    lightRow.pack(fill="x", pady=10)

    # Choose correct traffic light
    explanation = ""
    if actualProb < 0.33:
        trafficLightLabel.config(image=greenImg)
        explanation = "Your relative risk for developing a cardiovascular disease is low.\n Your health markers do not correlate with measurements\n that factor
    elif actualProb < 0.66:
        trafficLightLabel.config(image=amberImg)
        explanation = ""
    else:
        trafficLightLabel.config(image=redImg)
        explanation = ""

    # Pack image + explanation side by side
    trafficLightLabel.pack(side="left", padx=10)
    explanationTrafficLabel.config(text=explanation)
    explanationTrafficLabel.pack(side="left", padx=10)
```

Doing it this way ensures that labels are only created once, but the text can still be configured and updated. It also means the explanation can appear to the right of the traffic lights.

I can now add the other text for the other traffic lights and do something similar for the heart age comparison by adding the correct text.

```
def regressionModel():
    lightRow = tk.Frame(resultContainerTraffic)
    heartRow = tk.Frame(resultContainerHeartAge)
    # Traffic light label inside its own container
    trafficLightLabel = tk.Label(lightRow)
    heartAgeDisplay = tk.Label(heartRow , font=titleFont, borderwidth=2, relief="solid")

    #explanationLabelTraffic
    explanationTrafficLabel = tk.Label(lightRow, font=bodyFont, borderwidth=1, relief="solid")

    #explanation for Heart
    explanationHeartLabel = tk.Label(heartRow,font = bodyFont,borderwidth = 1, relief= "solid")

    #functions for display
    def showHeartAge():
        hideAllRightSide()
        resultContainerTraffic.pack_forget()
        resultContainerHeartAge.pack_forget()
        heartAgeDisplay.config(text = heartAge)
        explanation = ""
        if abs(heartAge - int(inputs[0])) <= 3:
            explanation = "Your relative risk for developing cardiovascular disease is low, and your health markers\n do not correlate with measurements that factor\n int
        elif abs(heartAge - int(inputs[0])) > 3 and abs(heartAge - int(inputs[0])) <= 10:
            explanation = "Your relative risk for developing a cardiovascular disease should be brought to your attention.\n Although you don't fall within the dangerous
        else:
            explanation = "Your relative risk for developing a cardiovascular disease is high. If you haven't already,\n consult a medical practitioner for an assessment

        #packing the row
        heartRow.pack(fill = "x",pady = 10)

        #packing containers
        explanationHeartLabel.config(text = explanation)
        resultContainerHeartAge.pack(pady=10)
        heartAgeDisplay.pack(side = "left", padx=2,pady=2)
        explanationHeartLabel.pack(side = "left", padx = 10)
```

There are three features that I now need to implement so that the program is more usable. Firstly, the general button should now become the button that opens the output page, as you may want to change the font size after you get the results. To do this I can create a general variable that acts as a flag to display either the general menu pre or post output calculation. This makes it very easy to adjust my code, as I only need to add some if statements compared to altering the structure of my entire code.

```
1   import tkinter as tk
2   import tkinter.font as tkFont
3   from tkinter import messagebox
4   import pandas as pd
5   from sklearn.model_selection import train_test_split
6   from sklearn.preprocessing import StandardScaler
7   from sklearn.metrics import accuracy_score
8
9   # FUNCTIONS ------------------------------------------------------------------
10  hasModelRan = False
```

As you can see, I've created this flag at the start of the program.

```
def generalButtonCommand():
    #forgets all the menus except the general menu
    if(hasModelRan == False): #checks if the regression model has ran
        rightSideContainer1.pack_forget()
        rightSideContainer2.pack_forget()
        rightSideContainer3.pack_forget()
        rightSideContainer4.pack_forget()
        rightSideContainer5.pack_forget()
        resultContainer.pack_forget()

        rightSideContainer1.pack(side="right", anchor="ne", padx=10, pady=10)#packs the general menu
    else:
        rightSideContainer1.pack_forget()
        rightSideContainer2.pack_forget()
        rightSideContainer3.pack_forget()
        rightSideContainer4.pack_forget()
        rightSideContainer5.pack_forget()
        resultContainer.pack_forget()

        resultContainer.pack(padx=10, pady=10)
```

I've then altered the generalButtonCommand function so that it utilises this flag, and changes functionality depending on the flags state. Therefore, once the regression model is called, I can change the state of this flag.

```
def regressionModel():
    global hasModelRan
    hasModelRan = True
```

Additionally, I want to force full screen on the user after the output is revealed. This is because the original window becomes too small to display all the necessary information, and there are still images and videos that need to be added to the display. I can do this with a single Tkinter line:

```
        messagebox.showerror("Input Error", "\n".join(errors))
    else:
        regressionModel()
        root.state("zoomed")
```

This then will make the menu full screen after regressionModel is called.

Lastly, I need to add a loading screen as you load the regression model, as this can take up to 10 seconds.

```
else:
    messagebox.showinfo("Loading...", "Calculating your diagnosis. Please wait.")
    regressionModel()
    root.state("zoomed")
```

This will inform the user that the regression model is working to calculate their cardiovascular diagnosis, rather than them being unsure what happened.

I can test all these features now to see if they work as intended.

Test video: accessibilityTest.mp4

As you can see, the features all work as intended! I now need to make the traffic light display call upon the regression model running.

```
trafficLightButton.pack(side="left", expand=True, padx=10)
heartAgeButton.pack(side="left", expand=True, padx=10)

showRiskLight()
#packing main container
resultContainer.pack(padx=10, pady=10)
```

This can easily be done by calling the function after I wrote it, making it the default display.

Additionally, I was originally going to add a paragraph that explains what medical feature likely contributed most to your diagnosis. However, I will scrap this feature. Firstly, the program is already a predictor not a certified diagnosis system, outlining a specific feature for the user when the diagnosis is supposed to act as a guide could only be damaging and misleading. On top of that, a lot of amber and red-light diagnosis will be due to multiple factors that are all outside the healthy ranges; the human body is very complex so saying that 1 feature contributed to their diagnosis would also be misleading.

Lastly, I need to prioritise essential features that are more impactful for the game's development. The diary tool created in the next stage is essential for the user's progress post diagnosis, which is more important than potentially providing closure for their diagnosis.

Since I'm not including this feature, I want to add the video listed below to the right of the short explanation.

I will use the following YouTube video to detail the risk factors and solutions to potential risk factors of cardiovascular disease. It's important that I reference this video in my program to prevent breaking any copyright infringement.

https://www.youtube.com/watch?v=h413NHcx7eo

This code then creates the frames for the videos:

```
#video player

# Heart Row
heartVideoFrame = tk.Frame(heartRow, borderwidth=1, relief="solid")
heartVideoPanel = tk.Frame(heartVideoFrame, width=300, height=180, bg="black")
heartVideoPanel.pack(pady=10, padx=10)

heartVideoFrame.pack(pady=10, padx=10, side="right")

heartVLC = vlc.Instance("--no-xlib")
heartPlayer = heartVLC.media_player_new()
heartPlayer.set_hwnd(heartVideoPanel.winfo_id())

# Light Row
lightVideoFrame = tk.Frame(lightRow, borderwidth=1, relief="solid")
lightVideoPanel = tk.Frame(lightVideoFrame, width=300, height=180, bg="black")
lightVideoPanel.pack(pady=10, padx=10)

lightVideoFrame.pack(pady=10, padx=10, side="right")

lightVLC = vlc.Instance("--no-xlib")
lightPlayer = lightVLC.media_player_new()
lightPlayer.set_hwnd(lightVideoPanel.winfo_id())
```

Now I can use the VLC module[42] to import the video and run it using buttons that can play, pause and stop the video. This is very intuitive for the user, ensuring that the program remains user-friendly.

```
# Playback functions for heart row
def playHeartVideo():
    media = heartVLC.media_new("cardiovascularVideo.mp4")
    heartPlayer.set_media(media)
    heartPlayer.play()

def pauseHeartVideo():
    heartPlayer.pause()

def stopHeartVideo():
    heartPlayer.stop()

# Playback functions for light row
def playLightVideo():
    media = lightVLC.media_new("cardiovascularVideo.mp4")
    lightPlayer.set_media(media)
    lightPlayer.play()

def pauseLightVideo():
    lightPlayer.pause()

def stopLightVideo():
    lightPlayer.stop()

# Buttons
tk.Button(heartVideoFrame, text="Play", font=bodyFont, command=playHeartVideo).pack(side="left", padx=5)
tk.Button(heartVideoFrame, text="Pause", font=bodyFont, command=pauseHeartVideo).pack(side="left", padx=5)
tk.Button(heartVideoFrame, text="Stop", font=bodyFont, command=stopHeartVideo).pack(side="left", padx=5)

tk.Button(lightVideoFrame, text="Play", font=bodyFont, command=playLightVideo).pack(side="left", padx=5)
tk.Button(lightVideoFrame, text="Pause", font=bodyFont, command=pauseLightVideo).pack(side="left", padx=5)
tk.Button(lightVideoFrame, text="Stop", font=bodyFont, command=stopLightVideo).pack(side="left", padx=5)
```

I can now test if this works through the usage of a screen recording. I want both the videos to appear in either display to the right of the diagnosis.

Test Video: cardioVideoTest.mp4

The video shows that the videos display correctly in the window.

[42] (GeeksforGeeks, 2025)

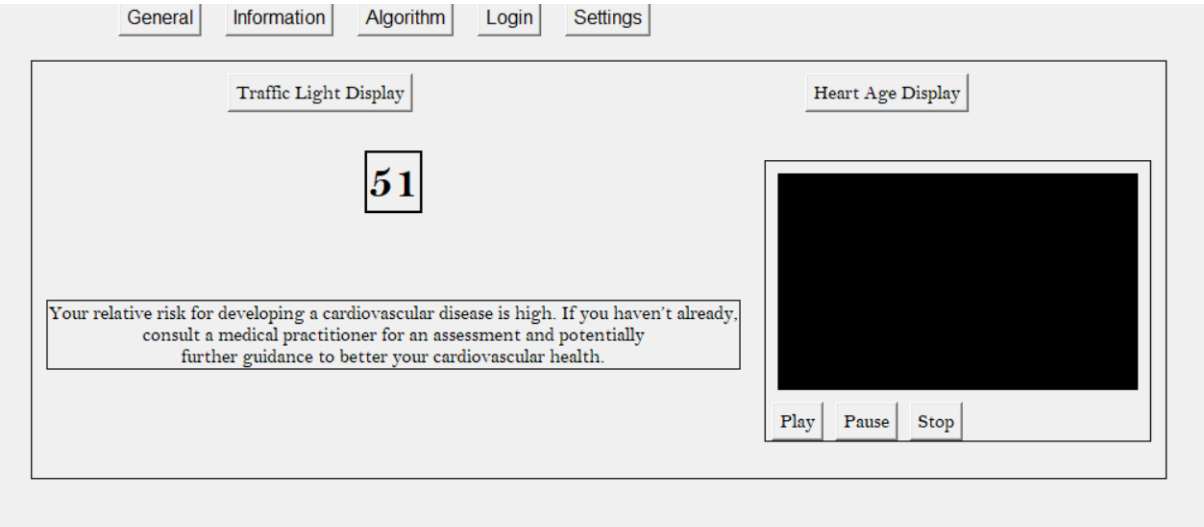
I should now add a paragraph that accompanies this video. I want to add it below the video to prevent clustering of texts to keep the UI user-friendly and intuitive. I also want to make the heart age comparison number larger and appear above the text. Since it's not as long as the traffic light image, it doesn't have to be to the left of the text.

```
#packing containers
explanationHeartLabel.config(text = explanation)
resultContainerHeartAge.pack(pady=10,padx = 2 )
heartAgeDisplay.pack(side = "top", padx=2,pady=2)
explanationHeartLabel.pack(side = "left", padx = 10)
```

```
smallFont = tkFont.Font(family= "Bell MT", size=9)
smallFontBold = tkFont.Font(family="Bell MT", size=9,weight = "bold")
HeartDisplayFont = tkFont.Font(family = "Bell MT",size = 30,weight = "bold")
```

```
fontSizes = [titleFont.cget("size"),bodyFont.cget("size"),smallFont.cget("size"),smallFontBold.cget("size"),HeartDisplayFont.cget("size")]

if userValidation ==True:
    if(fontSizes[1] > userInput):
        #need to reduce font sizes
        difference = fontSizes[1] - userInput
        for i in range(len(fontSizes)):
            fontSizes[i] -= difference
    else:
        difference = userInput - fontSizes[1]
        for i in range(len(fontSizes)):
            fontSizes[i] += difference

titleFont.config(size = fontSizes[0])
bodyFont.config(size = fontSizes[1])
smallFont.config(size = fontSizes[2])
smallFontBold.config(size = fontSizes[3])
HeartDisplayFont.config(size = fontSizes[4])
```

This then allows it to be dynamically changed but it will also appear larger.



The text I will use is below:

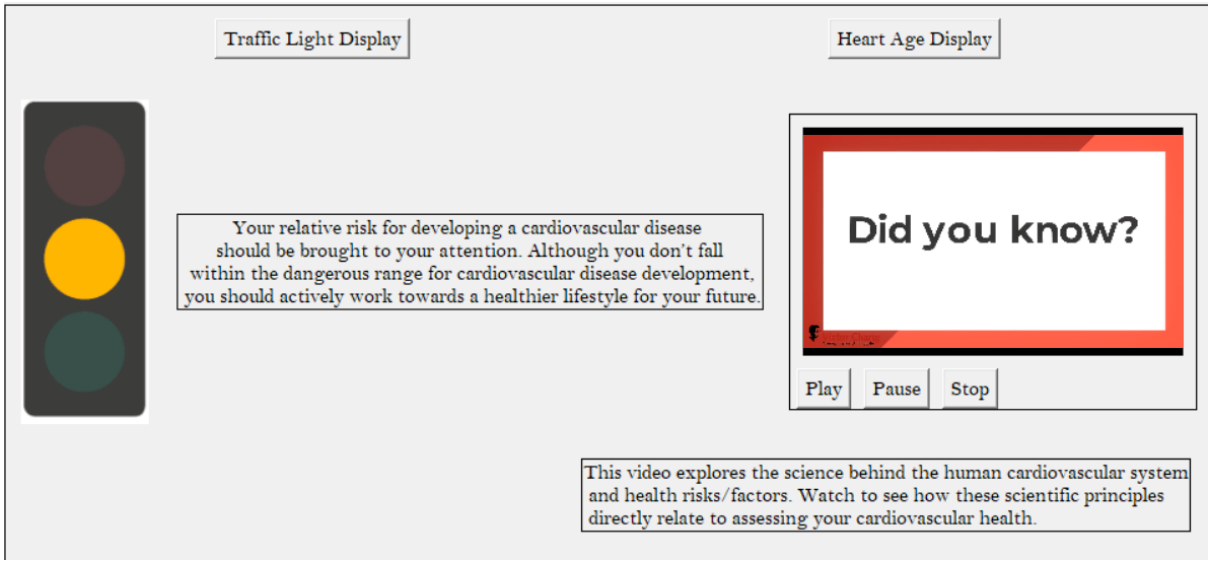
This video explores the science behind the human cardiovascular system and health risks/factors. Watch to see how these scientific principles directly relate to assessing your cardiovascular health.

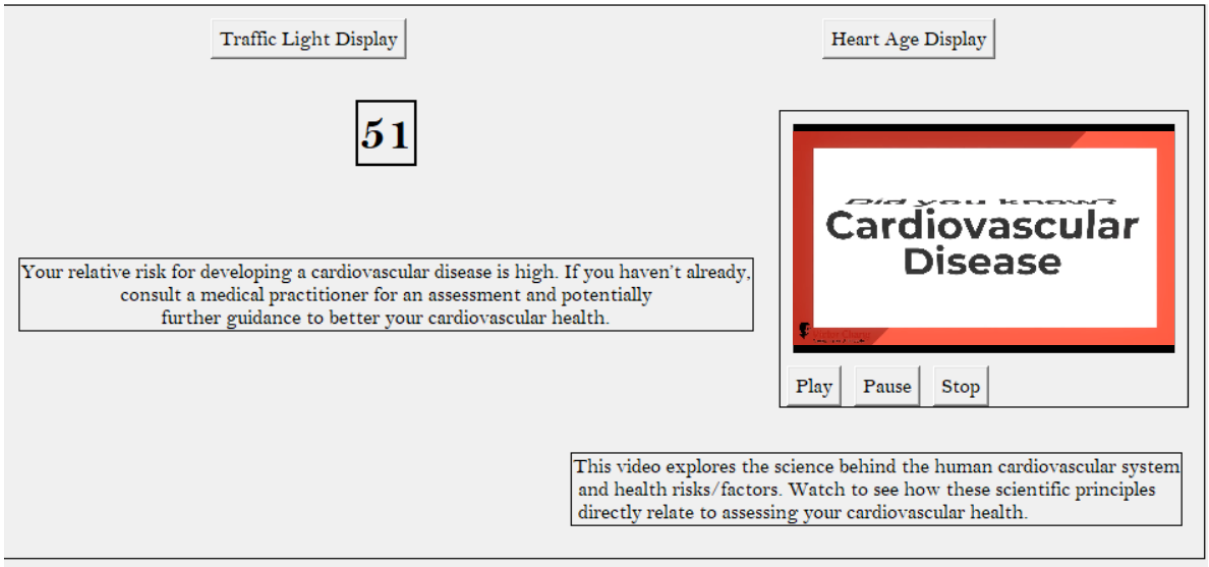
I can then add this into a frame that is added to the two result containers for heart age and the traffic lights:

```
#supporting text
videoText = "This video explores the science behind the human cardiovascular system\n and health risks/factors. Watch to see how these scientific principles\n directly relate to a


videoRowHeart = tk.Frame(resultContainerHeartAge)
videoRowLight = tk.Frame(resultContainerTraffic)

videoRowHeart.pack(padx=10,pady=10,side ="bottom",anchor = "e")
videoRowLight.pack(padx=10,pady=10,side = "right")

heartVideoTextLabel = tk.Label(
    videoRowHeart,
    text=videoText,
    font=bodyFont,
    justify="left",
    borderwidth = 1,
    relief = "solid"
)
heartVideoTextLabel.pack(fill = "x", padx=5, pady=5,side = "right")

# --- For Traffic Light Display --


lightVideoTextLabel = tk.Label(
    videoRowLight,
    text=videoText,
    font=bodyFont,
    justify="left",
    borderwidth = 1,
    relief = "solid"
)
lightVideoTextLabel.pack(fill = "x",padx=5, pady=5,side = "right")


root.mainloop()
```

This then outputs the text as intended, below the video.

Test:





I now want to implement the diary function, which comes under stage 4. To conclude stage 3, I will test all the features using the test plan, but I need to alter it considering I don't outline what may be the 'contributing' factors to their diagnosis.

All I need to do is remove the tests that mention testing contributing factors, which are the test numbers 3, 4, 5, 6, 7 and 8.

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data to a record in the data set (not boundary and healthy)	Valid	Heart age <= real heart age or green on traffic light system, with no outlying factor contributing to their health	
2.	Enter exact data to a record in the data set (not boundary and unhealthy).	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factor led to this or if none are outlying.	

Test 1:

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data to a record in the data set (not boundary and healthy)	Valid	Heart age <= real heart age or green on traffic light system, with no outlying factor contributing to their health	

For this test I found a record in the data set that had standard, healthy metrics for all the inputs:

age: 10950d (30y)

gender: Male (2)

height: 180 cm

Weight: 72 kg

systolic blood pressure: 120 mm Hg

diastolic blood pressure: 80 mm Hg

cholesterol: 1

glucose: 1

smoke: 0

alcohol: 0

active: 1

Test Video: stage4Test1.mp4

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data to a record in the data set (not boundary and healthy)	Valid	Heart age <= real heart age or green on traffic light system, with no outlying factor contributing to their health	Heart Age matched the real age and the traffic light appeared green.

For test 2 I can do something very similar, just by using an unhealthy record. This person has a relatively high BMI and their blood pressure is definitely very high, making them a standard unhealthy record to utilise.

Record:

age: 48

gender: Female (1)

height: 165

Weight: 80

systolic blood pressure: 150

diastolic blood pressure: 100

cholesterol:

glucose: Normal (1)

smoke:0

alcohol: 1

active: 0

Test Video: stage4Test2.mp4

Test no.	Description + test data	Type of test	Expected outcome	outcome
2.	Enter exact data to a record in the data set (not boundary and unhealthy).	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factor led to this or if none are outlying.	Heart age was very high (90) and traffic light returned as red.

Since this concludes my third stage of development with the following test plan provided, I can move onto creating the diary function which ties into this menu and stage.

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Enter exact data to a record in the data set (not boundary and healthy)	Valid	Heart age <= real heart age or green on traffic light system, with no outlying factor contributing to their health	Heart Age matched the real age and the traffic light appeared green.
2.	Enter exact data to a record in the data set (not boundary and unhealthy).	Valid	Heart age > real heart age or yellow/red on traffic light system. Identify which health factor led to this or if none are outlying.	Heart age was very high (90) and traffic light returned as red.

Stage 3 References

41 - intensiveblue (2022). *Traffic Light Sequence in the UK Explained*. [online] Intensive Driving School. Available at: https://www.intensive-driving-school.co.uk/traffic-lights-explained. [Accessed 27th Nov. 2025]

42- GeeksforGeeks (2020). *VLC module in Python An Introduction*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/vlc-module-in-python-an-introduction/ [Accessed 30 Nov. 2025].

Stage 4 – Create the diary tool (15th of November)

This stage involves creating a tool that allows the user to track their progress as they hopefully work towards a better lifestyle by following the advice given. The tool should show the measurements entered by the user and allow them to track their progress by seeing how measurements change over time alongside their diagnosis. It would use files on the persons computer which can be saved, accessed and updated easily.

Features I want to implement:

1. Allow the user to create a diary
2. Places their starting measurements in the diary as it's created
3. Saves whenever a user adds/ removes anything
4. Contains general diet and lifestyle advice initially based on their diagnosis

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	User selects to create a diary	Valid	If the user chooses to create the diary, the file is created accordingly	
1b.	User selects to create a diary	Valid	Upon opening the diary, the measurements and advice is already in it	
2.	User doesn't want to create a diary	Valid	No file is created	
3.	Add text to the diary	Valid	It saves next time and opens	
4.	Delete from the diary	Valid	It removes the text from the diary	

Now that I am developing the diary tool, I can create a sign-up section after you receive your diagnosis. This allows the user to create a login for the diary section of the program.

Firstly, I need to write a paragraph that will outline why you should use the diary tool and add this to a frame that will go below the display outputs. This is the paragraph I will use:

“Opening a diary within C.H.E.A.P is a great way to track your progress as you work towards a healthier lifestyle. C.H.E.A.P will provide the data you entered within your diary as well as a general plan for you.”

I can then create a username label with an entry box and the same for a password, allowing me to take in information from the user to have them log into the program.

```
#adding the area where you can sign up for diary
diaryToolFrame = tk.Frame(resultContainer, borderwidth=1, relief="solid")

# At the end of regressionModel()
diaryToolFrame.pack(padx=10, pady=10, side="bottom", fill="x")

diaryExplanationLabel = tk.Label(diaryToolFrame, text="Opening a diary within C.H.E.A.P is a great way to track your progress\n as you work towards a healthier lifest

# Create a frame for the input fields
diaryInputFrame = tk.Frame(diaryToolFrame, borderwidth=1, relief="solid")

# Username input
usernameFrame = tk.Frame(diaryInputFrame)
usernameLabel = tk.Label(usernameFrame, text="Username:", font=bodyFont)
usernameEntry = tk.Entry(usernameFrame, width=20, font=bodyFont)

usernameFrame.pack(pady=5)
usernameLabel.pack(side="left", padx=5)
usernameEntry.pack(side="left", padx=5)

# Password input
passwordFrame = tk.Frame(diaryInputFrame)
passwordLabel = tk.Label(passwordFrame, text="Password:", font=bodyFont)
passwordEntry = tk.Entry(passwordFrame, width=20, font=bodyFont)

passwordFrame.pack(pady=5)
passwordLabel.pack(side="left", padx=5)
passwordEntry.pack(side="left", padx=5)



# Sign up button
```

I then need to make a function for the sign-up button. This function should make sure the password and username are both present, with a minimum character count being present too.

```
def signUpCommand():
    errors = []

    userName = usernameEntry.get().strip()
    if(userName == ""):
        errors.append("User name must be present.")
    elif (len(userName) <= 5):
        errors.append("User name must be greater than 5 characters.")

    password = passwordEntry.get().strip()

    if(password == ""):
        errors.append("Password must be present.")
    elif (len(password) <= 5):
        errors.append("Password must be greater than 5 characters.")

    if errors:
        messagebox.showerror("Sign Up Error", "\n".join(errors))
    else:
        messagebox.showinfo("Success", "Information Saved.")
        global userUsername
        global userPassword
        userUsername = userName
        userPassword = password
```

As you can see, the sign-up button ensures that the user’s password and username are both greater than 5 characters. I don’t want to implement any characters checks or strength necessities as older people attempting to use the diary tool may want a simpler password that they can remember easily. Additionally, the information within the dairy isn’t very sensitive so a very secure diary system also isn’t necessary.

```
# Sign up button
signUpButton = tk.Button(diaryInputFrame, text="Sign Up for Diary", font=bodyFont, command = signUpCommand)
signUpButton.pack(pady=10)

# Pack the diary components
diaryExplanationLabel.pack(padx=5, pady=5, side="right")
diaryInputFrame.pack(padx=5, pady=5, side="left")
```

I can test that this button works, by first adding no information, and then data where the length is less than 5 and then a valid username and password.

Test video: signUpTest.mp4

This video shows that the command works as intended. I can now move onto the login tab, which is where the diary will be accessed and used.

I can create this tab the same way I created the resultContainer1.

I can also create similar labels and entry fields for the login page, as I need to collect the users input and test it against what the user inputted during sign up.

After this, I just need to check that the global variables that store the user's username and password are not empty. Since I use validation to check if they're acceptable, an empty variable indicates that they have either not signed up or failed to sign up correctly.

```
userUsername = ""
userPassword = ""

loginTitle = tk.Label(rightSideContainer4, text="Login to Diary", font=titleFont)
loginTitle.pack(pady=10,padx=10)#create the title for the diary login

usernameLoginFrame = tk.Frame(rightSideContainer4)
usernameLoginLabel = tk.Label(usernameLoginFrame, text="Username:", font=bodyFont)
usernameLoginEntry = tk.Entry(usernameLoginFrame, width=20, font=bodyFont)
usernameLoginFrame.pack(pady=5)
usernameLoginLabel.pack(side="left", padx=5)
usernameLoginEntry.pack(side="left", padx=5)
#create similar entry labels as the sign up area


passwordLoginFrame = tk.Frame(rightSideContainer4)
passwordLoginLabel = tk.Label(passwordLoginFrame, text="Password:", font=bodyFont)
passwordLoginEntry = tk.Entry(passwordLoginFrame, width=20, font=bodyFont, show="*")
passwordLoginFrame.pack(pady=5)
passwordLoginLabel.pack(side="left", padx=5)
passwordLoginEntry.pack(side="left", padx=5)

#check if the variables are not empty
def attemptLogin():
    AttemptUsername = usernameLoginEntry.get().strip()
    AttemptPassword = passwordLoginEntry.get().strip()
    if userUsername != "" and userPassword != "" and AttemptUsername == userUsername and AttemptPassword == userPassword:
        messagebox.showinfo("Success", "Login successful!")

    else:
        messagebox.showerror("Error", "Invalid username or password.")
```

I can then make the login button work as always intended.

```
# Add login functionality
def loginButtonCommand():
    # Hide all other containers and show login container
    hideAllRightSide()
    rightSideContainer1.pack_forget()
    rightSideContainer2.pack_forget()
    rightSideContainer3.pack_forget()
    rightSideContainer5.pack_forget()
    resultContainer.pack_forget()
    rightSideContainer4.pack(side="right", anchor="ne", padx=10, pady=10)
```

I can now test that this feature works. I don't need to use the test plan yet as the program never actually utilises a text file, but I can see if the entry label checks are working as intended.

Test video: loginTest.mp4

This test shows that the login menu is working as intended.

Lastly, I need to create the diary. This can be done by reading and writing to files using python, which is simple to implement. I can also write the users markers into the file automatically as week as a guide to a plan to better your cardiovascular health. I can also make it so that you can save entries and view what the previous entry was by opening the txt file under someone's username.

```
def showDiaryContent():
    # Clear the login frame first
    for widget in rightSideContainer4.winfo_children():
        widget.destroy()

    # Create main container for diary with two columns
    mainDiaryFrame = tk.Frame(rightSideContainer4)
    mainDiaryFrame.pack(fill="both", expand=True, padx=10, pady=10)

    # Left column for health markers
    leftColumn = tk.Frame(mainDiaryFrame, borderwidth=1, relief="solid")
    leftColumn.pack(side="left", fill="both", expand=True, padx=5, pady=5)

    # Right column for diary
    rightColumn = tk.Frame(mainDiaryFrame, borderwidth=1, relief="solid")
    rightColumn.pack(side="right", fill="both", expand=True, padx=5, pady=5)

    # Health markers in left column
    healthTitle = tk.Label(leftColumn, text="Your Health Summary", font=titleFont)
    healthTitle.pack(pady=10)

    healthDataText = f"""
    Age: {int(inputs[0]/365)} years
    Sex: {'Male' if inputs[1] == 2 else 'Female'}
    Height: {inputs[2]} cm
    Weight: {inputs[3]} kg
    BMI: {inputs[3] / ((inputs[2]/100) ** 2):.1f}
    Smoking: {'Yes' if inputs[4] == 1 else 'No'}
    Alcohol: {'Yes' if inputs[5] == 1 else 'No'}
    Physical Activity: {'Yes' if inputs[6] == 1 else 'No'}
    Guide: {"https://www.health.harvard.edu/healthbeat/10-small-steps-for-better-heart-health"}
    """
```

I plan on using this website: https://www.health.harvard.edu/healthbeat/10-small-steps-for-better-heart-health As a source to guide users towards a healthier cardiovascular system.
I must separately check if the optional inputs are inputted for this health display box, but this is simple as I can create individual labels for each of these inputs and add it to the leftColumn frame.

After this I can create a label for the title of the actual diary tool. This can go in a separate column so that the health metric appears on the left, and the dairy functionality appears on the right.

```
    healthLabel = tk.Label(leftColumn, text=healthDataText, font=bodyFont, justify="left")
    healthLabel.pack(pady=10, padx=10)

    # Add some spacing and additional health information if available
    if inputs[7] != -1:  # Systolic BP
        bpText = f"Systolic BP: {inputs[7]} mm/Hg\n"
        if inputs[8] != -1:  # Diastolic BP
            bpText += f"Diastolic BP: {inputs[8]} mm/Hg"
        bpLabel = tk.Label(leftColumn, text=bpText, font=bodyFont, justify="left")
        bpLabel.pack(pady=5, padx=10)

    if inputs[9] != -1:  # Cholesterol
        cholLevels = ["Normal", "Above Normal", "Well Above Normal"]
        cholLabel = tk.Label(leftColumn, text=f"Cholesterol: {cholLevels[inputs[9]-1]}", font=bodyFont, justify="left")
        cholLabel.pack(pady=5, padx=10)

    if inputs[10] != -1:  # Glucose
        glucLevels = ["Normal", "Above Normal", "Well Above Normal"]
        glucLabel = tk.Label(leftColumn, text=f"Glucose: {glucLevels[inputs[10]-1]}", font=bodyFont, justify="left")
        glucLabel.pack(pady=5, padx=10)

    # Diary content in right column
    diaryTitle = tk.Label(rightColumn, text="Your Health Diary", font=titleFont)
    diaryTitle.pack(pady=10)

    diaryPrompt = tk.Label(rightColumn, text="How are you feeling today? Track your health journey:", font=bodyFont)
    diaryPrompt.pack(pady=5)

    # Create a frame for the text area and scrollbar
    textFrame = tk.Frame(rightColumn)
    textFrame.pack(pady=10, padx=10, fill="both", expand=True)

    # Create scrollbar
    scrollbar = tk.Scrollbar(textFrame)
    scrollbar.pack(side="right", fill="y")
```

What I need to do is create three buttons to make the text box functional.

1. I need to create a saveEntry function that writes what you most recently wrote into a file, essentially saving it. This then feeds into the next button:

2. Viewing previous entries. The most important feature of a diary tool is being able to view previous entries, allowing you to track your progress or reflect on what you were talking about in previous events. This button will allow you to view the date and time of a entry as well as the contents of the entry.

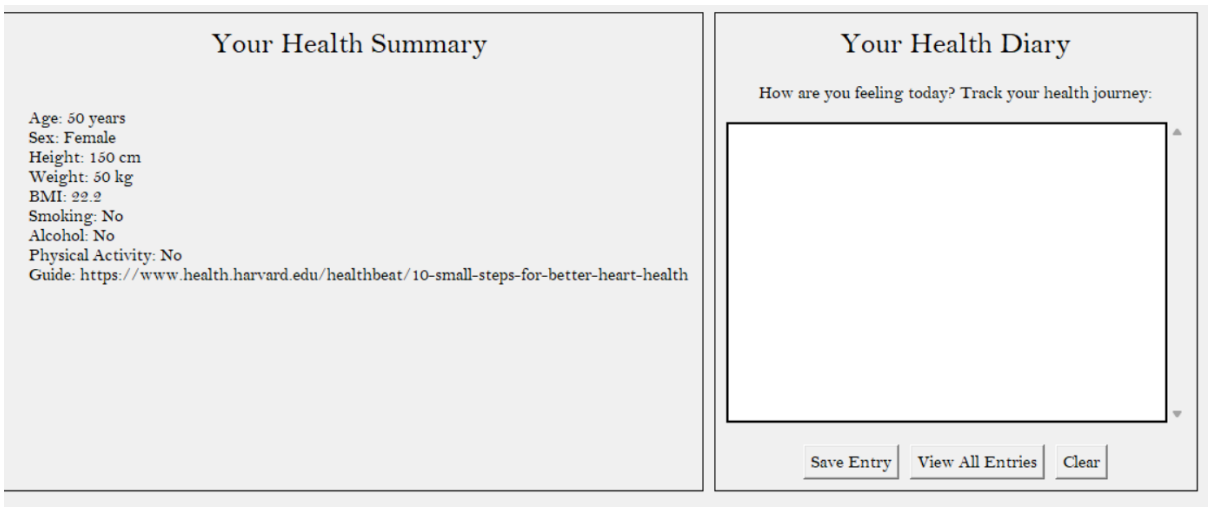
3. Lastly, the clear button will remove what you currently have written in the text box. This just makes it easier to remove anything instead of deleting it yourself.

I want to implement a scrolling text frame, as you may have a lot to say when adding an entry. I also need to make the view entry tab utilise this scroll bar[43].

[43] (GeeksrforGeeks, 2025)

```
# Create text widget for diary entry
global diaryText
diaryText = tk.Text(textFrame,
    width=50,
    height=15,
    font=bodyFont,
    yscrollcommand=scrollbar.set,
    wrap="word",
    bg="white",
    fg="black",
    borderwidth=2,
    relief="solid")
diaryText.pack(side="left", fill="both", expand=True)

# Configure scrollbar
scrollbar.config(command=diaryText.yview)

# Load previous diary entries
loadDiaryEntries()

# Buttons frame
diaryButtonsFrame = tk.Frame(rightColumn)
diaryButtonsFrame.pack(pady=10)

saveButton = tk.Button(diaryButtonsFrame, text="Save Entry", font=bodyFont, command=saveDiaryEntry)
saveButton.pack(side="left", padx=5)

viewButton = tk.Button(diaryButtonsFrame, text="View All Entries", font=bodyFont, command=viewAllEntries)
viewButton.pack(side="left", padx=5)

clearButton = tk.Button(diaryButtonsFrame, text="Clear", font=bodyFont, command=clearDiaryEntry)
clearButton.pack(side="left", padx=5)
```

Test:



As you can see, the health markers appear to the left of the display, and the diary tool appears on the right, with all the buttons available.

The save diary function should use the date time library to extract the current date and time, which can then be added to the saved entry. Additionally, I need to check if the entry is empty, which is simple as I can extract the content within the box using tkinter functions.

After this I can create a filename using the user's username. Using this filename, I can create a new file if one doesn't exist, using python file handling.

This can be seen below:

```
def saveDiaryEntry():
    entryContent = diaryText.get("1.0", tk.END).strip()
    if not entryContent:
        messagebox.showwarning("Empty Entry", "Please write something before saving.")
        return

    # Create filename based on username
    filename = f"{userUsername}_diary.txt"

    # Get current timestamp
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # Prepare entry with timestamp
    diaryEntry = f"\n--- Entry on {timestamp} ---\n{entryContent}\n"

    try:
        # Append to file (create if doesn't exist)
        with open(filename, "a", encoding="utf-8") as file:
            file.write(diaryEntry)

        messagebox.showinfo("Success", "Diary entry saved successfully!")
        diaryText.delete("1.0", tk.END)  # Clear the text area
    except Exception as e:
        messagebox.showerror("Error", f"Could not save diary entry: {str(e)}")
```

Next, the button that allows you to see previous entries should be implemented. Since the user may have not added anything previously to the file, I need to make it so that the functions checks if the file exists, and if not, the function should display a message that informs them of this.

If this check is passed, I can then attempt to read the information from the file and display this in the window. Additionally, this requires python file handling and using buttons that manipulate the window that I will create.

```
def viewAllEntries():
    """Display all diary entries in a new window"""
    filename = f"{userUsername}_diary.txt"
    if not os.path.exists(filename):
        messagebox.showinfo("No Entries", "You haven't written any diary entries yet.")
        return

    try:
        with open(filename, "r", encoding="utf-8") as file:
            content = file.read()

        # Create new window to display all entries
        entries_window = tk.Toplevel(root)
        entries_window.title("Your Diary Entries")
        entries_window.geometry("600x400")

        # Create a frame for the text area and scrollbar
        text_frame = tk.Frame(entries_window)
        text_frame.pack(pady=10, padx=10, fill="both", expand=True)

        # Create scrollbar
        scrollbar = tk.Scrollbar(text_frame)
        scrollbar.pack(side="right", fill="y")

        # Create text widget for displaying entries
        entries_text = tk.Text(text_frame,
            width=70,
            height=20,
            font=bodyFont,
            yscrollcommand=scrollbar.set,
            wrap="word",
            bg="white",
            fg="black",
            borderwidth=2,
            relief="solid")
        entries_text.pack(side="left", fill="both", expand=True)

        # Configure scrollbar
        scrollbar.config(command=entries_text.yview)

        # Insert content and make read-only
        entries_text.insert("1.0", content)
        entries_text.config(state="disabled")  # Make it read-only

        close_button = tk.Button(entries_window, text="Close", font=bodyFont, command=entries_window.destroy)
        close_button.pack(pady=5)

    except Exception as e:
        messagebox.showerror("Error", f"Could not read diary entries: {str(e)}")
```

Lastly, the clear diary button is very simple to create. I will just add a message asking if the user wants to delete their window for clarification (a mistake could delete a lot of meaningful text and information) and if so, I can use the tkinter delete method.

```
def clearDiaryEntry():
    """Clear the current diary text"""
    if messagebox.askyesno("Clear", "Are you sure you want to clear your current entry?"):
        diaryText.delete("1.0", tk.END)
```

I can now consider the test plan for this stage to see if the feature fully works as intended.

Because of how I implemented the diary tool, both test 1a and 1b can be tested simultaneously. To see if a file was created if you choose to open a diary you must attempt to open the diary after logging in; once the diary tool opens, you can see if the measurements and guide is provided.

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	User selects to create a diary	Valid	If the user chooses to create the diary, the file is created accordingly	
1b.	User selects to create a diary	Valid	Upon opening the diary, the measurements and advice is already in it	

Here's the test video that tests these features:

Test video: diaryFileTest.mp4

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	User selects to create a diary	Valid	If the user chooses to create the diary, the file is created accordingly	The diary is created
1b.	User selects to create a diary	Valid	Upon opening the diary, the measurements and advice is already in it	The user's metrics is displayed alongside a guide

Since these features worked as intended, I can move onto the next set of tests. Test 2 is non-essential, as the diary only saves information if the save entry button is pressed. On top of that, the diary menu is only displayed if you sign up and log in using the correct username and password, so the test is redundant.

Now I can move onto test 3 and 4. Both tests can be done simultaneously. For test 3 I can create an entry and save it, and then see if it appears the next time, I open it. For test 4, I will test the clear button to see if it successfully clears the text currently within the text box.

Test no.	Description + test data	Type of test	Expected outcome	outcome
3.	Add text to the diary	Valid	It saves next time and opens	
4.	Delete from the diary	Valid	It removes the text from the diary	

Test Video: diarySaveTest.mp4

As you can see in the video, the clear button removed the text from the video, and once the entry was saved, both the date-time and the message was displayed correctly.

Test no.	Description + test data	Type of test	Expected outcome	outcome
3.	Add text to the diary	Valid	It saves next time and opens	It saves and you can access the entry 25-36s
4.	Delete from the diary	Valid	It removes the text from the diary	It clears the text box 0.26s

Stage 4 references

1. GeeksforGeeks (2020). *PythonTkinter Scrollbar*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-tkinter-scrollbar/. [Accessed 1st Dec. 2025]

Stage 5 – Create the information and algorithm (UIs) (25th of November 2025)

These menus/UIs are there for the stakeholders who are confused on what they need to do to get an accurate reading or for stakeholders that are curious about how the program works.

The information menu will contain text that educates the user on how to get certain measurements and what they are, preventing any misconceptions and confusion. Additionally, it will tell them about cardiovascular diseases that the program is designed to diagnosis, raising awareness for those that may need it. The dataset that the program was trained against will be in the information section too, as it's important to be transparent about the program while providing credibility and reasoning for your conclusions.

Any general, common questions will also be identified and placed in this section, although most will likely be vague or just lead you to a practitioner as it's important to be medically verified to give advice on most situations.

The algorithm menu will be used to display how the algorithm works, with text-based explanations as well as graphical and mathematical explanations. I plan to allow the user to experiment with their inputs, but entering different values into a mock input section, allowing them to see how this changes the diagnosis in real time. This allows students to study logistic regression in a practical situation while also exploring how different factors lead to different diagnoses.

Although extreme data would normally be rejected, allowing this to happen in this tab can show the user how extreme data can skew a result massively, so it is important this is allowed.

Features I want to implement:

1. An informative menu that educates the user
2. An algorithm tab that allows you to see how the algorithm works
3. Allows the user to enter custom values
4. Updates the result as the user enters values

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Open the information menu	Valid	Contains all the text required	
2.	enter erroneous data into any field in the algorithm tab	Erroneous	Rejects the inputs and provides an error message	
3.	enter valid data into the fields	Valid	A diagnosis is provided, with the calculations being displayed accordingly	
4.	Enter valid data into all fields then change a value to another valid data value	Valid	Diagnosis updates accordingly based on the regression calculations.	
5.	Enter characters into any field	Erroneous	Rejects the inputs and an error message is provided	
6.	Enter boundary data on the unhealthy range for all fields	Boundary	Accepts the data and provides a diagnosis, likely very high risk of cardiovascular disease	
7.	Enter boundary data on the healthy range for all fields	Boundary	Accepts the data and provides a diagnosis, but likely very high risk of cardiovascular disease	

8.	Enter extreme data outside the range of the dataset	Extreme	Accepts the data and provides a diagnosis	

Due to time constraints, I am choosing to remove the algorithm half of this stage Firstly, I chose to remove this side of the stage due to the complexity of its implantation compared to the need and reward for including it within the program; if we consider its purpose, it was created so that students understand how logistic regression works in a practical sense. However, this is non-essential as the main stakeholders for the program are those seeking medical guidance, who are likely over 50 years old.

Keeping the information tab is essential, as it's important that any questions held by the users are answered within the program's description. Comparatively, it is much easier to implement as it only involves using frames and labels to add text to the menu.

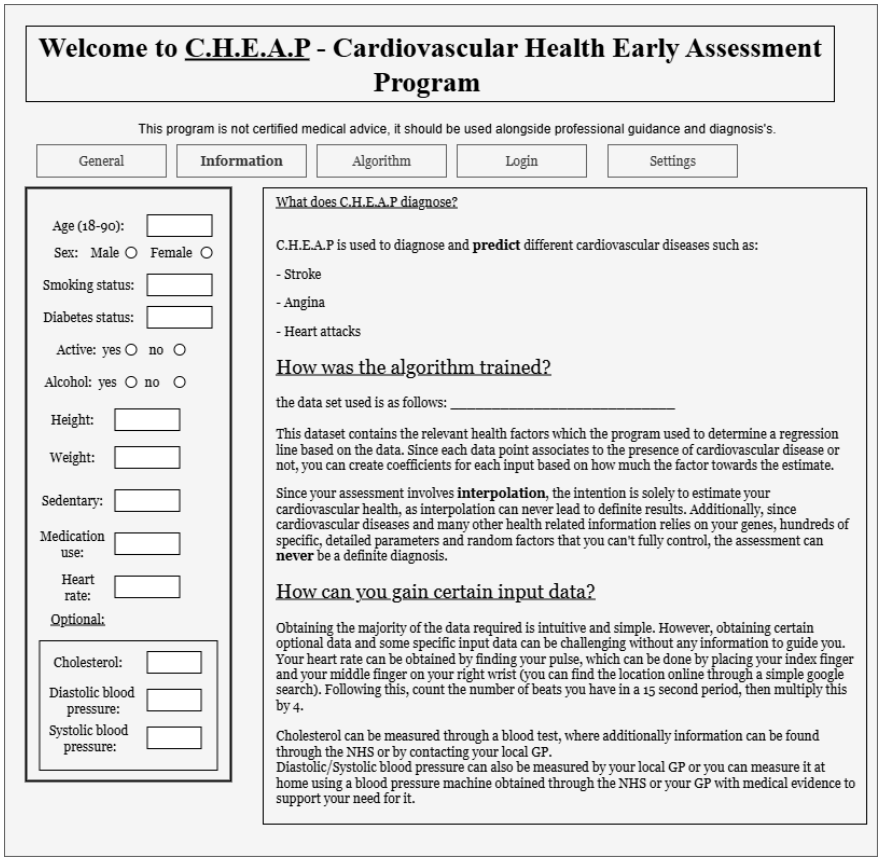
```
informationButton = tk.Button(menus,text = "Information",font = (bodyFont,12))
#algorithmButon = tk.Button(menus,text = "Algorithm",font = (bodyFont,12))
loginButton = tk.Button(menus,text = "Login",font = (bodyFont,12),command =loginButtonCommand)
settingsButton = tk.Button(menus,text = "Settings",font = (bodyFont,12),command = settingsButtonCommand)

#placing the buttons

menus.pack(pady=10)


generalButton.pack(side="left",expand=True,padx= 10)
informationButton.pack(side="left",expand=True,padx= 10)
#algorithmButon.pack(side="left",expand=True,padx= 10)
```

The only way this affects my program is the fact that the algorithm button needs to be removed. Which is easily done.

This results in the test table only containing the following:

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Open the information menu	Valid	Contains all the text required	

A lot of the information in my GUI design is what I aim to provide for the user:



Implementing this information in frames and labels can be done like this:

```
# Information Tab in rightSideContainer2
rightSideContainer2 = tk.Frame(root)  # Information menu


# Main bordered frame
infoWrapper = tk.Frame(
    rightSideContainer2,
    bd=2,
    relief="groove",
    padx=15,
    pady=15
)
infoWrapper.pack(anchor="nw", padx=10, pady=10, fill="both")

#define the content area
infoWidth = 700
wrapLen = infoWidth - 40


# SECTION 1 – What does C.H.E.A.P diagnose?
section1 = tk.Frame(infoWrapper)
section1.pack(anchor="w", pady=(0, 20), fill="x")

tk.Label(section1, text="What does C.H.E.A.P diagnose?", font=smallFontBold).pack(anchor="w")
tk.Label(
    section1,
    text="C.H.E.A.P is used to diagnose and predict different cardiovascular diseases such as:\n"
         "- Stroke\n"
         "- Angina\n"
         "- Heart attacks",
    font=bodyFont,
    justify="left",
    wraplength=wrapLen
).pack(anchor="w")
```

Firstly, I want to create wrapper to go around the information so that it is formatted better. I can then create first section, which stores the list of cardiovascular diseases the program tests against and is used for.

```
# Information Tab in rightSideContainer2
rightSideContainer2 = tk.Frame(root)  # Information menu


# Main bordered frame
infoWrapper = tk.Frame(
    rightSideContainer2,
    bd=2,
    relief="groove",
    padx=15,
    pady=15
)
infoWrapper.pack(anchor="nw", padx=10, pady=10, fill="both")

#define the content area
infoWidth = 700
wrapLen = infoWidth - 40


# SECTION 1 — What does C.H.E.A.P diagnose?
section1 = tk.Frame(infoWrapper)
section1.pack(anchor="w", pady=(0, 20), fill="x")

tk.Label(section1, text="What does C.H.E.A.P diagnose?", font=smallFontBold).pack(anchor="w")
tk.Label(
    section1,
    text="C.H.E.A.P is used to diagnose and predict different cardiovascular diseases such as:\n"
        "- Stroke\n"
        "- Angina\n"
        "- Heart attacks",
    font=bodyFont,
    justify="left",
    wraplength=wrapLen
).pack(anchor="w")
```

This next container can do something very similar; it just provides different information. The purpose of this section is to show that the output is an estimation, not a professional assessment that governs the right for medication usage. Including this section is very important so that the user's better understand the true purpose of the program.

```
# SECTION 2 — How was the algorithm trained?
section2 = tk.Frame(infoWrapper)
section2.pack(anchor="w", pady=(0, 20), fill="x")

tk.Label(section2, text="How was the algorithm trained?", font=smallFontBold).pack(anchor="w")

tk.Label(
    section2,
    text=
        "This dataset contains the relevant health factors which the program used to determine a "
        "regression line based on the data. Since each data point associates to the presence of "
        "cardiovascular disease or not, you can create coefficients for each input based on how "
        "much they factor towards the estimate.\n\n"
        "Since your assessment involves interpolation, the intention is solely to estimate your "
        "cardiovascular health, as interpolation can never lead to definite results. Additionally, "
        "cardiovascular diseases and many other health-related information rely on your genes, "
        "hundreds of specific parameters and random factors — the assessment can never be a "
        "definite diagnosis.",
    font=bodyFont,
    justify="left",
    wraplength=wrapLen
).pack(anchor="w")
```

Lastly, the third section just explains how you can get the measurements for certain inputs like cholesterol and blood pressure, as these can be hard to measure.

```
# SECTION 3 — How can you gain certain input data?
section3 = tk.Frame(infoWrapper)
section3.pack(anchor="w", fill="x")

tk.Label(section3, text="How can you gain certain input data?", font=smallFontBold).pack(anchor="w")

tk.Label(
    section3,
    text="Obtaining the majority of the data required is intuitive and simple. However, obtaining "
        "certain optional data and some specific input data can be challenging.\n\n"
        "• Cholesterol: measured through a blood test through the NHS or GP.\n\n"
        "• Blood pressure: measured using a blood pressure machine from the NHS or GP.",
    font=bodyFont,
    justify="left",
    wraplength=wrapLen
).pack(anchor="w")
```

The test plan is then very simple, as I just need to make sure the information is displayed properly.

Test video: informationMenu.mp4

Test no.	Description + test data	Type of test	Expected outcome	outcome
1.	Open the information menu	Valid	Contains all the text required	The information is formatted correctly

My full functional program video is:

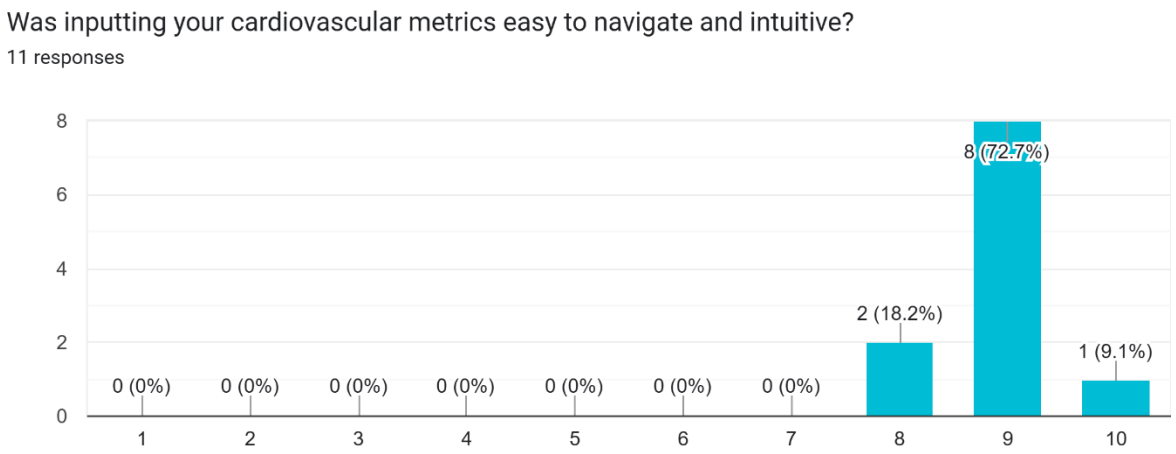
fullProgram.mp4

End Testing

In this section I will carry out the tests I mentioned in the post development test plan, starting with the usability tests.

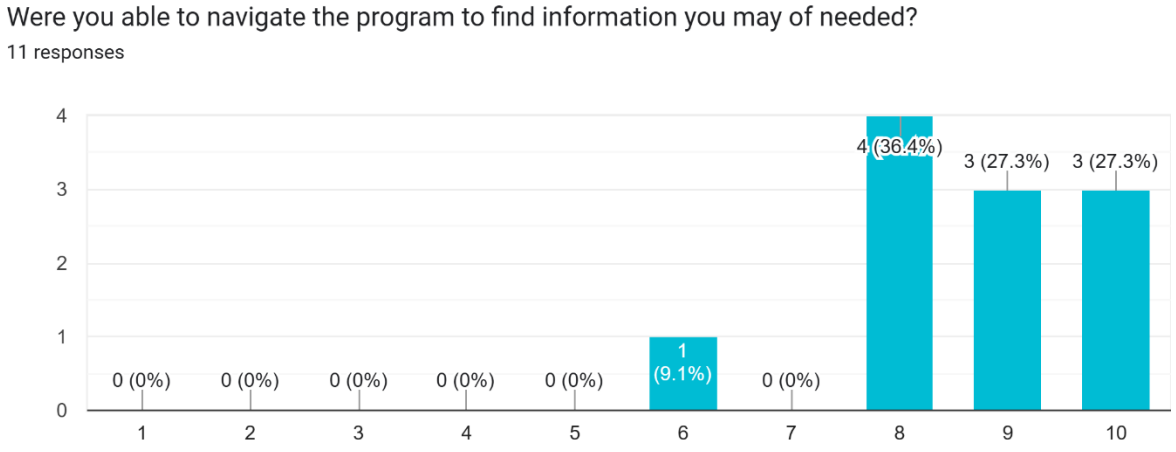
For this I will evaluate each of the responses to the questions I created in the survey during the design stage of my NEA.

1)



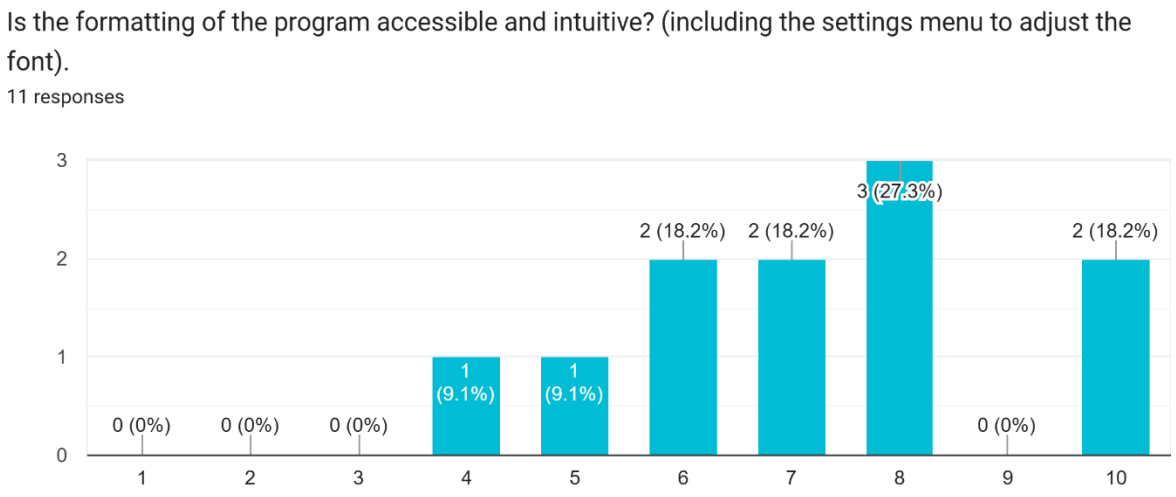
Based on the user feedback, the health metric input section was intuitive and easy to follow, considering the range of the feedback was only 2 and the lower bound was an 8. Additionally, the mean and mode of the data was a 9, indicating there was a consensus on the quality of this section.

2)



These scores indicate that holistically, information was accessible. The lowest score being a 6 isn't a concern, as the mean and the modal score was higher than this, sitting around an 8 for both. This indicates that this feature was well integrated and accessible for my stakeholders.

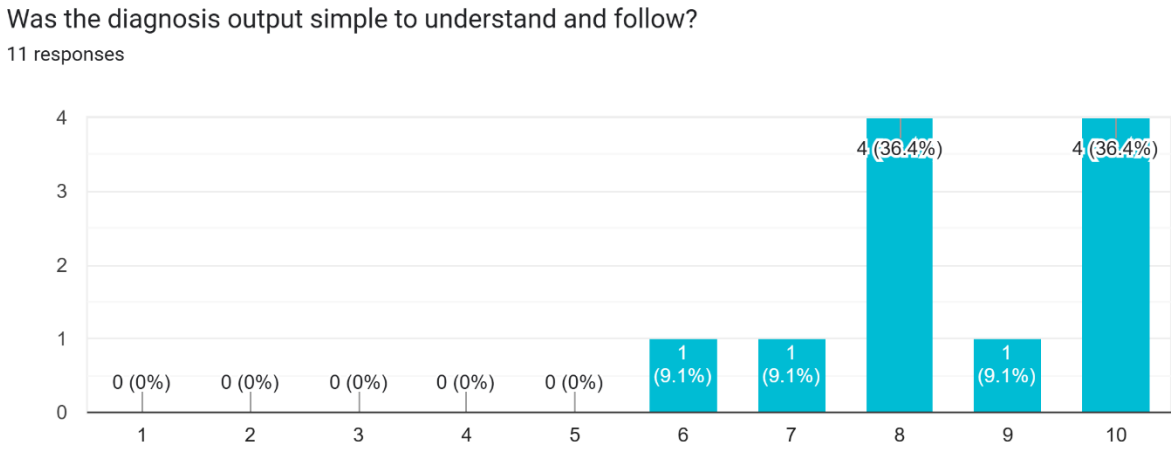
3)



This is the first question where a larger spread of answers was present, indicating a mixed bag of feedback. This may be due to personal preference within different stakeholders, which shouldn't be present in a program that is intended to be accessible for all. Since the question focused on formatting and font changing, I believe the lower scores were due to the poor container calibration as the font size got too large/too small. Certain text boxes would be cut off if you were in the largest font if you weren't in full screen. This can be seen in the following video:

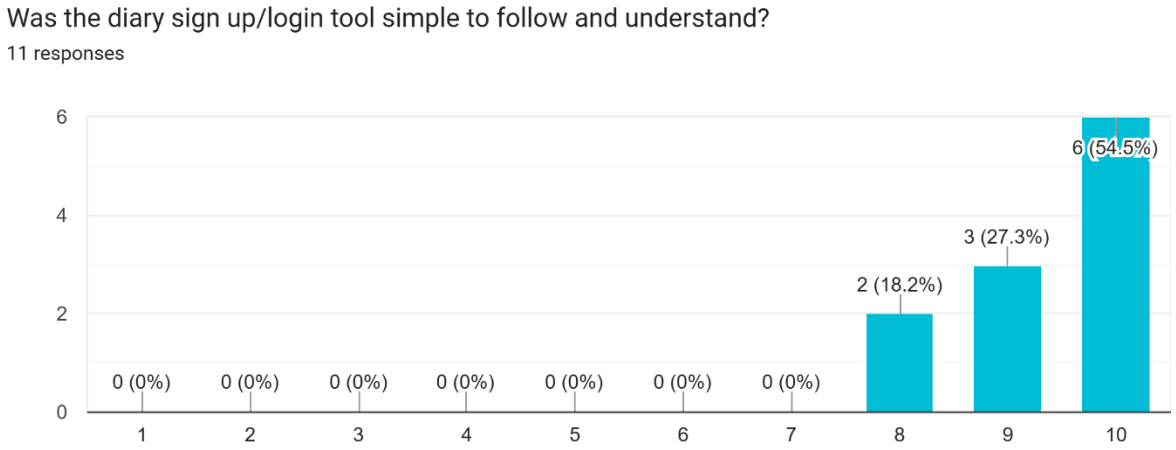
Video: UsabilityTestVideo.mp4

4)



Although the range in the scores is 4, there seems to be pooling on the scores 8 and 10, implying that the feature was well received by the stakeholders. The lower scores of 6 and 7, indicate that there could be improvements to this output page I could include, which I will detail in my evaluation and possibly come across in the final qualitative question on the survey.

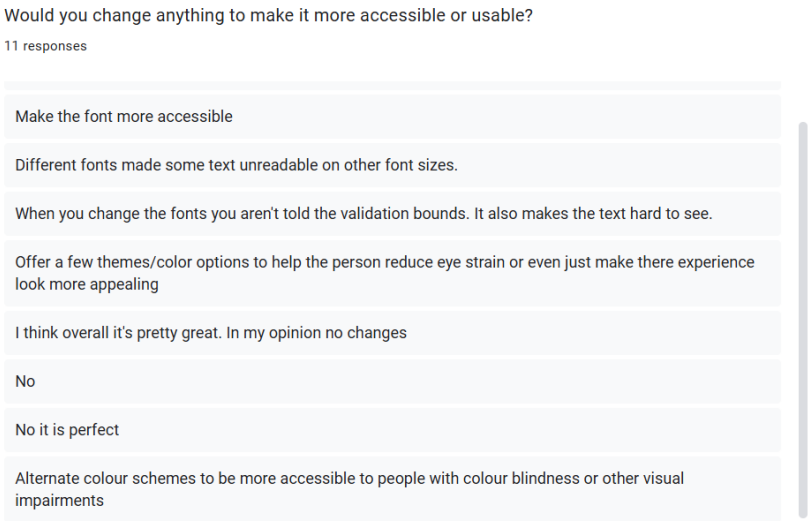
5)



The diary tool was clearly accessible and simple to use, which was as expected since it just requires a sign up and login page, which is very simple. Making this feature anymore complicated would likely reduce the accessibility for my stakeholders.

6)

Finally, the qualitative question that allows for more specific feedback follows:



Firstly, 2 users mentioned that the program appeared to be bland in terms of a colour scheme. Since the range of potential stakeholders (age) is very large, the younger group of my user base may wish the program to have a wider range of available colours. Interestingly, a user picked up on more colours to reduce colour blindness seclusion. Monochromasy is a form of colour blindness against black and white, which is something I didn't pick up on during the development of my program. Additionally, for certain users allowing for a colour scheme may make it more engaging.

Secondly, users appeared to pick up on the font size issue, which makes sense since that question received a large range of responses, with 1 falling as low as 5/10. Since a user may be visually impaired, they may increase the font size to make any text legible, but some of the containers didn't calibrate correctly for a smaller window size, causing text overlap and cutoff.

Later in my evaluation, I will mention how I can improve this feature, and potentially what new features I could introduce to make it easier for the user to operate my program.

I can now move onto the functionality tests.

Here is my functionality test plan:

Test no.	Description	Test Data	Expected outcome	outcome
1.	The program opens to a menu that displays general information.	N/A	Information is presented in a welcoming manner, making it intuitive to start.	
2a.	An intuitive and simple health metric input feature is implemented in the program.	Standard, healthy metric for a human, passing the validation checks for the program.	The inputs are taken and used to determine the user's cardiovascular health status.	
2b.	Following the previous test, there is an appropriate output outlining their cardiovascular health.	N/A	Their heart age/ traffic light score corresponds to a healthy individual with an appropriate text response.	

2c.	Alongside the information about the user's cardiovascular health, the health metrics that were likely the contributing factors are identified and outlined.	N/A	Since the user is healthy, there is likely no contributing factor identified.	
3.	Upon entering your health metrics, there is anonymous data storage, meaning that no data about the user is stored outside of the usage of the program.	N/A	There is no trace of data storage outside of logistic regression usage.	
4.(O)	A tool that allows you to see the view the mathematical side of the program, where you can enter health metrics and see visually how it effects the logistic regression model.	Various health metrics that the user can enter to visually analyse the logistic regression model.	The logistic regression model output (probability) varies corresponding to the altered health metrics.	
5.(O)	A tool that recommends diet changes to fit a healthier lifestyle.	N/A	Regardless of your health metrics it recommends what a healthy diet consists of.	
6.(O)	A diary tool that allows you to track your progress or record anything you wish.	Input information about you/ your progress and test if you're able to see previous entries	Previous entries are accessible to the user, and you can enter anything you wish to track your progress.	

1) To begin my tests, I will carry out test 1 within the video as follows:

Video: functionalityTest1.mp4

The video shows that the program opens to a menu that contains the input section as well as a section that contains general information about the program. Additionally, buttons are positioned at the top of the screen indicating what different menus are available to the user.

1.	The program opens to a menu that displays general information.	N/A	Information is presented in a welcoming manner, making it intuitive to start.	Worked as expected.

2a)

This test aims to see if you're able to take a user's inputs for the logistic regression model. The following video will also be used for test 2b and 2c, as once the inputs are taken in by the program, the output provided will test against the previously mentioned tests.

Video: functionalityTest2.mp4 {0 – 20s}

The video shows that the inputs have validation incorporated and they are taken and used for the diagnosis. Additionally, the compulsory and optional inputs are separated within the input menu, accordingly, making it easy to understand what is and isn't required.

2a.	An intuitive and simple health metric input feature is implemented in the program.	Standard, healthy metric for a human, passing the validation checks for the program.	The inputs are taken and used to determine the user's cardiovascular health status.	Worked as expected.

2b)

This test uses the same video as the previous test.

Video: functionalityTest2.mp4 {27 – 41s}

As you can see from the video, the user's cardiovascular health is displayed with their predicted heart age and their traffic light cardiovascular status. On top of that, a short paragraph accompanies their diagnosis, in case both previous outputs don't indicate clearly the state of their cardiovascular health.

Following the previous test, there is an appropriate output outlining their cardiovascular health.	N/A	Their heart age/ traffic light score corresponds to a healthy individual with an appropriate text response.	Worked as expected.

2c)

This test uses the same video as the previous test.

Video: functionalityTest2.mp4 {27 – 41s}

As you can see, I chose to not implement this feature within my program and indicated this, with my justification for scrapping this feature on pp. 170.

2c.	Alongside the information about the user's cardiovascular health, the health metrics that were likely the contributing factors are identified and outlined.	N/A	Since the user is healthy, there is likely no contributing factor identified.	N/A

3)

The reason this was an essential feature is because a lot of my stakeholders requested this in my survey, likely since medical information can be sensitive to a lot of people. Moreover, it's an essential feature as it is easy to implement; if the program doesn't store the user's inputs outside of the program code, it can't be accessed/utilised elsewhere.

Contrary to previous tests, I don't need a video of my program to prove this test. Since this involves program code, I can show that the user's information is only within the program code. The user's health metrics are displayed next to the diary, but it isn't stored within the text file to make sure that they aren't accessible. Additionally, previous entries from a user held within a text file are behind a username and password login, which can be kept private.

Video: functionalityTest3.mp4

Within this video you can see that the user's inputs are taken and passed into the regression model, with no external code saving implemented. Additionally, for the diary tool the user's health metrics are pulled directly from the array created as you enter your inputs, not from an external source.

3.	Upon entering your health metrics, there is anonymous data storage, meaning that no data about the user is stored outside of the usage of the program.	N/A	There is no trace of data storage outside of logistic regression usage.	No external saving of metrics is done.

4)

I chose to not implement this feature within my program due to time pressure, but also further reasoning found on pp. 188. To summarise, the feature was created for the usage of medical and mathematical students, but my main stakeholders are those looking for medical guidance that are not considering how the output is calculated.

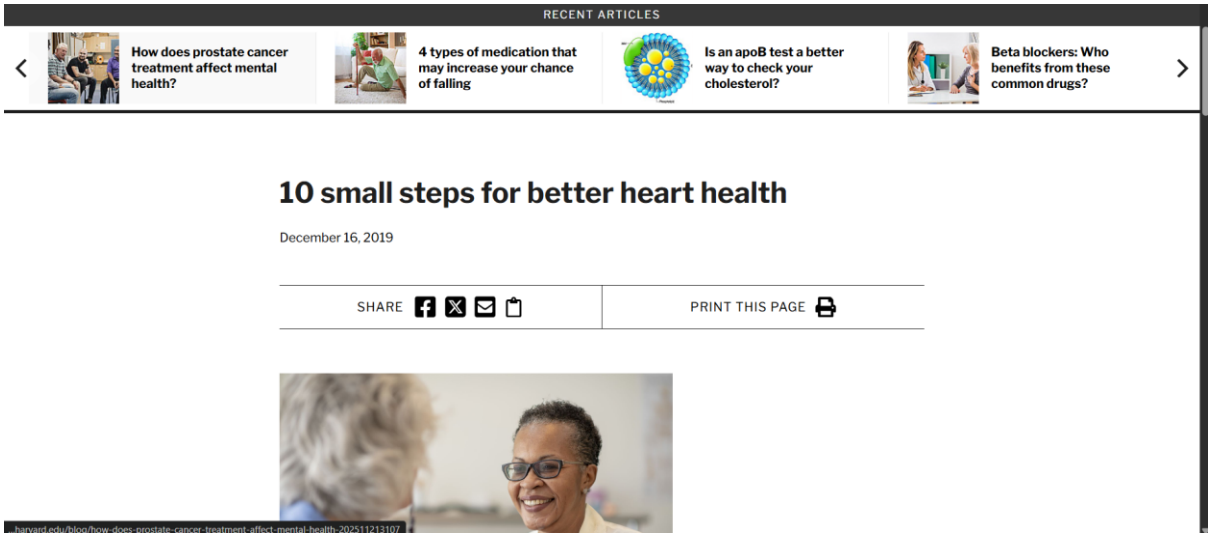
4.(O)	A tool that allows you to see the view the mathematical side of the program, where you can enter health metrics and see visually how it effects the logistic regression model.	Various health metrics that the user can enter to visually analyse the logistic regression model.	The logistic regression model output (probability) varies corresponding to the altered health metrics.	N/A

5)

I added a guide to the dairy section which indicates how you can work towards a better lifestyle in terms of your diet and daily habits.

Video: functionalityTest5.mp4

As you can see, a URL to this website is displayed:



This then aims to guide the user towards a healthier diet.

5.(O)	A tool that recommends diet changes to fit a healthier lifestyle.	N/A	Regardless of your health metrics it recommends what a healthy diet consists of.	Worked as expected.

6)

This test tests the diary tool I implemented. Here is the video testing this feature:

Video: functionalityTest6.mp4

As you can see from the video, the dairy tool includes the required features and works as intended.

6.(O)	A diary tool that allows you to track your progress or record anything you wish.	Input information about you/ your progress and test if you're able to see previous entries	Previous entries are accessible to the user, and you can enter anything you wish to track your progress.	Worked as expected.

<u>Here is my full functionality test plan completed:</u>

Test no.	Description	Test Data	Expected outcome	outcome
1.	The program opens to a menu that displays general information.	N/A	Information is presented in a welcoming manner, making it intuitive to start.	Worked as expected.
2a.	An intuitive and simple health metric input feature is implemented in the program.	Standard, healthy metric for a human, passing the validation checks for the program.	The inputs are taken and used to determine the user's cardiovascular health status.	Worked as expected.
2b.	Following the previous test, there is an appropriate output outlining their cardiovascular health.	N/A	Their heart age/ traffic light score corresponds to a healthy individual with an	Worked as expected.

			appropriate text response.	
2c.	Alongside the information about the user's cardiovascular health, the health metrics that were likely the contributing factors are identified and outlined.	N/A	Since the user is healthy, there is likely no contributing factor identified.	N/A
3.	Upon entering your health metrics, there is anonymous data storage, meaning that no data about the user is stored outside of the usage of the program.	N/A	There is no trace of data storage outside of logistic regression usage.	Worked as expected.
4.(O)	A tool that allows you to see the view the mathematical side of the program, where you can enter health metrics and see visually how it effects the logistic regression model.	Various health metrics that the user can enter to visually analyse the logistic regression model.	The logistic regression model output (probability) varies corresponding to the altered health metrics.	N/A
5.(O)	A tool that recommends diet changes to fit a healthier lifestyle.	N/A	Regardless of your health metrics it recommends what a healthy diet consists of.	Worked as expected.
6.(O)	A diary tool that allows you to track your progress or record anything you wish.	Input information about you/ your progress and test if you're able to see previous entries	Previous entries are accessible to the user, and you can enter anything you wish to track your progress.	Worked as expected.

I can now move onto carrying out the robustness tests from my robustness test plan, which is as follows:

Test no.	Description + Data	Test Type	Expected outcome	outcome

1.	Enter outside of range data 10 times, repeatedly encountering the error display	Erroneous	The message continuously displays and doesn't allow the user in	
2.	Enter a string 100 characters long as the age, height and weight of the user.	Erroneous	The inputs are still rejected.	
3.	Enter a number for the user's age, ending with ".0"	Erroneous	Whole integers only are accepted, so it is rejected	
4.	Repeatedly swap between menu buttons rapidly (10+ times).	Erroneous	The menus swap accordingly, leading to no errors	
5.	After a valid diagnosis, swap the health metrics drastically to replicate another user.	Valid	The new cardiovascular status based on this user is displayed	
6.	Enter a 100+ character string for the username and password for the diary tool.	Valid	The input is accepted and used as the username/ password	
7.	Attempt to change the font size of the program to 100.	Erroneous	The font size has an upper limit which doesn't allow this.	
8.	Copy pastes a 1000+ character string into the diary and save it as an entry.	Valid	The entry saves and is accessible.	
9.	Repeat the previous step 10+ times and attempt to visualise each entry to the diary.	Valid	All entries are accepted and visible to the user.	
10.	Attempt to enter non-English characters from another alphabet.	Valid	The entries are still accepted.	

1)

This test aims to brute force the error by repeatedly submitting an outside of range input. I will choose inputs that fall outside of all the ranges:

Age: 100

Height: 18cm

Weight: 18kg

Systolic blood pressure: 1500 mm / Hg

Diastolic blood pressure: 1200 mm/ Hg

Cholesterol levels: 1000 mg/dL

Glucose levels: 1000 mg/dL

Video: robustnessTest1.mp4

The video indicates clearly that the program couldn't be brute forced and the error message persisted.

1.	Enter outside of range data 10 times, repeatedly encountering the error display	Erroneous	The message continuously displays and doesn't allow the user in	Worked as expected.

2)

For the second test, I'll see if an extremely long string will affect the program. Since this test is designed to test the range checks for the inputs, I will use a number over characters, as characters are already not accepted. I will use the number 9 * 10^100 for the user's age, and the rest will be within bounds, acceptable inputs.

Video: robustnessTest2.mp4

As you can see from the video, the program still rejected the very long input, showing that the input is still checked properly.

2.	Enter a string 100 characters long as the age, height and weight of the user.	Erroneous	The inputs are still rejected.	Worked as expected.

3)

This test is very similar to the previous test, except I'm testing to see if the program does only accept integers, even with a decimal value of 0. The input I will use is '67.0'.

Video: robustnessTest3.mp4

As you can see, the input wasn't accepted as expected. However, I would change the text to display the input must be an 'integer', not a number, as decimal numbers are still clearly numbers.

3.	Enter a number for the user's age, ending with ".0"	Erroneous	Whole integers only are accepted, so it is rejected	Worked as expected.

4)

This test aims to see if rapidly swapping between the menus at the top of the program would break the program, causing a bug or issue. I will swap between the top menus that appear as you open the program.

Video: robustnessTest4.mp4

4.	Repeatedly swap between menu buttons rapidly (10+ times).	Erroneous	The menus swap accordingly, leading to no errors	Worked as expected.

5)

The test is to see if the diagnosis updates even after a user alters their health metrics, which it should, and a new cardiovascular status should be displayed.

I'll use the two users as my cases:

Case 1:

age: 18 years

gender: Female

height: 170cm

Weight: 65kg

systolic blood pressure: 120 mm /Hg

diastolic blood pressure: 80 mm/ Hg

cholesterol: 100 mg / dL

glucose: 90 mg / dL

smoke: No

alcohol: No

active: Yes

Case 2:

age: 87 years

gender: Male

height: 150 cm

Weight: 82 kg

systolic blood pressure: 127 mm /Hg

diastolic blood pressure: 93 mm /Hg

cholesterol: 160 mg / dL

glucose: 90 mg / dL

smoke: No

alcohol: Yes

active: No

Video: robustnessTest5.mp4

As you can see from the video, when you alter the user's inputs, the output display breaks and the old display isn't removed. This means that the user wouldn't be able to alter their metrics without restarting the program.

To fix this issue, I would completely clear the frame that contains the information of the user's information. This means that when the new information is created by the regression model it is the only thing that occupies the container.

5.	After a valid diagnosis, swap the health metrics drastically to replicate another user.	Valid	The new cardiovascular status based on this user is displayed	The old display isn't cleared, causing a double image.

6)

For both the username and password I will enter the string JamesCook copy pasted 100 times and see if this allowed me to log into the program. I chose not to include a hard capped username and password as people like to keep their system secure, potentially creating very long passwords.

Video: robustnessTest6.mp4

As you can see by the video, there appears to be a character limit when attempting to create files using python file handling.

To fix this in the future, I would introduce a hard limit to the length of a username and password, potentially to around 20 characters. This would prevent any extremely long usernames/passwords causing errors and issues. Luckily, I believe users would likely use shorter and easier to remember usernames/passwords, so this wouldn't be an issue people would run into.

6.	Enter a 100+ character string for the username and password for the diary tool.	Valid	The input is accepted and used as the username/ password	Large usernames/passwords don't allow for python file handling.

7)

This test is to see if the font size can be changed outside of the designated range.

Video: robustnessTest7.mp4

As you can see from the video, you're unable to enter extremely large font sizes, as there is an upper limit to this. I do wish the limit was displayed to the user for extra clarity, but I didn't. This would be an easy fix, as it could go in the box where you could enter available font sizes.

7.	Attempt to change the font size of the program to 100.	Erroneous	The font size has an upper limit which doesn't allow this.	Worked as expected.

8)

This test is like test 6, I intend to see if a large entry can be saved. This should be allowed, as a user may want to write a lot to track certain metrics, diet changes or lifestyle changes.

I'm going to use the same text I used in test 6 but with triple the number of characters, securing a string of at least 1000 characters.

Video: robustnessTest8.mp4

The video shows that the entry was saveable.

8.	Copy pastes a 1000+ character string into the diary and save it as an entry.	Valid	The entry saves and is accessible.	Worked as expected.

9)

This test is an extension of test 8, and it is used to see if the case stills hold for multiple entries. I will use the same string I used in test 8 for this test:

Video: robustnessTest9.mp4

The video shows that despite the increase in the number of entries, all were possible and can be seen within the appropriate menu.

9.	Repeat the previous step 10+ times and attempt to visualise each entry to the diary.	Valid	All entries are accepted and visible to the user.	Worked as expected.

10)

This test just involves entering non-roman characters, which should be saveable as the string is saved as an entry to the text file, and strings aren't restricted to roman characters.

Video: robustnessTest10.mp4

The video shows that the Arabic alphabet can still be displayed within the entry box.

10.	Attempt to enter non-English characters from another alphabet.	Valid	The entries are still accepted.	Worked as expected.

Here is the full, filled in test plan with the results from each individual test.

Test no.	Description + Data	Test Type	Expected outcome	outcome
1.	Enter outside of range data 10 times, repeatedly encountering the error display	Erroneous	The message continuously displays and doesn't allow the user in	Worked as expected.
2.	Enter a string 100 characters long as the age, height and weight of the user.	Erroneous	The inputs are still rejected.	Worked as expected.
3.	Enter a number for the user's age, ending with ".0"	Erroneous	Whole integers only are accepted, so it is rejected	Worked as expected.
4.	Repeatedly swap between menu buttons rapidly (10+ times).	Erroneous	The menus swap accordingly, leading to no errors	Worked as expected.
5.	After a valid diagnosis, swap the health metrics drastically to replicate another user.	Valid	The new cardiovascular status based on this user is displayed	The old display isn't cleared, causing a double image.
6.	Enter a 100+ character string for the username and password for the diary tool.	Valid	The input is accepted and used as the username/ password	Large usernames/passwords don't allow for python file handling.
7.	Attempt to change the font size of the program to 100.	Erroneous	The font size has an upper limit which doesn't allow this.	Worked as expected.
8.	Copy pastes a 1000+ character string into the diary and save it as an entry.	Valid	The entry saves and is accessible.	Worked as expected.
9.	Repeat the previous step 10+ times and attempt to visualise	Valid	All entries are accepted and visible to the user.	Worked as expected.

	each entry to the diary.			
10.	Attempt to enter non-English characters from another alphabet.	Valid	The entries are still accepted.	Worked as expected.

Evaluation

In my evaluation I need to explore each of my success criteria and identify if this was fully met, partly met or not met/implemented in my program. I will begin by exploring the criteria that I labelled as essential.

Essential:

Criterion:

'An **opening page** containing text that details the function of the program in a large, easy-to-read font'

I believe this feature was **fully met** – 9/10

As you open the program, the general menu is opened displaying the input menu and additional information about the program. Additionally, the tabs at the top of the program are very intuitively named and accessible. The proof of this can be found on **page 192**. Although the menu received possible feedback and it was fully implemented. The menu displayed uses a suitable font size for varied stakeholders upon opening the program, and the font used is accessible for the users.

I could've included more information/features within this opening page, but since simplicity and intuitiveness were a priority for my stakeholders, making it more complicated and congested may have decreased accessibility.

Criterion:

'A Simple, intuitive **user interface** that allows users to enter their data, with some boxes being optional'

I believe this feature was **partially met-** 7/10

Overall, the user interface is simple and intuitive, making it easy to use upon opening the program. Additionally, input information can be easily found using buttons that are readily available for the user. The menus available for the user are simple in terms of how they're named but also their function; no medical jargon was used as this was specifically requested by a user on **page 5**. The optional and compulsory inputs are separated in their own boxes, making it clear what inputs are required and what aren't. On top of this, the method if input taking is simple, with text boxes allowing the user to type their health metrics and with a simple validation system feeding errors in the case that their information isn't sufficient.

However, the reason this criterion wasn't fully met is due to a lack of information and implementation issues with the settings menu. Despite this only being 1 menu causing errors, this menu is essential for creating a simple and effective user interface. As you can see from the video found on **page 193**, a large font and font size leads to text overlap and deletion. This means that users that may be visually impaired would be unable to increase the font size without running into sizing issues. Furthermore, the upper and lower limit of the available font sizes isn't displayed, making it more difficult for the user to understand what they're capable of.

Improvements:

1. Firstly, the easiest fix required would be displaying the bounds for available font sizes, providing clarity for my users for what they're capable of. This can be done by adding the text within the label that contains the entry boxes. This would be easily maintainable. As the text can be altered or deleted by removing the additional label I would add.

2. To fix the font calibration issue, I could brute force validate how different fonts appear with different font sizes, ensuring that only sizes and fonts that allow the menus I use to exist without overlap or deletion. Although this would require brute force extensive testing, font calibration for different menus is difficult in a simple interface like tkinter. This feature would be a **top priority** and wouldn't require significant time to implement. One issue with this implementation is that it wouldn't be very maintainable, as any further required changes would require a similar approach of brute force testing. To better this approach, I could use another library that allows for better rendering and font calibration than tkinter. This may allow me to dynamically alter the size of containers as the font size increases, making it work.

3. **If I had more time and if I conducted more rigorous research**, I would add alternate colour schemes that the user's could pick within the settings menu. A rare form of colour blindness known as achromatopsia effects black and white visuality, which I didn't believe was possible. Additionally, since the age range of my stakeholders is so large, I could implement a range of colour schemes to make the program more engaging for a younger audience (18-40). These features would be easy to implement using font manipulation while making a large contribution for my stakeholders, so they would be **essential.** Additionally, maintaining these features

would be simple, as you can edit and add more fonts that use various colours quite easily.

Criterion:

'A direct and logical **evaluation of their cardiovascular health**'

I believe this feature was **fully met** – 9/10

I was able to implement the two most requested forms of cardiovascular status feedback found on **page 4**, meaning that the user's preferences were integrated in my program. Additionally, to accompany both the traffic light and heart age system, a short paragraph outlines what these mean to provide context to the images. This is important as potentially the images may not render for different devices. Furthermore, a video accompanies the user's output providing clarity to what cardiovascular diseases are, and what may have potentially caused them. This video can be found on **page 191**.

Improvements:

What I would have added to make the output page better match my requirements is I would have included varied diet plans that aimed to reduce/prioritise different aspects of a healthy diet depending on what the user requires. This would have been easy to implement and maintain, as it would have mostly required further research into how you can create these diet plans, and then implementing this into my program would just require a few extra labels which could've been added at various stages in my menu.

Criterion:

'Displaying which **specific factors likely contributed to the result**'

I believe this was **not met** – N/A

I chose to not implement this within my program, as I believed that different features required prioritising and that this feature wasn't as essential as I previously thought. I mentioned how this feature isn't an essential feature on **page 170**. Since this feature was being implemented before the diary feature, I chose to prioritise this feature as it's more essential to allow my stakeholders to track their growth. Also, I believe it is an easier to implement feature as it only requires python file handling.

Improvements:

Adding this criterion would require me to do further analysis on the user's health metrics, to where I could identify which factor would be an outlying factor. This wouldn't be difficult to implement, as I could create a "healthy" user with standard, healthy metrics stored for all

the required input fields; I can then compare what the user's inputs were against this sample healthy user, and if the absolute difference in certain fields are too large I can add it to a 'contributing factor' array. To then see if there were 1 or 2 outlying factors, I could check what the length of the list was and if it was longer than a certain length (2), I could consider them to have no outlying factors. This would also be easy to maintain as well as it utilises array handling which can be reused. In future improvements to the program, I would incorporate this within the output section so that the user has clarity to what may of contributed.

Criterion:

'An appropriate follow up that provides **advice** and **guidance** related to your **cardiovascular health**.'

I believe this was **partially met** – 5/10

I believe this was partially met as a lot of the requirements for this criterion were met, but I could have included more information to assist the user and laid it out better. Firstly, the video I provided does outline cardiovascular diseases and what they could be caused by, but this is very general. Additionally, there is general lifestyle and diet advice found within the diary tool. On top of this, the diary tool itself is a form of advice and guidance, as I detail how journalling and recording your progress would be beneficial to know how much progress you're making.

Improvements:

Features that were either underdeveloped or missing include specific, varied dietary plans tailored to different cardiovascular needs for my users. This could then be displayed within the output page, providing extra information my stakeholders could use to better their cardiovascular health. Implementing this wouldn't be difficult and it would be maintainable, as it would only require configuring text labels that display below the traffic light/heart display frames.

Criterion:

'**Anonymous data storage** or a **lack of storing user's data**'

I believe this was **fully met** – 10/10

This feature is easy to check for and to implement, and you can see my implementation and testing on **page 198**. Since my stakeholders mentioned anonymous data storage in my survey (page 6), implementing this as an essential feature made the most sense, since to not store the user's data I simply need to not actively create some form of long-term data storage. As mentioned in page 198, the user's health metrics are displayed next to the diary, not within the diary so the user is given the choice of adding it to their diary or not to maintain confidentiality.

Improvements:

If I were to incorporate some form of data storage, I could do so anonymously. I could then use this data for research purposes, like identifying the most common user archetype for my program to target more specific advice and guidance to my users. However, since this would then create complicated user right concerns, I would need to ensure there would be user consent to implement this feature, and since it wouldn't be a very beneficial feature this would only be something I would incorporate if it was requested by my stakeholders. This feature would also be maintainable, as it would likely utilise file handling, which can be altered or changed once you have access to the file, I used to store confidential information.

Since this was the last of the essential features I needed to implement, I can move onto the important and optional features, where I chose to not implement different features for a multitude of reasons. Firstly, here are the Important yet non-essential features.

Important:

Criterion:

'**Calculation display** through mathematical equations and potentially graphs.'

This criterion was **not met-** N/A

I chose to not implement this feature for reasons which can be found on **pages 188 and 199**. Despite this feature being less important for the majority of my stakeholders for my program, it would be a feature I would like to include in any future developments of my program since it widens the scope of my program to students and educators.

Improvements:

As a mathematician, the inner workings of the logistic regression are very interesting to me and I believe it would be too for medical students, but also to other mathematicians that may not be keen on cardiovascular health. This shows that this feature alone could broaden my potential stakeholders, leading to my project doubling as an advisory and educational program. I could use graphing libraries like matplotlib and seaborn within tkinter to dynamically update a graph that represents how the probability is generated from the logistic regression model using the sigmoid function. This would improve transparency, support learning, and help users better understand how the model reaches its conclusions.

Criterion:

'A tool that recommends **diet changes**'

I believe this was **partially met** – 4/10

Considering I implemented a link that indicates what dietary improvements/ lifestyle changes would be helpful to create a healthier lifestyle (**page 180)**, I can say that this criterion was partially met. Alongside this link, users can use the diary tool to track their dietary progress or any progress for that matter, meaning the users are able to track and receive dietary improvements.

However, the dietary recommendations are directly provided within the program, they are held within a link that leads to an external website. Additionally, this website contains vague and general guidance for your diet, and it doesn't provide a detailed plan.

Improvements:

I would wish to create multiple week by week dietary plans that differ based on what aspect of your diet needs improving the most. This would be maintainable and quite easy to implement within my program, but it would require vigorous planning and researching to create accurate and helpful plans. This therefore made it difficult to implement within my program, since I believed there were other features like the dietary tool that needed prioritising, as this tool provides freedom of expression for my users. To implement this into a future development of my program I would ensure that I have a planned time slot to where I can research and create these more tailored diet plans.

Criterion:

'A **diary tool** to track your progress'

I believe this was **fully met** – 10/10

This feature has been mentioned throughout other criterion since it is such a pivotal and useful feature that ties into other essential and important features for my program. Despite this feature being labelled as important since it isn't part of the necessary features for a diagnosis program, it was the feature I prioritised out of all the optional ones. As you can see on **page 186**, you're able to sign up and log into the diary, using your own unique username and password to ensure your information is secure. Once you log in, you are then able to save entries to your diary and view all your previous entries, meaning you can visually see your progress over time and understand where you were a few weeks or months ago. This feature was also taken well by my stakeholders in terms of usability, which can be seen on **page 194**. The simplicity of the feature is what makes it accessible and usable; certain features could have been more complicated, but the diary tool is useful for my program because the users have free reign over how they wish to use it. The dietary tool is also maintainable, as it utilises python text files which can be checked and investigated to see how my code effects the text files

Improvements:

There are certain features that work alongside the diary tool, like dietary plans, but since the dietary plans weren't as strong as the diary, the diary tool isn't as useful as it could have been. To improve the efficacy of the dietary tool, I would provide the user with more information that they can then talk about/document in their diary.

I can now move onto the optional and less important features, both of which I chose to not implement. These features were barely not considered limitations, as they would have potentially benefited the program by providing more functionality for my stakeholders, but implementing them would require a lot of complex programming and legality concerns.

Optional:

Criterion:

'The ability to **contact the NHS** through the program'

I believe this was **not met** – N/A

Implementing this within my program would provide my users with the direct ability to contact the NHS or their local GP to book a checkup following the cardiovascular status diagnosis, but there are many setbacks that makes this quite difficult to implement.

Firstly, since the NHS is funded by the government and an official healthcare system, implementing a feature that allows you to directly contact them would require some form of legal partnership. Establishing such an agreement would involve extensive administrative processes and potentially financial costs, such as licensing or compliance fees. This wouldn't be worth it since users can indirectly contact the NHS through different websites or telephone numbers.

Secondly, implementing and validating this within my program would be complex, likely leading to me either spending more time on the program then I have or cutting out other features which would be more impactful for my stakeholders.

As a result, this functionality was excluded to allow greater focus on features that directly improve accessibility, usability, and the accuracy of cardiovascular risk assessment.

Criterion:

'The ability to **share your progress and results** with people of your choosing'

I believe this feature was **not met – N/A**

Similar to the previous criterion, this would have been a useful inclusion for my program, but the complexity of implementing it wouldn't be worth the little benefit.

Firstly, confidential data storage was already a concern for some of my stakeholders (see **page 6**), so I assume a lot of my stakeholders wouldn't wish to share their progress through the program. Additionally, implementing this within my program would require a 'friends' system where you can create an account and befriend other people. This would require authentication, access permissions and networking logic that would take a considerable time to plan and implement within my program. Despite the complexity, it wouldn't be a very useful feature; my stakeholders will be able to share their progress/results using third party apps like WhatsApp or email without hassle, which may be preferred due to familiarity.

I would therefore allow users to export their results/diary as a PDF, which they can send directly from the program to contacts or through email addresses. This would also be maintainable as it avoids user accounts, servers and long-term data storage, making it much less of a security liability. It also keeps the feature separated from other essential features within the program, as this module could be improved or implemented without affecting other aspects of the program.

Limitations:

- It takes ≈ 8-10 seconds for my laptop to use the logistic regression model to compute the user's cardiovascular status, and on a slower device or smaller device, this many take significantly longer. I could either attempt to implement a better timer that informs the user how long it would take to compute their diagnosis, or I could implement a more efficient algorithm. However, this logistic regression model is optimised, and the thing that can vary is the number of iterations I use and the learning rate. This would make the program run faster, but the accuracy may decrease, so in future developments, I would experiment with different parameter values across a range of devices to find a balance between speed and accuracy that best suits user preferences.
- Since the program relies on external resources such as images, text files, CSV files, and multiple Python libraries, all of which must be stored in the same directory, distributing the program to stakeholders is challenging. They would need to download the required python libraries unless I am able to force this upon downloading the program.
- As mentioned earlier in my usability test plan on **page 195,** the GUI is simple and intuitive but not appealing, and since my age range is 72 years, the younger audience

may prefer a variety of colour schemes. In future developments, I would implement multiple colour schemes or themes to better accommodate different user preferences.

- On top of this, even without the font size calibration issues mentioned on **page 193**, entering full screen leads to smaller containers that aren't necessarily tailored to varied GUI dimensions. In future developments of the program I would try and have the containers dynamically update as the dimensions of the GUI menu change. This would then allow the user to change their window size, and the font sizing and menu sizing would adjust alongside it.
- Due to time constraints, I only ever attempted logistic regression for the dataset, when other algorithms like KNN were available for usage. In future developments of the program, I would like to compare the accuracy of KNN and logistic regression and how long each take to run within my program, as KNN may have been the better approach, but I never tried to implement it.

Conclusion:

Overall, I'm happy with how my program turned out. The most essential feature (the logistic regression model integration) was successful allowing me to provide a statistically significant diagnosis for my stakeholders. Additionally, I believe the GUI was intuitive and basic enough for the wide range of ages that would use my program.

I believe certain features I included were ambitious in their programming complexity like the mathematical tab to display how logistic regression works, so throughout my development plan I should have considered the time taken to develop certain features more closely. However, I planned the most essential features regardless of their complexity to be completed first, which allowed me to have a fully functional program which could afford losing certain less important features. This approach ultimately resulted in a robust and usable final program.